PUBLICATION EDITION

Platform Engineering · DevOps · SRE 实战教材 —— 每一个结论都有执行记录

企业级生产环境高保真仿真

工程问题 → 技术原理 → 架构方案 → 实现 → 实验验证 → 结果分析 → 局限与启示

13 章
从平台基线到能力考核，E0–E12 完整的知识体系

78 幅图
机制示意图、实测数据图、真实截图与终端记录，分类标注

572 处
正文陈述与原始证据逐条自动比对

8 起
计划外的真实事故与事件，连同复盘写进正文

实验仓库：github.com/miku-wwl/devops-delivery（分支 v2/publication，目录 v2/）

实验环境：一台笔记本 · Windows 11 + WSL2 + Docker + Kind（Kubernetes v1.37，1 + 3 节点）

编写方式：Claude Code 在本机逐阶段实际执行实验，按真实输出、指标与截图编写；实验环境的能力验证不等于真实企业的生产经验

第二册 · Publication Edition · 2026-11

目录

阅读指南 6

第 1 章 平台基线：在一台电脑上模拟三可用区集群（E0） 8

1.1 学习目标 8
1.2 原理 8
1.3 设计 11
1.4 实现与验证 14
1.5 局限：这个集群不是什么 17
1.6 常见问题 17
1.7 练习 18
1.8 本章小结 18

第 2 章 多服务系统：storefront 与它的平台（E1） 19

2.1 学习目标 19
2.2 原理 20
2.3 系统设计 24
2.4 实现要点 26
2.5 验证 28
2.6 排障记录：第一次部署暴露的问题 32
2.7 局限 34
2.8 常见问题 34
2.9 练习 35
2.10 本章小结 35

第 3 章 服务网格：身份、加密与 L7 流量（E2） 37

3.1 学习目标 37
3.2 原理 38
3.3 安装：从 Git 部署网格 40
3.4 实验 1：线上看到了什么 41
3.5 实验 2：为什么负载不均 42
3.6 实验 3：网格的代价，以及如何测准它 44
3.7 实验 4：重试在层与层之间叠加 47
3.8 实验 5：开启 STRICT 之后，什么会坏 49
3.9 一次真实的事故：扣了款，订单却失败了 49
3.10 追查：900 毫秒的停顿 51
3.11 局限 52
3.12 常见问题 52
3.13 练习 53

3.14 本章小结 53

第 4 章 负载与容量规划（E3） 55

4.1 学习目标 55
4.2 原理 56
4.3 准备：一个可以信任的测量工具 57
4.4 实验 1：观测的成本 58
4.5 实验 2：一个比业务还贵的配置错误 58
4.6 实验 3：第一次阶梯负载，以及三个不该相信的数字 60
4.7 实验 4：CPU 花在哪里——剖析、优化与金丝雀对比 62
4.8 实验 5：利特尔定律检验的是测量 64
4.9 实验 6：容量模型——先预测，再验证 65
4.10 局限 71
4.11 常见问题 72
4.12 练习 72
4.13 本章小结 73

第 5 章 弹性与调度（E4） 75

5.1 学习目标 75
5.2 原理 76
5.3 实验 1：把副本数交给 HPA 78
5.4 实验 2：HPA 的反应，以及它看不见的过载 79
5.5 实验 3：按队列积压扩容（KEDA） 82
5.6 实验 4：CPU 限制与限流 84
5.7 实验 5：吵闹的邻居与 CPU 请求 85
5.8 实验 6：节点内存压力与驱逐 86
5.9 实验 7：优先级与抢占 89
5.10 实验 8：配额、默认值与拓扑分布 89
5.11 局限 90
5.12 常见问题 91
5.13 练习 92
5.14 本章小结 92

第 6 章 多租户与安全治理（E5） 94

6.1 学习目标 94
6.2 原理 95
6.3 实验 1：租户与身份 96
6.4 实验 2：权限矩阵 97
6.5 实验 3：审计日志回答“谁做了什么” 98
6.6 实验 4：网络隔离 99
6.7 实验 5：准入策略：Kyverno 与内置策略 101

6.8 实验 6：Argo CD 的项目与权限 102
6.9 实验 7：外部密钥管理 103
6.10 局限 104
6.11 常见问题 105
6.12 练习 105
6.13 本章小结 106

第 7 章 故障注入与传播（E6） 107

7.1 学习目标 107
7.2 原理 108
7.3 实验 1：依赖变慢与失败：两种韧性模式 109
7.4 实验 2：节点失效 111
7.5 实验 3：可用区分区 112
7.6 实验 4：GameDay 与复盘 114
7.7 局限 115
7.8 常见问题 116
7.9 练习 116
7.10 本章小结 117

第 8 章 数据可靠性与灾难恢复（E7） 118

8.1 学习目标 118
8.2 原理 119
8.3 准备：备份目标、容量与插件 121
8.4 实验 1：PostgreSQL 的故障切换 122
8.5 事故：脑裂之后，一个副本悄悄停止了复制 124
8.6 实验 2：时间点恢复：找回被误删的订单 125
8.7 实验 3：Velero：备份“已完成”，数据却不在 126
8.8 实验 4：OpenBao 的快照与恢复演练 127
8.9 实验 5：整个集群损毁之后 127
8.10 局限 129
8.11 常见问题 130
8.12 练习 131
8.13 本章小结 131

第 9 章 发布治理（E8） 132

9.1 学习目标 132
9.2 原理 133
9.3 实验 1：合并请求门禁 135
9.4 实验 2：第 7 层入口下重做扩容台阶 136
9.5 实验 3：5% 的金丝雀、预览与撤回 138
9.6 实验 4：按项目的同步冒用 139

9.7 实验 5：变更冻结与紧急变更 141
9.8 部署事件与 DORA 指标 142
9.9 实验 6：空集群上的同步波次 143
9.10 局限 146
9.11 常见问题 147
9.12 练习 147
9.13 本章小结 148

第 10 章 可观测性与事故响应（E9） 149

10.1 学习目标 149
10.2 原理 150
10.3 实验 1：一条永远在响的告警 152
10.4 实验 2：告警离开集群 152
10.5 实验 3：被删除的 Pod 的日志，和谁读了哪个密钥 153
10.6 实验 4：事故演练——一个测试参数进入了生产 154
10.7 实验 5：尾部采样与 exemplar 157
10.8 局限 159
10.9 常见问题 160
10.10 练习 160
10.11 本章小结 161

第 11 章 控制面的规模边界（E10） 162

11.1 学习目标 162
11.2 原理 163
11.3 实验 1：一万个虚拟 Pod 的调度（K1、K2） 166
11.4 实验 2：etcd 里的一个 Pod（K3） 167
11.5 实验 3：一万个 Pod 的 LIST（K4） 169
11.6 实验 4：一个失控的客户端（K5）与事故 E10-1 169
11.7 规模边界 174
11.8 局限 174
11.9 常见问题 175
11.10 练习 176
11.11 本章小结 176

第 12 章 基础设施即代码与云迁移（E11） 177

12.1 学习目标 177
12.2 原理 178
12.3 实验 1：接管已有的门禁（I1） 179
12.4 实验 2：状态加密，锁真的锁住了吗（I2） 180
12.5 实验 3：手工改动能被发现吗（I3） 181
12.6 实验 4：Argo CD 管理自己（I4） 182

12.7 事故 E11-1：恢复脚本毁掉了唯一的解封密钥 183
12.8 云迁移分析：只陈述可核实的事实 185
12.9 局限 186
12.10 常见问题 186
12.11 练习 187
12.12 本章小结 187

第 13 章 能力考核（E12） 189

13.1 学习目标 189
13.2 考核的设计原则 190
13.3 架构设计题 190
13.4 故障排查题 190
13.5 实操题：关闭事故 E11-1 的行动项 5 192
13.6 局限 193
13.7 第二册的边界 193

附录 A 练习参考答案 194

附录 B 术语表 209

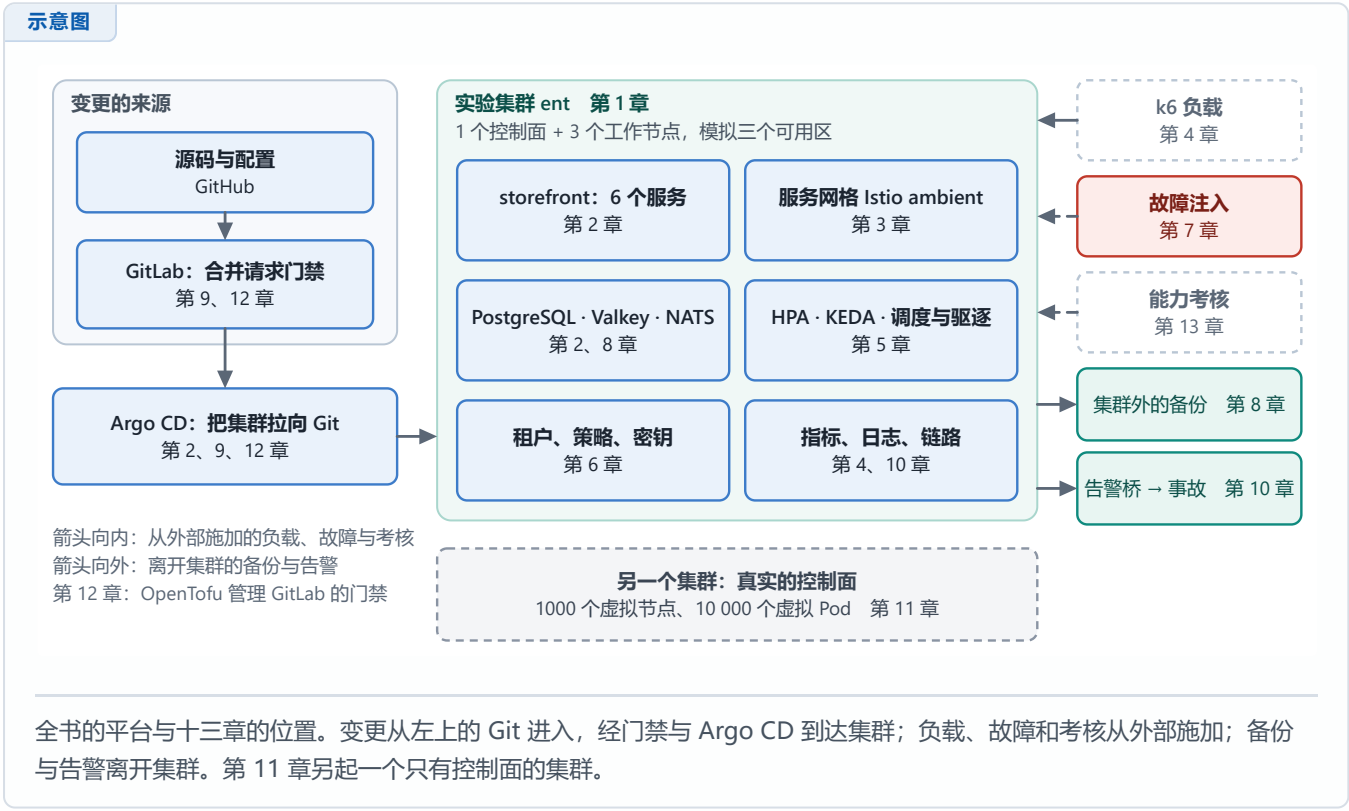
附录 C 代码与实验索引 210

附录 D 证据索引 215

阅读指南

这本书用一台笔记本上的真实软件，搭建一个尽可能接近企业生产的多服务平台，然后对它做实验：测它的容量，让它扩容，把它隔离成租户，向它注入故障，删掉它再重建，给它立规矩、装告警，最后把它放大到一万个 Pod。每一章回答一个工程问题，每一个结论都有执行记录。它不是工具手册——工具的用法在各自的文档里；它想讲清楚的是机制、取舍，以及“我怎么知道这是真的”。

全书地图



全书的平台与十三章的位置。变更从左上的 Git 进入，经门禁与 Argo CD 到达集群；负载、故障和考核从外部施加；备份与告警离开集群。第 11 章另起一个只有控制面的集群。

每一章怎样组织

每一章沿着同一条路走：**工程问题 → 技术原理 → 架构与实现 → 实验验证 → 结果分析 → 局限与工程启示。**章首的深色方框提出这一章要回答的工程问题，旁边列出实测得到的结论——可以先看结论，再带着“这是怎么得出来的”读下去。“原理”一节只讲后面的实验用得到的机制。每个实验先写下假设，再给出测量；章末的“局限”说明哪些事情没有测，“从实验环境到生产环境”的对照表说明这里的做法与真实生产的差别。最后是常见问题、练习（参考答案在附录 A）和小结。

实验遵循一条纪律：**假设写在实验之前，并且先提交到 Git。**所以书里有不少“假设被推翻”——它们按原样保留，因为预测为什么错，往往比预测对了更有教益。书中还记录了八起计划之外的真实事故与事件，其中最重的两起（第 11 章的 E10-1、第 12 章的 E11-1）都出自作者自己的操作，连同复盘写进了正文。

四种真实性标注

标注	含义	例子
[真实]	真实运行的软件、真实的请求与数据	Kubernetes、Argo CD、PostgreSQL、k6 产生的负载
[虚拟]	KWOK 模拟的节点与 Pod：只有控制面是真实的，没有容器在运行	第 11 章的 1000 个节点、10 000 个 Pod
[注入]	人为制造的故障或负载	冻结节点、切断复制、让一半请求返回 503
[模拟]	用实验手段代替的企业设施	“可用区”是节点的标签，“外部支付”是自己写的模拟器，“异地”备份在同一台机器上

图的五种标签

每幅图左上角的标签说明它是什么，避免把示意当成测量。

标签	含义
示意图	解释机制与结构的概念图，不含测量数据（图中出现的实测数字会在图注中说明）
实测数据	由实验的原始记录绘制的图表；每幅图的数据来源登记在仓库的 `v2/manual/pub/figures.py` 与 `diagram_defs.py`，并由程序与正文、证据核对
真实截图	实验当时的 Grafana 页面，未经修改
终端记录	实验脚本输出的原样渲染，标题栏写明记录时间；有删节处会标明
模型曲线	把公式画成曲线，不是测量

证据在哪里

正文里的上标数字（例如[3]）是证据编号，每章单独编号，对应附录 D 中列出的原始记录文件——终端输出、查询结果、时间线和复盘。全部文件连同配置与脚本都在实验仓库 `github.com/miku-wwl/devops-delivery` 的 `v2/` 目录下（分支 `v2/publication`）。正文中的具体陈述另有程序逐条与这些文件比对。附录 C 按章列出每个实验的脚本和配置在仓库中的位置；仓库的 `v2/README.md` 说明怎样在自己的机器上复现。

边界

全书的数字都来自一台笔记本（16 个硬件线程，虚拟机分得 15.3 GiB 内存）上的一次或几次运行，只代表这里测到的范围。本书验证的是“在受控环境中，能否正确地设计、实施、测量和排查”；它不能替代生产经验：真实生产有更大的规模、真实的用户、跨团队的协作、合规审计和长期运行才会出现的问题。书中没有虚构任何生产规模或工作经历。

第 1 章

平台基线：在一台电脑上模拟三可用区集群（E0）

第二册的所有实验都运行在同一台笔记本上。要让这些实验有意义，第一步不是安装更多软件，而是先回答三个问题：这台机器到底有多少资源；“节点”和“可用区”在这里指什么；哪些数字是内核真正执行的限制，哪些只是调度器的记账。本章建立一个 1 个控制面、3 个工作节点的集群 `ent`，把每个工作节点做成一台“4 核 3 GiB 的小虚拟机”，并逐层测量它的资源占用。测量中有两个发现会贯穿全书：kubelet 看不到节点容器的真实大小；Kind 默认根本不做内存驱逐。

工程问题

一台 16 线程、虚拟机只分得 15.28 GiB 内存的笔记本，怎样当作“三个可用区、每个节点 4 核 3 GiB”的集群来做实验——并且让调度器、kubelet 和内核对“这个节点有多大”的看法一致？

实测结论

- Kind 的节点是容器：不加修正时，kubelet 认为每个节点拥有整台虚拟机的 15642 MiB 内存，而且 Kind 默认关闭了内存驱逐。
- 修正之后三条线对齐：调度器按 2666 MiB 放 Pod，kubelet 在 2922 MiB 开始驱逐，内核在 3072 MiB 才动用 OOM killer。
- 集群创建用了 43 秒，脚本可以重复执行；CPU 用 cpuset 固定，而不是用配额限制，避免了周期性的限流。

1.1 学习目标

学完本章，你应当能够：

- 说清楚 Windows、WSL2 虚拟机、Docker、Kind 节点容器、Pod 这五层的关系，以及每一层由谁限制资源；
- 写出节点可分配资源（allocatable）的公式，并解释驱逐信号 `memory.available` 是怎样算出来的；
- 区分三种“资源真相”：内核执行的限制、调度器记的账、kubelet 的驱逐判断；
- 为一台共享的机器设计资源预算，并标出哪些是真实的、哪些是模拟的。

1.2 原理

1.2.1 一台机器上的“多台机器”

第一册已经用过 Kind（第 2 章）：每个 Kubernetes 节点是一个 Docker 容器，容器里运行 systemd、containerd 和 kubelet，Pod 是这个容器里的又一层容器。第二册仍然这样做，但要把它当作一台台“机器”

来对待，所以必须先看清楚这几层是怎样嵌套的：

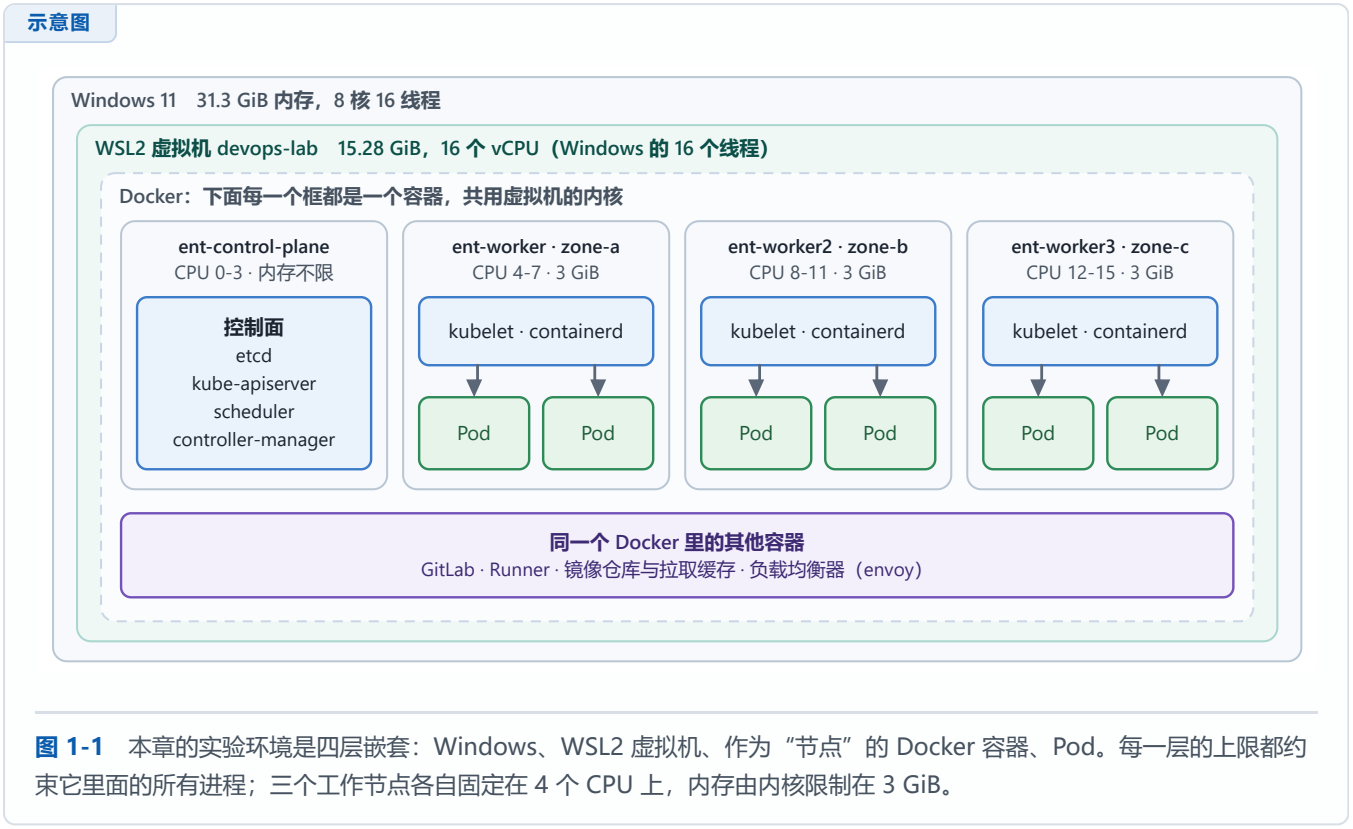


图 1-1　本章的实验环境是四层嵌套：Windows、WSL2 虚拟机、作为“节点”的 Docker 容器、Pod。每一层的上限都约束它里面的所有进程；三个工作节点各自固定在 4 个 CPU 上，内存由内核限制在 3 GiB。

每一层都可以限制下一层的资源，但限制的手段不同：

层	谁决定它的大小	手段
WSL2 虚拟机	Windows 的 WSL 配置（默认为主机内存的 50%）	虚拟机的内存与 vCPU 数
节点容器	本章的 `create.sh`	cgroup v2：`cpuset.cpus` 固定 CPU，`memory.max` 限制内存
Pod 与容器	Pod 的 requests / limits	kubelet 在节点容器内部再建一层 cgroup

这里的关键是 cgroup 的嵌套：子 cgroup 的用量计入父 cgroup，**任何一层的上限都约束它下面的所有进程。**节点容器的 `memory.max` 是 3 GiB，那么这个节点上所有 Pod、加上 kubelet 和 containerd 自己，合起来不能超过 3 GiB，超过时由内核的 OOM killer 在这个节点内部挑一个进程杀掉。

1.2.2　kubelet 眼中的节点

问题在于，kubelet 并不知道自己被关在一个 3 GiB 的容器里。它通过 cAdvisor 读取 `/proc/meminfo` 和 `/proc/cpuinfo` 来确定节点的容量（capacity），而这两个文件在容器中**没有被隔离**，显示的是整个虚拟机的数值。于是每个工作节点都会声称自己有 16 个 CPU、15.28 GiB 内存，三个节点加起来就是 48 个 CPU、46 GiB 内存，而真实的机器只有三分之一。调度器只相信 kubelet 上报的数字，它会把 Pod 放到一个早已满了的节点上。

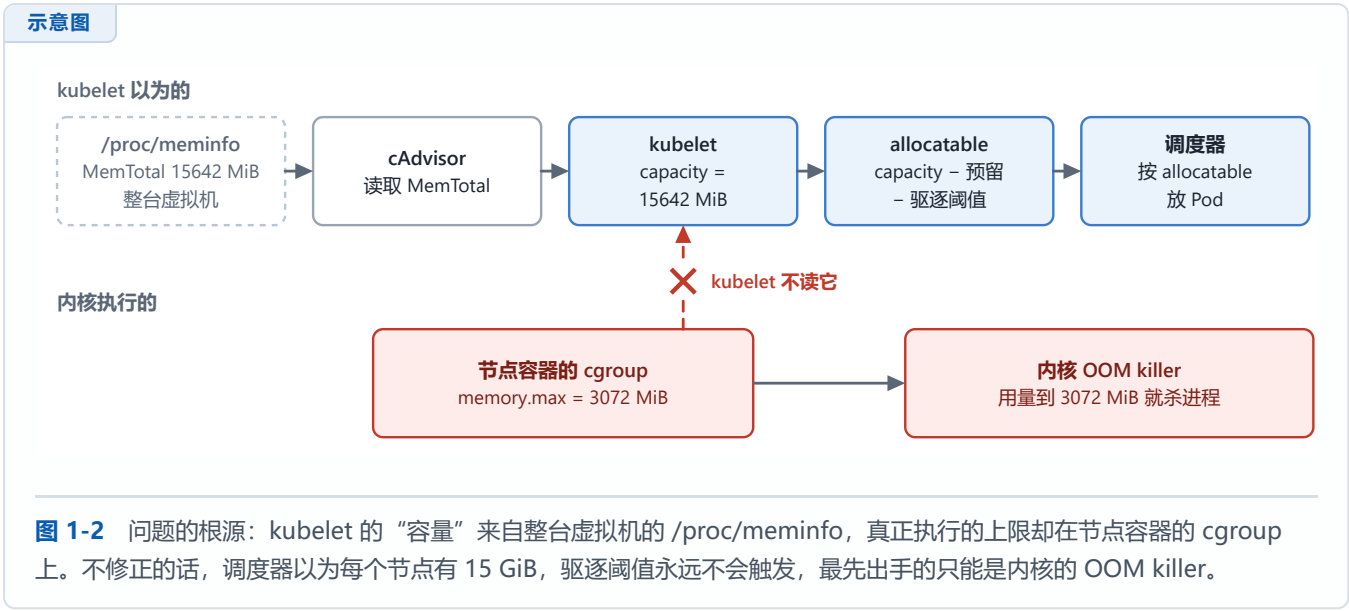


图 1-2 问题的根源：kubelet 的“容量”来自整台虚拟机的 /proc/meminfo，真正执行的上限却在节点容器的 cgroup 上。不修正的话，调度器以为每个节点有 15 GiB，驱逐阈值永远不会触发，最先出手的只能是内核的 OOM killer。

kubelet 对外公布的是**可分配资源（allocatable）**，计算公式是：

$$\text{allocatable} = \text{capacity} - \text{kube-reserved} - \text{system-reserved} - \text{驱逐阈值（eviction-hard）}$$

`kube-reserved` 留给 kubelet、containerd 等节点组件，`system-reserved` 留给操作系统；驱逐阈值则是“剩余内存低于这个值就开始驱逐 Pod”的那条线（只对内存这类不可压缩的资源有意义）。调度器只把 Pod 的 **requests** 之和与 allocatable 比较，从不看实际用量。

驱逐（eviction）是 kubelet 自己的保护机制：它每隔一段时间（默认 10 秒）计算一次信号

$$\text{memory.available} = \text{capacity} - \text{workingSet}$$

其中 workingSet 是节点根 cgroup 的内存用量减去可以随时回收的非活跃文件页（`inactive_file`）。信号低于阈值时，kubelet 按 QoS 等级和超出 requests 的程度选择 Pod 驱逐。它和内核 OOM 的区别在于：驱逐是“有序撤离”，Pod 会被优雅终止、在别处重建，节点会被打上 `MemoryPressure` 污点，不再接收新 Pod；OOM 则是“当场击毙”，内核杀掉一个进程，不通知任何人，Pod 的容器以 137 退出后原地重启。

1.2.3 让 kubelet 看到真实的尺寸

既然 kubelet 读到的容量是假的，就用它能理解的参数把“不属于这个节点的部分”扣掉。以 4 个 CPU、3072 MiB 的节点为例（虚拟机的 MemTotal 为 15642 MiB）：

- **CPU**：`system-reserved cpu = 16 − 4 = 12`，再扣 `kube-reserved cpu = 250m`，allocatable 为 3.75 个 CPU。这 12 个 CPU 是“幻影容量”，节点并不拥有它们。
- **内存**：幻影容量不能放进 `system-reserved`，因为驱逐信号是相对于 capacity 计算的。如果驱逐阈值仍是默认的 100 MiB，只有 workingSet 超过 15542 MiB 时才会驱逐，而内核早在 3072 MiB 时就动手了。所以要把阈值本身平移：希望在用量达到 2922 MiB（离内核上限还有 150 MiB）时开始驱逐，就设

$$\text{memory.available} < 15642 - 2922 = 12720\ \text{MiB}$$

此时 allocatable 为 $15642 - 256 - 12720 = 2666$ MiB。

实测数据

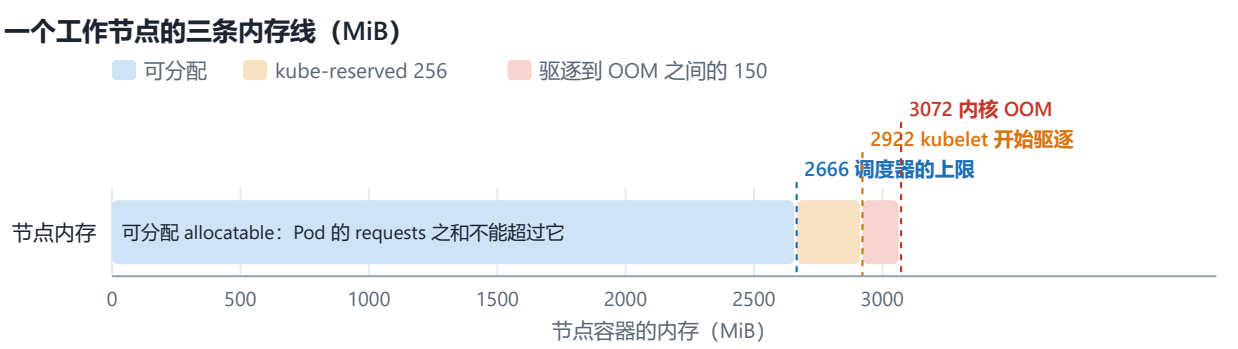


图 1-3 一个工作节点的三条内存线（MiB）。把预留和驱逐阈值设成“从虚拟机的 15642 MiB 倒推”的值之后，三个角色的判断依次发生：调度器不会把 requests 放过 2666 MiB；实际用量到 2922 MiB 时 kubelet 按顺序驱逐 Pod；只有驱逐来不及时，内核才在 3072 MiB 杀进程。

这样，三种“资源真相”第一次对齐了：内核在 3072 MiB 执行硬限制；kubelet 在 2922 MiB 开始有序驱逐；调度器最多把 2666 MiB 的 requests 放到这个节点上，剩下的 256 MiB 留给节点组件。`create.sh` 在执行时读取 `nproc` 和 `/proc/meminfo` 计算这些值，所以换一台机器也成立。

原理

三种资源真相。 讨论“节点有多少内存”时，要先问是谁的视角。**内核**执行 cgroup 的上限，超过就 OOM，不讲道理；**调度器**只做 requests 的加法，它不知道实际用量；**kubelet** 用实际用量（workingSet）对照驱逐阈值，在内核动手之前有序地腾出空间。生产事故中常见的“节点明明还有空间却调度不上”“明明 requests 很小却被 OOM”，都是把这三者混为一谈的结果。

1.3 设计

1.3.1 资源预算

预算从实测出发。暂停第一册的 `lab` 集群之前，虚拟机已用 8188 MiB、可用 7454 MiB；暂停后已用 3517 MiB、可用 12124 MiB，三个 v1 节点释放了约 4.6 GiB[3]。剩下的主要是 GitLab（2.98 GiB），第二册继续用它托管 GitOps 配置仓库。

层	空集群实测	预算	性质
虚拟机自身（内核、systemd、dockerd）	约 0.5 GiB	0.6 GiB	[真实]
GitLab、Runner、推送用镜像仓库	3.0 GiB	3.0 GiB，压测期间可停止	[真实]
4 个拉取缓存、负载均衡控制器	0.08 GiB	0.1 GiB	[真实]
控制面节点	0.70 GiB	不设上限，目标 ≤ 1.5 GiB，持续测量	[真实]

层	空集群实测	预算	性质
3 个工作节点	0.14–0.20 GiB/个	3 × 3 GiB，由内核强制	`[模拟]` 节点尺寸
余量（页缓存、k6、工具进程）	—	≥ 1.2 GiB	—

空集群的数字来自 [4]：创建 60 秒后，控制面节点 699 MiB，三个工作节点分别为 199、142、139 MiB，虚拟机已用 4656 MiB、可用 10985 MiB。合计接近 15.28 GiB 的上限，说明预算是紧的。为此定一条规则：**压测与故障实验开始前，若虚拟机可用内存低于 1 GiB，先停止 GitLab**（Argo CD 暂时无法拉取配置，但已部署的应用不受影响）。

CPU 的分配见下表。这颗 Ryzen 7 8845HS 有 8 个同构的 Zen 4 核心，每个核心两个硬件线程（0/1 是核心 0，2/3 是核心 1，以此类推，见 [5] 中的 `lscpu` 输出），所以每个工作节点固定两个完整的物理核心，彼此不共享执行单元：

节点	可用区	CPU（cpuset）	物理核心	内存上限
ent-control-plane	—	0-3	0、1	不限
ent-worker	zone-a	4-7	2、3	3 GiB
ent-worker2	zone-b	8-11	4、5	3 GiB
ent-worker3	zone-c	12-15	6、7	3 GiB

为什么用 cpuset 固定 CPU，而不是用 `--cpus 4` 这样的配额？配额是“每 100 ms 最多用 400 ms 的 CPU 时间”，进程仍然看得到 16 个 CPU。Go 等运行时会按 16 个 CPU 启动工作线程，然后在配额用完时被集体暂停（throttling），这是容器环境特有的问题，真实的 4 核虚拟机不会这样。cpuset 让节点里的进程只看到 4 个 CPU（下文的测量证实了这一点），更接近一台小虚拟机。

主机上的 GitLab、Docker 守护进程和 Windows 自身不受 cpuset 约束，它们的活动会给测量带来噪声。后面每个性能实验都会记录同时段的负载与 PSI（压力停滞信息），并在结论中注明。

1.3.2 可用区是什么，不是什么

三个工作节点带有标签 `topology.kubernetes.io/zone=zone-a/b/c`、`topology.kubernetes.io/region=local-1` 和 `node.kubernetes.io/instance-type=sim-4c3g`。这些都是云上节点的标准标签，调度器的拓扑分布约束、存储的拓扑感知、服务的拓扑感知路由都依赖它们，所以后面的实验可以原样使用。

但它们是 `[模拟]` 的：三个“可用区”共享同一个内核、同一块磁盘、同一个网桥和同一个电源。真实可用区之间有独立的供电与网络，有 1–2 毫秒的往返延迟，会整体失效。本书在故障实验中用停止节点容器、注入网络分区来模拟可用区故障，所得结论只适用于“控制面与调度如何反应”，不适用于“真实数据中心如何失效”。

1.3.3 网络与入口

第一册的集群在 Docker 网络 `kind` 上。第二册用环境变量 `KIND_EXPERIMENTAL_DOCKER_NETWORK=kind-ent` 把 `ent` 放在独立的网络 `kind-ent`（172.20.0.0/16）上，两个集群的地址互不干扰，`lab` 以后恢复时不会因为 IP 被占用而出问题。GitLab 和推送用镜像仓库同时接入两个网络。

云上的 `LoadBalancer` Service 由云厂商的负载均衡器实现，它在集群之外。`cloud-provider-kind` 模拟了这一点：它作为一个容器运行在虚拟机上，监视所有 Kind 集群中 `type: LoadBalancer` 的 Service，为每个 Service 启动一个 envoy 容器，并把这个容器的 IP 写回 Service 的 `status.loadBalancer`。读取它生成的配置[6]，可以看到它的行为和云上的网络负载均衡器（L4）几乎一样：

- 监听器是 `tcp_proxy`，只转发 TCP 连接，不解析 HTTP；
- 后端是三个工作节点的 NodePort，策略为 `RANDOM`；控制面节点带有 `node.kubernetes.io/exclude-from-external-load-balancers` 标签，被排除在外；
- 健康检查访问每个节点上 kube-proxy 的 `:10256/healthz`，每 2 秒一次；
- `healthy_panic_threshold` 为 0：没有健康的后端时直接拒绝连接，而不是“死马当活马医”地随机转发。

启动时设置 `--gateway-channel disabled`，关闭它自带的 Gateway API 实现，第 3 章（E2）由 Istio 提供 Gateway API。

1.3.4 镜像供应：拉取缓存

每个节点独立拉取镜像。三个工作节点加一个控制面，同一个镜像要从互联网下载四次；第 8 章（E7）的灾难恢复演练还要整个重建集群。Docker Hub 对匿名拉取有频率限制。企业里的做法是在内网放一个拉取缓存（Harbor、Artifactory、Nexus 的代理仓库）。这里用 Distribution（`registry:3.1.2`）的代理模式为 docker.io、ghcr.io、quay.io、registry.k8s.io 各起一个缓存，再在每个节点的 `/etc/containerd/certs.d/<registry>/hosts.toml` 中声明：

```
server = "https://registry-1.docker.io"

[host."http://mirror-docker:5000"]
  capabilities = ["pull", "resolve"]
```

containerd 先试缓存，缓存不可用时回退到 `server`，所以缓存故障不会让集群拉不到镜像（只是变慢）。四个缓存都经过了验证：从节点拉取一个小镜像后，对应缓存的存储目录中出现了这个仓库[4]。

1.3.5 审计日志

第 6 章（E5）要回答“谁在什么时候做了什么”，这依赖 API Server 的审计日志，而审计策略只能在 API Server 启动时指定，所以在建集群时就配置好。策略（`platform/cluster/audit-policy.yaml`）按“第一条匹配的规则生效”的顺序写，原则是：

1. 丢弃高频而无价值的请求：健康检查、API 发现、事件、租约（leader 选举和节点心跳）、认证与授权审查；
2. Secret、ConfigMap、令牌和 `exec`：无论谁操作，只记元数据，**永远不记请求体**，否则审计日志本身就成了泄密的渠道；
3. 系统组件（ServiceAccount、节点、控制器）：读操作不记，写操作只记元数据；
4. 人的写操作：记请求体；对 RBAC、准入策略、NetworkPolicy 的写操作，请求和响应都记；
5. 其余（人的读操作）：只记元数据。

实测的结果符合预期：集群创建后约 7.5 分钟内记录了 1077 个事件，其中 Metadata 1003 个、Request 44 个、RequestResponse 30 个，文件约 1.2 MB；`kubernetes-admin` 删除命名空间的事件带有请求体，创建 Secret 的事件没有[7]。日志按 100 MB 轮转、保留 5 个，最多占用约 600 MB。

1.4 实现与验证

1.4.1 创建集群

`create.sh` 先渲染 Kind 配置（渲染结果保存在 [8]），然后创建集群、启动镜像缓存、配置每个节点的 containerd、设置节点容器的 CPU 与内存、启动负载均衡控制器，最后安装 metrics-server。工作节点的 kubelet 参数通过 Kind 的 JSON 6902 补丁追加到 `JoinConfiguration` 中：

```
kubeadmConfigPatchesJSON6902:
  - group: kubeadm.k8s.io
    version: v1beta4          # Kubernetes 1.31 起，kubeadm 的参数列表是 name/value 数组
    kind: JoinConfiguration
    patch: |
      - op: add
        path: /nodeRegistration/kubeletExtraArgs/-
        value: {name: system-reserved, value: "cpu=12"}
      - op: add
        path: /nodeRegistration/kubeletExtraArgs/-
        value: {name: eviction-hard, value: "memory.available<12720Mi,nodefs.available<0%,..."}
```

为什么用 JSON 6902 的 `add ... /-`（追加到数组末尾），而不是 Kind 更常见的合并补丁？因为 Kind 生成的 `kubeletExtraArgs` 已经是一个数组（含 `node-ip`、`provider-id`、`node-labels`），合并补丁会把整个数组替换掉，节点就丢了自己的 IP 设置。同样的原因，API Server 的审计参数也是追加的。`eviction-hard` 必须完整地写出所有信号：命令行参数会整体替换配置文件中的 `evictionHard`，而 Kind 在配置文件里把磁盘相关的信号设为 `0%`（关闭磁盘驱逐，因为节点容器看到的是整个主机的磁盘）。

集群创建用了 43 秒[9]。第一次执行时有一个缺陷：`docker inspect -f '{{json .NetworkSettings.Networks."kind-ent"}}'` 在 Go 模板中是非法的（带连字符的名字不能作为字段名），命令报错，而脚本把“报错”当成了“已连接”，于是没有把 GitLab 和镜像仓库接入新网络。修正为 `{{json (index .NetworkSettings.Networks "kind-ent")}}` 后再执行一次，脚本只补做了缺失的步骤[2]。这个小插曲说明了幂等脚本的价值：可以放心地重复执行。

1.4.2 kubelet 的有效配置

写进配置的参数不一定生效，要看 kubelet 实际在用什么。kubelet 的 `/configz` 端点返回合并了配置文件和命令行参数之后的有效配置[4]：

```
== ent-control-plane
{"kubeReserved":null,"systemReserved":null,
 "evictionHard":{"imagefs.available":"0%","nodefs.available":"0%","nodefs.inodesFree":"0%"}, ...}
== ent-worker
{"kubeReserved":{"cpu":"250m","memory":"256Mi"},"systemReserved":{"cpu":"12"},
 "evictionHard":{"imagefs.available":"0%","memory.available":"12720Mi","nodefs.available":"0%",...},
 "evictionMinimumReclaim":{"memory.available":"100Mi"}, ...}
```

工作节点的参数全部生效。控制面节点的输出揭示了一个容易被忽略的事实：**Kind 默认的 kubelet 配置里没有 `memory.available` 阈值。**Kind 为了关闭磁盘驱逐写了 `evictionHard`，而只要设置了 `evictionHard`，kubelet 就不再为其中没写的信号套用默认值（默认的 `memory.available<100Mi` 也就消失了）；控制这一行为的字段 `mergeDefaultEvictionSettings` 在这里是 `false`[6]。结果是默认的 Kind 集群**从不因为内存压力驱逐 Pod**，内存耗尽时只有内核 OOM。在 Kind 上做的“驱逐实验”如果没有先检查这一点，结论很可能是错的。第 5 章（E4）的驱逐实验就建立在本章设置的阈值上。

节点上报的容量与可分配资源也与计算一致：

```
NODE                ZONE     CAP_CPU   ALLOC_CPU   CAP_MEM      ALLOC_MEM
ent-control-plane   <none>   16        16          16017492Ki   16017492Ki
ent-worker          zone-a   16        3750m       16017492Ki   2730068Ki
```

16017492 KiB 正是虚拟机的 15642 MiB；2730068 KiB 是 2666 MiB。驱逐信号也来自节点自己的 cgroup，而不是整个虚拟机：`ent-worker` 的 workingSet 为 199 MiB，`memory.available` 为 15442 MiB（= 15642 − 199，取整），与从节点 cgroup 文件直接算出的 201 MiB、`docker stats` 显示的 199.1 MiB 一致。这一点很重要：如果 kubelet 用的是虚拟机的整体内存，那么一个节点的压力会让所有节点同时驱逐。

1.4.3 节点内部的 cgroup 树

进入节点容器，可以看到 kubelet 建立的层级[10]：

```
.                                                    memory.max=3221225472   ← Docker 设置的节点上
限 (3 GiB)
kubelet.slice                                        memory.max=max
kubelet.slice/kubelet-kubepods.slice                 memory.max=16133476352  ← capacity − kube-re
served
kubelet.slice/kubelet-kubepods.slice/...-burstable   memory.max=max
kubelet.slice/kubelet-kubepods.slice/...-besteffort  memory.max=max
```

kubelet 给所有 Pod 的总 cgroup（kubepods）设置的上限是 capacity − kube-reserved − system-reserved = 15642 − 256 = 15386 MiB，比节点本身的 3 GiB 大得多，所以实际起作用的是节点容器的上限。这也验证了前面的推理：在 kubelet 看不到真实容量的环境里，**驱逐阈值是唯一能在内核 OOM 之前起作用的保护。**

OOM 的优先级（`oom_score_adj`，越小越不容易被杀）同样反映了 kubelet 的策略：containerd、kubelet、kube-proxy 为 −999，kindnet 为 −997。kindnet 和 kube-proxy 的 QoS 等级分别是 Burstable 和 BestEffort，按 QoS 本应排在前面被杀，但它们使用 `system-node-critical` 优先级类[6]，kubelet 对关键 Pod 一律给 −997；kube-proxy 进程启动后又按自己的默认值 `oomScoreAdj=-999` 改写了一次。

1.4.4 从 Pod 里看节点

在 `ent-worker` 上的 Pod 里执行几个命令[11]：

```
$ nproc                                  → 4
$ grep -c ^processor /proc/cpuinfo       → 16
$ cat /sys/fs/cgroup/cpuset.cpus.effective → 4-7
$ head -1 /proc/meminfo                  → MemTotal: 16017492 kB
```

`nproc` 通过 `sched_getaffinity` 得到 4，这是 cpuset 的效果；但 `/proc/cpuinfo` 仍列出 16 个处理器，`/proc/meminfo` 仍显示整个虚拟机的内存。应用如果用后两者来决定线程池大小或堆大小，就会严重高估。现代运行时大多已改为读取 cgroup 或 CPU 亲和性（例如 JVM 的容器支持、Go 1.25 起的容器感知 GOMAXPROCS），但 Python 的 `os.cpu_count()` 在这个 Pod 里返回 16，`len(os.sched_getaffinity(0))` 和 Python 3.13 新增的 `os.process_cpu_count()` 才返回 4[6]。第 4 章（E3）设置 Uvicorn 的进程数时会用到这一点。

1.4.5 负载均衡器的第一秒

冒烟测试（`platform/cluster/smoke.yaml`）在三个可用区各放一个 Pod，前面放一个 LoadBalancer Service。拓扑分布约束把三个 Pod 分别放到了三个节点上，负载均衡器拿到了 IP 172.20.0.12。但第一次测试的 30 个请求中，**9 个没有响应，其余 21 个全部落在同一个 Pod 上**[11]。

稍后再测，60 个请求经负载均衡器分布为 21/23/16，直接访问一个节点的 NodePort 分布为 15/26/19，都是均匀的。为了弄清第一次发生了什么，又单独重建 Service，从创建那一刻起计时[12]：IP 在 4.5 秒后分配；随后的 40 个请求全部成功，但前 2 秒内几乎只落在两个 Pod 上，第三个 Pod 在第 6.43 秒才第一次出现；之后的 90 个请求分布为 34/25/31。

可以确定的部分是：envoy 在第一次健康检查通过之前没有可用的后端，又没有恐慌路由，这期间的连接会被直接拒绝；第一次测试时 Deployment 和 Service 是同时创建的，比第二次多了 Pod 启动的过程。不能完全确定的是请求在前几秒集中的原因，可能与各节点的 kube-proxy 在不同时刻写入转发规则有关，本次没有采集足以证实的数据。对运维的启示是确定的：**新建的负载均衡器需要预热时间**，切换流量（例如蓝绿发布切换 DNS）之前要先确认健康检查已通过，并用真实请求验证分布。

1.4.6 暂停与恢复第一册的集群

第一次暂停 `lab` 时用了 `docker stop` 的默认超时（10 秒）：节点容器的 systemd 收到停止信号后要依次停止所有 Pod、containerd 和 kubelet，10 秒内没有完成，Docker 发出 SIGKILL，容器以 137 退出[3]。这相当于给三台机器同时断电。

第一册的成果不能因此受损，所以立刻做了一次恢复检查[13]：启动后 API Server 最初返回 `Forbidden: unknown`（它在 RBAC 缓存就绪之前就开始应答，脚本已改为先等待 `/readyz`）；随后三个节点 Ready，42 个 Pod 全部 Running，两个 Argo CD 应用均为 Synced/Healthy。etcd 的预写日志保证了断电后的一致性。之后改用 120 秒超时正常停止，用时 1 分 30 秒。

1.5 局限：这个集群不是什么

方面	本环境	真实生产
节点	Docker 容器，共享一个内核	独立的虚拟机或物理机，内核故障互不影响
节点容量	kubelet 读到的是整个虚拟机，靠参数修正	kubelet 读到的就是真实容量
可用区	只是标签；共享磁盘、网桥和电源；区间无延迟	独立的故障域，区间有 1–2 ms 延迟
磁盘	所有节点共用一块 NVMe，磁盘驱逐已关闭	每个节点有自己的磁盘与 IOPS 上限
负载均衡	主机上的一个 envoy 容器	高可用的托管服务，有自己的扩缩容与配额
噪声	Windows、GitLab、Docker 与实验共享机器	专用的节点池，但会有邻居租户

这些差异不会让本书的实验失去意义，但决定了结论的边界：调度、驱逐、控制器的行为是真实的 Kubernetes 行为；吞吐量、延迟这类绝对数字只属于这台机器。

1.6 常见问题

Q1：为什么不直接调大 WSL 的内存？ 可以，在 Windows 的 `%UserProfile%\.wslconfig` 中设置 `memory=20GB` 后执行 `wsl --shutdown` 即可。但这会改变主机的系统配置，影响所有 WSL 发行版，属于机器所有者的决定，本书没有这样做，而是在默认的 15.28 GiB 内设计预算。如果你调大了内存，只需重新执行 `create.sh`，阈值会按新的 MemTotal 自动计算。

Q2：控制面节点为什么不设内存上限？ API Server 的内存随对象数量、CRD 数量和 watch 连接数增长，静态 Pod 的 OOM 优先级又不高。在 3 GiB 的上限下，一旦超出，内核可能先杀 API Server，整个集群随之失去控制。本书选择不设上限、持续测量，把控制面的容量问题留给第 11 章（E10）在独立的集群上研究。

Q3：`cloud-provider-kind` 需要挂载 Docker 套接字，安全吗？ 能访问 Docker 套接字就等于拥有虚拟机的 root 权限。这里它是 Kubernetes SIG 官方维护的组件，只在本地实验环境运行；在共享或生产环境中，不应该把 Docker 套接字交给任何容器。

Q4：恢复 `lab` 集群时要注意什么？ `ent-ccm` 会监视所有 Kind 集群，它被设置为把负载均衡器放在 `kind-ent` 网络上，对 `lab` 不适用（第一册也没有使用 LoadBalancer）。同时运行两个集群还会超出内存预算。建议先停止 `ent`（`docker stop -t 120` 依次停止四个节点），再执行 `v1-lab.sh start`。

1.7 练习

练习

1. **（计算）** 把工作节点改为 2 个 CPU、4 GiB 内存，驱逐余量仍为 150 MiB，kube-reserved 不变。写出 `system-reserved` 和 `eviction-hard` 的值，以及节点的 allocatable。虚拟机的 MemTotal 仍为 15642 MiB。
2. **（分析）** 一个节点上 Pod 的 requests 之和为 2.5 GiB，实际用量为 2.8 GiB。此时 kubelet 会做什么？如果某个 Pod 在 3 秒内又分配了 400 MiB，接下来最可能发生什么？为什么？
3. **（动手）** 在一个默认配置的 Kind 集群中，用 `/configz` 确认没有 `memory.available` 阈值；然后只用 kubeadm 补丁（不用命令行参数）为所有节点加上 `memory.available<500Mi`，并说明为什么仍要保留磁盘相关的三个 `0%`。
4. **（设计）** 本章用 cpuset 固定 CPU。列出改用 `--cpus 4`（配额）后，节点内的 Go 程序、Python 多进程程序和 CPU 密集型 Pod 的行为会有什么不同，各举一个可以观测到差异的指标。
5. **（排查）** 新建的 LoadBalancer Service 前几秒的请求部分失败。列出你会依次检查的三个地方，以及每个地方用什么命令或日志确认。
6. **（思考）** 审计策略把 Secret 的所有操作都只记录元数据。审计人员提出：“我们需要知道 Secret 的值被改成了什么，才能判断是否被篡改。”你如何在不把密文写进审计日志的前提下满足这个需求？

参考答案见附录。

1.8 本章小结

- `ent` 集群由 1 个控制面和 3 个工作节点组成，三个节点带有 `[模拟]` 的可用区标签；每个工作节点由内核限制为 4 个固定 CPU、3 GiB 内存，相当于一台小虚拟机。
- kubelet 读到的容量是整个虚拟机的。用 `system-reserved` 扣掉幻影 CPU、用平移后的 `memory.available` 阈值扣掉幻影内存，使内核限制（3072 MiB）、驱逐起点（2922 MiB）和调度上限（2666 MiB）三者对齐，`/configz` 与节点状态证实全部生效。
- 测量发现 Kind 默认不做内存驱逐：设置了 `evictionHard` 就不再套用默认的 `memory.available` 阈值。
- 空集群占用约 1.2 GiB；加上 GitLab，虚拟机可用内存约 11 GiB，预算是紧的，压测前要检查余量。
- 负载均衡器、拉取缓存和审计日志都已就绪并经过验证；新建的负载均衡器在最初几秒可能拒绝连接或分布不均。
- 第一册的集群被暂停，经过一次意外的强制停止后已验证可以完整恢复。

第 2 章

多服务系统：storefront 与它的平台（E1）

第一册只有一个服务。真实的系统由许多服务组成，它们通过网络同步调用，通过消息异步协作，各自拥有数据。本章搭建第二册的被测系统 storefront：六个服务、一个三实例 PostgreSQL 集群、一个 Valkey、一个三节点 NATS JetStream 集群，外加 Prometheus、Tempo、Loki 组成的可观测性平台，全部由 Argo CD 从 Git 部署。系统刻意做得“小而完整”：一笔订单要经过价格查询、库存预留、支付、确认和异步通知，涉及分布式系统中最常见的几个难题——部分失败、重复请求、双写、跨服务追踪。第一次部署就暴露了八个问题，本章把它们连同证据一起记录下来，这比一次顺利的部署更有教学价值。

工程问题

把一个服务拆成六个之后，一笔订单要跨三个数据库、一个缓存和一个消息队列。怎样让它在部分失败时不丢、不重、可追踪——并且整个平台能从 Git 重建？

实测结论

- 平台从 Git 收敛用了 735 秒，其中等待 PostgreSQL 集群就占了 437 秒：同步波次把“谁依赖谁”变成了看得见的时间线。
- 一笔订单的链路包含 6 个服务、65 个 span，从网关一直延续到通知服务消费消息——追踪上下文跨过了消息队列。
- 两个副本并不意味着负载减半：长连接下 orders 的一个副本承担了 91.3% 的请求（第 3 章解决）。
- 第一次部署暴露了 8 个问题，每一个都留下了证据和修复——它们比顺利的部分更有教学价值。

2.1 学习目标

学完本章，你应当能够：

- 说明同步调用和异步消息各自带来的耦合，以及为什么“写数据库再发消息”是错的；
- 用 saga、补偿、幂等键和事务性 outbox 实现一个跨服务的业务流程，并指出它在哪些情况下仍会出错；
- 说出超时、截止时间、重试、重试预算、熔断、舱壁分别回答什么问题，并估算重试的放大倍数；
- 让链路追踪跨过消息队列，并从一条链路中读出时间花在了哪里；
- 用 app-of-apps、同步波次和 ApplicationSet 从 Git 引导一个完整的平台；
- 把“第一次部署出的问题”当作数据：每个问题都有现象、证据、根因、修复和回归测试。

2.2 原理

2.2.1 拆分服务的代价

把一个程序拆成多个服务，换来的是独立部署、独立伸缩和故障隔离；付出的代价是：原来的函数调用变成了网络调用，原来的一个数据库事务变成了多个数据库各自的事务。网络调用会失败、会变慢、会成功了却没有返回结果；多个事务无法一起提交或一起回滚。微服务设计的大部分技巧，都是在处理这两个代价。

服务之间有两种协作方式：

	同步调用（HTTP）	异步消息（NATS JetStream）
调用方是否等待	等待结果	只等消息被可靠保存
时间耦合	被调用方必须此刻可用	消费方可以晚一点再处理
失败的传播	被调用方慢，调用方跟着慢（第 7 章研究级联故障）	消费方慢，消息积压，生产方不受影响
适合	需要结果才能继续：价格、库存、支付	不影响当前请求的后续工作：通知、统计

storefront 两种都用。下单必须知道价格、库存和支付结果，所以是同步调用；发通知不应让顾客等待，所以是异步消息。

2.2.2 跨服务的业务流程：saga

下单涉及三个服务的数据：库存（inventory）、支付（payments）、订单（orders）。没有一个事务能同时覆盖它们。**saga** 的做法是把流程拆成一串本地事务，每一步失败时，用**补偿动作**撤销已经完成的步骤。storefront 采用编排式（orchestration）saga，由 orders 服务按顺序指挥：

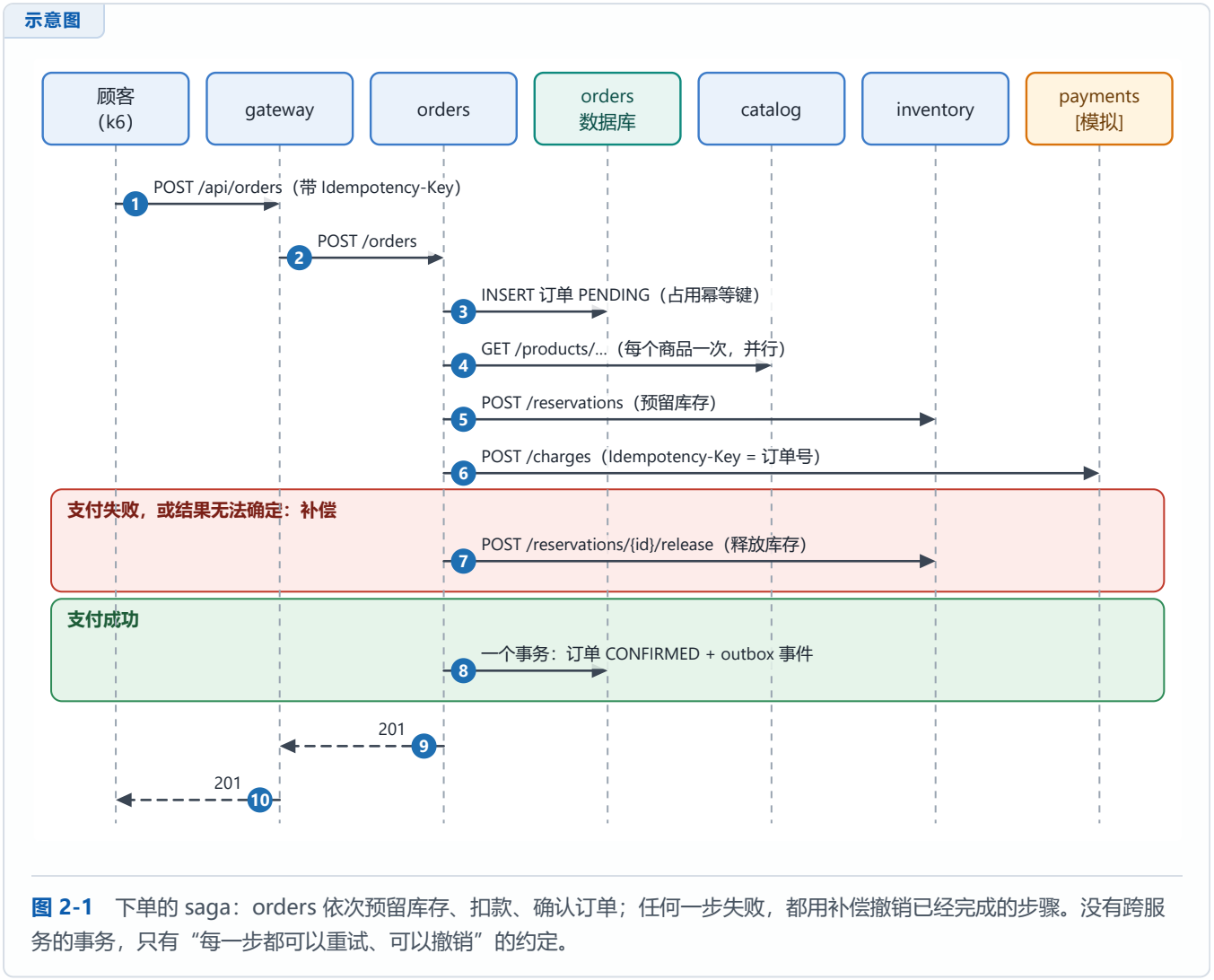


图 2-1 下单的 saga：orders 依次预留库存、扣款、确认订单；任何一步失败，都用补偿撤销已经完成的步骤。没有跨服务的事务，只有“每一步都可以重试、可以撤销”的约定。

saga 不提供隔离性：在补偿完成之前，其他请求能看到“库存已预留、订单尚未确认”的中间状态。这在电商中可以接受（预留本来就是暂时的），在转账这类场景中则需要更强的设计。

2.2.3 幂等键与“模糊结果”

网络调用有第三种结局：请求发出去了，没收到回答。可能是对方根本没收到，也可能是对方已经处理完、回答在路上丢了。这叫**模糊结果**（ambiguous outcome）。对查询来说重试无害；对“扣款”来说，盲目重试可能扣两次钱。

解决办法是**幂等键**：调用方为每个业务操作生成一个唯一的键，随请求发送；服务端记住每个键的结果，同一个键再来时直接返回第一次的结果。storefront 在三处使用它：

1. 顾客 → gateway → orders：`Idempotency-Key` 由客户端生成，orders 用数据库的唯一约束占用它（`INSERT ... status='PENDING'`）。并发的重复请求中只有一个能插入成功，其余返回已有的订单（`200` 和 `Idempotent-Replayed: true`）。
2. orders → inventory：预留以订单号为主键，重复预留返回已有结果。
3. orders → payments：以订单号为幂等键。payments 还为每个键加了一把带过期时间的锁：同一个键的两个请求同时到达时（典型情形：调用方超时后重试，而第一次请求还在处理），第二个得到 `409`，必须

稍后再问，而不是两个都去扣款。

幂等键把“重试是否安全”这个问题，从“调用方猜测对方的状态”变成了“服务端保证”。本章后面的排障记录中，正是它让一次真实的错误没有变成重复扣款。

2.2.4 双写问题与事务性 outbox

订单确认后要发一个 `OrderConfirmed` 事件。最直接的写法是“提交数据库事务，然后发消息”。这是错的：两步之间进程可能崩溃，消息就丢了；反过来“先发消息再提交”，事务可能回滚，消息却已发出。两个系统无法原子地一起写，这叫**双写问题**。

事务性 outbox 把消息先当作数据写进同一个数据库：

```
BEGIN;
UPDATE orders SET status = 'CONFIRMED', total_cents = $2 WHERE id = $1;
INSERT INTO outbox (event_id, subject, payload, headers) VALUES (...);
COMMIT;   -- 订单和事件要么都在，要么都不在
```

一个独立的中继（relay）读取 outbox 中未发布的行，发布到 NATS，再把它们标为已发布。中继可能在“发布成功”和“标记完成”之间崩溃，于是同一事件会再发一次：outbox 提供的是**至少一次**投递。重复由两层处理：消息携带 `Nats-Msg-Id`（等于事件 ID），JetStream 在 2 分钟的去重窗口内丢弃相同 ID 的消息；消费方再按事件 ID 去重。多个 orders 副本都运行中继，`SELECT ... FOR UPDATE SKIP LOCKED` 让它们各取不同的行，不会同时处理同一行。

示意图

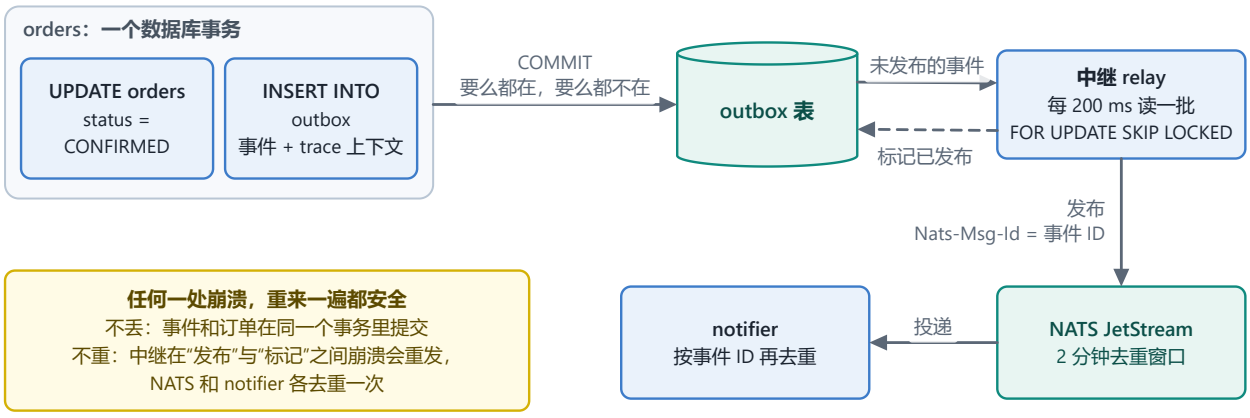


图 2-2 事务性 outbox 的三段：订单与事件在同一个事务里提交；中继把事件发布到 NATS，重复发布由消息 ID 去重；消费者自己再去重一次。任何一处崩溃，重来一遍都不会丢、也不会重。

2.2.5 韧性原语：每一个回答一个问题

服务 A 调用服务 B 时，B 可能慢、可能错、可能不在。共享库 `storefront.common` 为每次调用依次检查：

原语	回答的问题	storefront 的默认值（resilient 模式）
超时	一次尝试最多等多久？	1 秒
截止时间传播	整个请求还剩多少时间？	gateway 设 3 秒，经 `X-Deadline-Ms` 向下传递
重试（带全抖动的指数退避）	值得再试吗？什么时候？	最多 2 次，仅对幂等调用
重试预算	系统还负担得起重试吗？	重试最多占请求量的 20%
熔断器	对方此刻是不是坏了？	最近 20 次中至少 10 次、失败率 ≥ 50% 则断开 5 秒
舱壁	最多允许多少个调用同时等待它？	64

另有一个 **naive 模式**（`RESILIENCE_MODE=naive`）：30 秒超时、不加抖动地立即重试 3 次、没有预算、没有熔断和舱壁。它是第 7 章（E6）故障传播实验的对照组。

为什么需要重试预算？设调用链有 n 层，每层最多重试 r 次，最深一层在最坏情况下收到的请求是用户请求的 $(1+r)^n$ 倍。naive 模式下 gateway → orders → payments 两层各重试 3 次，就是 $4^2 = 16$ 倍：依赖本来只是慢，被重试活活压垮。预算把每一层的额外负载限制在 20%，两层合计不超过 $1.2^2 = 1.44$ 倍。全抖动（每次退避取 $[0, \min(\text{上限}, \text{基数} \times 2^k)]$ 中的随机值）则避免大量客户端在同一时刻一起重试。

截止时间传播解决的是另一种浪费：用户在 3 秒后已经放弃，下游却还在为这个请求查数据库。每个服务从请求头读出截止时间，给每次下游调用的超时取“自己的超时”和“剩余时间”中较小的那个；时间已经用完，就直接失败（`deadline`），不再发出调用。

2.2.6 追踪如何跨过消息队列

HTTP 调用的链路由 OpenTelemetry 自动传递：客户端在请求头中写入 `traceparent`，服务端读出它作为父上下文。消息队列没有这种自动机制，storefront 手工完成三步：

1. 下单请求在写 outbox 时，把当时的上下文（`traceparent`）存进 outbox 行的 `headers` 列；
2. 中继发布时，以这个上下文为父，创建一个 `PRODUCER` 类型的 span（`send orders.confirmed`），并把新的上下文写进 NATS 消息头；
3. 通知服务读出消息头，以它为父创建 `CONSUMER` span（`process orders.confirmed`）。

结果是：一笔订单从网关请求到通知发出，在 Tempo 中是同一条链路，事件在 outbox 和流中等待的时间也直接显示为 span 之间的空隙。

2.3 系统设计

2.3.1 服务与数据

组件	规格	说明
gateway、catalog、inventory、payments、orders	各 2 个副本，requests 50m CPU / 96 Mi	同一个镜像，启动命令选择服务
notifier	1 个副本	第 5 章（E4）由 KEDA 按积压伸缩
PostgreSQL（CloudNativePG 1.30.1）	3 个实例，每个可用区一个，PostgreSQL 18	每个服务一个数据库、一个登录角色（共享集群上的“每服务一库”）
Valkey 9.1.3	1 个实例，AOF 持久化，`maxmemory-policy noeviction`	三个逻辑库三种用途，见下文
NATS 2.15.2	3 节点 JetStream 集群（Raft），流 ORDERS 3 副本	文件存储，去重窗口 2 分钟

Valkey 的三种用途对“丢失”的容忍度完全不同：catalog 的缓存丢了只是变慢；payments 的幂等记录丢了可能导致重复扣款；notifier 的去重记录丢了会重复发通知。所以淘汰策略选 `noeviction`：内存满时写入失败（catalog 能容忍），而不是悄悄淘汰幂等记录。生产环境应把缓存和“记录”放在不同的实例上，本章的排障记录会再次说明这一点。

示意图

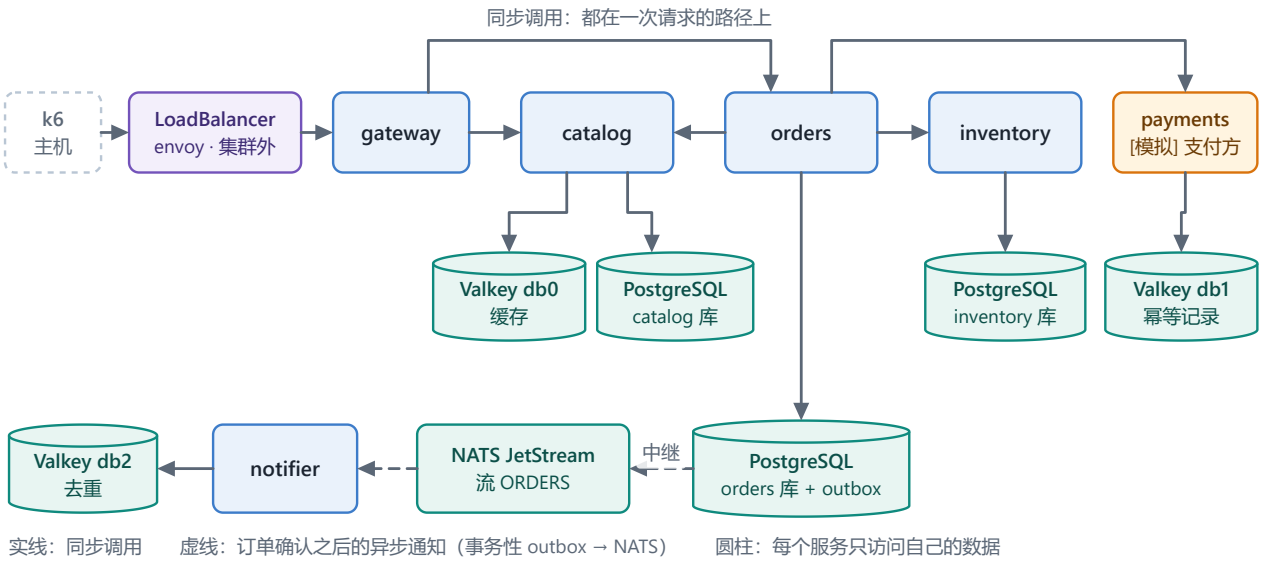


图 2-3 storefront 的服务与数据：同步调用（实线）在请求路径上，订单确认后的通知经事务性 outbox 与 NATS 异步送达（虚线）。每个服务只访问自己的数据。

2.3.2 每个服务的运维面

所有服务共享同一套运维接口（`common/web.py`），这是平台对服务的“契约”：

- `/healthz`：存活探针，只说明进程能应答，**从不检查依赖**；
- `/readyz`：就绪探针，启动完成、未在关闭、自身就绪检查通过（例如数据库连接池可用）；
- `/metrics`：RED 指标（按路由模板计数的请求数、错误、延迟直方图）和依赖调用指标（按结果计数的尝试、重试、被拒绝的调用、熔断器状态、舱壁占用、连接池等待）；
- `/admin/faults`：`[注入]` 故障的开关（延迟、错误率），供第 7 章使用；
- 每个请求的 `X-Request-ID` 和 trace ID 都写进 JSON 日志。

Pod 的安全上下文全部加固：非 root（UID 10001）、只读根文件系统、去掉所有 capability、`Runtime Default` seccomp；ServiceAccount 不挂载令牌（服务不需要访问 Kubernetes API）。停止时先 `preStop` 等待 5 秒，让端点的删除传播到各节点，再由 uvicorn 在 20 秒内处理完已接收的请求。

2.3.3 平台从 Git 引导

第一册的配置仓库只放了一个服务的部署配置。第二册把整个平台都放进 Git（`gitops/`，发布到本地 GitLab 的 `root/platform-gitops`）：

```
gitops/
  bootstrap/root.yaml             唯一需要手工 apply 的清单：根应用
  bootstrap/argocd-values.yaml    Argo CD 本身的 Helm 参数（引导时安装）
  platform/apps/                  每个平台组件一个 Application（带同步波次）
  platform/values/                各组件的 Helm 参数
  platform/manifests/             Valkey、PostgreSQL 集群、看板等清单
  charts/storefront-service/      通用服务 Chart
  envs/prod/storefront/           环境配置：common.yaml + 每个服务一个文件
```

app-of-apps：根应用指向 `platform/apps/`，那里的每个文件又是一个 Application。重建平台只需要两步：安装 Argo CD，apply 根应用。**同步波次**决定顺序：先装带 CRD 的 kube-prometheus-stack（波次 −3），再装 CloudNativePG 运营商（−2），然后是 Tempo、Loki、Collector、NATS、Valkey（−1），PostgreSQL 集群（0），最后是服务（1）。波次能起作用有一个前提：Argo CD 自 1.8 起默认不评估子 Application 的健康状态，根应用会把所有子应用视为立即就绪。`argocd-values.yaml` 用一段 Lua 健康检查恢复了它，这一点很容易被忽略。

ApplicationSet 为 `envs/*/storefront/services/*.yaml` 中的每个文件生成一个 Application：增加一个服务就是增加一个文件。第 9 章（E8）会在同一机制上增加 staging 环境。

密钥不进 Git。 数据库角色密码、Grafana 管理员密码由 `bootstrap.sh` 用 `openssl rand` 生成，保存在 WSL 的 `~/.lab-secrets/v2/`（权限 600），直接创建为 Kubernetes Secret；Argo CD 读取配置仓库用的是只读、30 天过期的项目访问令牌。第 6 章（E5）会把这些密钥移到外部密钥管理系统。

示意图

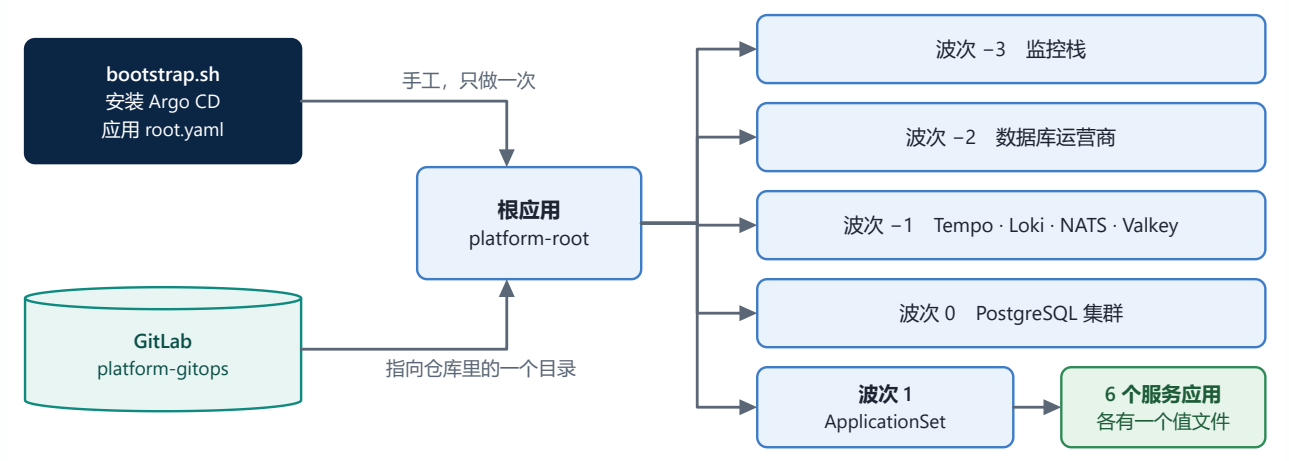


图 2-4　平台从 Git 引导（app-of-apps）：手工只做两件事——安装 Argo CD、应用根应用。根应用指向一个目录，目录里的每个文件是一个应用；同步波次决定先后，ApplicationSet 为每个服务生成一个应用。

2.4 实现要点

2.4.1 一次依赖调用的完整流程

`common/upstream.py` 的核心循环按固定顺序做检查（有删节）：

```
async def request(self, method, path, *, json=None, headers=None):
    headers = {**context.outgoing_headers(), **(headers or {})}   # 请求 ID 与截止时间
    idempotent = method in ("GET", "HEAD") or "Idempotency-Key" in headers
    if self.budget:
        self.budget.deposit()                          # 每个请求为预算存入 0.2 个令牌
    async with self.bulkhead:                          # 舱壁满了立即失败，不排队
        return await self._attempts(method, path, json, headers, idempotent)

async def _attempts(self, method, path, json, headers, idempotent):
    attempt = 0
    while True:
        remaining = context.remaining_seconds()
        if remaining is not None and remaining <= 0:
            raise self._reject("deadline")             # 上游已放弃：不再发出调用
        if self.breaker and not self.breaker.allow():
            raise self._reject("circuit_open")         # 熔断：快速失败
        timeout = min(self.policy.timeout_s, remaining or inf)
        ...                                             # 发出一次尝试，记录结果与耗时
        if outcome == "ok":
            return resp
        retryable = outcome in ("timeout", "connect_error") or status in {429, 500, 502, 503, 504}
        if not (idempotent and retryable and attempt < self.policy.retries):
            break
        if self.budget and not self.budget.try_withdraw():
            break                                      # 预算用完：放弃重试
        attempt += 1
        await asyncio.sleep(self.policy.backoff(attempt))   # 全抖动退避
```

失败最终变成 `UpstreamError`，由统一的异常处理器转换为状态码：熔断、舱壁满、预算用完返回 `503`（附 `Retry-After`），超时和截止时间用完返回 `504`，连接失败和重试后仍错误返回 `502`。调用方因此能从状态码区分“对方过载、请稍后”和“对方坏了”。

2.4.2 saga 的失败分支

orders 的下单函数中，每一个可能失败的步骤都写明了补偿（`orders/main.py`）：

```
try:
    r = await up["inventory"].request("POST", "/reservations", ...)
except UpstreamError as e:
    await release(order_id)       # 没收到回答，不代表没预留：照样补偿（释放是幂等的）
    await fail(order_id, "FAILED", f"inventory {e.kind}", 503, total)
    raise
...
if p.status_code == 402:          # 支付被拒
    await release(order_id)
    return await fail(order_id, "PAYMENT_DECLINED", "declined by provider", 402, total)
if p.status_code != 201:          # 409：同一个键的另一次尝试还在进行，结果未知
    await release(order_id)
    return await fail(order_id, "FAILED", f"payments HTTP {p.status_code}", 503, total)
```

最后一个分支是一个有意保留的缺口：支付结果未知时订单被标为失败、库存被释放，但那次仍在进行的扣款可能随后成功。真实系统用**对账**（reconciliation）补上：定期查询支付服务（`GET /charges/{key}`），把“已扣款却失败”的订单找出来退款或补单。第 7 章（E6）会让这个缺口真实地出现。

2.4.3 测试

测试分三层[3]：

- **韧性原语**：退避的指数增长与上限、预算把 100 个请求的重试限制在 20 次、熔断器的状态机（含半开状态的试探失败）、舱壁拒绝而不排队；
- **依赖调用**（用 `httpx.MockTransport` 模拟对方）：503 后重试成功、没有幂等键的 POST 不重试、预算阻止重试风暴（10 个请求只重试 2 次，而不是 30 次）、熔断后不再发出调用、截止时间被传递、过期的截止时间立即失败；
- **服务**：把真实的 catalog、inventory、payments 应用在进程内连起来，测试 saga 的成功、重放、缺货、拒付补偿和支付不可达的补偿；
- **集成**（`compose.test.yaml` 提供 PostgreSQL 18.6、Valkey 9.1.3、NATS 2.15.2）：迁移只执行一次；30 个并发预留争抢 5 件库存，恰好 5 个成功、25 个得到 409，库存不会被超卖；一笔订单经 outbox、NATS 到达通知服务。

2.5 验证

2.5.1 第一次从 Git 收敛

引导脚本用了 2 分 41 秒（生成密钥、构建并推送镜像、创建配置仓库与只读令牌、安装 Argo CD、apply 根应用，[4]）。之后的一切由 Argo CD 完成，`wait-apps.sh` 记录了时间线[5]：

时刻	事件
t+0 s	根应用开始同步，波次 −3：kube-prometheus-stack
t+161 s	波次 −2：CloudNativePG 运营商
t+188 s	波次 −1：Tempo、Loki、Collector、NATS、Valkey
t+261 s	波次 0：PostgreSQL 集群 pg-main
t+698 s	pg-main 就绪；波次 1：ApplicationSet 生成 6 个服务应用
t+735 s	15 个应用全部 Synced + Healthy

实测数据

第一次从 Git 收敛：同步波次的时间线（秒）

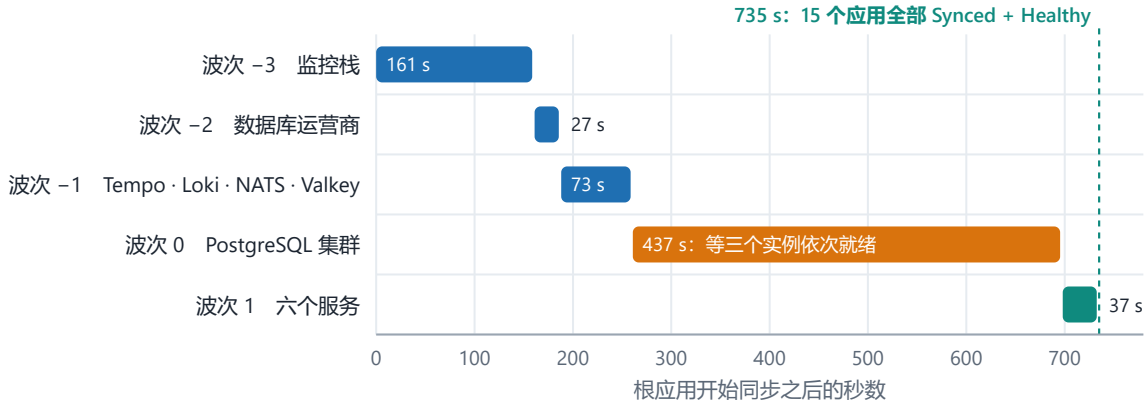


图 2-5 第一次从 Git 收敛的时间线。每一条是一个同步波次从开始到下一波次开始的时间：整个 735 秒里，437 秒在等 PostgreSQL 集群就绪（第一个实例初始化，两个副本依次加入），而 6 个服务自己只用了 37 秒。

整个平台从 Git 到可用用了 735 秒，其中 PostgreSQL 集群就占了 437 秒（t+261 到 t+698）。第一个副本的初始化日志停在一行：`pg_basebackup: initiating base backup, waiting for checkpoint to complete`。主库自己的检查点日志给出了完整的答案[6]：

```
03:54:52.034  checkpoint starting: force wait                    ← 第一个副本的 pg_basebackup 请求
03:59:22.339  checkpoint complete: wrote 2900 buffers (35.4%) ... write=269.958 s, total=270.306 s
04:00:09.532  checkpoint starting: force wait                    ← 第二个副本
04:00:09.625  checkpoint complete: wrote 0 buffers (0.0%) ... total=0.094 s
04:05:09.742  checkpoint starting: time   ... wrote 177 buffers ... write=17.729 s
04:10:09.111  checkpoint starting: time   ... wrote  93 buffers ... write=9.367 s
```

`pg_basebackup` 默认请求一个“平缓”（spread）检查点。平缓检查点在进度领先计划时，每写一个缓冲区就休眠 100 毫秒，计划的终点是 `checkpoint_completion_target × checkpoint_timeout` = 0.9 × 5 分钟 = 270 秒。所以它的耗时约为“脏缓冲区数 × 0.1 秒”，最多 270 秒：后面几次定时检查点 93 个缓冲区用了 9.4 秒、177 个用了 17.7 秒，正好符合。第一个副本加入时，主库刚完成 initdb 和三个数据库的创建，有 2900 个脏缓冲区，检查点被拉满到 270 秒；第二个副本加入时没有脏缓冲区，检查点只用了 0.094 秒，副本在 48 秒后就启动了（[7]：两个副本实例分别在 03:59:26 和 04:00:14 启动）。

结论：**新建副本的耗时可能被一个与数据量无关的检查点主导。** `pg_basebackup --checkpoint=fast` 可以避免等待，代价是一次集中的写入；另一种做法是从备份或卷快照创建副本，第 8 章（E7）计算“重建一个副本需要多久”时会再回到这里。

2.5.2 一笔订单的完整链路

通过负载均衡器下一单，然后用订单号在 Tempo 中搜索（TraceQL `{ span.order.id = "..." }`），再把链路打印成树（`experiments/e1/trace_tree.py`，[2]，节选）：

```
65 spans, 6 services: catalog, gateway, inventory, notifier, orders, payments; total 320.8 ms
      0.0 ms    156.6 ms  gateway   SERVER   POST /api/orders
     12.4 ms    137.0 ms      orders    SERVER   POST /orders  order.status=CONFIRMED
     26.1 ms     27.2 ms        orders    CLIENT   GET                       ← 两个价格查询并行
     27.7 ms     41.9 ms        orders    CLIENT   GET
     72.1 ms     25.4 ms        orders    CLIENT   POST                      ← 预留库存
     85.6 ms      2.7 ms            inventory CLIENT UPDATE stock SET available = available - $2 ...
     99.7 ms     36.2 ms        orders    CLIENT   POST                      ← 支付
    138.3 ms      0.9 ms        orders    CLIENT   BEGIN;
    142.6 ms      1.9 ms        orders    CLIENT   INSERT INTO outbox ...
    144.7 ms      1.9 ms        orders    CLIENT   COMMIT;
    267.2 ms     10.0 ms        orders    PRODUCER send orders.confirmed   ← 中继发布
    283.0 ms     37.8 ms          notifier  CONSUMER process orders.confirmed
```

几个值得注意的地方：

- 顾客在 156.6 ms 时已经拿到了 `201`；事件在 outbox 中等了约 118 ms（144.7 ms 提交，267.2 ms 发布）才被中继取走，又过了约 6 ms 被通知服务开始处理。这两段等待就是异步解耦的代价，在链路中直接可见。中继在 outbox 为空时每 200 ms 轮询一次，这解释了大部分等待。
- 同一个请求的日志也带着同一个 trace ID（`8435aeb0...`）和请求 ID（`e1-trace-demo`），可以在 Loki 中按任一个查到这笔订单在每个服务中的日志。
- 用同一个幂等键重复下单，得到 `200` 和 `Idempotent-Replayed: true`，没有产生第二笔订单。

真实截图

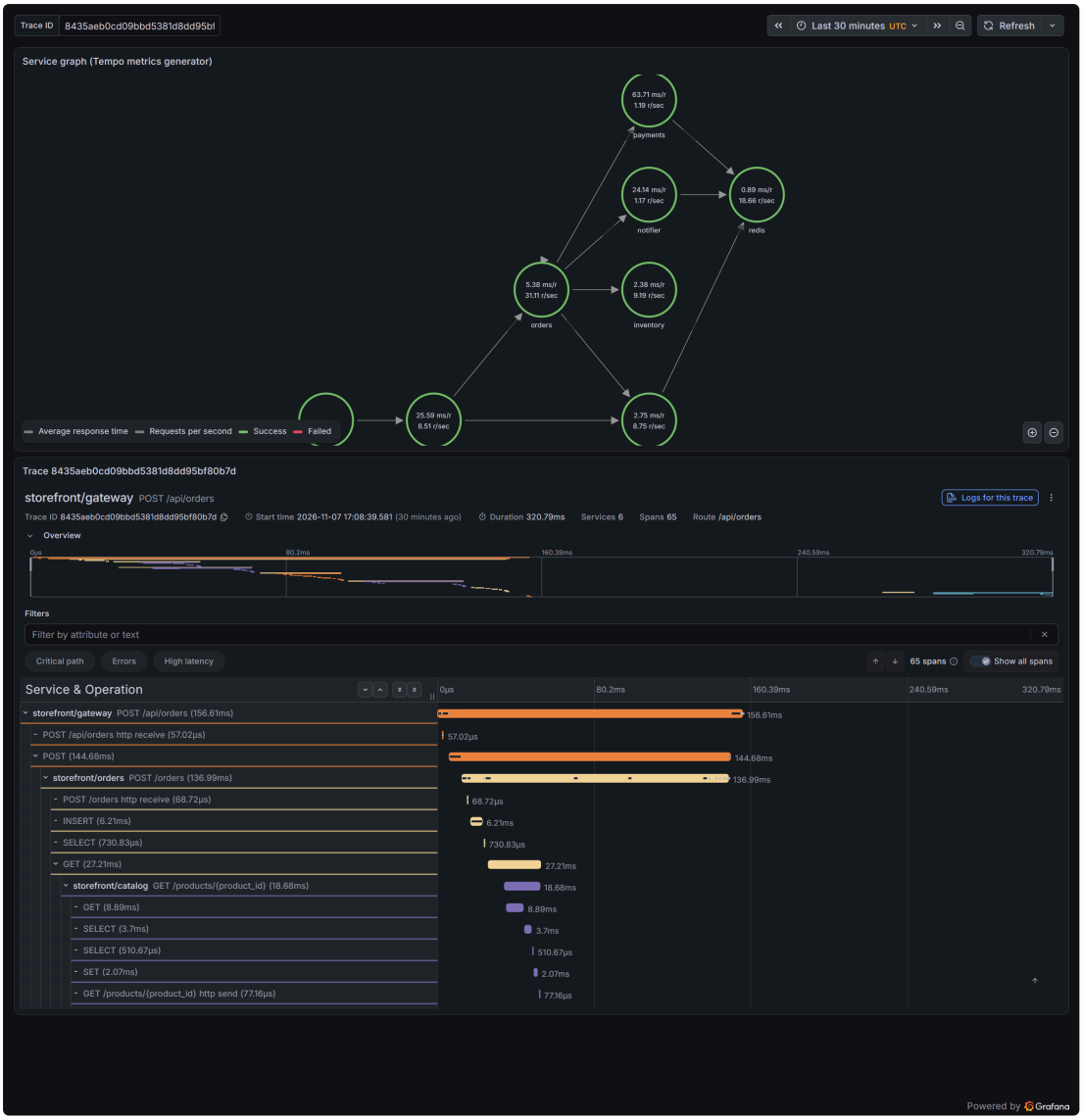


图 2-6 一笔订单的链路：上方是 Tempo 生成的服务依赖图，下方是链路瀑布图，6 个服务、65 个 span、320.79 ms；右侧末端的两段是中继发布和通知消费（Grafana 截图）

2.5.3 基线流量

k6 以开放模型（constant-arrival-rate，每秒固定数量的迭代，不因系统变慢而减少）运行 5 分钟：70% 浏览（列表 + 一个商品）、25% 下单、5% 查询订单。k6 运行在主机上，固定在控制面所用的 CPU 0-3，不占用工作节点的 CPU。结果[8]：

```
rate=20/s duration=5m iterations=6001 dropped=0
requests=10183 (34.6/s) failed=0.00%
browse: p50=10.4 p95=17.0 p99=27.5 max=93.6 ms
order:  p50=107.7 p95=216.2 p99=291.2 max=789.2 ms
checks: 100.00% passed
```

`dropped=0` 说明负载生成器跟上了设定的速率，结果可信（开放模型下，生成器自己跟不上时会丢弃迭代，这是判断压测是否有效的第一个指标）。下单的延迟主要来自 `[模拟]` 的支付服务：对数正态分布、中位数 40 ms、p99 250 ms，理论上的 p95 为 146.2 ms[9]，再加上价格查询、库存预留和两次数据库事务。此时各 Pod 的 CPU 用量只有 0.004–0.16 核[10]，系统远未饱和，第 4 章（E3）会一直加压到拐点。

服务端的直方图给出的是估计值，精度取决于桶的边界（第一册第 9 章）。这里有一个现成的对照：同一个支付服务的 p95，旧的桶（100 ms 与 250 ms 之间没有边界）估计为 213 ms，加密后的桶估计为 165 ms，理论值是 146 ms 加上约 4 ms 的 Valkey 操作；连接池等待的 p95，旧桶（第一个桶是 0–5 ms）恒为 4.75 ms，新桶为 0.22 ms（同一记录）。看板截图中可以看到这个台阶出现在新版本上线的那一刻。

真实截图

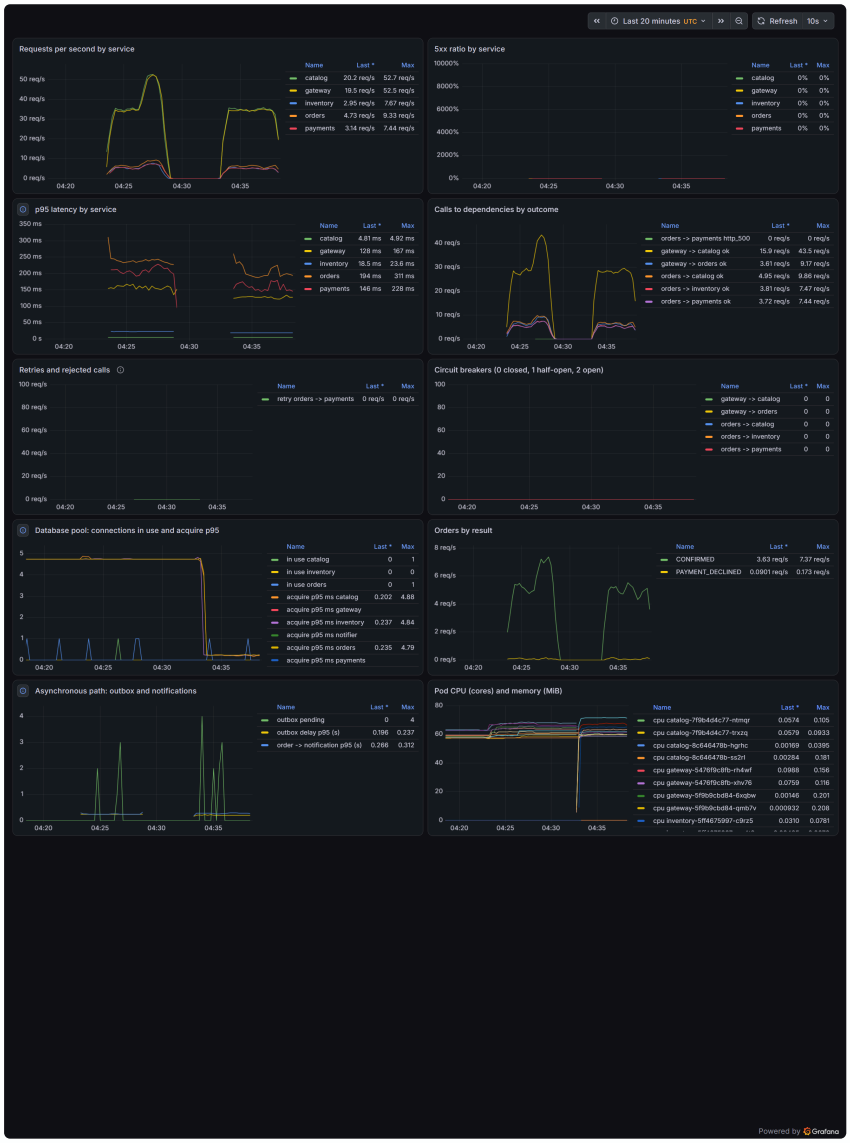


图 2-7 storefront 看板：各服务的请求速率、错误率、p95 延迟，依赖调用的结果、重试与熔断器，连接池，订单结果（Grafana 截图）

2.6 排障记录：第一次部署暴露的问题

系统第一次真正运行，就暴露了一组单元测试和集成测试都没有发现的问题。它们都是真实发生的，每一个都留下了证据。

#	现象	根因	修复	证据
1	notifier 启动后重启两次	它比 orders 先启动，流 ORDERS 还不存在，`pull_subscribe` 抛出 `stream not found`，进程退出	等待流出现（指数退避），期间就绪探针返回 503；增加回归测试	`converge.log`、`app-tests-2.log`
2	Argo CD 控制器被 OOM 杀死 3 次，无人察觉	启动时同步整个集群的资源缓存，内存峰值超过 768 Mi 的上限（内核日志 `anon-rss:783016kB`，启动后 6 秒被杀）	上限 1 GiB、请求 512 Mi、`GOMEMLIMIT=850MiB`、并行处理数 10/5；以同样的触发条件验证	`argocd-oom.log`
3	一个节点的工作集达到可分配内存的 86%	多个组件的 requests 远低于实际用量（Tempo 请求 192 Mi，实际 719 MiB），调度器按 requests 把它们放到了同一节点	按实测调整 requests；去掉不需要的边车（Loki 规则边车单独就占 71 MiB）	`rightsize.log`
4	每个依赖调用都打一行日志	httpx 以 INFO 级别记录每个出站请求，一笔订单 4 行	httpx 日志级别改为 WARNING	`one-order-trace.log`
5	Tempo 的服务依赖图是空的	打开了指标生成器，但它的处理器要在 `overrides` 中按租户启用，默认一个都没有	启用 `service-graphs` 和 `span-metrics`	`tempo-service-graph.log`、`tempo-service-graph-after.log`
6	看板上出现一次 orders → payments 的 HTTP 500	payments 在保存支付结果后释放幂等锁，Valkey 读超时（200 ms）使请求失败；orders 的重试命中已保存的结果，被“重放”	释放锁改为尽力而为（锁本来就会过期）；Valkey 超时按用途区分：缓存 0.2 s，记录 1 s；增加回归测试	`app-tests-3.log`
7	连接池等待的 p95 恒为 4.75 ms；payments 的 p95 为 213 ms	直方图桶太粗：所有观测值都落在第一个桶（0–5 ms），插值得到 0.95 × 5 ms；100–250 ms 之间没有边界	连接池等待改用亚毫秒级的桶，延迟桶加密：分别变为 0.22 ms 和 165 ms	`latency-buckets.log`
8	同一服务两个副本的 CPU 相差十几倍	见下一节	留给第 3 章（E2）	`baseline-distribution.log`

第 6 条值得多说几句。一次本可以变成“重复扣款”的事故，因为幂等键而只是一次多余的错误：payments 已经保存了扣款结果，只是在释放锁时超时，返回了 500；orders 按策略重试（带同一个幂等键），payments 直接返回已保存的结果。修复的关键在于区分 Valkey 在不同服务中的角色：对 catalog 它是可以丢弃的缓存，超时应该短，宁可绕过它；对 payments 它是记录，超时太短反而制造错误。**同一个依赖，在不同的调用方那里可能需要不同的超时。**

第 2 条说明了平台组件本身也需要容量规划。Argo CD 的应用控制器为它管理的集群维护一份完整的资源缓存，内存随集群中对象和 CRD 的数量增长，而不是随应用数量增长；它被杀死后会自动重启、重新同步，表

面上一切正常，只有内核日志和容器的 `lastState` 记录了这件事。**一个会自己恢复的组件，可能长期带着问题运行而无人知道。**

第 3 条中有一个容易被忽略的约束：Tempo、Prometheus、Loki 和 PostgreSQL 使用 local-path 存储，卷建在某一个节点上，Pod 只能调度到那个节点。调整 requests 只影响以后的调度，不会把它们搬走。**存储的位置是调度的约束。**

2.6.1 负载在副本之间严重不均

基线的 5 分钟内，各服务两个副本承担的请求比例（服务端计数，`experiments/load/distribution.sh`，[10]）：

服务	调用方	副本 A	副本 B
catalog	2 个 gateway Pod + 2 个 orders Pod	49.0%	51.0%
gateway	k6 的大量连接（经负载均衡器）	57.4%	42.6%
orders	2 个 gateway Pod	91.3%	8.7%
inventory	2 个 orders Pod	92.1%	7.9%
payments	2 个 orders Pod	88.3%	11.7%

kube-proxy 的负载均衡发生在 TCP 连接建立的时候（第一册第 5 章），而调用方的 HTTP 客户端会复用连接（keep-alive）。每个调用方 Pod 只保持少数几个连接，之后的请求都沿着这些连接走，落到哪个副本取决于连接建立时的随机选择。表中的规律与此一致：调用方和连接越多（catalog、gateway），分布越均匀；只有两个调用方 Pod 的服务严重倾斜。而且倾斜会沿调用链传递：承担了 91% 订单的那个 orders 副本，也发出了绝大部分对 inventory 和 payments 的调用。这不是 storefront 的缺陷，而是 L4 负载均衡与长连接的固有矛盾。第 3 章（E2）先在压测期间统计每个 Pod 实际持有的连接来证实这个解释，再用服务网格的 L7 负载均衡解决它。

2.6.2 一次意外：主机重启之后

本章的工作被一次会话中断打断，WSL 虚拟机随之关闭。下一次启动时，平台“自己恢复”的过程又暴露了三件事[11, 12, 13, 14]：

1. **已暂停的第一册集群自己启动了。** Kind 创建节点容器时使用 `--restart=on-failure:1`，而 Docker 文档写明，手动 `docker stop` 只是暂停重启策略，到 Docker 守护进程重启为止；节点的 systemd 停止时退出码是 130，算作“失败”。三个 v1 节点于是随虚拟机一起启动，多占了约 5 GiB 内存。`v1-lab.sh stop` 现在会先把重启策略改为 `no`，`start` 时再恢复。
2. **ent 的控制面没有回来。** 两次 `wsl.exe` 调用之间有几秒钟没有会话，WSL 终止了发行版，所有容器被正常关闭；再次启动时，Docker 按重启策略拉起控制面，却因为 systemd 还没清理完上一次的 cgroup 作用域（`unable to start unit docker-….scope ... got failed`）而失败，且 `on-failure:1` 只重试一次。工作节点都在运行，API Server 却不可达。手动 `docker start` 后 12 秒恢复。

3. **PostgreSQL 发生了一次计划外的故障转移。** 控制面恢复后，CloudNativePG 运营商发现主库 pg-main-1“不健康”（实例刚刚重启），立即把 pg-main-3 提升为主库（事件 `FailingOver`、`Failover Target`），旧主库作为副本重新加入。新主库在时间线 2 上，三个实例全部就绪；4071 笔订单都在，outbox 中没有未发布的事件，新的下单请求正常。由于关机时没有流量，没有写入处于风险中。

第 3 条说明，一次“整个集群重启”在运营商眼里与“主库故障”无法区分：主库只是还没启动完，就被换掉了。CloudNativePG 的 `failoverDelay` 可以让它多等几秒再判断，代价是真正的故障时恢复变慢。第 8 章（E7）会有意地制造主库故障，测量这个取舍。

2.7 局限

- 数据库、缓存和消息系统都运行在集群内，与服务共享三个模拟节点；它们的性能数字不代表托管数据库或专用服务器。
- payments 是 `[模拟]` 的外部服务，延迟是按对数正态分布（中位数 40 ms、p99 250 ms）人为生成的，用来代表第三方的不可控延迟。
- saga 没有实现对账；支付结果未知时，系统会把订单标为失败，这在第 7 章被有意利用。
- 服务数量（6 个）远少于真实企业系统，但通信模式（同步扇出、异步事件、共享缓存、共享数据库集群）具有代表性。

2.7.1 从实验环境到生产环境

本章的做法	生产环境通常的做法	差别意味着什么
六个服务共用一个镜像和一套公共库	每个服务独立的代码库、镜像与发布节奏	这里一次构建更新全部服务；真实团队要面对版本并存与接口契约的兼容
payments 是自己写的模拟器，延迟人为生成	外部支付网关：限流、回调、对账文件	“结果不明”在真实网关上更常见，对账是必需的流程，而不是可选的补充
一个 PostgreSQL 集群上每服务一个库；Valkey 单实例三种用途	按数据的重要性分开实例，缓存与“记录”不共用	淘汰策略、故障域和备份要求不同的数据放在一起，迟早互相拖累
中继每 200 ms 轮询 outbox	变更数据捕获（读取 WAL）或 LISTEN/NOTIFY	轮询简单、延迟固定；变更数据捕获延迟更低，但多一个要运维的系统

2.8 常见问题

Q1：为什么六个服务用一个镜像？ 它们共享同一个代码库和依赖；一个镜像意味着只构建、扫描、签名一次。每个服务仍然有自己的 Deployment 和镜像标签，可以独立发布：只给 orders 换一个新标签，其他服务不受影响。代价是任何一个服务的代码改动都会产生新的镜像标签（本章标签取最后一次修改 `apps/storefront` 的提交）。

Q2：为什么就绪探针不检查所有依赖？ 第一册第 5 章讲过：依赖一抖动，所有副本同时变为未就绪，整个服务从负载均衡中消失，比依赖本身的故障还严重。storefront 的就绪检查只看“自己能否工作”：连接池是否

可用（catalog、inventory、orders），是否已连上消息系统并完成订阅（notifier）。

Q3：outbox 中继每 200 ms 轮询一次，为什么不用 PostgreSQL 的 LISTEN/NOTIFY 或 CDC? 轮询最简单、最容易理解，代价是平均约 100 ms 的额外延迟和空闲时的少量查询。对通知这种场景足够。延迟敏感的系统常用变更数据捕获（CDC，例如 Debezium 读取 WAL）代替轮询。

Q4：Valkey 为什么不做高可用? 单实例是有意的简化，也是一个已知的单点。第 7 章会让它失效，观察 catalog（降级）、payments（失败）、notifier（暂停）三种不同的反应，这正好说明了“同一个依赖，不同的角色”。

2.9 练习

练习

1. **（计算）** 调用链为 client → gateway → orders → payments，三层调用方都采用 naive 策略（失败后立即重试 3 次）。payments 完全不可用时，一个用户请求最多会让 payments 收到多少次请求？换成 resilient 策略（每层最多重试 2 次、重试预算 20%）后，在稳态下最多多少?
2. **（分析）** orders 在“提交订单与 outbox”之后、返回响应之前崩溃。客户端会看到什么？它重试时会发生什么？事件会不会丢失、会不会重复?
3. **（设计）** 为 saga 写一个对账任务：找出“支付已成功、订单却是 FAILED”的情况并修复。说明它查询什么、多久运行一次、修复动作是什么，以及如何保证修复本身是幂等的。
4. **（动手）** 把 outbox 中继的空闲轮询间隔从 200 ms 改为 1 秒，用 `trace_tree.py` 对比一笔订单的 PRODUCER span 与 COMMIT 之间的间隔，并在看板上观察 `outbox delay p95` 的变化。
5. **（排障）** 某个服务的 `upstream_rejected_total{reason="bulkhead_full"}` 突然增长，而依赖服务的延迟正常。列出三种可能的原因和区分它们的方法。
6. **（思考）** Argo CD 控制器被 OOM 杀死后自动恢复，应用状态始终显示 Synced/Healthy。你会为平台组件建立什么样的监控或告警，让这类“自愈掩盖的故障”被发现?
7. **（设计）** 本章的同步波次让服务等待 PostgreSQL 完全就绪（三个实例），用了 437 秒。服务其实只需要主库可用。给出两种缩短等待的办法，并说明各自的风险。

参考答案见附录。

2.10 本章小结

- storefront 由六个服务组成，同步调用用于需要结果的步骤，异步消息用于后续工作；数据按服务划分到 PostgreSQL 的不同数据库，缓存与记录共用一个 Valkey（已知的简化）。
- saga 加补偿处理跨服务的业务流程；幂等键让重试变得安全；事务性 outbox 解决双写问题，配合消息 ID 去重实现“至少一次 + 去重”。
- 韧性原语各回答一个问题：超时、截止时间、重试、预算、熔断、舱壁；naive 模式保留为第 7 章的对照组。

- 平台从 Git 引导：两步重建，735 秒收敛，其中 PostgreSQL 副本因为平缓检查点占了 437 秒。
- 一笔订单的链路跨越 6 个服务和一次消息传递，outbox 与流的等待时间在链路中直接可见。
- 第一次部署暴露的八个问题都留下了证据并已修复；负载在副本之间严重不均，这是 L4 负载均衡遇上长连接的必然结果，下一章用服务网格解决。
- 一次意外的主机重启表明，“自动恢复”本身也会出错：暂停的集群被重启策略拉起，控制面因 cgroup 竞争没有回来，PostgreSQL 把正在启动的主库当作故障并完成了一次计划外的故障转移（无数据丢失）。

第 3 章

服务网格：身份、加密与 L7 流量（E2）

上一章留下了两个问题：服务之间的流量是明文的，任何能接触到节点网络的人都能读到请求头和订单内容；负载在副本之间严重不均，一个副本承担了九成以上的请求。服务网格同时回答这两个问题，但它不是免费的：多了一层代理，就多了一层延迟、一层资源开销，以及一层“你没有配置、它却在做”的默认行为。本章用 Istio 的 ambient 模式给 storefront 加上网格，每一步都与上一步做对照测量：线上抓包证明加密，连接普查证明失衡的原因，交替进行的 A/B 实验测量网格的真实代价，故障注入揭示重试在不同层之间的叠加。

工程问题

第 2 章留下两个问题：服务之间的流量是明文的；长连接让请求压在一个副本上。不改应用代码，服务网格能解决到什么程度，又要付出什么代价？

实测结论

- 加入 ambient 网格不需要重启任何 Pod；节点之间线上可读的服务间请求行从 140 行变为 0。
- 只加密（L4）解决不了负载不均：inventory 仍是 97.2% / 2.8%；经过 waypoint（L7）之后，各服务的两个副本都在 46.9%–53.1% 之间。
- 代价要交替测量才测得准：L4 让浏览的 p50 增加约 2.4 ms，waypoint 再加约 1.3 ms；相隔 15 分钟的第一次对比得出的 +32 ms 主要是漂移。
- 网格的重试与应用的重试相乘：加一条“重试 3 次”的路由，payments 收到的调用变成 4 倍。

3.1 学习目标

学完本章，你应当能够：

- 说明服务网格把哪些能力从应用代码中移出来，sidecar 与 ambient 两种数据面的结构差异；
- 解释 HBONE、SPIFFE 身份和 mTLS 如何在不修改应用的情况下加密并认证服务间的流量，并用抓包验证；
- 区分 L4 与 L7 负载均衡，用连接普查和请求分布证明长连接下 L4 失衡的原因；
- 设计能控制漂移的性能对照实验（交替 A/B、微基准、按跳分解），测量网格的真实代价；
- 识别不同层次的重试如何叠加，以及网格默认行为带来的“看不见的”重试；
- 预判开启 STRICT mTLS 会让哪些不在网格中的调用方失效。

3.2 原理

3.2.1 网格把什么从应用里拿走

上一章的共享库为每个服务实现了超时、重试、熔断、指标和链路追踪。这在一个团队、一种语言的系统里可行；当一个企业有几十个团队、五六种语言时，同样的能力要实现五六遍，版本还各不相同。服务网格把一部分网络能力下沉到应用之外的代理中：

能力	应用库能做	网格能做	谁更合适
服务间加密与身份认证	很难（每个服务管理证书）	自动签发、轮换证书，透明加密	网格
按身份的访问控制	需要自己验证调用方	基于身份与 L7 属性的策略	网格
L7 负载均衡、流量拆分	客户端负载均衡（各语言各写一遍）	统一实现	网格
统一的 RED 指标	每个服务自己暴露	代理统一产生	两者互补
超时、重试、熔断	知道业务语义（是否幂等、能否降级）	不知道语义，只看状态码	**应用为主**，网格谨慎使用
链路追踪的上下文传递	必须由应用转发 `traceparent`	只能生成自己的 span	应用

最后两行是本章实验要证明的要点：网格不知道一个 POST 能不能重试，也无法替应用传递追踪上下文。

3.2.2 sidecar 与 ambient

Istio 有两种数据面。**sidecar 模式**在每个 Pod 里注入一个 Envoy 容器，所有进出流量都经过它：每个 Pod 多一个进程，加入或升级网格都要重启 Pod。**ambient 模式**把网格拆成两层：

- **ztunnel**（零信任隧道）是每个节点一个的 L4 代理，用 Rust 编写。istio-cni 节点代理在每个加入网格的 Pod 的网络命名空间里写入 iptables 规则，把出站流量重定向到 15001、入站流量重定向到 15006、DNS 重定向到 15053，再把 Pod 交给 ztunnel 接管。ztunnel 负责 mTLS、L4 授权和 TCP 指标，不解析 HTTP。
- **waypoint** 是可选的 Envoy 代理，按命名空间或服务部署。只有需要 L7 能力（按请求的负载均衡、路由、重试、L7 授权、HTTP 指标）时，流量才经过它。

加入网格只需要给命名空间打标签 `istio.io/dataplane-mode=ambient`，**不重启任何 Pod**；需要 L7 时再部署 waypoint，并用 `istio.io/use-waypoint` 标签让流量经过它。

示意图

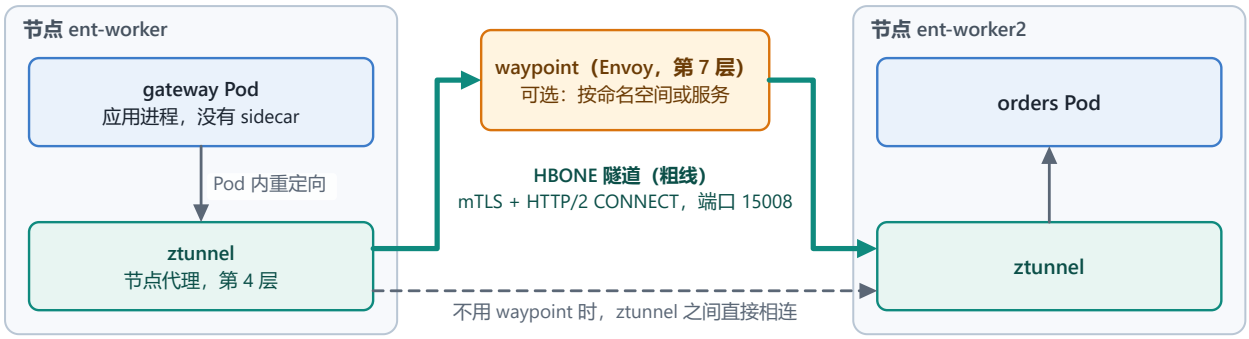


图 3-1 ambient 模式的数据路径：Pod 里没有 sidecar，流量被重定向到本节点的 ztunnel，节点之间用 HBONE（mTLS 上的 HTTP/2 CONNECT，端口 15008，图中的粗线）传输；需要第 7 层能力时，流量再经过一个 waypoint。

3.2.3 HBONE 与工作负载身份

ztunnel 之间用 **HBONE**（HTTP-Based Overlay Network Environment）通信：一条到对端节点 15008 端口的 mTLS 连接，其中每个应用连接是一个 HTTP/2 `CONNECT` 流。证书由 istiod 签发，标识的是**工作负载身份**而不是 IP：

```
spiffe://cluster.local/ns/storefront/sa/orders
```

身份来自 Pod 的 ServiceAccount。这正是第 2 章给每个服务一个独立 ServiceAccount 的原因：网格里的授权策略写的是“谁”（身份），而不是“从哪个 IP 来”（IP 会变）。

3.2.4 L4 与 L7 负载均衡

kube-proxy 和 ztunnel 都在**连接**建立时选择后端。一个调用方保持 k 条长连接，每条连接随机落到 N 个副本之一，之后它的所有请求都沿着这些连接走。当 k 很小（第 2 章的连接普查显示只有 1–3 条）时，“所有连接恰好落到同一个副本”的概率并不小：$k=2$、$N=2$ 时是 $2\times(1/2)^2=50\%$。再考虑到每条连接承载的请求数不同，失衡几乎是必然的。

waypoint 在**请求**层面做负载均衡：它终止客户端的 HTTP 连接，为每个请求重新选择后端（Istio 默认使用最少请求算法 `LEAST_REQUEST`），自己与每个后端维持连接池。代价是多一跳 Envoy，以及 HTTP 解析的开销。

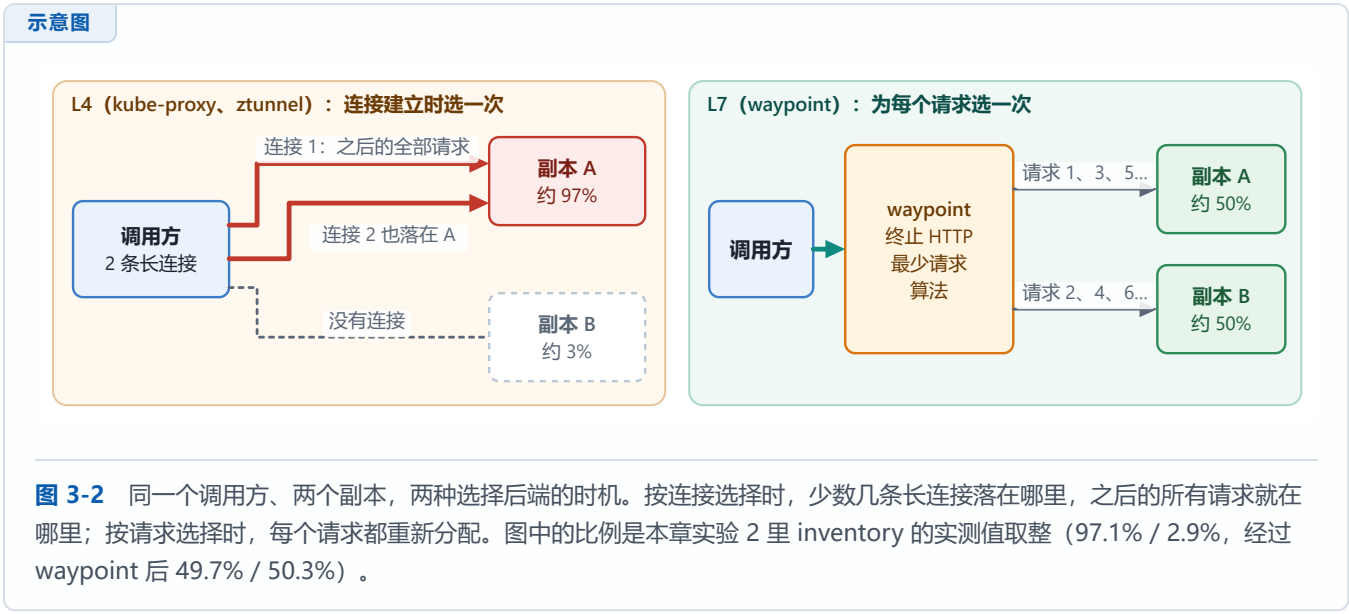


图 3-2 同一个调用方、两个副本，两种选择后端的时机。按连接选择时，少数几条长连接落在哪里，之后的所有请求就在哪里；按请求选择时，每个请求都重新分配。图中的比例是本章实验 2 里 inventory 的实测值取整（97.1% / 2.9%，经过 waypoint 后 49.7% / 50.3%）。

另一种思路是让客户端定期关闭长连接（例如每 N 个请求或每 T 秒重建一次），用更多的连接建立换来更均匀的分布；gRPC 的客户端负载均衡则是在客户端为每个请求选择后端。这些都是不用网格时的替代方案。

3.3 安装：从 Git 部署网格

3.3.1 选择版本

版本的选择记录在 `platform/versions.env` 中，每一条都有理由：

- **Istio 1.31.4**，而不是已经发布 16 天的 1.31.3。通常我倾向于避开刚发布一两天的版本，但 1.31.4 修复了 GHSA-rr2c-p22p-6r9r（CVSS 8.2）：在 ambient 模式下，一个运算符优先级错误会让端口级的 STRICT PeerAuthentication 被跳过，明文流量可以到达本应只接受 mTLS 的端口。这恰好是本章要验证的能力，安全修复优先于“发布时间”。
- **Gateway API v1.6.3**，而不是最新的 v1.7.0：Istio 1.31 的 `go.mod` 依赖的是 `sigs.k8s.io/gateway-api v1.6.0`。CRD 的版本应与实现它的控制器相匹配，v1.6 线的最新补丁是 1.6.3。选择 experimental 通道，是因为本章要用的 HTTPRoute 重试（GEP-1731）在 v1.6 中还属于实验特性（v1.7 才进入 standard 通道）。
- **Helm 仓库已经迁移。** 旧地址 `istio-release.storage.googleapis.com/charts` 的索引停在 1.30.8，1.31 的 Chart 只在新地址 `blob.istio.io/istio-release/charts` 上，`gcr.io` 上的 OCI 地址也没有 1.31.4[5]。依赖外部仓库的平台要预期这种迁移，企业里通常会把 Chart 镜像到自己的制品库。

3.3.2 Chart 的默认值是为大集群准备的

四个 Chart 的默认资源请求（`helm show values`）：istiod 500m CPU / 2 GiB，ztunnel 每个节点 200m / 512 Mi，istio-cni 每个节点 100m / 100 Mi。在本书的集群上，仅请求就会占去约 4.5 GiB 内存；更危险的是 ztunnel 以 `system-node-critical` 优先级运行，节点放不下时它会**抢占**应用的 Pod，而不是自己挂起。本书先设置很小的请求，安装后按实测修正。

一切由 Argo CD 完成：`gateway-api-crds` 和 `istio-base`（波次 −6）、`istiod` 和 `istio-cni`（−5）、`ztunnel`（−4）。安装后空载时的实测[6]：

组件	实例数	内存（每个）	Chart 默认请求（每个）
istiod	1	49 MiB	2048 Mi
ztunnel	4（每节点）	4 MiB	512 Mi
istio-cni	4（每节点）	15–17 MiB	100 Mi

空载的 ztunnel 只用 4 MiB，这是 Rust 实现的节点代理的优势；作为对照，sidecar 模式下每个 Pod 的 Envoy 通常要几十 MiB。有负载时 ztunnel 的用量会上升，后文的测量中有记录。

3.3.3 安装中遇到的两件小事

- **一次 HTTP 403。** 第一次把配置推送到 GitLab 时被拒绝（`RPC failed; HTTP 403`），一分钟后用完全相同的命令重试成功。令牌有效、权限正确、项目可访问，原因没有找到。记录在案，不做推测。
- **健康状态停留在旧值。** `gateway-api-crds` 应用同步完成后一直显示 Progressing，原因指向 CRD `referencegrants`，而这个 CRD 早已是 Established。读 Argo CD 3.6.1 的 CRD 健康检查脚本可以确认：Established 的 CRD 应判为 Healthy。手动刷新后 10 秒内变为 Healthy。可以确定的是，健康状态是在 CRD 刚创建、尚未 Established 的那一刻计算的，之后约 1.5 分钟没有被重新评估；更深的原因没有核实。后面的同步波次在此期间一直在等待，**一个过时的健康状态会阻塞整条波次链。**

3.4 实验 1：线上看到了什么

加密要在线上验证，而不是相信配置。`experiments/e2/capture.sh` 在节点 ent-worker 的网络命名空间里运行 tcpdump（一个 netshoot 容器加入节点容器的网络命名空间），抓取 eth0 上 10 秒的流量，同时通过负载均衡器发出 40 次商品查询和 40 次下单。eth0 上的包都是发往或来自**其他节点**的 Pod 的，也就是跨节点的服务间流量。

加入网格前后各抓一次[4]：

	发往 8080 的包	发往 15008（HBONE）的包	线上可读的服务间请求行	线上可读的外部请求行
无网格	501	0	140（`GET /products/{id}` 76、`POST /reservations` 64）	68
ambient	234	342	**0**	56

无网格时，请求行和请求头（`X-Request-ID`、`Idempotency-Key`）在线上一览无余[7]。加入网格后，服务之间的请求全部进入 15008 端口的 mTLS 隧道，线上再也读不到。

但还有 56 行明文：它们都是 `/api/...` 路径，即从负载均衡器进来的外部请求。负载均衡器（集群外的 envoy 容器）不在网格中，它经 NodePort 把明文转发给 gateway Pod 所在的节点，这一段跨节点的流量仍

是明文，到达目的 Pod 后才由 ztunnel 接收。**网格只保护网格内部的流量。**入口要么使用网格内的入口网关，要么在边缘终止 TLS；第 6 章（E5）会在 STRICT 模式下完整地解决这个问题。

加入网格不需要重启：命名空间打上标签后，istio-cni 在几秒内为每个已经在运行的 Pod 写入重定向规则并交给 ztunnel（[8] 中各 Pod 的启动时间和重启次数都没有变化，istio-cni 的日志见 [9]）。

3.5 实验 2：为什么负载不均

3.5.1 连接普查

第 2 章把副本间的失衡归因于“L4 负载均衡 + 长连接”，但没有直接证据。证据要从“谁和谁保持着连接”中找。这里有一个陷阱：在**客户端** Pod 里查看连接表，看到的目的地址是 Service 的 ClusterIP，因为 kube-proxy 的地址转换（DNAT）发生在节点的网络命名空间里，Pod 看不到转换后的后端。所以 `experiments/load/connections.sh` 在每个**服务端** Pod 里读取 `/proc/net/tcp`（镜像中没有 `ss` 和 `netstat`，用 Python 解析），列出连到 8080 端口的已建立连接，再把对端 IP 映射回 Pod 名称。

无网格时，压测进行到第 120 秒的普查结果（节选，[3]）：

```
server pod (port 8080)          client                          established
inventory-5ff4675997-c9rz5      orders-6b8b5f4579-zq89c         1
inventory-5ff4675997-c9rz5      orders-6b8b5f4579-z9ctm         1
orders-6b8b5f4579-z9ctm         gateway-5476f9c8fb-rh4wf        1
orders-6b8b5f4579-z9ctm         gateway-5476f9c8fb-xhv76        3
orders-6b8b5f4579-zq89c         gateway-5476f9c8fb-rh4wf        1
payments-569c98ff59-4kbv5       orders-6b8b5f4579-z9ctm         2
payments-569c98ff59-94k6l       orders-6b8b5f4579-zq89c         1
gateway-5476f9c8fb-rh4wf        10.240.1.1                      11
```

两个 orders 副本各自只保持一条到 inventory 的连接，而这两条连接恰好都落在了副本 c9rz5 上；另一个 inventory 副本没有任何连接。同一轮的请求分布中，c9rz5 承担了 97.1%（剩下的 2.9% 来自偶尔新建的连接）。这就是失衡的直接原因。

普查还顺带揭示了一件事：gateway 的对端不是负载均衡器，而是节点地址（10.240.1.1 是节点在 Pod 网段上的网关地址）。经 NodePort 进入的流量在 `externalTrafficPolicy: Cluster` 下被做了源地址转换，**真实的客户端 IP 丢失了**，这对按客户端限流和审计都有影响（第 6 章）。

3.5.2 三种模式下的分布

同样的负载（20 次迭代/秒，3 分钟），三种模式下每个服务两个副本承担的请求比例[3, 10, 2]：

服务	无网格	ambient L4（ztunnel）	ambient L7（waypoint）
catalog	14.4% / 85.6%	57.8% / 42.2%	50.0% / 50.0%
gateway	53.5% / 46.5%	50.7% / 49.3%	49.4% / 50.6%
orders	87.4% / 12.6%	11.2% / 88.8%	46.9% / 53.1%

服务	无网格	ambient L4（ztunnel）	ambient L7（waypoint）
inventory	97.1% / 2.9%	97.2% / 2.8%	49.7% / 50.3%
payments	86.7% / 13.3%	15.0% / 85.0%	48.2% / 51.8%

实测数据

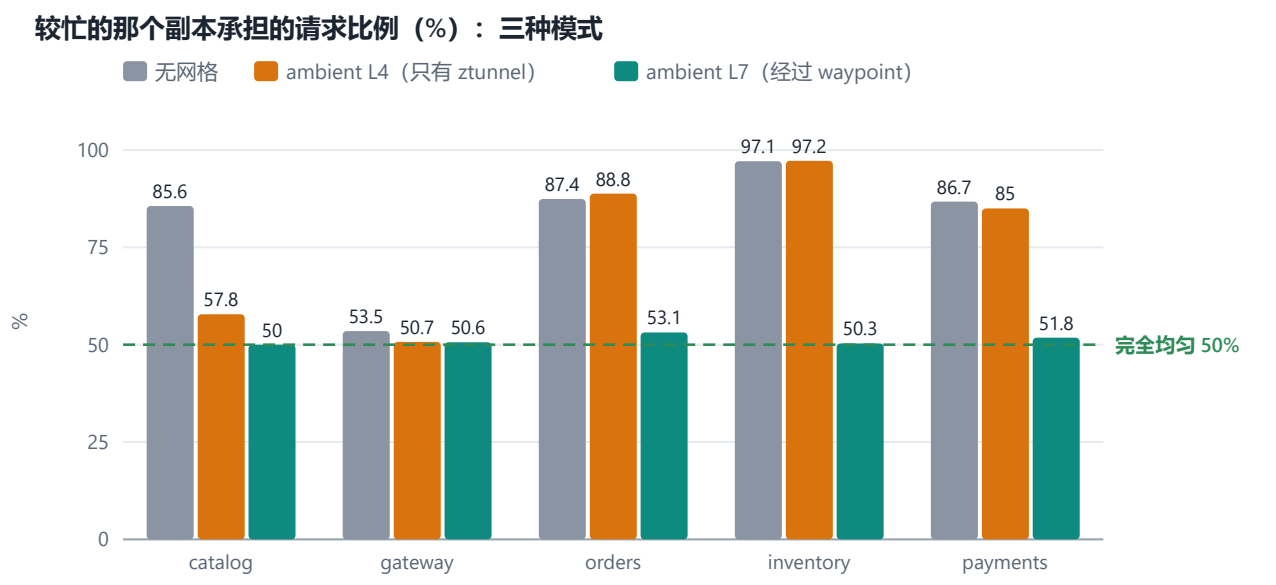


图 3-3 三种模式下，两个副本中较忙的那个承担的请求比例。50% 是完全均匀。只有 ztunnel（L4）时，不均的程度和没有网格时一样——只是换了一个副本“中签”；经过 waypoint（L7）之后，五个服务全部回到 50% 附近。gateway 的调用方是 k6 的大量连接，所以三种模式下都均匀。

结论很清楚：

- **ztunnel 不改变分布。**它是 L4 代理，同样在连接建立时选择后端，L4 模式下的失衡与无网格时同一个量级，只是哪个副本“中签”不同。加密与身份认证并不附带负载均衡的改善。
- **waypoint 让所有服务回到接近 50/50。**L7 模式下的连接普查显示，每个后端 Pod 的对端都只剩下 waypoint（每个后端 2–4 条连接）：客户端的长连接终止在 waypoint 上，waypoint 为每个请求重新选择后端。
- gateway 在三种模式下都比较均匀，因为它的调用方是 k6 的大量连接（经负载均衡器），连接数多，随机分配本身就接近均匀。这与 k 越大越均匀的推论一致。

真实截图

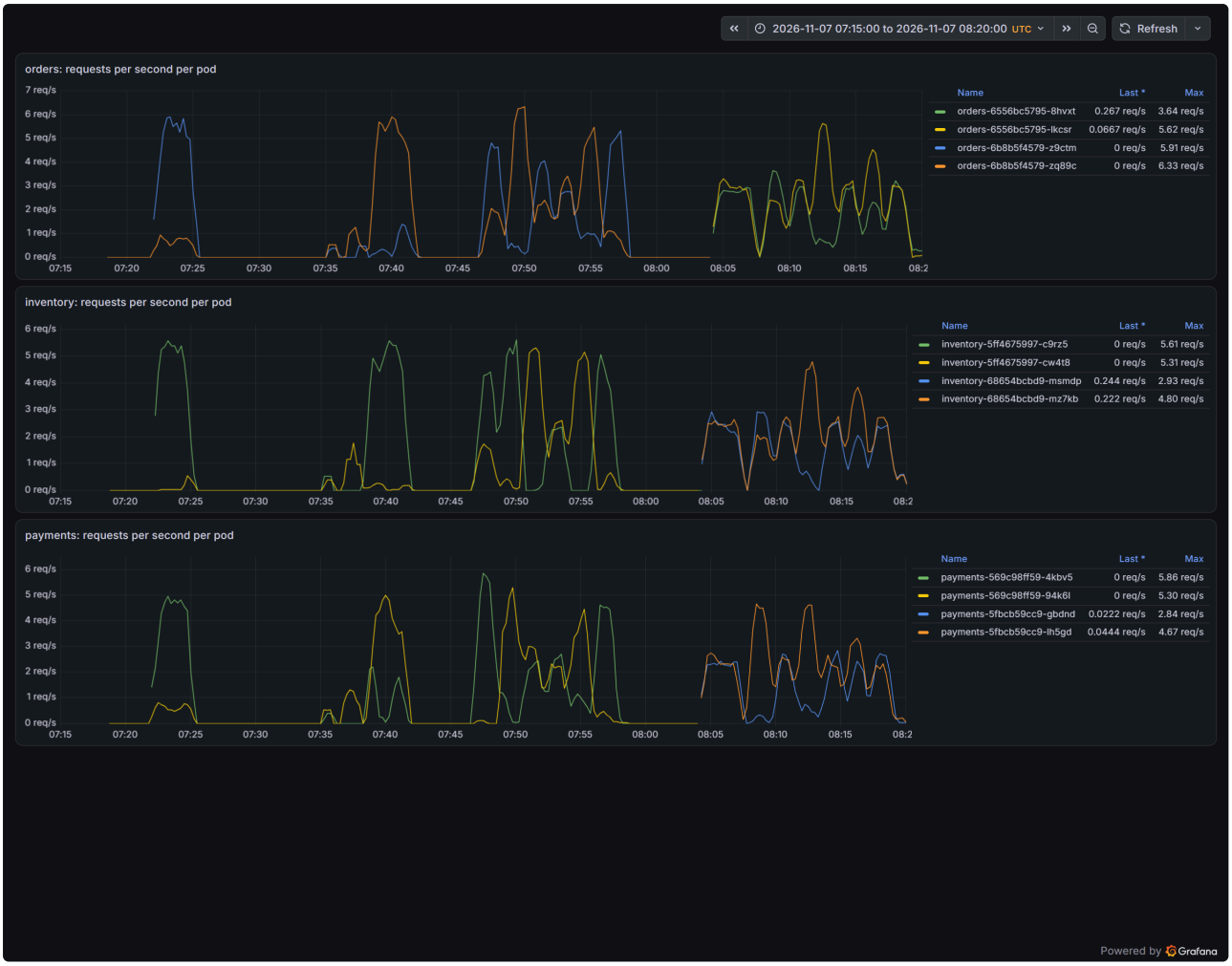


图 3-4 orders、inventory、payments 每个 Pod 的每秒请求数，07:15–08:20 UTC。08:03 之前（无网格与 ambient L4）每个服务都是一条线高、一条线贴近零；08:03 部署 waypoint 后新的 Pod 上两条线基本重合；08:07–08:18 是 L4 与 L7 的交替实验，L4 阶段两条线又会分开（Grafana 截图，看板 storefront-balance）

这满足了本章的验收标准：有网格（L7）时，长连接上的请求均匀分布。

3.6 实验 3：网格的代价，以及如何测准它

3.6.1 第一次测量是错的

最直接的做法是分别跑一次“无网格”和“ambient L4”，比较结果。两次运行相隔 15 分钟[3, 10]：

	无网格（07:21）	ambient L4（07:38）	差
浏览 p50 / p95	10.8 / 19.2 ms	14.8 / 27.7 ms	+4.0 / +8.5 ms
下单 p50 / p95	106.0 / 225.0 ms	138.4 / 253.4 ms	**+32.4** / +28.4 ms

下单路径慢了 32 ms，这比通常所说的“每跳亚毫秒级”大了两个数量级。在下结论之前，先做三件事来检验它。

一、按跳分解。 `experiments/e2/compare-windows.sh` 从 Prometheus 取两个时间窗口内每个服务的服务端耗时、每个依赖调用的客户端耗时和每种 SQL 语句的耗时（[11]，节选）：

```
== server side
  p50  catalog /products/{product_id}                 2.6 ms ->      2.7 ms  (   +0.1)
  p50  orders /orders                                93.4 ms ->    127.2 ms  (  +33.8)
== client side: one attempt of a call to a dependency
  p50  gateway -> catalog                             7.6 ms ->     10.0 ms  (   +2.4)
  p50  orders -> catalog                              8.0 ms ->     12.6 ms  (   +4.5)
  p50  orders -> inventory                           15.2 ms ->     20.3 ms  (   +5.1)
  p50  orders -> payments                            54.0 ms ->     64.5 ms  (  +10.4)
== database: SQL statements as seen by the client
  p50  orders INSERT                                  2.0 ms ->      2.5 ms  (   +0.5)
  p50  inventory UPDATE                               1.0 ms ->      1.0 ms  (   +0.0)
```

catalog 自己处理一个请求的时间没有变（2.6 → 2.7 ms），调用它的一方却多花了 2.4–4.5 ms；SQL 语句几乎不变。这说明多出来的时间在两个应用之间的路径上，而不在数据库。

二、隔离出单跳成本。 `experiments/e2/hop_cost.py` 在一个 Pod 里对同一个目标连续做 2000 次往返（复用一条连接），分别从网格内（storefront 命名空间）和网格外（e2-bench 命名空间）的客户端发起，目标分别是网格内的 catalog Pod 和网格外的 Valkey，组成 2×2 的对照[12, 13]：

p50 往返时间	客户端在网格内	客户端在网格外	差
Valkey `PING`（服务端在网格外，只经过客户端的 ztunnel）	339 / 372 / 356 µs	170 / 252 / 174 µs	约 +0.1–0.2 ms
HTTP `/healthz`（服务端在网格内）	2007 / 1706 / 1623 µs	1572 / 1878 / 1082 µs	−0.2 至 +0.5 ms，噪声量级
每次新建连接再请求	10.9 ms	10.9 ms	0
16 条连接并发	12.0 ms	13.8 ms	服务端（单进程）成为瓶颈

单独测量时，客户端一侧的 ztunnel 每次往返只增加约 0.1–0.2 ms，按下单路径约 30 次往返计算，最多几毫秒，解释不了 32 ms。

三、控制漂移：交替进行 A/B。 两次单独的运行之间，主机的状态可能已经变了（Windows 上的其他进程、温度、后台任务）。`experiments/e2/ab-mesh.sh` 用命名空间标签在两种模式之间来回切换（加入或退出网格不需要重启 Pod），每种模式 90 秒，交替 3 轮。为了不让 Argo CD 把标签改回去，实验期间暂停了两个应用的自动同步，结束时（包括中断时）自动恢复[14]：

轮次	模式	浏览 p50	浏览 p95	下单 p50	下单 p95
1	无网格	12.8	23.0	122.4	249.5

轮次	模式	浏览 p50	浏览 p95	下单 p50	下单 p95
1	ambient	15.5	29.6	139.2	254.9
2	无网格	13.0	22.5	126.2	234.6
2	ambient	15.4	29.9	131.6	258.4
3	无网格	12.4	22.6	124.0	230.1
3	ambient	14.5	24.6	123.4	226.5

实测数据

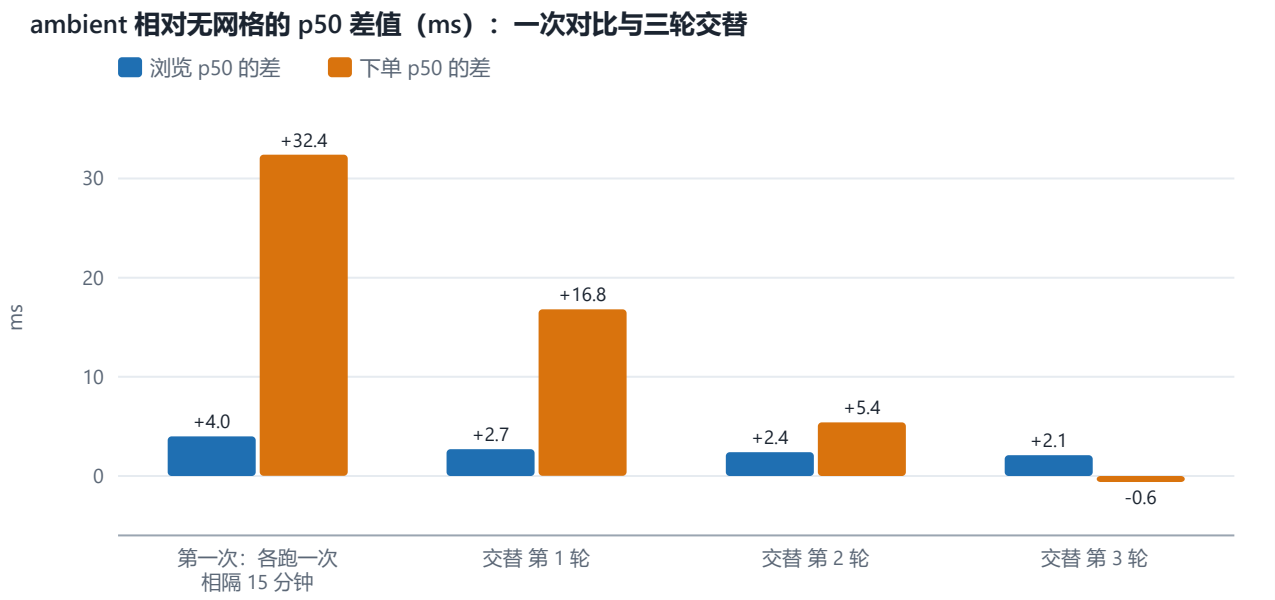


图 3-5 同一个问题的两种测法。第一次把“无网格”和“ambient”各跑一次，相隔 15 分钟：下单 p50 相差 32.4 ms。交替运行三轮之后，浏览的差值稳定在 2.1–2.7 ms，而下单的差值在 −0.6 到 +16.8 ms 之间摆动——它被 payments 人为生成的延迟淹没了，第一次的 +32 ms 大部分是两次运行之间的漂移。

配对的差：浏览 p50 为 +2.7、+2.4、+2.2 ms，三轮一致，平均 **+2.4 ms**；浏览 p95 平均 +5.3 ms。下单 p50 为 +16.9、+5.4、−0.6 ms，三轮之间的差别比差值本身还大：在这个实验的精度下，**无法确定网格是否让下单变慢。**下单延迟主要由 `[模拟]` 支付服务的随机延迟决定，90 秒内约 450 笔订单的中位数本身就有几毫秒的抽样波动。

再看第一次测量：无网格的那次下单 p50 是 106.0 ms，而交替实验中三次无网格的结果是 122–126 ms。主机在这十几分钟里变慢了，第一次测量的“+32 ms”大部分是**漂移**，不是网格。

> **原理**
>
> **性能对比要控制漂移。**两次单独的运行之间，被测系统以外的一切都可能变了。可靠的做法是：交替（A、B、A、B……）而不是先 A 后 B；做多轮，看配对差的一致性，而不是只看平均值；用微基准隔离出单个组件的成本，再用按跳分解检查它能否解释整体的变化。当结论在不同轮次之间不一致时，如实写“在这个精度下无法确定”，比给出一个漂亮的数字更有价值。

3.6.2 waypoint 的代价

同样的交替方法，比较 L4（只有 ztunnel）与 L7（经过 waypoint），切换的是命名空间标签 `istio.io/use-waypoint`（`experiments/e2/ab-waypoint.sh`，[15]）：

轮次	浏览 p50（L4 → L7）	下单 p50（L4 → L7）
1	12.9 → 13.9（+1.0）	117.2 → 124.0（+6.8）
2	13.0 → 14.0（+1.0）	118.0 → 121.3（+3.3）
3	12.6 → 14.4（+1.8）	123.6 → 129.4（+5.8）

waypoint 让浏览 p50 增加约 1.3 ms（一次 Envoy 转发），下单 p50 增加约 5.3 ms，三轮方向一致。下单路径上有 gateway → orders、orders → catalog、→ inventory、→ payments 四到五次经过 waypoint 的调用，每次约 1 ms，量级吻合。p95 的差在 −7.6 到 +35.1 ms 之间，同样无法确定。

资源方面，3 分钟 L7 负载期间 waypoint 用了 91m CPU、35 MiB 内存，ztunnel 每个工作节点 41–62m CPU、6–7 MiB[2]。在约 34 个请求/秒的负载下，整个网格（waypoint 加四个 ztunnel）的 CPU 约 0.25 核，略少于两个最忙的 gateway Pod 之和（约 0.30 核）。

3.7 实验 4：重试在层与层之间叠加

应用有自己的重试（第 2 章），网格也能重试。两层都重试时会发生什么？`experiments/e2/retry-matrix.sh` 让 payments 对每一次扣款都返回错误（`[注入]`，故障开关 `/admin/faults`），以每秒 2 笔的速度下 61 笔订单，在每个 Pod 的 `/metrics` 上读取精确的计数（Prometheus 的 15 秒抓取间隔会让小计数失真），统计每一层的调用次数[16]：

场景	orders 收到的下单（每笔订单）	orders → payments（应用发出的尝试）	payments 实际收到	网格增加的调用
S1：waypoint，payments 返回 500	44（0.72）	25	25	0
S2：waypoint，返回 503	11（0.18）	8	8	0
S3：同 S2，另加 HTTPRoute 重试 3 次	11（0.18）	7	28	**21（4 倍）**

场景	orders 收到的下单（每笔订单）	orders → payments（应用发出的尝试）	payments 实际收到	网格增加的调用
S4：只有 ztunnel（L4），返回 503	11（0.18）	6	6	0

实测数据

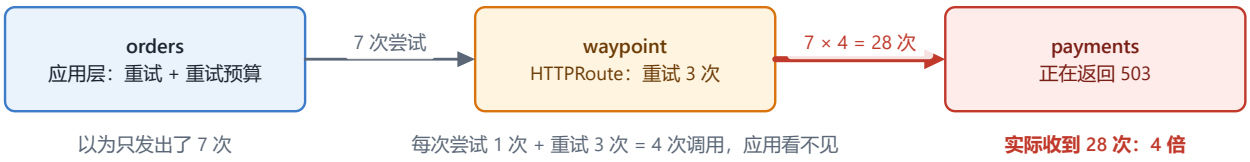


图 3-6 场景 S3 中的重试叠加（数字为实测）：应用认为自己只发出了 7 次尝试，waypoint 上的路由为每一次又重试 3 次，payments 实际收到 28 次。每一层都只看得见自己的重试。

这张表回答了三个问题。

一、应用的熔断器起了主要作用。61 笔订单中，大部分根本没有到达 payments：gateway 到 orders、orders 到 payments 的熔断器在失败率超过 50% 后断开，之后的请求被立即拒绝（`503`、`Retry-After`）。S1 是第一个场景，熔断器从闭合状态开始，放过了较多的请求；S2–S4 开始时熔断器已经断开，每 5 秒只放过少量试探。这也提醒了实验设计中的一个陷阱：**熔断器的状态会从一个场景带到下一个场景**，场景的先后顺序会影响结果。

二、网格的显式重试按倍数放大，而且应用看不见。S3 中，orders 发出了 7 次尝试，payments 却收到了 28 次：每次尝试在 waypoint 上被重试 3 次。orders 的重试预算和熔断器只统计它自己发出的 7 次，那 21 次额外的调用对它完全不可见。在真实的故障中，这意味着应用以为自己“只重试了一点点”，下游却在承受 4 倍的压力。如果两层都配置了重试，放大倍数是相乘的。

三、Istio 默认并不重试 503。我原本以为网格的默认策略会重试 503，S2 的测量结果却是 0 次额外调用。读 Istio 1.31.4 的源码（`pilot/pkg/networking/core/route/retry/retry.go`，[17]）：默认策略是 `NumRetries: 2`、`RetryOn: "connect-failure,refused-stream,unavailable,cancelled,retriable-status-codes"`，但**没有设置任何可重试的状态码**，所以只有连接失败、流被拒绝等“请求还没有到达应用”的情况会被重试。这种默认是安全的：请求没有被处理过，重试不会造成重复。测量纠正了我的假设，代码中的注释也随之改正。

> **原理**
>
> **重试只放在一层，并且放在知道业务语义的那一层。**应用知道哪个调用是幂等的、哪个失败值得重试、什么时候应该降级；网格只看得到状态码。网格适合处理“请求根本没送达”的连接级失败（这正是 Istio 的默认行为），应用处理业务级的重试，并用预算和熔断器限制总量。在网格上为 5xx 配置重试之前，先确认应用没有同时在重试，并把网格的重试也计入下游的容量规划。

3.8 实验 5：开启 STRICT 之后，什么会坏

默认的 PERMISSIVE 模式同时接受明文和 mTLS，所以加入网格不会破坏任何东西。真正的零信任要求 STRICT：只接受 mTLS。`experiments/e2/strict.sh` 在 storefront 命名空间上临时应用 STRICT 的 PeerAuthentication（手工、限时 45 秒，随后删除），分别从三类调用方测试[18]：

调用方	PERMISSIVE	STRICT	删除后
网格内：gateway Pod → orders	200	200	200
网格外：负载均衡器 → gateway	200	空响应（curl 退出码 52）	200
网格外：Prometheus → 各服务的 `/metrics`	全部 `up=1`	全部 `up=0`	全部 `up=1`

服务之间的调用不受影响；但是外部入口和监控都断了，而且是静默的：没有任何告警说“mTLS 拒绝了连接”，只是外部用户访问失败、所有服务的指标同时消失。STRICT 的前提是**所有合法的调用方都在网格里**：入口要换成网格内的入口网关，Prometheus 所在的命名空间也要加入网格（它的身份随后可以写进授权策略）。第 6 章（E5）在多租户与安全治理中完成这套改造。

3.9 一次真实的事故：扣了款，订单却失败了

在 07:40:24 的一次负载中，k6 报告了一个失败的请求（失败率 0.03%）。顺着它查下去，得到了第 2 章在设计时预言过的那个缺口，而这一次它真实地发生了。

从日志到链路。gateway 的访问日志里有一条 `POST /api/orders 200`，耗时 1106.8 ms；同一个请求 ID 在 orders 中是 `503`（1152.1 ms），在 payments 中是 `201`（1243.0 ms）。三个服务给出了三个不同的结果。用 trace ID 取出完整的链路（[19]，节选）：

```
      0.0 ms   1109.8 ms  gateway   SERVER   POST /api/orders
      3.2 ms   1012.1 ms    gateway   CLIENT   POST                      ← gateway 的第一次尝试，1 秒
超时
     16.6 ms   1153.8 ms      orders    SERVER   POST /orders  order.status=FAILED
     39.0 ms     24.4 ms        orders    CLIENT   POST                  ← 预留库存，成功
     66.5 ms   1036.3 ms        orders    CLIENT   POST                  ← 扣款，1 秒超时
     75.0 ms   1244.8 ms          payments  SERVER   POST /charges
    992.3 ms      0.1 ms            payments  INTERNAL POST /charges http receive   ← 917 ms 后才开始
读请求体
   1315.7 ms      1.4 ms            payments  CLIENT   SET           ← 保存扣款结果：approved
   1123.8 ms     17.8 ms        orders    CLIENT   POST                  ← 重试扣款：409（锁仍被持有）
   1148.2 ms     12.3 ms          inventory SERVER   POST /reservations/{order_id}/release   ← 补偿：
释放库存
   1164.0 ms      3.5 ms        orders    CLIENT   UPDATE orders SET status = $2 ...  ← 订单标为 FAIL
ED
   1060.3 ms     44.5 ms    gateway   CLIENT   POST                      ← gateway 的重试（同一个幂等
键）
   1098.9 ms      2.4 ms        orders    CLIENT   SELECT ... FROM orders ...       ← 重放：返回当时的
PENDING 订单
```

真实截图

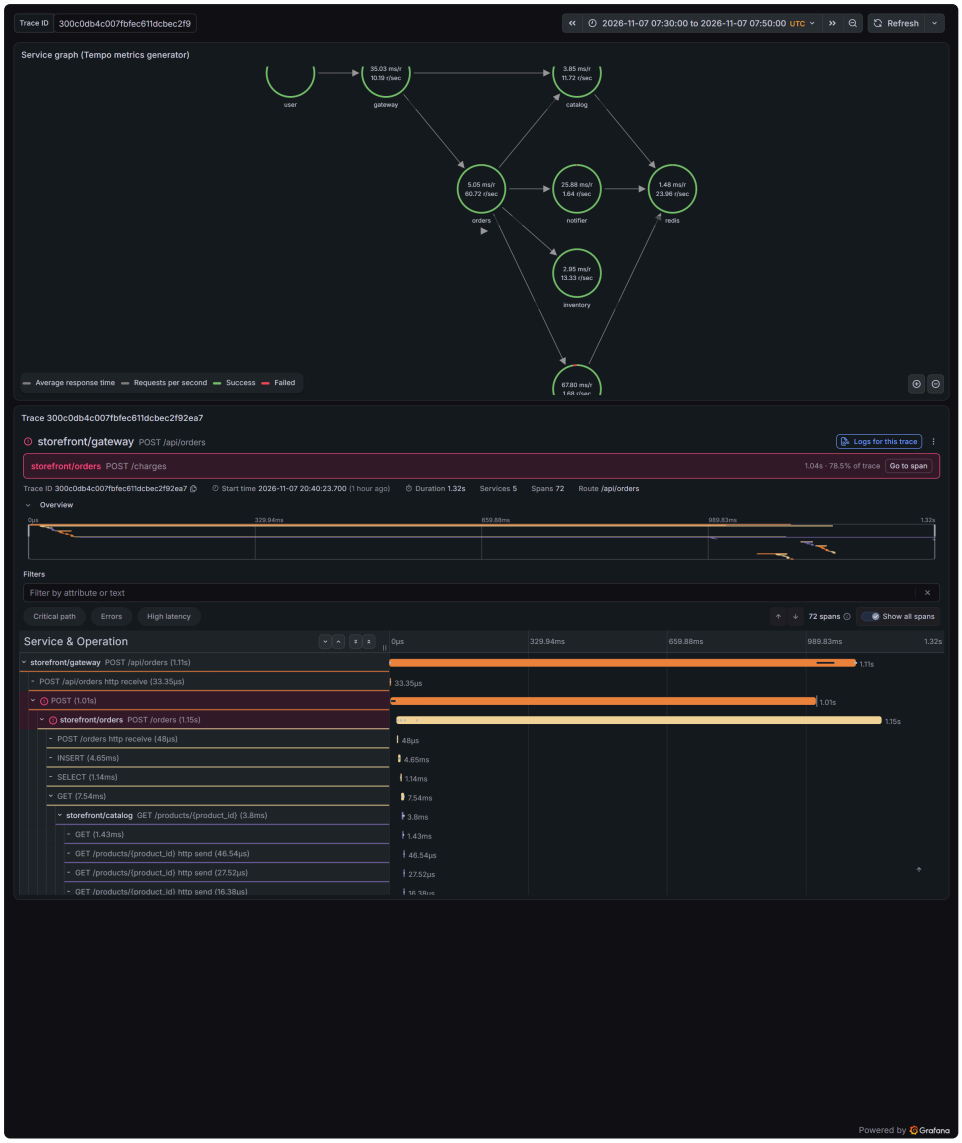


图 3-7 这笔订单的链路：Grafana 把 orders 对 payments 的调用（1.04 s，占整条链路的 78.5%）标为关键路径，gateway 的第一次尝试与 orders 的处理都带有错误标记；概览中段的大片空白就是 payments 的停顿（Grafana 截图）

时间线。payments 收到请求后约 917 ms 没有任何动作（原因见下一节），然后才处理并保存了“扣款成功”。orders 等不了那么久：它对 payments 的单次尝试超时是 1 秒，超时后带着同一个幂等键重试，payments 发现这个键的锁还被第一次请求持有，返回 409；orders 把 409 当作“结果不明”，释放了库存，把订单标为 FAILED。与此同时，gateway 对 orders 的单次尝试也超时了，它的重试被 orders 当作重复请求，“重放”了数据库中当时的状态：PENDING，状态码 200。k6 只接受 201 和 402，所以记了一次失败。

最终状态[20, 21]：订单 `5be7495e` 为 FAILED（`payments HTTP 409`），库存已释放；payments 中却有一笔 5625 分、状态为 approved 的扣款。顾客付了钱，却没有订单。数据库中所有 FAILED 的订单只有这一笔。

为什么各层的保护都“正确”地失败了？每一层的行为单独看都是对的：orders 的超时保护了自己，幂等锁防止了重复扣款，补偿释放了库存，gateway 的重放保证了幂等。问题在于“结果不明”被当成了“失败”。这

是分布式系统中最典型的一类事故：

改进	做法	代价
超时与依赖的延迟分布对齐	调用支付的超时取支付服务 p99.9 以上，或者在超时后先查询结果再决定	慢请求占用更久
结果不明时不做终局判断	409 或超时后，订单保持 PENDING，查询 `GET /charges/{key}`，再决定确认或退款	状态机更复杂，顾客要等待
对账	定期找出“支付成功、订单失败”的组合并修复（第 2 章练习 3）	修复有延迟
支付改为异步	下单只受理，扣款结果通过事件回来	系统设计改变

第 7 章（E6）会有意制造这类故障，并实现第二、三种改进。

3.10 追查：900 毫秒的停顿

事故链条的起点是 payments 在收到请求后“发呆”了 917 ms。这不是一次性的：另外两条慢链路中，第二个 payments Pod 在同一时刻也停顿了约 900 ms（读完请求体之后），07:51:35 的 inventory 也停顿了约 892 ms（读请求体之前）。07:51:35 正处于交替实验的**无网格**轮次，所以停顿与网格无关。

排除过的假设（每一条都有记录）：

假设	检验	结果
网格造成的延迟	07:51:35 的停顿发生在无网格时	排除
Valkey 的 AOF 刷盘阻塞	Valkey 日志中没有 fsync 慢的警告	未发现证据
虚拟机内存不足、换页	`pswpin` 只有 72，内存和 I/O 的压力指标很低，内核没有任务挂起的记录	未发现证据
连接普查的 `kubectl exec` 干扰了应用（观察者效应）	每 20 ms 探测一次 payments 的 `/healthz`，期间 3 次 `kubectl exec` 进入同一个 Pod[22]	排除：探测的最大延迟 23.7 ms

加装仪表。排除法走到尽头，就该让系统自己说话。共享库新增了一个事件循环看门狗（`common/loopwatch.py`）：一个协程每 50 ms 醒来一次，记录晚醒了多久（`event_loop_lag_seconds`）；一个独立线程在循环超过 250 ms 没有心跳时，记下循环线程当时的调用栈，并在停顿结束时报告这段时间内进程用掉的 CPU 和循环线程在运行队列上等待的时间（来自 `/proc/self/task/<tid>/schedstat`）。三个数字区分三种情形：CPU 高是在忙着计算，运行队列等待高是“想跑却没有 CPU”（例如整个虚拟机被宿主机挂起），两者都不高则是在等待 I/O 或锁。GC 回调同时记录每次垃圾回收的时长（`python_gc_pause_seconds`）。

看门狗的第一个版本自己就有缺陷：每个服务启动时都报告了一次“停顿”。心跳的起点设在应用对象创建时，而不是事件循环开始运行时，从导入模块到 uvicorn 启动的这段时间被当成了停顿；CPU 的基线又是 0，报告的是进程启动以来的全部 CPU 时间。修复后增加了回归测试（先等待 0.4 秒再启动应用，断言只报告一次故意制造的停顿），新版本启动时不再误报[23]。

15 分钟的持续负载（20 次迭代/秒，30588 个请求，[24, 25]）：

- 没有任何服务的事件循环停顿超过 250 ms；晚醒超过 100 ms 的情况每个服务只有 0–2 次。
- 但找到了另一个延迟来源：catalog、gateway、orders 各做了 6–10 次完整（第 2 代）的垃圾回收，**平均每次 91–92 ms**，期间事件循环完全停止。这类服务的 p99 延迟中，有一部分就来自这里。第 4 章（E3）会测试 `gc.freeze()` 等调优手段。
- 约 900 ms 的停顿在这段时间内没有再出现。它的原因**至今没有找到**，看门狗留在每个服务中，下一次发生时会记录下调用栈和调度数据。

如实记录“没有找到原因”是这一节的要点之一。排障不总能在一次会话中结束；重要的是排除过什么、为什么排除、留下了什么手段等待下一次。

3.11 局限

- 所有“节点”在同一台机器上，跨节点的流量不经过真实的网络设备，没有真实的网络延迟；网格在真实网络上的相对开销会更小（网络本身更慢），绝对开销（CPU、内存）则相近。
- 负载只有约 34 个请求/秒，远低于 ztunnel 和 waypoint 的设计容量；本章测量的是低负载下每次转发的延迟，不是网格的吞吐上限。
- 只部署了一个 waypoint 副本，它是这个命名空间所有 L7 流量的单点。生产中应部署多个副本并配置 PodDisruptionBudget，或按服务部署 waypoint 以缩小故障范围。
- 外部入口仍是集群外的负载均衡器直连 gateway 服务，没有使用网格内的入口网关；第 6 章（E5）在启用 STRICT 时一起解决。

3.11.1 从实验环境到生产环境

本章的做法	生产环境通常的做法	差别意味着什么
本章只有一个 waypoint 副本（第 7 章改为两个）	waypoint 多副本、跨可用区、带 PodDisruptionBudget	单副本的 L7 代理是整个命名空间的单点：第 7 章的节点失效实验为此付出了 5.5 分钟
负载每秒几十个请求	成千上万的连接与请求	ztunnel 与 waypoint 的容量要单独压测；本章毫秒级的开销不能直接外推
PERMISSIVE，STRICT 只做了短暂的试验	全网格 STRICT，入口与监控抓取纳入网格或显式放行	STRICT 会切断网格外的调用方（实验 5），上线前要先盘点谁在网格外
网格版本固定，一次安装	控制面与数据面按金丝雀方式分批升级	升级过程中的版本偏差，本书没有测试

3.12 常见问题

Q1：加入 ambient 网格真的不需要重启吗？ 是的，本章的记录显示加入前后各 Pod 的启动时间和重启次数都没有变化。但已经建立的连接会被重置：交替实验中每次切换模式，catalog 和 inventory 的就绪探针都会

短暂失败（连接池中经过 ztunnel 的数据库连接被断开），所以实验在每次切换后等待 20 秒。生产中加入或退出网格应安排在低峰期，并确认客户端会重连。

Q2：为什么不直接用 sidecar 模式？ sidecar 模式更成熟，L7 功能对每个 Pod 都可用；代价是每个 Pod 一个 Envoy（内存与 CPU 随 Pod 数线性增长），加入、升级网格都要重启 Pod。ambient 把常驻成本降到每个节点一个 ztunnel（空载 4 MiB），只在需要 L7 的地方付出 waypoint 的成本。本书的资源预算很紧，ambient 更合适；这也是 Istio 社区推荐的新方向。

Q3：怎样在 L4 和 L7 之间选择？ 只需要加密和身份的服务停在 L4；需要按请求负载均衡、流量拆分（第 9 章的金丝雀）、L7 授权或 HTTP 指标的服务才使用 waypoint。本章的数据是：每次经过 waypoint 约 1 ms，换来的是负载从 97/3 回到 50/50。

Q4：网格自带的指标能代替应用的指标吗？ 只能部分代替。waypoint 能给出每个服务的请求量、错误率和延迟（RED），但不知道业务语义（订单是被拒付还是失败）、看不到应用内部（连接池等待、重试预算、熔断器状态）。两者互补：网格提供统一的、与语言无关的基线，应用提供业务与内部状态。

3.13 练习

练习

1. **（计算）** 一个客户端对一个 3 副本的服务保持 3 条长连接，每条连接独立、均匀地落到某个副本上。至少有一个副本没有任何连接的概率是多少？由此说明为什么 L4 负载均衡在连接数与副本数相当时几乎必然失衡。
2. **（分析）** 本章的连接普查为什么必须在服务端 Pod 中进行？如果只能在客户端一侧观察，用什么方法得到“客户端连接到了哪个后端 Pod”？
3. **（实验设计）** 解释交替 A/B 为什么比“先测 A、再测 B”可靠，它仍然没有控制哪些因素？你如何判断一个配对差是否可信？
4. **（排障）** 开启 STRICT 后 Prometheus 抓不到 storefront 的指标。给出两种解决方案，并分析各自的代价。
5. **（设计）** 为 storefront 制定一套重试策略：哪一层重试、对哪些失败重试、如何避免层与层之间的放大。说明你的策略在 payments 完全不可用时，最多会让 payments 收到多少请求。
6. **（复盘）** 针对“扣了款，订单却失败了”的事故，提出至少三项改进，并说明每一项分别消除了事故链条中的哪一环。
7. **（动手）** 设计一种不需要进入应用容器的连接普查方法，说明它的优缺点。

参考答案见附录。

3.14 本章小结

- Istio 1.31.4 的 ambient 模式经 GitOps 安装；版本选择有记录（安全修复优先、CRD 与控制器版本匹配、Chart 仓库已迁移）。空载时 ztunnel 每节点 4 MiB，Chart 默认值则会请求约 4.5 GiB。

- 线上抓包证明了加密：服务间的明文请求行从 140 行变为 0，HBONE 隧道承载了跨节点的流量；来自集群外负载均衡器的入口流量仍是明文，网格只保护网格内部。
- 连接普查证明了失衡的原因：每个客户端只有 1–3 条长连接，L4 负载均衡在连接建立时就决定了一切。ztunnel 不改变这一点；waypoint 让所有服务回到约 50/50，满足本章的验收标准。
- 网格的代价要用交替实验测：ambient L4 让浏览 p50 增加约 2.4 ms，waypoint 再增加约 1.3 ms（下单约 5.3 ms）；第一次单独测量得到的“+32 ms”主要是主机漂移。
- 重试会在层与层之间相乘：网格上的显式重试让下游收到 4 倍的请求，应用的预算和熔断器却看不见；Istio 默认只重试连接级失败，不重试 503。应用的熔断器在 payments 完全失败时把 61 笔订单对它的调用压到 6 次。
- STRICT 会切断所有不在网格中的调用方：外部入口和 Prometheus 同时失效。
- 一次真实的事故（扣款成功、订单失败）完整地重现了“结果不明被当作失败”的缺口；它的起因之一是一次约 900 ms 的进程停顿。停顿的原因没有找到，但加装的看门狗找到了另一个延迟来源：平均 91 ms 的完整垃圾回收。

第 4 章

负载与容量规划（E3）

“这套系统能扛多少流量”是每个平台团队都会被问到的问题，而大多数回答是猜的。本章用测量回答它：先量出“观测本身”要花多少资源，再用阶梯负载找到延迟的拐点和最先饱和的资源，用排队论的三条定律（利特尔定律、利用率定律、响应时间随利用率的变化）检验测量是否自洽，最后建立一个容量模型，让它预测一次还没有做过的实验，再用实验检验预测。所有数字只属于这台笔记本；可以迁移的是方法。

工程问题

“这套系统能扛多少流量”不该靠猜。怎样先用一个模型写下预测，再用实验检验它，并且知道第一个撑不住的是哪个组件、为什么？

实测结论

- 预测先于实验提交：两个副本时网关最先饱和，预测的饱和点是 172 迭代/秒；实测在 180 迭代/秒，最忙的网关 Pod 用到 0.938 核，失败率 13.6%。
- 一个 Python 服务的 Pod 是单线程的事件循环，最多用满 1 个 CPU：容量由“副本数 ÷ 每个请求的 CPU 时间”决定。
- 网关的 CPU 只有 3.7% 花在我们自己的代码上；改掉一个中间件，每个请求的 CPU 少了 25%——这要靠并排对比才测得出来。
- 三个不该相信的数字都出在测量链路上：被 `increase()` 隐藏的 503、负载均衡器 1024 个文件描述符的上限、观测自身的开销。

4.1 学习目标

学完本章，你应当能够：

- 用利特尔定律、利用率定律检验一组性能数据是否自洽，并用它们推算系统内部看不到的量；
- 解释响应时间为什么在利用率接近 100% 时急剧上升，从而理解“拐点”和“余量”；
- 设计开放模型的阶梯负载实验，识别负载生成器自身的瓶颈；
- 找出最先饱和的资源，并理解单进程异步服务（Python 事件循环）的容量上限；
- 量化“观测”本身的成本，为采样率做出有依据的选择；
- 建立一个可验证的容量模型，并把故障域（可用区）纳入余量的计算。

实验性质

本章的实验都是 `[真实]` 运行的：真实的进程、真实的请求与数据。`[模拟]` 的只有两样：三个“可用区”（同一台电脑上三个 Kind 节点的标签），以及外部支付服务 payments（延迟按对数正态分布人为生成，它决定了下单请求的大部分耗时）。本章没有 `[注入]` 任何故障：负载均衡器的崩溃、熔断器的打开和各种瞬态，都是负载本身引起的。

4.2 原理

4.2.1 三条定律

容量规划不需要复杂的数学，但离不开三条朴素的关系。设一个服务在一段时间内的吞吐量为 X（每秒完成的请求数），平均响应时间为 R，平均同时在处理的请求数为 N：

利特尔定律（Little’s Law）：

$$N = X \cdot R$$

它对任何稳定的系统都成立，不需要假设到达的分布。它的用途在于“三个量知道两个，就能算出第三个”，也可以用来检验测量：如果网关显示每秒 34 个请求、平均 15 ms，那么任一时刻平均只有约 0.5 个请求在处理中；如果你测到的同时在途请求是 5，说明某个数字测错了。

利用率定律（Utilization Law）：

$$U = X \cdot D$$

D 是**服务需求**，即处理一个请求所消耗的资源时间（例如 CPU 毫秒）。一个单线程的进程，每个请求要 5 ms CPU，那么它每秒最多处理 $1/D = 200$ 个请求；在每秒 100 个请求时利用率是 50%。D 可以直接从监控中算出：一段时间内的 CPU 用量除以请求数。

响应时间随利用率增长：最简单的排队模型（M/M/1，一个服务者、随机到达）给出

$$R = \frac{S}{1 - U}$$

其中 S 是不排队时的服务时间。利用率 50% 时响应时间是 $2S$，80% 时 $5S$，90% 时 $10S$，95% 时 $20S$。真实系统的数字不会恰好如此，但形状是一样的：**在利用率接近 100% 之前，延迟就已经开始急剧上升。**这就是“拐点”，也是为什么容量规划总是给资源留出余量，而不是让它跑到 100%。

模型曲线

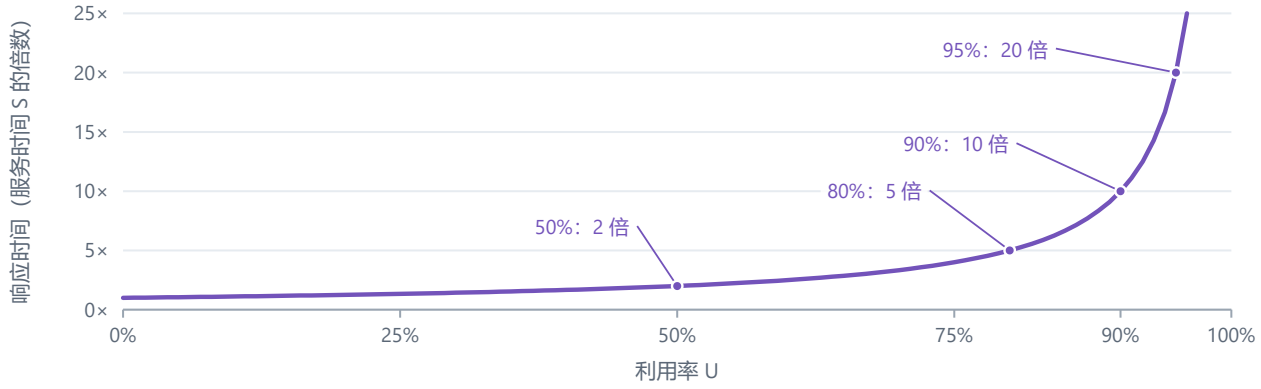


图 4-1 响应时间律画成曲线（这是公式，不是测量）。利用率从 50% 到 80%，响应时间从服务时间的 2 倍变为 5 倍；越过 90% 之后，每多一点负载都换来成倍的等待。容量规划谈的“拐点”和“余量”，说的就是不要让任何一个组件长期停在曲线的右端。

4.2.2 Python 服务的特殊之处

storefront 的每个服务是一个 uvicorn 进程，所有请求都在一个事件循环线程上处理。异步让一个进程可以同时等待几百个网络调用，但**计算**只能在这一个线程上串行进行，加上全局解释器锁（GIL），一个 Pod 最多只能用满约 1 个 CPU 核。所以一个服务的容量上限近似为

$$X_{\max} \approx \frac{\text{副本数}}{D_{\text{CPU}}}$$

扩容的办法是增加副本（或每个 Pod 运行多个工作进程），而不是给 Pod 更多的 CPU 配额。第 1 章把每个节点做成了 4 核，一个节点上最多也就同时用满 4 个这样的进程。

4.2.3 开放模型与协调遗漏

负载生成有两种模型。**封闭模型**是固定数量的虚拟用户，每个用户等上一个请求完成后再发下一个：系统变慢时，请求的发送速度也随之下降，过载被“自动”掩盖了，这叫**协调遗漏**（coordinated omission）。**开放模型**按固定速率发送请求，不管系统是否跟得上，这才像真实世界中互不相识的用户。本书始终使用开放模型（k6 的 `constant-arrival-rate`）。当系统跟不上时，k6 需要越来越多的虚拟用户来维持速率；虚拟用户达到上限（`MAX_VUS`）后，它会丢弃迭代（`dropped_iterations`），这本身就是过载的信号。

4.2.4 观测不是免费的

链路追踪、指标和日志都要消耗 CPU、内存和网络：在应用里创建与序列化 span，在采集器里批处理，在后端里存储和索引。在低负载时这些开销不显眼；在容量规划中，它们和业务处理一样要计入 D。本章第一个实验就是测量它。

4.3 准备：一个可以信任的测量工具

`experiments/e3/window_stats.py` 对任意一个时间窗口，从 Prometheus 中算出每个服务的请求速率、CPU 核数、**每个请求的 CPU 毫秒数**（即利用率定律中的 D）、最忙的 Pod 的 CPU、平均在途请求数（利特尔定律中的 N）、事件循环延迟的 p99 和连接池等待的 p95，以及每个节点和平台组件（waypoint、ztunnel、采集器、Tempo、PostgreSQL、Prometheus）的 CPU。

第一次使用它就发现了一个测量缺陷：通知服务几乎没有业务请求，在途请求数却稳定在 1.00。原因是计数器 `http_server_inflight_requests` 把 `/metrics` 也算了进去，而读取这个计数器的恰恰是一次正在进行的 `/metrics` 请求，所以它永远不会低于 1。用这样的数据检验利特尔定律，会得出完全错误的结论。修正后，运维接口不再计入在途请求，并增加了测试断言[3]。另一个限制是 kube-prometheus-stack 不保留 cAdvisor 中节点根 cgroup 的序列（`id="/"`），节点的 CPU 用该节点上所有容器 CPU 之和代替，不包括 kubelet 和 containerd 自身。

4.4 实验 1：观测的成本

在同样的负载（20 次迭代/秒，3 分钟）下，依次把采样率设为 1.0、0.1、1.0（A-B-A 设计：两次 A 之间的差异反映了漂移），每次都经过 Git 发布（`experiments/e3/tracing-cost.sh`，[4]）。每个请求的 CPU 毫秒数：

服务	采样 1.0（第一次）	采样 0.1	采样 1.0（第二次）
gateway	10.37	7.60	8.87
catalog	6.34	4.27	5.00
orders	44.99	31.92	35.88
inventory	18.73	12.23	13.48
payments	16.06	12.00	12.82

把采样率降到 10%，各服务每个请求的 CPU 下降约 6%–25%。两次 1.0 之间本身就相差 10%–25%，所以这个实验只能给出一个范围，不能给出精确的百分比。可以确定的是：在 Python 中，**完整的链路追踪约占业务处理 CPU 的十分之一到四分之一**，这是采样存在的理由。此后的负载实验都使用 10% 的头部采样（在入口决定，下游跟随父级的决定），第 10 章（E9）再讨论尾部采样。

真正的意外在平台组件里：

组件	采样 1.0	采样 0.1	采样 1.0
Tempo	1.472 核	1.201 核	1.468 核
六个服务合计	约 1.07 核	约 0.77 核	约 0.87 核

链路追踪的后端比被追踪的六个服务加起来还要“贵”，而且几乎不随采样率变化。存储十分之一的 span 不应该还要 1.2 个核。

4.5 实验 2：一个比业务还贵的配置错误

Tempo 是 Go 程序，先看它的运行时指标[5]：

```
go_gc_gomemlimit_bytes 8.388608e+08          ← 软内存上限 800 MiB（第 2 章按实测设置的）
go_memstats_heap_alloc_bytes 1.14314336e+09  ← Go 统计的堆：1.14 GB
go_memstats_next_gc_bytes 1.143137904e+09    ← 下一次 GC 的目标：就是现在
process_resident_memory_bytes 1.31301376e+08 ← 实际驻留内存：131 MB
go_gc_duration_seconds_count 131650 → 132196 ← 30 秒内 546 次 GC
```

堆统计为 1.14 GB，超过了 800 MiB 的软上限，GC 一刻不停地试图把堆压到上限以下：每秒 18 次。可是实际驻留内存只有 131 MB，堆里的大部分东西根本没有占用物理内存。

第一个修正（错误的）。既然软上限带来了 GC 风暴，就去掉它。结果更糟：Tempo 启动后十几秒就被内核以 OOM 杀死（几分钟内重启了 5 次），日志停在 `completing block`（把预写日志转换为存储块），而且每次

重启都重放同一份预写日志、在同一个地方被杀，陷入崩溃循环[6]。没有软上限时，GC 的目标是“当前堆的两倍”，真实使用的内存可以一直涨到 1 GiB 的硬上限。

第二个修正（没有解决问题）。把块的大小从默认的 500 MB 降到 100 MiB、时长从 30 分钟降到 5 分钟，恢复 900 MiB 的软上限。Tempo 不再崩溃，但堆仍是 1.105 GB，GC 每秒 21 次，CPU 0.84 核。块的大小不是原因。

找到原因：看堆里装的是什么。Go 程序可以通过 pprof 输出堆的剖析数据。按“在用字节数”排序（原始剖析数据 [7]，统计结果 [8]）：

```
heap profile: 63: 1078081808 [6530: 2081847848] @ heap/1048576
  1073.7 MB  main.main
     1.0 MB  ...tsdb/agent.newStripeSeries <- ...generator/storage.New
```

1073.7 MB 是在 `main.main` 中一次分配的。这是**内存压舱物**（memory ballast）：Tempo 的 Helm Chart 默认以 `-mem-ballast-size-mbs=1024` 启动，分配一块 1 GiB、从不使用的字节数组。在 Go 1.19 引入软内存上限之前，这是降低 GC 频率的常用技巧：GC 的目标是堆的两倍，一块很大的“死重”让目标变高，GC 就少了；它从不被写入，所以不占物理内存。压舱物与软上限不能同时使用：压舱物本身就超过了软上限，GC 永远达不到目标；去掉软上限，GC 的目标又变成 2 GiB 以上，真实内存在到达目标前就撞上了 1 GiB 的容器上限。

最终修正：关闭压舱物（`memBallastSizeMbs: 0`），软上限设为容器上限的 90%（900 MiB）。结果：

	压舱物 + 800 MiB	修正后
Go 堆	1.14 GB	67.5 MB
30 秒内 GC 次数	546	1
CPU（无负载时）	1.34 核	0.027 核
重启	—	0

一个配置错误占了工作节点 12 个核中的 1.3 个，比整个业务系统还多。几点值得记住：

- 它是两个各自“正确”的设置叠加出来的：Chart 的默认压舱物（在没有软上限的年代是好做法），和我在第 2 章按实测加上的软上限（在没有压舱物时是好做法）。**组合起来的行为要单独验证。**
- 它不随负载变化，所以在功能测试中看不出来，只有在按组件统计 CPU 时才显形。
- 排查的顺序是：资源统计（谁贵）→ 运行时指标（GC 频率、堆与驻留内存的差距）→ 剖析数据（堆里是什么）。在剖析数据出来之前的两次“修正”都是猜测，其中一次让情况变得更糟。
- 同一组参数中还有 `reportingEnabled: true`：Tempo 默认向 Grafana Labs 发送匿名的使用统计。本实验环境不向外部发送数据，一并关闭。第 6 章（E5）会系统地检查各组件的“回传”设置。

4.6 实验 3：第一次阶梯负载，以及三个不该相信的数字

`experiments/e3/step-load.sh` 依次以每秒 20、60、100……300 次迭代各运行 2 分钟（开放模型），每一档启动一个新的 k6 进程，统计窗口跳过每档的前 30 秒。采样率 10%，storefront 版本 f3d9043[9]：

档位（迭代/秒）	请求/秒	失败率	浏览 p50	浏览 p95	网关 CPU（两个 Pod）	最忙的网关 Pod	事件循环延迟 p99
20	34.7	0.12%	19.2 ms	41.9 ms	0.397 核	0.224 核	6.8 ms
60	103.9	5.35%	30.8 ms	250 ms	1.192 核	0.653 核	47 ms
100	166.7	65.9%	451 ms	5.3 s	1.857 核	0.938 核	403 ms
140	238.8	57.8%	286 ms	3.5 s	1.892 核	0.948 核	239 ms
180	298.4	100%	—	2.1 s	0.291 核	0.149 核	66 ms
220–300	372–441	100%	—	1.3–1.7 s	0.25–0.31 核	0.12–0.16 核	66–77 ms

从 100 迭代/秒起，最忙的网关 Pod 停在 0.94–0.95 核：一个 Python 进程用不了第二个核，请求在事件循环里排队，循环延迟 p99 达到 403 ms。拐点在 60 与 100 之间，最先饱和的是网关。这一结论后来被验证了，但表中另有三处数字经不起追问：60 迭代/秒时网关只用了 0.65 核，却有 5% 的请求失败；180 迭代/秒以上几乎全部失败，网关的 CPU 却降到了 0.3 核；同一时段，几个与负载无关的服务“不就绪”了。三处异常分属三种不同的机制。

4.6.1 异常一：Prometheus 看不见的 503

60 迭代/秒这一档，k6 记录 226 次商品列表和 284 次商品详情失败；网关的指标中，这两个路由在整个窗口内没有一个 5xx。日志给出了时间线[10]：这一档开始后的第 1–3 秒，catalog 读 Valkey 缓存连续超时（它把超过 200 ms 的缓存读当作失败，改从数据库读）；第 3 秒，两个网关 Pod 对 catalog 的熔断器同时打开，5 秒后关闭。这 5 秒内网关直接返回 503。

503 被计数了，只是 Prometheus 算不出来。prometheus_client 在某个标签组合第一次出现时才创建那条时间序列，所以 `code="503"` 的序列诞生时已经是 118（另一个 Pod 是 106），此后再没变过。`increase()` 只能看到序列存在之后的变化，从“没有序列”到 118 的那一跳它看不到，于是结果是 0：

```
qm2pv /api/products 503: first seen 09:32:15 = 118, at the end = 118
fdhx7 /api/products 503: first seen 09:32:30 = 106, at the end = 106
increase over the window: /api/products 503 = 0
```

118 + 106 = 224，k6 记录 226；商品详情 144 + 138 = 282，k6 记录 284。一次只在开头爆发的故障，恰好在它自己创建的序列里隐身了。对于需要告警的组合（路由 × 错误码），应在启动时预先创建并置零；Prometheus 也可以借助客户端暴露的“创建时间”在序列开头补一个零（`created-timestamp-zero-ingestion` 特性，本实验没有启用）。另一个教训是：服务端的指标不是唯一的真相，k6 在客户端一侧的失败计数才是用户看到的。

为什么每一档的开头会出现这样的爆发？每一档都是一个新的 k6 进程，120 个虚拟用户同时建立新连接，沿途所有连接池在同一时刻扩容，只给 200 ms 的缓存读首先失败。确切的机制本章没有找到（实验 3 的异常三中排除了一种猜测）。方法上的修正是：之后的阶梯负载在一个 k6 进程里完成，先用 30 秒爬升，档与档之间用 10 秒过渡，连接和连接池保持温热（`experiments/e3/step-ramp.sh`）。熔断器对这种短暂爆发的敏感度，第 7 章（E6）再讨论。

4.6.2 异常二：负载均衡器自己倒下了

180 迭代/秒这一档有 35 220 次网络层失败：23 578 次“connection refused”，9 465 次 EOF，2 106 次“connection reset”；网关只收到每秒 85 个请求。客户端与网关之间只有一个组件：cloud-provider-kind 为 LoadBalancer 服务创建的 Envoy 容器（Envoy 1.33.2，TCP 代理）。它的日志[11]：

```
[09:38:38.275][error][envoy_bug] socket(2) failed, got error: Too many open files
[09:38:38.287][error][envoy_bug] envoy bug failure: false. Details: Client socket failure
[09:38:38.289][critical][backtrace] Caught Segmentation fault, suspect faulting address 0x0
[09:38:40.290][info][main] starting main dispatch loop
```

容器以 Docker 默认的 1024 个文件描述符（软限制）启动 Envoy；每个被代理的连接要占用两个（客户端一侧和上游一侧），约 500 个并发连接时 `socket(2)` 失败，Envoy 随后崩溃，容器里的循环 1–2 秒后重启它，所有连接中断，客户端同时重连，几秒后再次崩溃：8 分钟里崩溃了 106 次。崩溃之间的空档就是那 2 万多次“connection refused”。

用同样的负载可以稳定复现（[12]：60 秒内崩溃 11 次）。`platform/cluster/lb-limits.sh` 通过宿主机上的 PID 用 `prlimit` 把软限制提高到 65 536（同时作用于正在运行的 Envoy 和负责重启它的 bash 循环），此后所有负载脚本都先执行它。修正后重做[13]：崩溃 0 次，网络层失败从 17 114 次降到 1 次，剩下的失败都是网关过载返回的 HTTP 错误，这才是被测系统本身的行为。

压测工具链是实验的一部分。把一个数字当作“系统的极限”之前，要检查路径上的每一个组件，包括负载生成器和负载均衡器。云上的负载均衡器没有这个特定的限制，但每一个代理都有某种上限（连接数、文件描述符、内存），要知道它们在哪里。

4.6.3 异常三：与负载无关的 Pod 也“不就绪”了

负载均衡器反复崩溃期间，ent-worker 和 ent-worker2 上所有依赖数据库的 Pod 都变成了 NotReady：catalog 的两个副本、orders 的两个副本、inventory 和 payments 各一个；ent-worker3 上的 Pod 一直就绪[14]。探针失败是因为 `/readyz` 在 1 秒的期限内没有返回。

为了找出慢在哪一跳，复现时在每个节点上同时测量[15, 16]：

测量	ent-worker / ent-worker2	ent-worker3
节点 → Pod 的 `/healthz`（不访问依赖）	2–7 ms	2–6 ms
节点 → Pod 的 `/readyz`（连接池 + `SELECT 1`）	约 950 ms 或 3 s 超时	5–56 ms
Pod 内部执行 `SELECT 1`（经过本节点的 ztunnel）	0.9–4.8 s，部分失败	2–7 ms

测量	ent-worker / ent-worker2	ent-worker3
节点自己的网络命名空间 → PostgreSQL（不经过 ztunnel）	9–21 ms	—

进程本身是健康的（`/healthz` 很快），网络也是健康的（同一台节点不经过 ztunnel 直接连数据库一直很快，网卡丢包计数为 0，catalog Pod 在整个过程中只重传了 12 个报文段）。慢的是经过本节点 ztunnel 的路径。PostgreSQL 主库和 Valkey 都在 ent-worker3 上，那里的 Pod 访问它们不需要跨节点。受影响的两个节点正是运行网关 Pod 的节点：每次 Envoy 重启，都有数百个新连接同时涌向网关，而 ztunnel 为每个 Pod 打开的监听套接字只有 128 的积压队列，网关 Pod 的网络命名空间里记录了 257 次监听队列溢出（`Listen Overflows`）。

一个自然的猜测是“同时建立大量新连接时 ztunnel 会变慢”。在没有其他负载时，从同一节点上分别位于网格内外的两个 Pod 同时向 Valkey 建立 200 个连接[17]，经过 ztunnel 的 p95 是 64–93 ms，不经过的是 114–123 ms：猜测被否定了。ztunnel 在这段时间没有记录任何相关的错误，它内部究竟卡在哪里，本章没有找到答案。可以确定的结论是：

- **节点级代理是同一节点上所有 Pod 的共同命运。**流向网关的连接风暴，拖慢了同一节点上 catalog 访问数据库的连接；两者在应用层面毫无关系。
- **就绪检查包含共享依赖时，所有副本会一起消失。**catalog 的两个副本恰好都在受影响的节点上（拓扑分布约束是软约束 `ScheduleAnyway`），数据库检查一失败，catalog 整个从服务中摘除，网关收到的是“no healthy upstream”。第 5 章（E4）重新审视副本分布，第 7 章（E6）讨论就绪检查应当检查什么。

4.7 实验 4：CPU 花在哪里——剖析、优化与金丝雀对比

网关是第一个饱和的服务，要提高容量，要么增加副本，要么降低每个请求的 CPU 毫秒数 D。先看 D 花在哪里。

4.7.1 剖析工具本身的陷阱

py-spy 是 Python 的采样剖析器，它从外部读取进程内存中的调用栈，不需要修改代码或重启。`experiments/e3/profile.sh` 在承载 Pod 的 Kind 节点容器中运行它（节点容器是特权容器，看得见 Pod 的进程），Pod 本身不需要任何额外权限。

第一次剖析的结果[18]说网关在 89% 的时间里“活跃”，而这个进程的主线程实际只用了 0.43 核。py-spy 判断线程是否空闲，依据的是操作系统的线程状态；可它在节点的 PID 命名空间里运行，解释器报告的却是 Pod 命名空间中的线程号：`py-spy dump` 显示 `Thread 1 (active): "MainThread"`，而节点上这个线程的编号是 143005（[19]，记录于另一个网关 Pod）。两者对不上，空闲的事件循环被当成了繁忙。改用 `--gil`，只保留某个线程持有 GIL 的样本：uvloop 在等待 I/O 时会释放 GIL，空闲的网关在 5 秒内只有 5 个样本，默认模式是 501 个。新的脚本同时从 `/proc` 读出每个线程在同一窗口内的 CPU 时间，两种数据可以互相核对。

4.7.2 结果：成本分散在框架里

60 迭代/秒时，网关主线程用 0.428 核，持有 GIL 的时间占 25%；其余的 CPU 花在释放了 GIL 的 C 代码里（uvloop、系统调用）。按组件划分持有 GIL 的样本（[20]，非阻塞模式下有四分之一到三分之一的读取失败，比例只是近似）：

组件	网关	catalog	orders
HTTP 客户端（httpx、httpcore、h11）	32.2%	—	21.8%
Web 框架（FastAPI、Starlette、Pydantic）	19.1%	28.5%	18.3%
OpenTelemetry 插桩 + SDK	16.7%	22.6%	13.4%
HTTP 服务器（uvicorn）+ 事件循环的 C 代码	13.7%	22.1%	13.0%
数据库驱动 / 缓存客户端	—	10.0%	14.3%
我们自己的代码	3.7%	2.5%	5.3%
失败的 import	3.4%	0	1.7%

实测数据

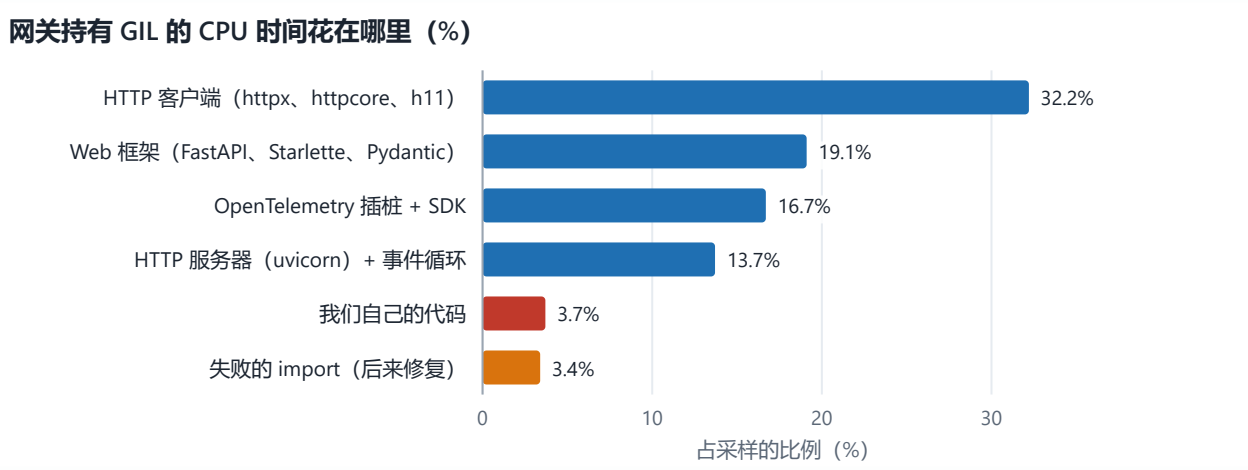


图 4-2 网关的 CPU 构成（表中“网关”一列）。业务代码只占 3.7%：一个只做转发的网关，成本几乎全在 HTTP 客户端、框架和插桩里。想让它更便宜，要改的是这些层的用法，而不是业务逻辑。

没有一个突出的热点，成本散布在几十个文件里；业务代码本身只占 3%–5%，围绕它的框架、客户端和插桩才是大头。其中有两项可以几乎零风险地去掉：

- **`@app.middleware("http")`。**Starlette 用 `BaseHTTPMiddleware` 实现这种中间件，它把下游应用放进一个额外的任务里运行，再通过内存流来回传递消息。仅 `starlette/middleware/base.py` 就占网关样本的 5.6%。改写为纯 ASGI 中间件，行为不变；顺带修正了一个缺口：未处理的异常以前不计数，现在记为 500（新增一个回归测试，30 个测试通过，[21]）。
- **每个请求都失败一次的 `import sniffio`。**httpcore 每次创建锁或事件时都尝试 `import sniffio`；anyio 4.x 不再依赖它，镜像中没有这个包，而失败的 import 不会被缓存，每次都要沿 `sys.path` 重新搜

索文件系统。依赖中加入 `sniffio==1.3.1`。

（栈中还能看到 FastAPI 0.162 自带的遥测包装层 `fastapi/telemetry`。它检测到应用已经接入 OpenTelemetry 插桩，只做透传，没有重复创建 span。）

4.7.3 效果要并排测

最直接的做法是先测旧版本、再测新版本。可是这台机器的漂移太大：同一个旧版本，在第一次阶梯负载中每个请求 11.3 ms CPU，三小时后测是 7.4 ms，差别比预期的效果还大。`experiments/e3/ab-canary.sh` 换了一种做法：复制网关的 Deployment，用新镜像起一个副本（金丝雀），标签相同，所以 Service 把流量同时分给两个旧 Pod 和一个新 Pod。三个 Pod 在同一时间、承受同一股流量，漂移对它们的影响相同；两个旧 Pod 之间的差异就是这个方法本身的噪声（A/A 对照）。

	旧 Pod 1	旧 Pod 2	新 Pod
第一次（新 Pod 在另一个节点上）	7.42 ms/请求	7.39 ms/请求	5.53 ms/请求
第二次（新 Pod 与一个旧 Pod 同节点）	7.42 ms/请求	7.26 ms/请求	5.48 ms/请求

两次的 A/A 差异都在 2% 以内，新版本每个请求的 CPU 少了约 25%，服务端的 p50 延迟低 1–2 ms[22, 23]。新版本随后经 GitOps 发布到全部六个服务（storefront fd38028）。

节省的 25% 比剖析数据暗示的（5.6% + 3.4%）多得多。剖析器只能把时间记在 Python 栈帧上，而 `BaseHTTPMiddleware` 为每个请求多创建的那个任务，以及内存流的每一次唤醒，消耗的是事件循环自身的 C 代码，甚至是在释放 GIL 之后，Python 剖析器很难归因。**剖析告诉你该看哪里，只有 A/B 对比能告诉你一个改动省了多少。**

4.8 实验 5：利特尔定律检验的是测量

`window_stats.py` 给出每个服务的平均在途请求数 N（Prometheus 每 15 秒读一次在途计数器），也能由吞吐量和平均响应时间算出 $X \cdot R$。40 迭代/秒时两者对不上[24]：

服务	计数器的平均值	$X \cdot R$
catalog	0.17	0.15
网关	0.76	1.70
orders	0.29	1.09
payments	0.33	0.63

对稳定的系统，利特尔定律不会错，所以错的是测量。`experiments/e3/little_check.py` 在 Pod 内部每秒读 20 次服务自己的 `/metrics`，用同一份数据算出 N 和 $X \cdot R$[25]：

服务	每秒 20 次采样的平均值	$X \cdot R$	如果每 15 秒读一次，可能读到
网关	0.788	0.747	0.00–1.17
orders	0.600	0.631	0.17–1.00
payments	0.326	0.332	0.17–0.67

采样足够密时，两者相差不到 5%，定律成立。在途请求数每隔几毫秒就变化一次，每 15 秒读一次，一个 90 秒的窗口里每个 Pod 只有 6–7 个读数，取决于读数落在什么相位，同样的流量可以读出 0 到 1.17 之间的任何值。catalog 是一个例外：它的请求只有 2.4 ms，采样请求和业务请求共用一个事件循环，采样的时刻并不独立于被测的请求（读到 0.088，$X \cdot R$ 是 0.035）。

结论是：对变化很快的量，平均值要用时间积分来算，而不是对瞬时值抽样。对已完成的请求，`rate(http_server_request_duration_seconds_sum)` 恰好就是 $X \cdot R$，即平均在途请求数；在途计数器适合发现“卡住不动的请求”，不适合求平均。

4.9 实验 6：容量模型——先预测，再验证

4.9.1 模型

每个服务由若干个单线程进程组成，其中最忙的那个先饱和。设每次迭代对服务 s 发出 k_s 个请求，每个请求消耗 D_s 毫秒 CPU，最忙的副本承担该服务 CPU 的比例为 b_s（负载均衡从来不是完全均匀的），那么在每秒 X 次迭代时，最忙副本的 CPU 是

$$C_s = b_s \cdot k_s \cdot D_s \cdot X$$

它达到 0.95 核（实验 3 测得的单进程上限）时饱和，即 $X_{\text{sat}} = 0.95/(b_s k_s D_s)$。全部工作节点的 CPU 用两个低负载窗口拟合成一条直线：固定部分加上每次迭代的边际 CPU。`experiments/e3/capacity_model.py` 用优化之后、负载为 20 和 40 迭代/秒时的数据[26]得到[27]：

服务	k（请求/迭代）	D（ms）	b	最忙副本每次迭代的 CPU	预测的饱和点
gateway	1.61	5.95	0.58	5.53 ms	172 迭代/秒
orders	0.28	23.37	0.50	3.31 ms	287 迭代/秒
catalog	1.69	3.10	0.52	2.75 ms	345 迭代/秒
inventory	0.23	8.71	0.51	1.03 ms	927 迭代/秒
payments	0.25	7.88	0.51	1.00 ms	952 迭代/秒

工作节点合计：1.01 核，加每次迭代 32.0 ms。

在跑更高的档位之前，四条可检验的预测连同它们的容差先写进了 [1] 并提交到 Git（提交 98be3f2）。先写下预测再做实验，是为了防止事后“解释”数据：任何模型都能事后拟合，只有事先的预测能被证伪。

4.9.2 验证

第二阶段在一次 k6 运行中从 60 升到 200 迭代/秒[28]：

档位	浏览 p50	浏览 p95	失败率	最忙网关 Pod：实测 / 预测	最忙 orders Pod	工作节点合计：实测 / 预测
60	13.2 ms	51.0 ms	0.11%	0.289 / 0.33 核（−12%）	0.190 核	3.19 / 2.93 核
80	11.5 ms	26.2 ms	5.84%	0.366 / 0.44 核（−17%）	0.231 核	3.68 / 3.57 核
100	13.5 ms	29.8 ms	0	0.480 / 0.55 核（−13%）	0.301 核	4.56 / 4.21 核
120	14.9 ms	39.5 ms	0	0.607 / 0.66 核（−8%）	0.358 核	5.39 / 4.85 核
140	14.8 ms	36.1 ms	0	0.583 / 0.77 核（−24%）	0.391 核	5.64 / 5.49 核
160	20.9 ms	81.4 ms	0	0.801 / 0.89 核（−10%）	0.452 核	6.37 / 6.13 核
180	130 ms	1.18 s	13.6%	0.938 / 0.95 核	0.527 核	7.14 / 6.77 核
200	221 ms	1.66 s	33.2%	0.533 核（熔断）	0.352 核	4.36 核

实测数据

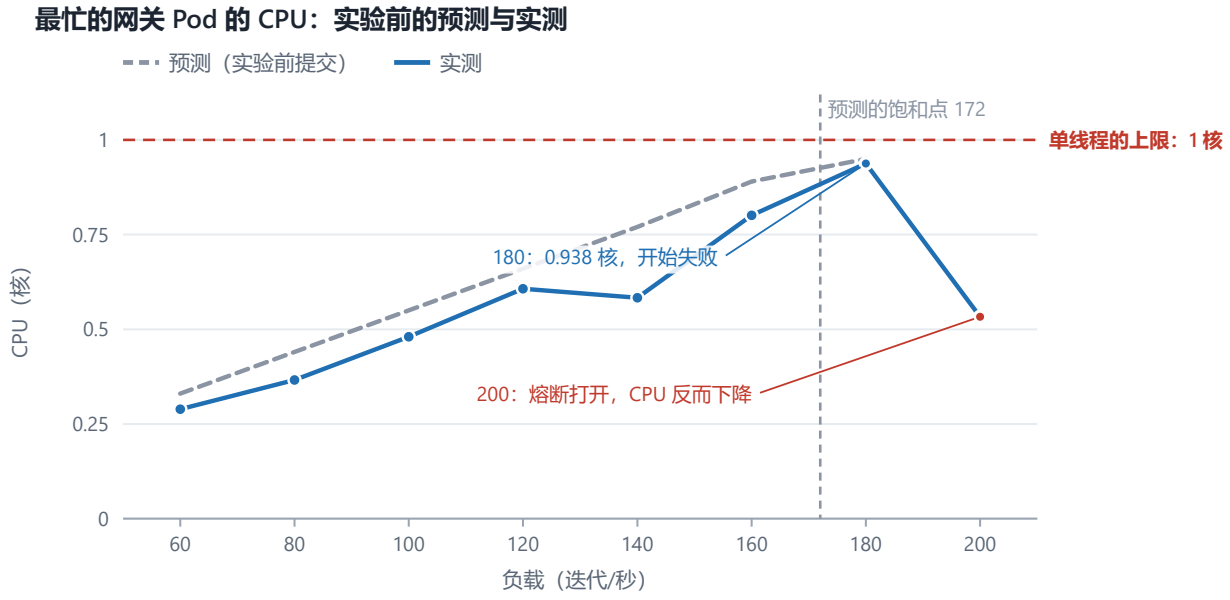


图 4-3 预测与实测。虚线是实验之前提交的预测，实线是阶梯负载中实测的 CPU：饱和之前实测比预测低 8%–24%，而饱和发生在预测的范围之内——180 迭代/秒时最忙的网关 Pod 用到 0.938 核，贴近单线程进程 1 核的上限。200 迭代/秒时 CPU 反而下降，因为熔断器打开后大部分请求被快速拒绝。

实测数据

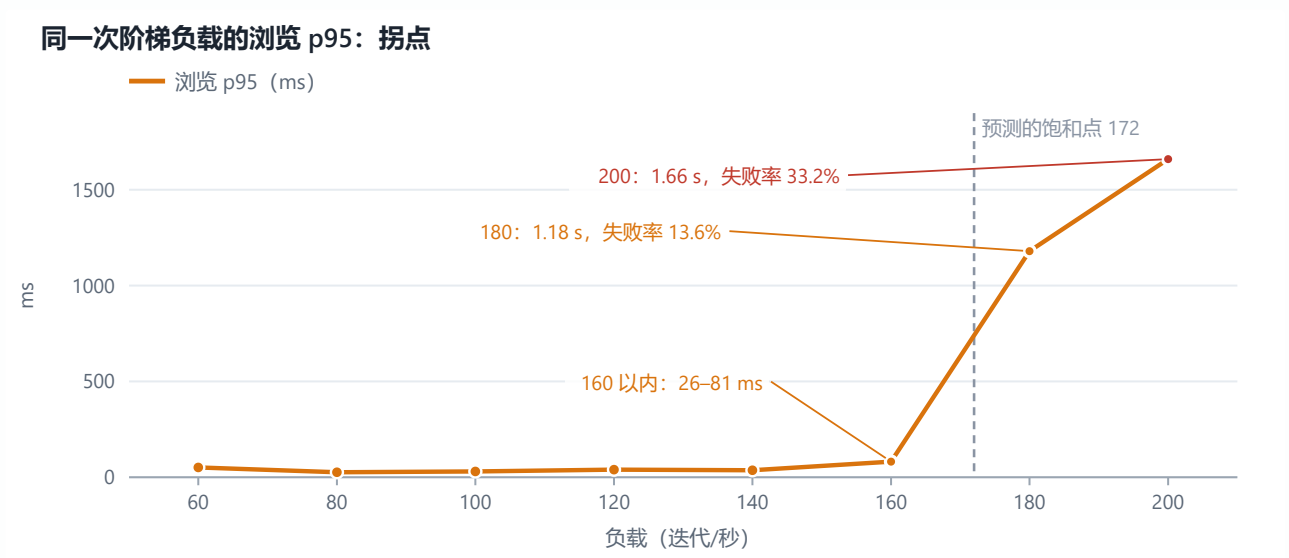


图 4-4 同一次运行的延迟。160 迭代/秒以内，浏览的 p95 在 26–81 ms 之间；越过饱和点，它在一个档位之内变成 1.18 秒。这就是上面那条模型曲线在真实系统里的样子：拐点之前几乎没有征兆。

逐条对照：

- **P1（最忙网关 Pod 的 CPU 随负载线性增长，误差 ±15%）：部分通过。**6 个档位中 4 个在容差之内，80 和 140 迭代/秒分别偏低 17% 和 24%。模型在每一档都偏高。网关的总 CPU 与模型相符（每次迭代 8.1–9.6 ms，模型是 9.58 ms），偏差主要来自最忙副本的份额：第一阶段是 0.58，第二阶段是 0.50–0.53。这个份额取决于 k6 的连接恰好怎样落在两个 Pod 上，是模型中最不稳定的参数。模型的偏差方向是保守的：它预测的饱和比实际早。
- **P2（网关最先饱和，在 152–192 迭代/秒之间达到 0.90 核以上）：通过。**180 迭代/秒时最忙的网关 Pod 是 0.938 核；第二忙的服务（orders）从未超过 0.53 核。
- **P3（120 迭代/秒以内浏览 p95 低于 50 ms，180 以上失败）：部分通过。**60 迭代/秒这一档的 p95 是 51.0 ms，以 1 ms 之差没有通过；80–120 迭代/秒是 26–40 ms；180 和 200 迭代/秒的 p95 达到 1.2 s 和 1.7 s，失败率 13.6% 和 33.2%，与预测一致。
- **P4（工作节点不是瓶颈，160 迭代/秒时合计 6.13 核，误差 ±15%）：通过。**实测 6.37 核（+4%），最忙的节点 2.70 核（共 4 核）。

拐点的形状与排队论一致：最忙的网关 Pod 在 0.6 核以下时，浏览延迟几乎不变（p50 13–15 ms）；0.80 核（160 迭代/秒）时 p50 升到 21 ms、p95 升到 81 ms；0.94 核（180 迭代/秒）时队列失控，网关的平均在途请求数（$X \cdot R$）从 18 跳到 98，请求等到 3 秒的截止时间耗尽，熔断器打开。网关的平均响应时间（其中约 34 ms 是等待下游的时间）在 120 迭代/秒时是 40 ms，160 时 62 ms，180 时 303 ms。按 M/M/1，请求在网关事件循环中的排队时间是 $S \cdot U/(1-U)$；以每个请求约 5 ms 的 CPU 计，利用率 0.61 时约 8 ms，0.80 时约 20 ms，实测的增量（约 6 ms 和 28 ms）与之相近。0.94 时 M/M/1 给出约 80 ms，实测增量约 270 ms：实际的上限是 0.95 核而不是 1 核，相对于这个上限，利用率已经是 0.99，排队时间趋于无穷。200

迭代/秒时，打开的熔断器把 catalog 的请求压到了 180 时的一半，网关自己的 CPU 反而下降：系统以快速失败的方式“卸载”了负载。

还有两类现象不属于模型，但真实存在[29, 30]：

- **60 → 80 迭代/秒的过渡中，约 2 秒内多个熔断器打开**，这一档 5.84% 的请求失败。被保留下来的链路显示，ent-worker3 上的一个 catalog Pod 在这 2 秒里做什么都慢：请求晚到 270–310 ms，一次 Redis GET 用了 140–364 ms，一次简单的 SELECT 用了 262–334 ms；同一时刻，网关重试到另一个 catalog 副本，只用了 8 ms。PostgreSQL 的检查点和 Valkey 的慢日志在这个时刻都没有异常，原因没有找到。档位切换时的这类瞬态在本章出现了三次，它们不是“冷启动”（这一次连接都是热的），而与负载的上升有关。另一个副本只用 8 ms 就答复了，这说明换一个副本重试是有效的，第 7 章（E6）会用到这一点。
- **第一档中有 11 个请求一直等到 k6 的 60 秒超时**，还有 4 个连接被重置。负载均衡器的日志记录了 3 次无法与 NodePort 建立连接（`UF,URX`），健康检查全部成功。网关从未记录过这些请求。这是本章第三次见到这种“请求消失”的尾部事件，约占请求的千分之一，原因同样没有找到。它们对 p95 几乎没有影响，却决定了 p99.9 和最大值。

4.9.3 少一个副本：第二次预测

第二阶段的偏差主要来自最忙副本的份额 b。只有一个网关副本时 $b = 1$，预测只取决于网关的总需求：每次迭代 1.61 × 5.95 = 9.58 ms，在 99 迭代/秒时达到 0.95 核。这次预测同样在实验之前写进 [1]（P5）。副本数要通过 GitOps 改：Argo CD 开启了自愈，`kubectl scale` 的修改几秒钟后就会被改回去。`experiments/e3/one-replica.sh` 先发布 `replicas: 1`，跑完阶梯负载，再发布 `replicas: 2`；配置仓库的历史中留下了每一次改动（两次实验共四个提交）。实验做了两次，第二次加上了对网关调用下游的统计和节点 CPU 调度的测量[31, 32]：

档位	第一次：网关 CPU	浏览 p95	失败率	第二次：网关 CPU	浏览 p95	失败率	预测
40	0.371 核	24.3 ms	0.04%	—	—	—	0.38 核
60	0.518 核	21.3 ms	0	—	—	—	0.57 核
80	0.609 核	22.7 ms	0	0.623 核	42.0 ms	4.86%	0.77 核
100	0.811 核	41.0 ms	0	0.759 核	72.5 ms	3.34%	0.95 核
110	—	—	—	0.962 核	1.58 s	33.4%	饱和
120	0.815 核	1.14 s	24.4%	0.959 核	1.37 s	31.2%	饱和

- **P5a（40、60、80 迭代/秒时 CPU 误差在 ±15% 以内）：部分通过。**40 和 60 迭代/秒时偏差 −3% 和 −10%；80 迭代/秒时两次都偏低约 20%。
- **P5b（80 迭代/秒时低于 0.90 核，100 迭代/秒时达到 0.90 核以上）：没有通过。**100 迭代/秒时网关只有 0.76–0.81 核，仍能服务（失败 0–3%，浏览 p95 41–73 ms）；第二次在 110 迭代/秒时达到 0.96 核

并饱和。按服务能力算，饱和点在 100 与 110 之间，与预测的 99 相差不到 10%；但“CPU 达到 0.90 核”这个判据没有在预测的档位出现。

- **P5c（40、60 迭代/秒时浏览 p95 低于 50 ms，120 迭代/秒时失败）：通过。**

第一次实验中，网关在 120 迭代/秒时只用到 0.815 核就已经失败了，第二次却达到了 0.96 核。0.81 核这个上限没有复现。第一次没有测量 CPU 调度，等到要分析时，那段时间的 Prometheus 数据已经被按大小的保留策略删除了，所以原因无法确定。

第二次实验回答了“饱和时请求在等什么”[33]：

档位	网关 CPU	网关看到的 catalog 调用耗时	catalog 自己的处理时间	隔舱拒绝 catalog 调用	网关所在节点每个 CPU 上平均等待的任务数
80	0.62 核	10.6 ms	2.2 ms	0	0.24–0.25
100	0.76 核	13.2 ms	2.3 ms	0	0.32–0.34
110	0.96 核	635 ms	3.3 ms	每秒 57.8 次	0.42–0.45
120	0.96 核	630 ms	2.8 ms	每秒 69.5 次	0.46–0.48

实测数据

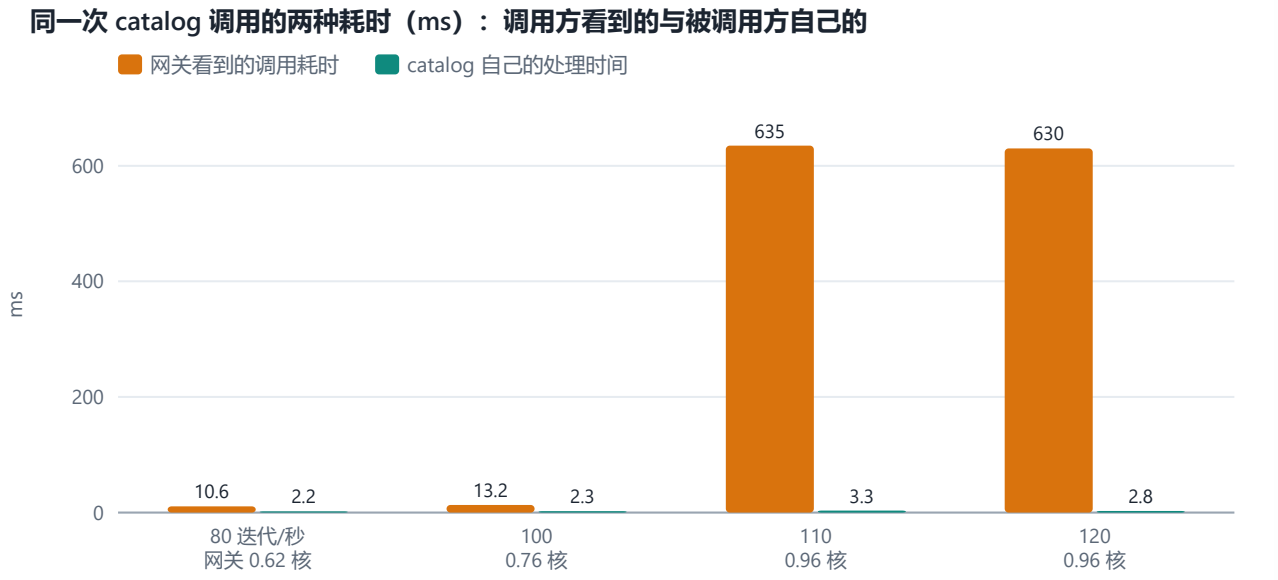


图 4-5 排队发生在调用方。网关的 CPU 到 0.96 核之后，它“看到”的 catalog 调用从十几毫秒变成 630 毫秒以上，而 catalog 自己处理一个请求始终只要 2–3 毫秒：响应早已到达，只是网关的事件循环没有空闲去读它。只看被调用方的延迟，会完全错过这次饱和。

110 迭代/秒时，catalog 处理一个请求只要 3.3 ms，网关这边却要等 635 ms。多出来的 630 ms 都花在网关自己的事件循环里：CPU 已经用满，就绪队列很长，等待 catalog 应答的协程要排很久才能继续执行。按利特尔定律，每秒约 98 次调用、每次 0.635 秒，同时进行的调用平均约 62 个，而网关为每个下游设的隔舱上限是 64 个并发调用（`common/resilience.py`），波动时不断越过上限，于是每秒有 58–70 次调用被直接拒

绝，用户看到的是快速返回的 503。饱和之后最先失败的是隔舱，不是 3 秒的截止时间（截止时间拒绝每秒只有 0.3 次）。

网关所在的 ent-worker3 上还运行着 PostgreSQL 主库、Valkey，以及 catalog、orders、inventory 的副本。它的 4 个 CPU 各忙 53%–71%，平均有 0.24–0.48 个任务在等 CPU；同一时刻的 ent-worker 只忙 34%，平均等待 0.09 个。网关主线程自己排队等 CPU 的时间没有测到：采样脚本的字段下标写错了，样本全部作废，脚本已修正。看门狗在 100 迭代/秒时报告了两次停顿，平均阻塞 337 ms，其中事件循环线程运行了 179 ms、等 CPU 26 ms，剩下约 130 ms 它既没有运行也不在等 CPU：可能阻塞在某个系统调用上，也可能在等另一个线程释放 GIL，本章没有区分出来。可以确定的是：单个网关的上限仍然是它的 CPU（0.96 核），节点上的争用增加了等待，但没有阻止它达到 0.96 核。

真实截图



图 4-6 一个网关副本时的阶梯负载（80→120 迭代/秒，2026-11-09 01:19–01:30 UTC）：唯一的网关 Pod 的 CPU 升到 0.96 核（红线为 1 核），110 迭代/秒起网关的 p95 跳到约 2 秒、每秒数十个 5xx、在途请求（$X \cdot R$）约 80；ent-worker3 的 CPU 明显高于另外两个节点（Grafana 原始截图，看板 Storefront: load and saturation）

4.9.4 余量与可用区

本实验环境的三个节点就是三个 `[模拟]` 可用区（第 1 章）：它们共享同一个内核、同一块磁盘和同一个电源，真实可用区之间的独立性这里没有。两个网关副本分在两个可用区时，失去其中一个可用区就只剩一个副本，能承受的速率从约 170 迭代/秒降到约 100 迭代/秒。容量规划不能只看全部健康时的数字。

设定一个目标：最忙的网关进程不超过 0.7 核。由实验 6 第二阶段的数据，这时延迟还几乎不变；到 0.8 核时 p95 已经翻倍。网关每次迭代合计约 9.2 ms CPU，所以：

- 一个副本：0.7 / 0.0092 ≈ 76 迭代/秒；
- 两个副本（最忙的承担约 53%）：0.7 / (0.53 × 0.0092) ≈ 144 迭代/秒；
- 三个副本、每个可用区一个：全部健康时约 210 迭代/秒（设最忙的副本承担约 36%），失去一个可用区后还剩两个，约 144 迭代/秒。

要在失去一个可用区之后仍然满足目标，副本数必须按“可用区数 + 余量”来定，而且分布约束必须是硬性的（`whenUnsatisfiable: DoNotSchedule`）。本章使用的软约束（`ScheduleAnyway`）允许副本挤在两个可用区里，catalog 就是这样，才会在异常三中一起被摘除。第 5 章（E4）在扩缩容和调度中继续这个话题。

4.10 局限

- **一台笔记本。**所有数字只属于这台机器。同一个旧版本相隔三小时测得的 CPU 开销相差 50%（11.3 ms 与 7.4 ms），k6 虽然被限制在单独的 4 个核上，内存带宽、缓存和磁盘仍与集群共享。比较两个版本时只用并排的方法，比较不同时间的数字时要先怀疑漂移。
- **负载是简化的。**固定的请求组合（70% 浏览、25% 下单、5% 查询订单），只有 18 个商品（缓存几乎总能命中），恒定的到达速率；真实流量有突发、有昼夜变化，热门商品的分布也不同。
- **模型只考虑 CPU。**在本章的负载范围内，数据库（最多 0.69 核）和连接池平时都不是瓶颈；但在档位上升的瞬态和接近饱和时（160–180 迭代/秒），catalog 读缓存超时后改读数据库，连接池从 1–2 个连接涨到 10 个，获取连接平均要等约 450 ms。缓存失效会把负载转移到数据库上，这种“回退负载”不在模型之内。
- **单进程的 Python 服务。** “每个 Pod 约 1 核”的上限来自这种架构。每个 Pod 运行多个工作进程，或使用其他语言，结论都会不同；方法（先测 D，再算饱和点，再验证）不变。
- **测量数据会过期。**Prometheus 的保留期配置为 3 天或 3 GiB，先到先生效：实验期间它保存了 10.8 万条序列、每秒写入约 7 500 个样本，按大小删除了三次最旧的数据块，实际只保留了约 31 小时[34]。本章的结论都以实验当时记录的文字证据为准，事后无法再从 Prometheus 中查询那些时段。
- **剖析是近似的。**非阻塞模式下有四分之一到三分之一的采样读取失败，`--gil` 只看得到持有 GIL 的时间。
- **有三类现象没有找到原因：**档位上升时持续 1–3 秒的瞬态（本章出现了三次）；约千分之一的请求一直等到客户端超时；负载均衡器反复崩溃期间，经过 ztunnel 的连接停顿。本章只记录了它们的证据和排除过的假设。

4.10.1 从实验环境到生产环境

本章的做法	生产环境通常的做法	差别意味着什么
固定的请求组合（70% 浏览、25% 下单、5% 查询订单），18 个商品	按生产的访问分布建模，或回放真实流量	缓存命中率、热点和数据量都会改变每个请求的成本
单进程的 Python 服务，一个 Pod 最多 1 核	多进程、多线程或编译型语言，一个 Pod 可以用多核	“副本数 ÷ 每请求 CPU”的模型要按运行时重新标定
客户端与被测系统在同一台电脑上，k6 固定在工作节点之外的 CPU	负载机与被测系统分开，经过真实的网络	本章的延迟不含真实网络；客户端自身也可能成为瓶颈
每个档位几分钟，每种配置运行一两次	长时间浸泡、多次重复、覆盖每日峰谷	内存泄漏、垃圾回收抖动和周期性任务需要更长的观测才会出现

4.11 常见问题

为什么不直接给网关 Pod 更多的 CPU? 一个 Python 进程同一时刻只能在一个核上执行 Python 代码，最忙的网关 Pod 停在 0.94–0.95 核，再多的 CPU 配额也用不上。提高容量要靠更多的进程：更多的副本，或者每个 Pod 多个工作进程。CPU 请求与限制对调度和延迟的影响在第 5 章（E4）讨论。

容量规划应该看平均延迟还是 p95? 两者都要看，但用途不同。平均值服从利特尔定律，可以用来检验测量、推算在途请求数；p95、p99 才反映用户的体验，也最先显示出拐点。本章的拐点判断都以 p95 和失败率为准。

k6 的 `dropped_iterations` 什么时候出现? 开放模型下，k6 需要越来越多的虚拟用户来维持速率；虚拟用户达到 `MAX_VUS` 后，到时间却没有空闲用户可用的迭代就被丢弃。它说明系统已经过载，这时 k6 报告的延迟只代表实际发出的那部分请求。

为什么在途请求数要用 `rate(..._sum)` 而不是在途计数器? 计数器的瞬时值每隔几毫秒变化一次，每 15 秒抓取一次只得到少数快照（实验 5）。直方图的总和是所有请求耗时的累加，它对时间的变化率恰好是平均在途请求数。

金丝雀对比和 A/B 交替（第 3 章）有什么不同? 两者都在对抗漂移。交替实验让两种配置在相邻的时间段里轮流运行，适合不能同时存在的配置（例如网格开或关）；金丝雀让两个版本在同一时刻承受同一股流量，适合可以共存的版本，漂移被完全抵消，还能用两个旧副本做 A/A 对照。

4.12 练习

练习

1. **（计算）** 一个服务每个请求消耗 6 ms CPU，由两个单进程副本承担，最忙的副本承担 58% 的 CPU，每次迭代向它发出 1.6 个请求。在每秒多少次迭代时，最忙的副本达到 0.95 核？如果增加到 3 个副本、最忙的副本承担 40%，饱和点变为多少？为什么不是原来的 1.5 倍？

2. **（分析）** 60 迭代/秒那一档，k6 记录了 510 次商品请求失败，网关指标的 `increase()` 却是 0。解释原因，并给出两种在指标层面避免这一问题的方法。
3. **（实验设计）** 为什么并排的金丝雀比“先测旧版、再测新版”可靠？金丝雀方法仍有哪些未被控制的因素？至少举出三个，并说明本章处理了其中的哪两个、如何处理的。
4. **（排障）** 负载均衡器反复崩溃期间，ent-worker 上 catalog 的就绪探针超时，而 `/healthz` 很快。列出你会依次检验的假设，以及能区分它们的测量。
5. **（计算）** 网关每秒完成 64 个请求，平均响应时间 26.5 ms，平均在途请求数是多少？Prometheus 每 15 秒抓取一次在途计数器，90 秒窗口内的平均值是 0.76。说明差异的原因，以及如何从 Prometheus 正确地得到平均在途请求数。
6. **（设计）** 本实验环境的三个节点就是三个可用区。用本章的模型和测量，计算网关在失去一个可用区后还能承受的速率；为“失去一个可用区后，最忙的网关进程不超过 0.7 核”这一目标，给出网关的副本数和分布要求。
7. **（批判）** py-spy 的默认模式说网关 89% 的时间在工作，而网关主线程只用了 0.43 核。解释原因。`--gil` 模式又会漏掉哪些 CPU？

参考答案见附录。

4.13 本章小结

- 观测有成本：完整的链路追踪约占 Python 服务 CPU 的 6%–25%（这台机器的漂移很大，只能给出范围），之后的实验使用 10% 的头部采样。
- 配置错误可能比业务本身还贵：Tempo 的内存压舱物与软内存上限叠加，垃圾回收每秒 18 次、占用 1.34 核；看了堆剖析才找到原因，修正后降到 0.027 核。
- 第一次阶梯负载中，网关最先饱和，单个 Python 进程停在 0.94–0.95 核。表中的三处异常各有机制：熔断器返回的 503 在它们自己创建的时间序列里对 `increase()` 隐身；实验环境的负载均衡器受 1024 个文件描述符的限制，在约 500 个连接时崩溃（8 分钟 106 次），修正后为 0；在负载均衡器反复崩溃期间，同节点上经 ztunnel 访问数据库的连接停顿数秒，依赖数据库的就绪检查让 catalog 的两个副本同时被摘除（ztunnel 内部的原因没有找到）。
- 剖析之前先要验证剖析工具：py-spy 跨 PID 命名空间时把空闲当成繁忙，改为只取持有 GIL 的样本，并对照线程的 CPU 时间。成本分散在框架、客户端和插桩中，业务代码只占 3%–5%。两项零风险的修改，经金丝雀并排对比，每个请求的 CPU 少了 25%，比剖析数据暗示的多。
- 利特尔定律在密集采样下误差小于 5%；每 15 秒抓取一次的在途计数器不能用来求平均，要用 `rate(…_sum)`。
- 容量模型先提交预测、再做实验：两个副本时网关在 180 迭代/秒饱和（预测 172 ± 20），节点 CPU 误差 4%；模型中最不稳定的参数是最忙副本的份额。一个副本时，100 迭代/秒仍能服务，110 迭代/秒饱和（预测 99），但“达到 0.90 核”的判据没有在预测的档位出现。饱和后最先失败的是隔舱：网关的事件循环排队，让 3 ms 的 catalog 调用在网关看来要 635 ms。

- 失去一个可用区，网关的容量约减半（约 170 → 约 100 迭代/秒）。要在失去一个可用区后仍保证最忙的进程不超过 0.7 核，网关需要 3 个副本，并且严格地分布在 3 个可用区。
- 没有找到原因、但已记录证据和排除过的假设的现象：负载上升时 1–3 秒的瞬态，约千分之一的请求一直等到客户端超时，第一次单副本实验中 0.81 核的上限。另外，Prometheus 按大小的保留策略只留下了约 31 小时的数据：结论要靠实验当时记录的证据，而不是事后去查。

第 5 章

弹性与调度（E4）

第 4 章回答了“这套系统能扛多少”；本章回答“负载变了、资源不够了、出了故障时，平台会怎样自动应对”。Kubernetes 用一组彼此独立的机制处理这些问题：按指标增减副本的自动扩缩容，决定 Pod 放在哪里的调度器，决定资源不够时谁先让路的优先级与驱逐，以及按请求量分配 CPU 的内核调度。每一种机制都有自己的延迟和自己的判断依据。本章逐一测量它们的时间线，并检验它们在一起工作时是否符合预期。

工程问题

负载变了、资源不够了——平台会“自动应对”吗？自动扩缩容要多久才起作用，资源争抢的时候又是谁让路？

实测结论

- HPA 不是瞬时的：利用率停在容差边缘（102%–114%）时，它等了 5 分 26 秒才扩容；一旦决定，新 Pod 8 秒就绪。
- 扩容不等于分担：在第 4 层入口和长连接之下，扩出的 Pod 一个请求也没收到，20.5% 的请求失败，而 HPA 看到的平均值“一切正常”。
- 只按队列积压扩缩的 KEDA，在恒定负载下每隔 300 秒把副本缩到 1 个；再加一个反映到达速率的触发器，才稳定在 3 个。
- 内存压力下，kubelet 先驱逐“超出请求最多”的 Pod，而不是 BestEffort；压力状态持续了近 10 分钟。

5.1 学习目标

学完本章，你应当能够：

- 区分 CPU 请求（requests）在调度、CPU 分配和自动扩缩容中的三种作用，以及 CPU 限制（limits）带来的限流；
- 解释 HPA 从负载变化到新副本就绪之间每一段延迟的来源，并据此设定目标利用率和副本范围；
- 在 GitOps 中让 HPA 或 KEDA 接管副本数，而不与 Argo CD 的自愈相互抵消；
- 用队列积压（而不是 CPU）驱动扩容；
- 预测节点内存不足时哪个 Pod 先被驱逐，以及高优先级 Pod 会抢占谁；
- 用配额、默认值和拓扑分布约束为多个团队共享的集群设定边界。

实验性质

本章的实验都是 `[真实]` 运行的。三个“可用区”是 `[模拟]` 的（第 1 章）；为了让通知服务跟不上订单，它的“发送通知”被调慢为每条 200 ms，这是 `[模拟]` 的外部服务延迟。驱逐、抢占和邻居干扰实验中占用内存或 CPU 的 Pod 是有意制造的负载，它们只运行在实验期间，脚本结束时删除。

5.2 原理

5.2.1 CPU 请求的三种作用，与 CPU 限制

一个容器的 `resources.requests.cpu` 同时被三个互不相干的组件使用：

1. **调度器**把它当作“这个 Pod 需要的 CPU”：一个节点上所有 Pod 的请求之和不能超过节点的可分配量（本实验环境每个工作节点 3.75 核）。请求只是记账，Pod 实际用多少由下面两项决定。
2. **内核**按请求分配 CPU：kubelet 把请求换算成 cgroup 的权重（`cpu.weight`），CPU 空闲时谁都可以多用，CPU 不够用时按权重比例分配。请求为 500m 的 Pod 权重约为 20，请求为 50m 的约为 2，没有请求的 BestEffort Pod 只有 1。
3. **HPA** 的目标利用率是“实际用量 ÷ 请求”。请求定得不真实，百分比就失去意义：请求 50m、实际用 0.5 核，利用率是 1000%。

`resources.limits.cpu` 是另一回事：它变成 CFS 的配额（`cpu.max`），在每个 100 ms 的周期里，容器最多运行“限制 × 100 ms”的 CPU 时间，用完就被暂停到下一个周期开始。平均用量远低于限制的服务，也可能在某个周期内的突发中用完配额，于是请求凭空多等几十毫秒。这就是 CPU 限流（throttling）。

5.2.2 HPA 的算法与它的延迟

HPA 控制器每 15 秒计算一次：

$$\text{期望副本数} = \left\lceil \text{当前副本数} \times \frac{\text{当前指标}}{\text{目标指标}} \right\rceil$$

比值与 1 的差在 10% 以内时不动（容差）。扩容默认立即执行（每 15 秒最多翻倍或加 4 个），缩容要求过去 300 秒内的期望值都更低（稳定窗口），以免来回抖动。从负载上升到新副本开始分担流量，要经过一串延迟：kubelet 采集容器 CPU，metrics-server 汇总（本环境每 15 秒），HPA 下一次计算，Deployment 创建 Pod，调度器选择节点，拉取镜像、启动容器，就绪探针通过，最后网格与负载均衡把新连接分给它。本章测量这条链上的每一段。

示意图

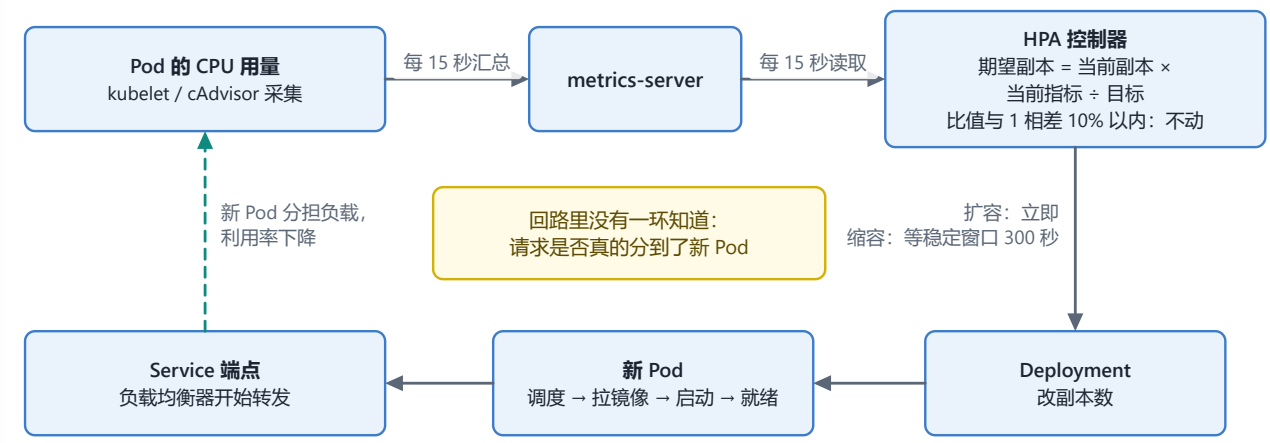


图 5-1 HPA 的控制回路。从负载升高到新副本接到请求要走完整条链，每一段都有自己的周期；缩容还要再等一个稳定窗口。回路里没有任何一环“知道”请求是否真的分到了新 Pod。

5.2.3 KEDA：按事件源扩缩容

CPU 不是所有服务的好指标。通知服务大部分时间在等待外部接口，CPU 很低，积压却可能越来越多。KEDA 读取事件源（这里是 NATS JetStream 消费者的未处理消息数），把它作为外部指标交给自己创建的 HPA：期望副本数约等于“积压 ÷ 每个副本的目标积压”。KEDA 每隔 `pollingInterval` 秒读一次事件源，这是它特有的一段延迟。

5.2.4 资源不够时谁让路：QoS、驱逐与抢占

- **QoS 类别**由请求和限制决定：所有容器的请求等于限制是 Guaranteed，有请求但不全相等是 Burstable，什么都没写是 BestEffort。
- **节点压力驱逐**：kubelet 发现节点可用内存低于阈值（本环境是工作集超过约 2922 MiB，第 1 章），按顺序驱逐 Pod：先看用量是否超过请求，再看优先级，最后看超出请求的多少。BestEffort 的请求为 0，它的全部用量都算“超出”。
- **抢占**：调度器为一个放不下的高优先级 Pod 寻找节点，在那里驱逐尽量少、优先级尽量低的 Pod。没有指定 PriorityClass 的 Pod 优先级为 0。
- **拓扑分布约束**：把副本均匀地放在可用区之间；`ScheduleAnyway` 是建议，`DoNotSchedule` 是硬性要求，宁可不调度也不打破均匀。

示意图

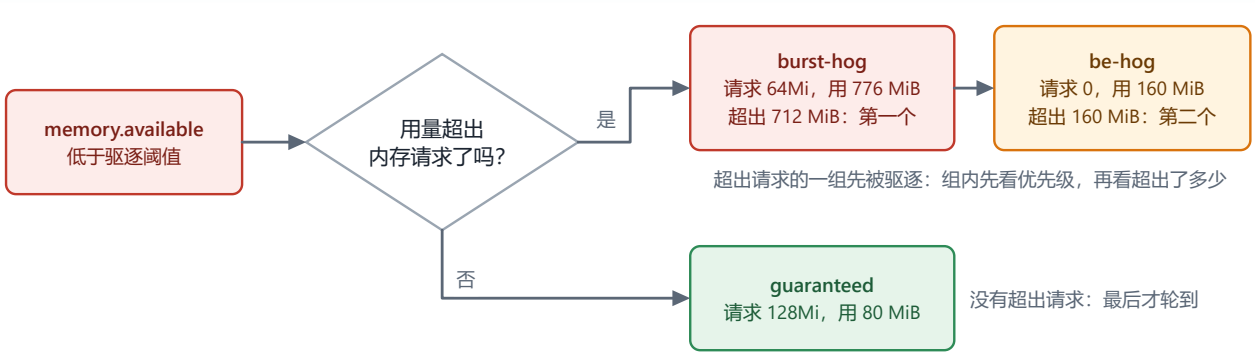


图 5-2 节点内存不足时 kubelet 的驱逐顺序（用实验 6 的三个 Pod 举例）。先看用量是否超出请求，再看优先级，最后看超出了多少——QoS 类别不是排序键。没有内存请求的 BestEffort Pod 只超出 160 MiB，排在超出 712 MiB 的 Burstable Pod 之后。

5.3 实验 1：把副本数交给 HPA

到第 4 章为止，网关的副本数写在 Git 中（`replicas: 2`），Argo CD 开启了自愈：任何人用 `kubectl scale` 改了副本数，几秒钟后就会被改回去。HPA 正是要改副本数的那个“人”。如果 Deployment 的清单里仍然有 `replicas` 字段，HPA 每扩容一次，Argo CD 就把它改回 2，两个控制器会永远互相抵消。所以 Chart 增加了一个可选的 HPA，启用时 Deployment 的清单里不再有 `replicas` 字段（`gitops/charts/storefront-service/templates/`）。

网关的 CPU 请求同时改为 500m。原来的 50m 只是一个占位数字：第 4 章测得网关在 60 迭代/秒时每个 Pod 约用 0.28 核，按 50m 计算利用率是 560%，HPA 会立刻扩到上限。500m 接近计划负载下的真实用量；目标利用率 100%，即让每个 Pod 平均用 0.5 核。按第 4 章的数据，两个副本时较忙的那个承担约 53%，它会在 0.53 核左右，仍在延迟平坦的区间之内。

这次交接本身有代价（[2]，观察脚本每 2 秒记录一次）：

```
01:52:51 | hpa none | deploy 2 2 | 9c2jd:Running:true twsgl:Running:true
01:52:58 | hpa 2 1  | deploy 2 1 | gm44w:Pending:false qhww7:Pending:false 9c2jd:Running:true twsgl:
Running:true
01:53:05 | hpa 2 1  | deploy 2 2 | gm44w:Running:false qhww7:Running:true 9c2jd:Running:true
01:53:15 | hpa 2 2  | deploy 2 2 | gm44w:Running:true qhww7:Running:true
```

Argo CD 默认用客户端方式应用清单（相当于 `kubectl apply`）：清单中去掉的字段会从对象上删除，`replicas` 被删除后取默认值 1，Deployment 立刻缩到 1 个副本。HPA 在同一秒里发现“副本数低于最小值 2”并改回 2（事件：`New size: 2; reason: Current number of replicas below Spec.MinReplicas`）。由于 CPU 请求同时变了，这还是一次滚动更新。合在一起，有约 7 秒只剩一个就绪的网关副本。在没有负载的时候这无关紧要；在高峰期做这个交接，相当于主动去掉一半容量。可以避免它的做法有两种：用服务端应用（server-side apply，字段的归属可以转移而不被删除），或者先让 Argo CD 忽略 `replicas` 的差异（`ignoreDifferences`）再从清单中删除它。

5.4 实验 2：HPA 的反应，以及它看不见的过载

`experiments/e4/hpa-timeline.sh` 用一次 k6 运行给网关加一个负载台阶，同时每约 2 秒记录 HPA 的决定、Deployment 的副本数和每个 Pod 的状态（`experiments/e4/watch.sh`），结束后收集 HPA 与 Pod 的事件，以及每个新 Pod 的创建、调度、就绪时间。本环境的相关设置都是默认值：metrics-server 每 15 秒汇总一次，HPA 每 15 秒计算一次，容差 10%，缩容稳定窗口 300 秒。

5.4.1 台阶到 120 迭代/秒：卡在容差边缘

第一次台阶从 30 升到 120 迭代/秒（[3]；脚本在 k6 结束后中断了，因为我在它运行时修改了脚本文件，bash 是边读边执行的；事件与 Pod 时间是之后补收集的）。按第 4 章的数据，120 迭代/秒时两个网关 Pod 平均各用约 0.55 核，即请求的 110%，正好在容差的边上。观察到的利用率在 102%–114% 之间摆动，HPA 直到台阶开始 5 分 26 秒后才扩到 3 个（`New size: 3; reason: cpu resource utilization (percentage of request) above target`）。一旦决定扩容，执行很快：

时刻	事件
01:57:17	负载升到 120 迭代/秒
02:02:43	HPA 决定扩到 3 个，Pod 创建并调度到 ent-worker2
02:02:45	容器启动（镜像已在节点上）
02:02:49	启动探针第一次失败（进程还没开始监听）
02:02:51	Pod 就绪：从决定到就绪 8 秒
02:04:27	负载回到 30 迭代/秒
02:08:28	缩回 2 个：距离最后一次“需要 3 个”的计算约 300 秒（稳定窗口）

实测数据

台阶到 120 迭代/秒：HPA 的一次扩容与缩容（秒，从负载升高起算）

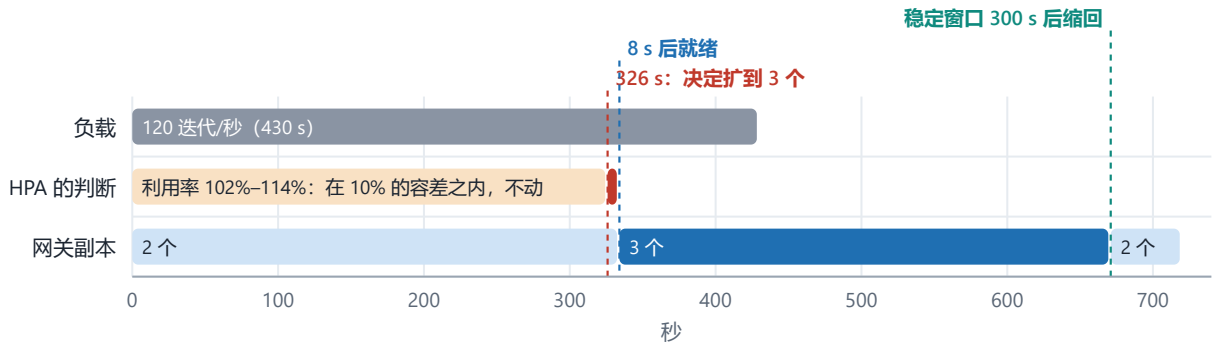


图 5-3 这次扩容的时间线。负载升高后的 326 秒里，利用率一直在目标值的 102%–114% 之间——落在 HPA 10% 的容差之内，它什么也不做；越过容差之后，从决定到新 Pod 就绪只用了 8 秒。负载回落后，HPA 又等满 300 秒的稳定窗口才缩回。慢的不是执行，是判断。

延迟一直正常（120 迭代/秒时浏览 p95 31.8 ms），因为两个副本本来也撑得住。这说明容差有它的盲区：利用率在目标的 100%–110% 之间时，HPA 什么也不做。目标值的含义是“允许长期停留的水平”，而不是“开始扩容的水平”。

5.4.2 台阶到 170 迭代/秒：扩出来的 Pod 没有流量

第二次台阶升到 170 迭代/秒，约为目标的 150%[4]。HPA 这次反应很快：台阶开始 44 秒后扩到 3 个（利用率 116%），8 秒后就绪；2 分 44 秒后扩到 4 个。可是这一档有 20.5% 的请求失败，浏览 p95 接近 1 秒。每 30 秒的记录[5]显示，网关每秒约 300 个请求、每秒 60–140 个 5xx，全部是隔舱或熔断器的拒绝；网关的 CPU 合计只有 1.4–1.5 核，扩到 3 个、4 个之后也没有增加。按 Pod 拆开[6]就清楚了：

02:21:30–02:22:30	请求/秒	CPU	5xx/秒
gateway-…-gm44w（原有）	147	0.79 核	36.0
gateway-…-qhww7（原有）	142	0.92 核	21.4
gateway-…-j72rh（02:19 扩出）	0	0.01 核	—
gateway-…-d5j6r（02:21 扩出）	0	0.01 核	—

实测数据

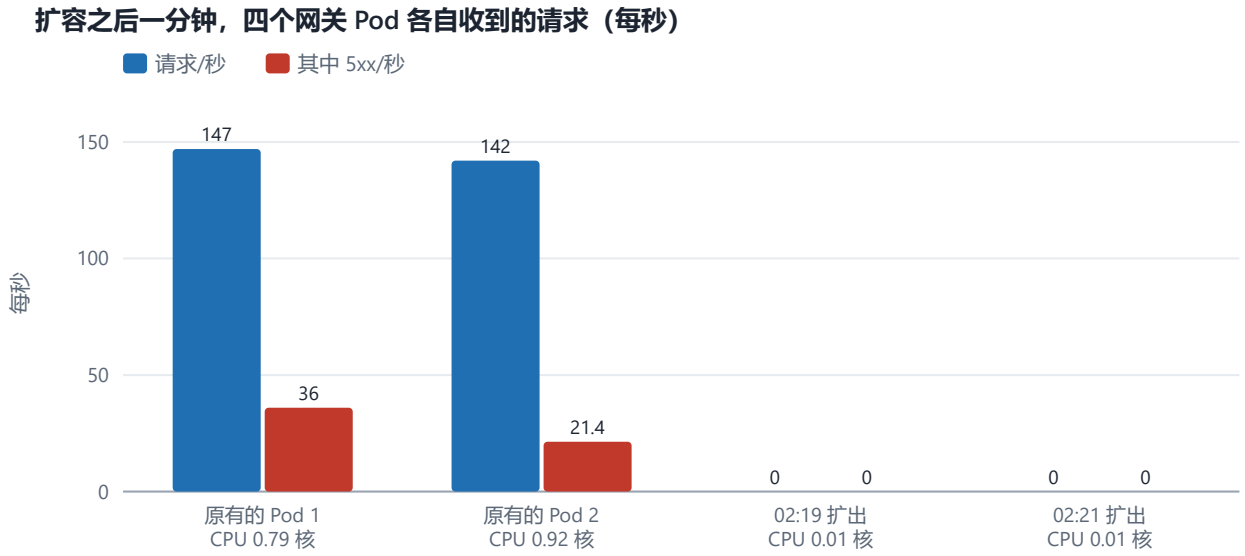


图 5-4 扩容之后的一分钟里四个网关 Pod 各自的负载。两个原有的 Pod 仍在过载并返回 5xx，两个新 Pod 一个请求也没有收到：已有的长连接不会自己搬家。HPA 计算的是四个 Pod 的平均利用率，于是认为问题已经解决。

扩出来的两个 Pod 在整个高负载阶段**一个请求也没有收到**。原因是第 3 章讲过的 L4 负载均衡与长连接：k6 的每个虚拟用户在第一次迭代时建立一条保持连接（keep-alive），之后一直复用它；负载均衡器、NodePort 都在连接建立时决定后端。台阶开始的头几秒里所有虚拟用户都已经连上了原来的两个 Pod，新 Pod 就绪时已经没有新连接了。HPA 看到的是四个 Pod 的平均值：两个 0.8–0.9 核、两个 0.01 核，平均约 0.43 核，正好在目标附近，“一切正常”；用户看到的是五分之一的请求失败。

真实截图



图 5-5 170 迭代/秒的台阶（2026-11-09 02:18:31–02:23:41 UTC）：网关的 HPA 从 2 个扩到 4 个，通知服务的 KEDA 从 1 个扩到 5 个（右上）；两个原有网关 Pod 的 CPU 升到 0.84 和 0.92 核，02:19 和 02:21 扩出的两个 Pod 始终不到 0.1 核（左中）；网关的 5xx 按 1 分钟速率最多每秒 106 个（左下）（Grafana 原始截图，看板 Storefront: load and saturation）

这个实验得出三点：

- **扩容不等于分担负载。**流量要能到达新的副本：要么客户端会重新建立连接，要么入口按请求（L7）而不是按连接分配。本环境的外部入口是 L4，第 9 章（E8）用网关 API 在入口做 L7 路由。
- **HPA 看的是平均值，容量却由最忙的那个 Pod 决定**（第 4 章的份额 b）。负载不均时，平均利用率会掩盖已经饱和的 Pod。
- **会主动拒绝请求的服务，CPU 会低估需求。**被隔舱或熔断器快速拒绝的请求几乎不花 CPU。对这种服务，按请求速率或排队情况扩容比按 CPU 更可靠。

5.4.3 让客户端重新连接：这次失败的是负载生成器

为了直接验证“是保持连接把流量钉在了旧副本上”，同样的台阶又做了一次，让 k6 在每次迭代时新建连接（`noVUConnectionReuse`，[7]）。这个对照没有做成，失败的是负载生成器自己：32 546 个错误中有 32 518 个是 `connect: cannot assign requested address`。k6 从同一个地址、向同一个负载均衡器地址每秒建立上百个新连接，客户端关闭的连接要在 TIME_WAIT 状态停留 60 秒，本地端口很快就用完了（加上失败的连接尝试本身也占用端口）。网关只收到一部分请求，其中大部分以 5xx 结束，HPA 始终没有扩容。原因在压测工具一侧，结论不能用于判断网关。要做这个对照，需要更多的客户端地址，或者干脆在入口按请求分配流量；第 9 章（E8）在入口使用 L7 路由。

另外，这几次实验之间，主机的 CPU 主频降了下来：笔记本在持续负载下降频，Windows 的“处理器性能”计数器显示只有标称频率的 65%（3.8 GHz 中的约 2.5 GHz，[8]）。同样的流量下，每个服务每个请求的 CPU 都变成了 1.5–2 小时前的约两倍（网关 4.47 → 8.94 ms，[9]）。这正是第 4 章强调要并排比较的原因；也说明自动扩缩容在这种情况下是有用的：同样 90 迭代/秒的负载，网关在主频下降后被扩到了 4 个副本。

5.5 实验 3：按队列积压扩容（KEDA）

通知服务的 CPU 很低，它的“负载”是 JetStream 消费者中未处理的消息。KEDA 2.21.0 经 GitOps 安装[10]，通知服务的 ScaledObject 以“每个副本 50 条积压”为目标，1–5 个副本，每 10 秒读一次 NATS 的监控接口。安装时有一个细节：客户端方式应用会把整个对象存进 `last-applied-configuration` 注解，而一个对象的注解总共最多 262 144 字节；KEDA 的 ScaledJob CRD 有 324 010 字节，写不进去（`metadata.annotations: Too long`，[11]），所以 KEDA 的 Argo CD 应用用服务端应用。为了让一个副本跟不上，通知服务的“发送”被调慢为每条 200 ms、同时 2 条，即每个副本每秒 10 条（`[模拟]` 的慢速外部服务）；170 迭代/秒时每秒约有 35 笔订单确认，即 35 条消息（实测 35.3 笔/秒，通知服务处理 35.2 条/秒），需要 4 个副本。

与网关的台阶在同一次运行中[4]：

时刻	通知服务
02:18:31	负载升到 170 迭代/秒
02:18:46	扩到 2 个（每个副本积压 100 条），8 秒后就绪
02:19:16	扩到 4 个（每个副本 193.5 条）
02:19:31	扩到 5 个（上限），02:19:37 全部就绪：台阶开始后 66 秒
02:19:47 起	每个副本的积压 0–4.4 条，积压消化完毕
02:23:41	负载回到 30 迭代/秒
02:24:17、02:24:32	缩到 3 个、1 个

KEDA 比 CPU 扩容快得多：第一次扩容只用了 15 秒，因为积压在台阶开始后立刻增长，而且它每 10 秒读一次。缩容的时刻值得细看：它恰好是 02:19:16 和 02:19:31 那两次“需要更多副本”的计算之后 300 秒（稳定

窗口），与负载何时下降无关。积压消化完之后，每个副本只剩几条积压，按“积压 ÷ 50”计算的期望副本数早就是 1 了。也就是说，**积压衡量的是“排着多少”，不是“来得多快”**：消费者跟得上时积压接近零，指标就认为一个副本足够。负载如果在 02:24 之后仍然保持，通知服务也会缩回 1 个，积压重新积累，再扩容。

5.5.1 稳定负载下的振荡

上面的推断可以直接检验：负载在 90 迭代/秒保持 12 分钟[12]，通知服务需要约 2 个副本才跟得上。

时刻	通知服务副本	每个副本的积压
02:52:52	负载升到 90 迭代/秒	
02:53:11	1 → 3	134
02:58:21	3 → **1**	1.7
02:58:36	1 → 3	121
02:58:49	3 → 5	67
03:03:45	5 → **1**	1.6
03:04:01	1 → 2	92
03:04:15	2 → 4	99.5
03:05:02	负载回到 30 迭代/秒	

实测数据

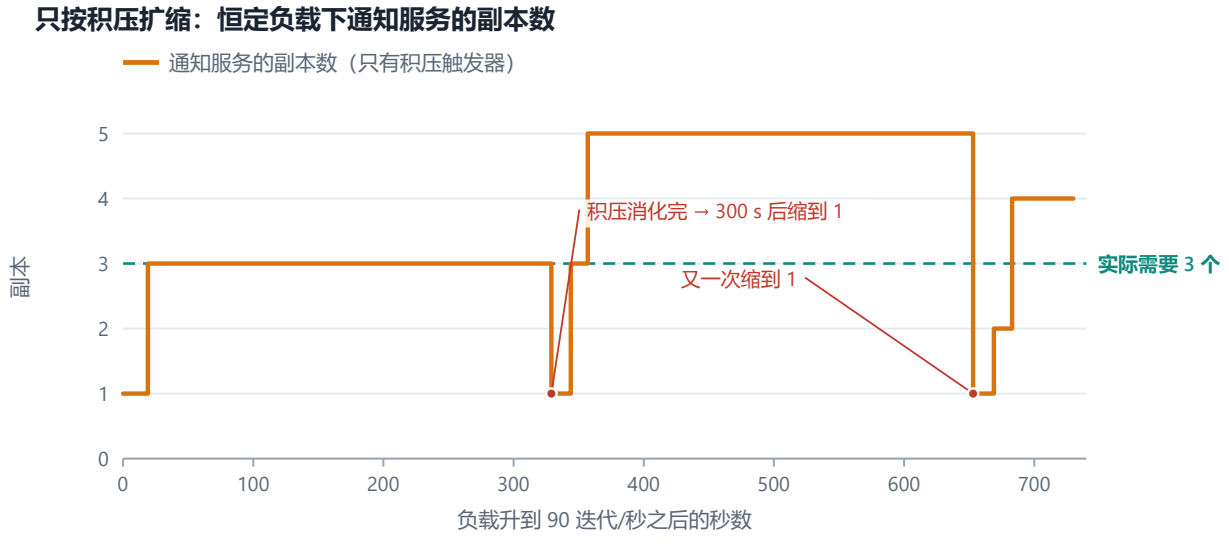


图 5-6 只按积压扩缩时的副本数。负载没有变化，副本数却在 1 和 5 之间来回：积压被消化完之后，“每个副本的积压”接近 0，KEDA 等满 300 秒的稳定窗口就缩到 1 个；一个副本处理不过来，积压重新出现，再扩容。积压衡量的是“排着多少”，不是“来得多快”。

负载一直没有变，通知服务却两次被缩到 1 个副本，又在 15–30 秒内扩回去：一个周期约等于稳定窗口（300 秒）的锯齿。每次缩到 1 个，积压都重新涨到一两百条，通知的延迟随之升高。

5.5.2 修正：再加一个反映到达速率的触发器

到达速率可以直接从订单服务的指标得到：每确认一笔订单就产生一条通知。通知服务的 ScaledObject 增加第二个触发器，用 KEDA 的 Prometheus 触发器按“每秒确认的订单数 ÷ 8”计算副本数（每个副本每秒能处理 10 条，8 是 80% 的利用率），积压触发器保留，用来应对突发；KEDA 取两个触发器中要求副本最多的那个（`gitops/envs/prod/storefront/services/notifier.yaml`）。同样的 12 分钟、90 迭代/秒再做一次，每分钟记录到达速率、发送速率、副本数和“订单确认到通知发出”的 p95[13, 14]：

每分钟的通知 p95	只有积压触发器	积压 + 到达速率
台阶开始后的头两分钟（从 1 个副本扩容）	最高 22.2 s	最高 15.7 s
之后的副本数	3 → 1 → 5 → 1 → 4，两次缩到 1	5 → 3，之后一直是 3
每次缩到 1 个之后的那一分钟	19.7 s、20.9 s	—（没有缩到 1）
其余时间	0.48–0.72 s（缩容后恢复的那一分钟 2.1 s）	0.48–0.73 s

每秒约 22–23 笔确认，按每个副本 8 条计算需要 3 个副本，这正是修正后稳定下来的副本数。积压衡量“排着多少”，到达速率衡量“来得多快”；扩缩容需要后者来决定稳态，前者只适合处理突发。两种配置在台阶开始时都要从 1 个副本扩上来，头一两分钟的积压是一样的；要避免它，需要更高的最小副本数，或者在已知的高峰之前提前扩容。

两次运行中 HTTP 请求分别有 2.7% 和 4.5% 失败，几乎全部是网关自己返回的 503 和 504（每秒 4.0 次和 7.0 次，[15]）。503 正好等于网关的快速拒绝，主要是对 catalog 的熔断器打开，其次是隔舱已满[16]；504 是上游超时。这些失败发生在主频下降之后的同步请求路径上。通知服务是异步的消费者，不在这条路径上；第二次失败更多的原因本章没有分析。

5.6 实验 4：CPU 限制与限流

`experiments/e4/ab-limit.sh` 复制网关的 Deployment，给副本加上 300m 的 CPU 限制（请求也设为 300m，请求高于限制是无效的），让它与两个不设限制的网关 Pod 同时承受 60 迭代/秒的流量，方法与第 4 章的金丝雀对比相同。内核为这个容器设置的配额是 `cpu.max: 30000 100000`：每 100 ms 的周期内最多运行 30 ms（[17]，按 Pod 的统计见 [18]）：

	不设限制（2 个 Pod）	限制 300m
平均 CPU	0.207–0.288 核	0.282 核
延迟 p50	16.4 ms	21.9 ms
延迟 p99	262 ms	475 ms
事件循环延迟 p99	5–7 ms	95.5 ms

	不设限制（2 个 Pod）	限制 300m
被限流的周期	—	67%

容器自己的 `cpu.stat` 在实验期间增加了 1816 个周期，其中 1218 个被限流，累计被暂停 44.8 秒。平均用量（0.28 核）低于限制（0.3 核），三分之二的周期却被限流：请求是一簇一簇到达的，一簇请求在一个周期的前 30 ms 就用完配额，这个周期剩下的时间里整个进程被暂停，排在后面的请求只能等到下一个周期。对单线程的事件循环服务，限流直接表现为事件循环的停顿，p99 从约 6 ms 升到 96 ms。连这个副本启动的过程（导入模块、建立连接池）就已经被限流了 71 个周期。

对延迟敏感的服务，CPU 限制几乎只有坏处：保证份额靠请求（cgroup 权重），限制只会在突发时把进程停下来。如果出于成本或公平需要限制，应当把它设在峰值之上，并监控被限流的周期比例（`container_cpu_cfs_throttled_periods_total`）。实验 8 中，LimitRange 正是在没有人察觉的情况下给每个容器加上了这样的限制。

5.7 实验 5：吵闹的邻居与 CPU 请求

`experiments/e4/noisy.sh` 在 50 迭代/秒的稳定负载下，在 ent-worker 上依次放一个 CPU“消耗者”：4 个死循环进程，正好占满这个节点的 4 个 CPU。ent-worker 上有一个网关 Pod（请求 500m）和 orders、inventory、payments 各一个（请求 50m）；同一服务在其他节点上的副本同时承受同样的流量，是天然的对照[19]：

阶段	网关 p50（ent-worker / 其他节点）	事件循环延迟 p99：网关	inventory	orders	payments
没有消耗者	16.3 / 16.2 ms	8.1 / 7.7 ms	4.4 / 4.9 ms	4.9 / 5.2 ms	4.2 / 4.2 ms
BestEffort 消耗者	17.6 / 16.6 ms	9.5 / 8.6 ms	13.3 / 4.8 ms	20.5 / 7.7 ms	12.9 / 5.9 ms
Burstable 消耗者（请求 2 核）	130.8 / 13.9 ms	88.8 / 7.2 ms	72.2 / 2.6 ms	162.5 / 3.7 ms	50.0 / 2.5 ms

没有消耗者时，两边几乎相同。BestEffort 的消耗者（没有请求）对请求 500m 的网关几乎没有影响，请求 50m 的服务事件循环延迟增加了 2–3 倍，但仍在十几毫秒。同样的进程只要声明请求 2 核，就变成了真正的灾难：这个节点上网关的 p50 是 131 ms，同一时刻其他节点上的网关只有 14 ms；orders 的 p50 达到 1.6 秒、p99 达到 6 秒（超时上限），整个下单流程被拖垮。网关对 orders 的熔断器打开，每秒拒绝 14.3 次调用（前两个阶段为 0，[20]），连其他节点上的 orders 副本每秒也只收到约 1 个请求（没有消耗者时约 8 个）。

实测数据

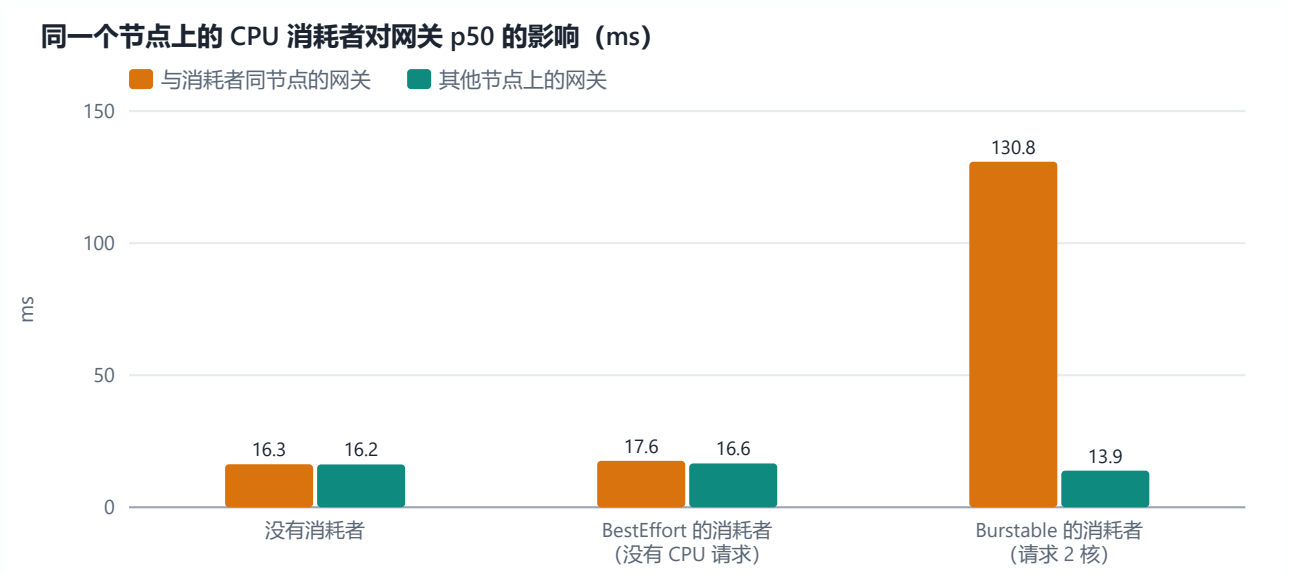


图 5-7 同一个 CPU 消耗者，声明不同的 CPU 请求。没有请求时，它只能用别人剩下的 CPU，同节点的网关几乎不受影响；声明 2 核的请求之后，内核按权重分配 CPU 时间（网关请求 500m，权重 20；消耗者请求 2 核，权重 79），网关在争抢中只分到一小份，p50 从 16 ms 变成 131 ms。CPU 请求不只是给调度器看的数字，它是争抢时的权重。

原因在 cgroup 的层次与权重[21]。kubelet 把请求换算成权重：50m 为 2，500m 为 20，2 核为 79；QoS 分组的权重是组内所有请求之和换算的结果。BestEffort 的 Pod 都在一个权重为 1 的分组里，与 Burstable 分组（计入消耗者的 2 核请求时为 125，消耗者删除、kubelet 更新之后为 47）竞争 CPU，它只能捡别人剩下的时间。声明了请求的消耗者进入 Burstable 分组，与服务的 Pod 按各自的权重分配 CPU：79 对网关的 20、对 50m 服务的 2。“请求”不只是调度器的记账，它在 CPU 不够时就是每个 Pod 的份额。给关键服务真实的请求，给批处理作业更低的请求（或者干脆隔离到专用节点），比在服务上设 CPU 限制有效得多。

5.8 实验 6：节点内存压力与驱逐

`experiments/e4/eviction.sh` 在 ent-worker 上用 `nodeName` 直接放三个 Pod（不经过调度器），用真实的内存占用制造压力：

Pod	QoS	内存请求	行为
guaranteed	Guaranteed	128Mi（等于限制）	保持 80 MiB
burst-hog	Burstable	64Mi，没有限制	从 100 MiB 起每秒增长 4 MiB
be-hog	BestEffort	无	保持 150 MiB；第一个 hog 停止后，脚本发信号让它每秒增长 3 MiB

这个设计检验的是驱逐排序的规则本身。kubelet 先看用量是否超过请求，再看优先级，最后看超出请求的多少（`experiments/e4/evict_rank.py` 按同样的规则给节点上的 Pod 排序），QoS 类别不在排序键里。如果

“BestEffort 先走”的说法成立，第一个被驱逐的应当是 be-hog；按规则，应当是超出请求最多的 burst-hog。预测和判据在实验前提交[22]。

第一次运行没有做成：脚本在 Pod 还处于 Pending 时就判断“没有运行中的 hog”而结束了。它却记录了一个值得知道的结果：用 `nodeName` 绑定的 Pod 绕过了调度器，绕不过 kubelet 自己的准入检查。ent-worker 可分配 2666 MiB 内存，已被请求 2402 MiB，请求 300Mi 的 Guaranteed Pod 被 kubelet 直接拒绝，原因是 `OutOfmemory`[23]。第二次运行把它改为 128Mi。

实验前这个节点的排序里[24]，超出请求的业务 Pod 只有一个：数据库副本 pg-main-1，工作集 290 MiB，请求 256Mi（kube-proxy 没有请求，也算超出，但它的优先级是 system-node-critical，排在所有优先级为 0 的 Pod 之后）。也就是说，这个节点如果在平时遇到内存压力，第一个被驱逐的会是数据库。

第二次运行的时间线（每约 2.5 秒采样一次可用内存、节点状态和 Pod 状态，[25]）：

时刻	事件
07:48:52	三个 Pod 都在运行，节点可用内存 13 577 MiB（阈值 12 720 MiB）
07:51:44	**驱逐 burst-hog**：“Container hog was using 794192Ki, request is 64Mi, has larger consumption of memory”。be-hog 保持在 160 MiB，没有被选中
07:51:52	节点状态变为 MemoryPressure；两秒后的采样中，节点已带上 `node.kubernetes.io/memory-pressure:NoSchedule` 污点
07:51:53	探测：绑定到这个节点的一个新 BestEffort Pod 被 kubelet 拒绝，状态 `Failed`、原因 `Evicted`：“Pod was rejected: The node had condition: [MemoryPressure].”
07:52:02	be-hog 开始每秒增长 3 MiB
07:56:26	**驱逐 be-hog**：“Container hog was using 954968Ki, request is 0”
08:01:32	MemoryPressure 解除，污点移除：第二次驱逐后 5 分 6 秒

实测数据

节点内存压力下的两次驱逐（秒，从 07:48:52 起算）

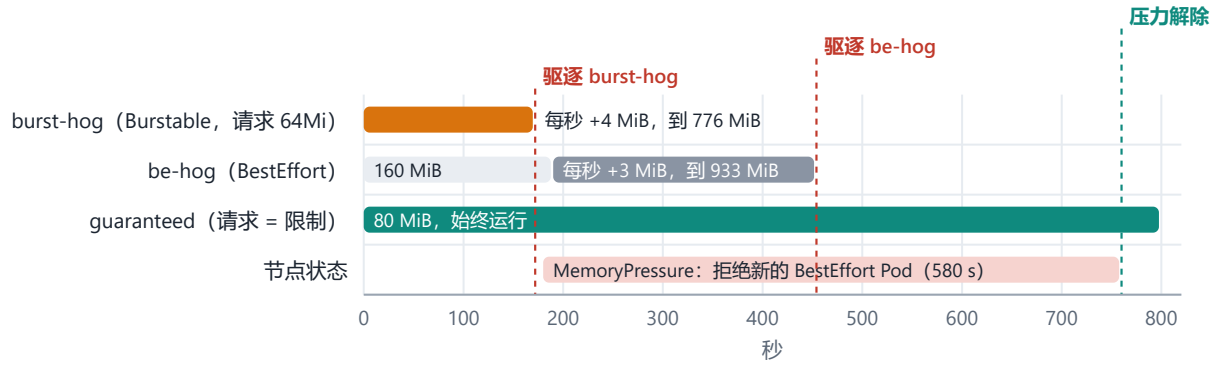


图 5-8 两次驱逐的时间线。超出请求最多的 burst-hog 先被驱逐，BestEffort 的 be-hog 开始增长之后才是第二个；请求等于限制的 Pod 自始至终没有受影响。节点的 MemoryPressure 状态持续了 580 秒：这段时间里它不接受新的 BestEffort Pod。

六项预测全部符合（OOM 的核对记录在 [26]）：

	预测	结果
V1	先驱逐 burst-hog，尽管 be-hog 是 BestEffort	符合：被驱逐时 burst-hog 用 776 MiB（超出请求 712 MiB），be-hog 用 160 MiB
V2	be-hog 开始增长后第二个被驱逐	符合，被驱逐时用 933 MiB
V3	其他 Pod 不受影响	符合：Guaranteed Pod 一直运行；实验命名空间之外没有驱逐事件；节点上其他 Pod 的状态和重启次数都没有变化
V4	kubelet 在越过阈值后很快动作，不发生内核的 OOM	符合：两次驱逐时 kubelet 观测到的可用内存比阈值低 1.8 MiB 和 9.2 MiB；没有容器因 OOM 被杀，内核日志中也没有 OOM 记录
V5	压力期间，新的 BestEffort Pod 在准入时被拒绝	符合
V6	压力状态在第一次驱逐后约 20 秒内出现，最后一次压力后约 5 分钟解除	符合：8 秒后出现，第二次驱逐后 5 分 6 秒解除

预测之外还有几点值得注意：

- **驱逐管理器的反应是即时的，慢在统计数据。**kubelet 统计接口中的可用内存大约每 10 秒更新一次。第一次越过阈值的那次更新（12 718 MiB）正是驱逐消息里的数字，驱逐紧接着发生。从越过阈值到驱逐，最长是一个统计周期，这期间内存继续增长的量就是超调。本实验的增长只有每秒 3–4 MiB，超调不到 10 MiB；增长快得多的进程（例如一次分配几百 MiB）会在 kubelet 看到之前就撞上内存上限，那时出手的是内核的 OOM killer，而不是有序的驱逐。驱逐阈值要留出“最快的增长速度 × 统计周期”的余量。
- **驱逐之前，内核已经在回收页缓存。**hog 增长的那几分钟里，同一节点上 pg-main-1 的页缓存从 411 MiB 降到 145 MiB，第二阶段又降到 115 MiB，工作集从 290 MiB 降到 161 MiB，而它的匿名内存（RSS）一直在 41–47 MiB[27]。内存压力最先伤害的不是被驱逐的 Pod，而是同节点上依赖页缓存的服务：数据库失去的缓存要靠之后的磁盘读取重新填满（本实验没有测量这对查询延迟的影响）。在 Kind 中，回收是由节点容器自己的 3 GiB 上限触发的：实验后虚拟机仍有约 5 GiB 可用内存[28]，节点 cgroup 自身记录了 6011 次触及上限（计数没有时间戳）。在真实节点上，起作用的是整机的空闲内存水位，页缓存同样最先被回收。
- **几秒钟的压力，换来近 10 分钟的 MemoryPressure。**两次真正低于阈值的时间各只有几秒，节点却从 07:51:52 到 08:01:32 一直处于 MemoryPressure：压力消失后，状态还要保持 `evictionPressure TransitionPeriod`（默认 5 分钟），以免来回抖动。这期间节点带着 NoSchedule 污点，调度器不会再往这里放没有相应容忍度的 Pod，kubelet 也拒绝绑定到这里的 BestEffort Pod。被拒绝的 Pod 状态是 `Failed`、原因 `Evicted`，看起来和被驱逐的 Pod 一样，其实它从未运行过。
- **被驱逐的 Pod 不会重启，日志也拿不回来。**两个 hog 停在 `Failed`（容器状态 `ContainerStatus Unknown`），`kubectl logs` 报错“container is terminated”。由 Deployment 管理的 Pod 会在别处重建，直接创建的 Pod 就没有了。本环境只收集服务主动通过 OTLP 发送的日志，这两个 Pod 的标准输出不在 Loki 里[29]；节点级的日志采集留给第 10 章（E9）。

时间与预期的差别：第一次驱逐在三个 Pod 都运行之后 172 秒，在预期的 50–200 秒之内；第二次在信号之后 264 秒，比预期的 150–250 秒稍长。原因还是页缓存的回收：第一次驱逐后节点的工作集只有 2159 MiB，be-hog 要增长约 770 MiB 才能让工作集回到阈值，按每秒 3 MiB 计算约 257 秒。

对实践的含义：给重要的 Pod，尤其是数据库，设置不低于实际用量的内存请求，或者干脆让它成为 Guaranteed，它就排到所有超出请求的 Pod 之后；请求很低的 Burstable Pod 并不比 BestEffort 更安全，超出得越多越先被驱逐。

5.9 实验 7：优先级与抢占

`experiments/e4/preemption.sh` 先建两个 PriorityClass：批处理 `e4-batch-low`（值 −10，`preemptionPolicy: Never`）和关键任务 `e4-critical`（值 100 000）。然后用批处理的 pause 容器（每个请求 1 核，实际不用 CPU）把三个工作节点的空余 CPU 占满：每个节点 3.75 核可分配、已请求 0.93–1.23 核，各放下 2 个，另外 6 个 Pending。最后创建 2 个关键任务 Pod，同样请求 1 核[30]：

```
02:35:19  critical-…-xmq4x  FailedScheduling  0/4 nodes are available: … 3 Insufficient cpu.
          preemption: found a potential placement for pod on node ent-worker3, preempting 1 victims
02:35:19  filler-…-kvfx8    Preempted         Preempted by pod 5d6b799e-… on node ent-worker3
02:35:20  critical-…-xmq4x  Scheduled         Successfully assigned e4-sched/critical-…-xmq4x to ent
-worker3
02:35:20  filler-…-2jmpb    FailedScheduling  … preemption: not eligible due to preemptionPolicy=Nev
er.
```

关键任务 Pod 创建后 1 秒被调度、3 秒后就绪；每个只抢占了一个批处理 Pod。被抢占的批处理 Pod 由它们的 ReplicaSet 重建，又因为 `preemptionPolicy: Never` 只能等待，不会反过来抢别人。

批处理的优先级设为 −10 而不是一个正数，是有意的。没有指定 PriorityClass 的 Pod（本书所有的 storefront 服务）优先级为 0；调度器选择牺牲者时，先保留优先级高的。如果批处理是 +100，比服务还高，关键任务需要腾地方时，被驱逐的会是 storefront 的 Pod。给批处理一个负的优先级，或者给所有在线服务一个明确的、更高的 PriorityClass，是共享集群的基本功。

5.10 实验 8：配额、默认值与拓扑分布

5.10.1 租户的配额与默认值

`experiments/e4/quota.sh` 为一个租户命名空间设置 ResourceQuota（最多 4 个 Pod、1 核请求、512Mi 内存请求、1Gi 内存限制）和 LimitRange（每个容器的默认请求 100m/64Mi、默认内存限制 128Mi、CPU 最多 500m）[31]：

- 一个没写任何资源的 Pod 被接受，LimitRange 为它补上了请求和限制：`{"limits":{"cpu":"500m","memory":"128Mi"},"requests":{"cpu":"100m","memory":"64Mi"}}`。**CPU 限制 500m 是没有人写过的**：LimitRange 只写了 `max.cpu`，没写默认限制时，Kubernetes 把最大值当作默认限制。结果这个租户的每个容器都带上了 CPU 限制，会在突发时被限流（实验 4）。

- 请求 2 核的 Pod 在准入时被拒绝：`must be less than or equal to cpu limit of 500m`，比较的正是那个被默认出来的限制。
- 6 个副本的 Deployment 本身被接受，但它的 ReplicaSet 只创建到配额用完为止：`exceeded quota: tenant, requested: pods=1, used: pods=4, limited: pods=4`，最终 3/6 就绪（另一个名额被第一个 Pod 占了）。配额拒绝的不是 Deployment，而是 Pod 的创建，所以要在 ReplicaSet 的事件里找原因。

5.10.2 拓扑分布：建议与要求

`experiments/e4/spread.sh` 把一个可用区（ent-worker2）设为不可调度，然后部署三组 3 副本的 Deployment[32]：

约束	ent-worker2 不可调度时	恢复调度 20 秒后
`ScheduleAnyway`（storefront Chart 的现状）	2 个可用区：1 + 2	不变（1 + 2）
`DoNotSchedule`	1 + 1，第 3 个 Pending	1 + 1 + 1（恢复的同一秒被调度）
`DoNotSchedule` + `nodeTaintsPolicy: Honor`	1 + 2	不变（1 + 2）

硬性约束下，第三个 Pod 宁可 Pending（“2 node(s) didn’t match pod topology spread constraints”）也不打破均匀：被禁止调度的节点仍然算作一个可用区，其中 0 个 Pod，再放一个就会使差距变成 2。加上 `nodeTaintsPolicy: Honor`，带污点（包括“不可调度”）的节点不再参与计算，Pod 可以挤进剩下的两个可用区。两种选择对应不同的风险：前者保证了失去任何一个可用区时副本不会全军覆没，代价是一个可用区不可用时扩容会卡住；后者优先保证副本数。还有一点：调度器只在放置新 Pod 时考虑分布，恢复之后并不会把已经挤在一起的 Pod 重新分散（那是 descheduler 的工作），软约束组和 Honor 组都停留在 1 + 2。第 4 章中 catalog 的两个副本落在同两个节点上，正是 `ScheduleAnyway` 加上不会自动重新分散的结果。

5.11 局限

- **主机在实验中途降频。**笔记本在持续负载下降到标称频率的 65%，同一服务每个请求的 CPU 变成约两倍。HPA 与 KEDA 的时间线、调度与抢占的时间不受影响；涉及绝对 CPU 的数字只在同一时段内比较（限流与邻居实验都是并排对照）。
- **入口是 L4。**外部流量经由 cloud-provider-kind 的 TCP 负载均衡器和 NodePort，新副本拿不到已有连接上的请求。生产环境的 L7 入口（或客户端的连接寿命限制）会改变实验 2 的结论；本章没有验证这一点，第 9 章（E8）会做。
- **负载生成器只有一个地址。**每次迭代新建连接的对照因客户端端口耗尽而失败。
- **只测了默认参数。**HPA 的容差、同步周期、稳定窗口，KEDA 的轮询间隔都用默认值或单一取值；调整它们的效果没有逐一测量。
- **驱逐实验只在一个节点上完整地做了一次**（实验 6；第一次运行因脚本的等待逻辑有误而中止），而且是在 Kind 节点这种“容器中的节点”上：节点的内存上限由 Docker 施加，kubelet 看到的总内存是整台虚拟

机的，驱逐阈值是第 1 章为此专门换算的；页缓存的回收也是由这个容器上限、而不是整机的内存水位触发的。增长速度只取了每秒 3–4 MiB，更快的增长下 kubelet 与内核 OOM killer 谁先动作没有测量。

- **日志只覆盖主动发送的服务。**OpenTelemetry Collector 只接收 OTLP，没有节点级的日志采集；被驱逐、被抢占的 Pod 和平台组件的标准输出不在 Loki 中（实验 6）。

5.11.1 从实验环境到生产环境

本章的做法	生产环境通常的做法	差别意味着什么
HPA 按 CPU 利用率扩容，metrics-server 每 15 秒采样	按请求速率、队列长度等更贴近负载的指标，配合定时或预测扩容	CPU 是滞后的信号：本章 5 分 26 秒的等待，在生产里可能就是一整个高峰
节点数固定为 3，Pod 只能挤进现有节点	集群自动伸缩器或 Karpenter 按需增加节点	本章没有“节点不够”的扩容路径，Pending 的 Pod 只能等
入口是第 4 层负载均衡（第 9 章改为第 7 层）	第 7 层入口，或让客户端定期重建连接	否则新副本接不到已有连接上的请求，扩容形同虚设
驱逐与抢占由专门制造的 Pod 触发，内存每秒增长 3–4 MiB	真实的内存泄漏、流量突增与批处理挤占	增长更快时，内核的 OOM killer 可能先于 kubelet 出手（本章没有测）

5.12 常见问题

HPA 的目标利用率应该设多少？它是“允许长期停留的水平”。在容差（默认 10%）之内 HPA 不会动作，所以真正开始扩容的点是目标的 1.1 倍；再加上负载不均（最忙的副本比平均高）和扩容的延迟（本章约 40 秒到几分钟），目标要比单个副本能承受的上限低得多。第 4 章测得网关在最忙的副本约 0.6 核以内延迟平坦，所以本章的目标是平均 0.5 核。

为什么不直接把 `minReplicas` 设大一点？这是最可靠的办法之一：预留的副本不需要反应时间，也不受“新副本拿不到连接”的影响。代价是闲置的容量。实践中两者结合：最小副本数按日常峰值和可用区冗余设定（第 4 章：失去一个可用区后仍能承受），HPA 负责超出部分。

CPU 限制什么时候是合理的？当你需要硬性的隔离（多租户的不可信工作负载）或者要按限制计费时。对延迟敏感、自己团队管理的服务，用请求保证份额，不设 CPU 限制；如果必须设，把它设在峰值之上并监控被限流的周期比例。内存限制则不同：超出时进程被杀，而不是被暂停，应当始终设置。

KEDA 和 HPA 能同时管理一个 Deployment 吗？不能，它们会争夺副本数。KEDA 本身就是通过创建一个 HPA 来工作的；需要同时按 CPU 和队列扩容时，在同一个 ScaledObject 中写两个触发器（KEDA 也支持 CPU 触发器），KEDA 取要求副本最多的那个。

优先级会影响已经在运行的 Pod 吗？只在两种时刻起作用：调度新 Pod 时（抢占），以及节点资源不足时（驱逐排序中，同样超出请求的 Pod 先驱逐优先级低的）。它不会让运行中的 Pod 得到更多 CPU，那是请求（权重）决定的。

5.13 练习

练习

1. **（计算）** 一个服务有 2 个副本，CPU 请求 500m，HPA 目标利用率 100%，容差为默认值。负载使每个 Pod 平均用 0.53 核时，HPA 会做什么？0.58 核时呢？如果希望平均 0.55 核时一定扩容，目标应当怎样设？
2. **（分析）** 实验 2 中扩出的网关 Pod 一个请求也没有收到。给出三种让新副本分担流量的做法，并说明各自的代价。
3. **（设计）** 为通知服务设计一个在稳定负载下不会来回振荡的扩缩容规则，并说明你会用什么实验验证它。
4. **（排障）** 一个团队说：“我们的服务平均 CPU 只有限制的 60%，延迟却有规律的尖峰。”你会看哪些指标？怎样确认是 CFS 限流？还要检查集群里的什么配置？
5. **（计算）** 一个 4 核的节点上，网关（请求 500m）、三个请求 50m 的服务，与一个请求 2 核、想用满 4 核的批处理作业同时竞争 CPU。按 cgroup 权重估算竞争时各自能分到的 CPU，并说明为什么网关的实际延迟比这个估算差得多。
6. **（分析）** 节点内存紧张时，一个没有请求的 Pod 用了 300 MiB，一个请求 64Mi 的 Pod 用了 500 MiB，一个请求与限制都是 300Mi 的 Pod 用了 200 MiB，三个 Pod 优先级相同。kubelet 会按什么顺序驱逐？如果第二个 Pod 的优先级更高呢？
7. **（设计）** 为多个团队共享的集群设计优先级与配额方案，覆盖在线服务、批处理、平台组件和实验性工作负载：每一类的 PriorityClass 与 `preemptionPolicy`、ResourceQuota 和 LimitRange 怎样设置？怎样避免实验 8 中“隐式的 CPU 限制”？

参考答案见附录。

5.14 本章小结

- 把副本数交给 HPA 本身就是一次变更：客户端方式应用删除 `replicas` 字段后，Deployment 先缩到 1 个，约 7 秒只剩一个就绪副本。用服务端应用，或者先 `ignoreDifferences` 再删除字段，可以避免这段空窗。
- HPA 从决定到新 Pod 就绪只要 8 秒，慢的是决定本身：利用率停在目标的 102%–114%，在 10% 的容差边缘，扩容等了 5 分 26 秒；缩容在最后一次“需要更多副本”的计算之后 300 秒（稳定窗口）才发生。目标利用率是允许长期停留的水平，而不是开始扩容的水平。
- 扩容不等于分担负载：L4 入口加上保持连接，扩出的两个网关 Pod 一个请求也没有收到，这一档 20.5% 的请求失败，而 HPA 看到的平均 CPU（约 0.43 核）“正常”。会快速拒绝请求的服务，CPU 低估了需求。
- 积压只说明“排着多少”：只按积压扩缩容，恒定负载下通知服务两次被缩到 1 个，之后一分钟的通知 p95 约 20 秒；加上按到达速率计算的触发器后，副本数稳定在 3，p95 保持在 0.48–0.73 秒。
- CPU 限制：平均用量（0.28 核）低于限制（0.3 核），仍有 67% 的周期被限流，事件循环延迟 p99 从约 6 ms 升到 96 ms。对延迟敏感的服务，用请求保证份额，不设 CPU 限制。

- 请求就是 CPU 不够时的份额：同一个占满 4 个 CPU 的消耗者，不声明请求时几乎无害；声明 2 核请求时，同节点网关的 p50 是 131 ms（其他节点 14 ms），orders 的 p99 达到 6 秒，网关对 orders 的熔断器打开。
- 驱逐按“是否超出请求 → 优先级 → 超出多少”排序，QoS 类别不是排序键：请求 64Mi、用了 776 MiB 的 Burstable Pod 先于只用 160 MiB 的 BestEffort Pod 被驱逐，事先提交的六项预测全部符合。kubelet 在第一次看到可用内存低于阈值时就动作（超调不到 10 MiB），但统计约 10 秒才更新一次；驱逐之前，内核已经回收了同节点数据库约 70% 的页缓存（411 → 115 MiB）；两次各几秒的压力，让节点近 10 分钟处于 MemoryPressure。
- 抢占在 1 秒内完成，每个关键任务 Pod 只牺牲一个批处理 Pod。批处理要用负的优先级和 `preemptionPolicy: Never`，否则优先级高于默认 0 的批处理会让在线服务成为牺牲者。
- 默认值会悄悄改变行为：LimitRange 只写 `max.cpu` 时，最大值成为每个容器的默认 CPU 限制。拓扑分布的硬约束宁可让 Pod Pending 也不打破均匀，调度器也不会在节点恢复后重新分散已经挤在一起的 Pod。
- 主机在实验中途降频到标称频率的 65%，每个请求的 CPU 翻倍：绝对数字只能在同一时段内并排比较。

第 6 章

多租户与安全治理（E5）

到第 5 章为止，这个集群只有一个团队、一个管理员：谁都能做任何事，任何 Pod 都能连到数据库。本章把它变成多个团队共享的平台：每个人有自己的身份，每个团队只能动自己的命名空间，网络默认不通，不合规的 Pod 进不来，秘密从专门的存储中按需取得。每一道边界都用越权的尝试去测试，并用审计日志回答“谁做了什么”。

工程问题

集群不再只有一个团队。怎样让每个团队只能动自己的东西——在 API、网络和密钥三个层面——并且事后能从记录里回答“谁做了什么”？

实测结论

- 越权测试全部符合预期：Kubernetes 权限 58 项、Argo CD 权限 24 项、网络可达性 45 项；预期在施加策略之前提交。
- 能创建 Pod，就能读同一命名空间里的 Secret：RBAC 对此无能为力，边界要靠命名空间和外部密钥管理来划。
- 网格改变了 NetworkPolicy 的规则：流量到达 15008 端口，经过 waypoint 时来源是 waypoint，新策略不切断已有的连接。
- 准入引擎是全集群的依赖：单副本、不限范围的 Kyverno 重启一次，全集群 34 秒不能创建 Pod。
- 密钥轮换时，Secret 8 秒内更新，挂载的文件 61 秒后更新，环境变量要等 Pod 重启。

6.1 学习目标

学完本章，你应当能够：

- 为多个团队设计命名空间、角色和委派授权，并用测试矩阵验证“不该做的做不了”；
- 从 API 服务器的审计日志中还原一段时间内谁做了什么、为什么被允许或拒绝，并把 GitOps 的变更追溯到 Git 中的作者；
- 在服务网格中正确编写 NetworkPolicy，并知道它对已有连接不起作用；
- 比较 Kyverno（webhook）与内置的 ValidatingAdmissionPolicy，设计不会拖垮整个集群的准入策略；
- 用外部密钥存储为每个团队提供各自的秘密，并测量轮换在 Secret、挂载文件和环境变量上的传播。

实验性质

本章的实验都是 `[真实]` 运行的。两个“团队”和四个用户是 `[模拟]` 的组织结构：用户是集群 CA 签发的客户端证书，不是来自企业身份系统（本环境没有单点登录）；insight 团队的服务只是一个最小的 HTTP 服务，用来检验隔离。所有密钥值都是实验中随机生成的测试值，记录中只出现它们的哈希。

6.2 原理

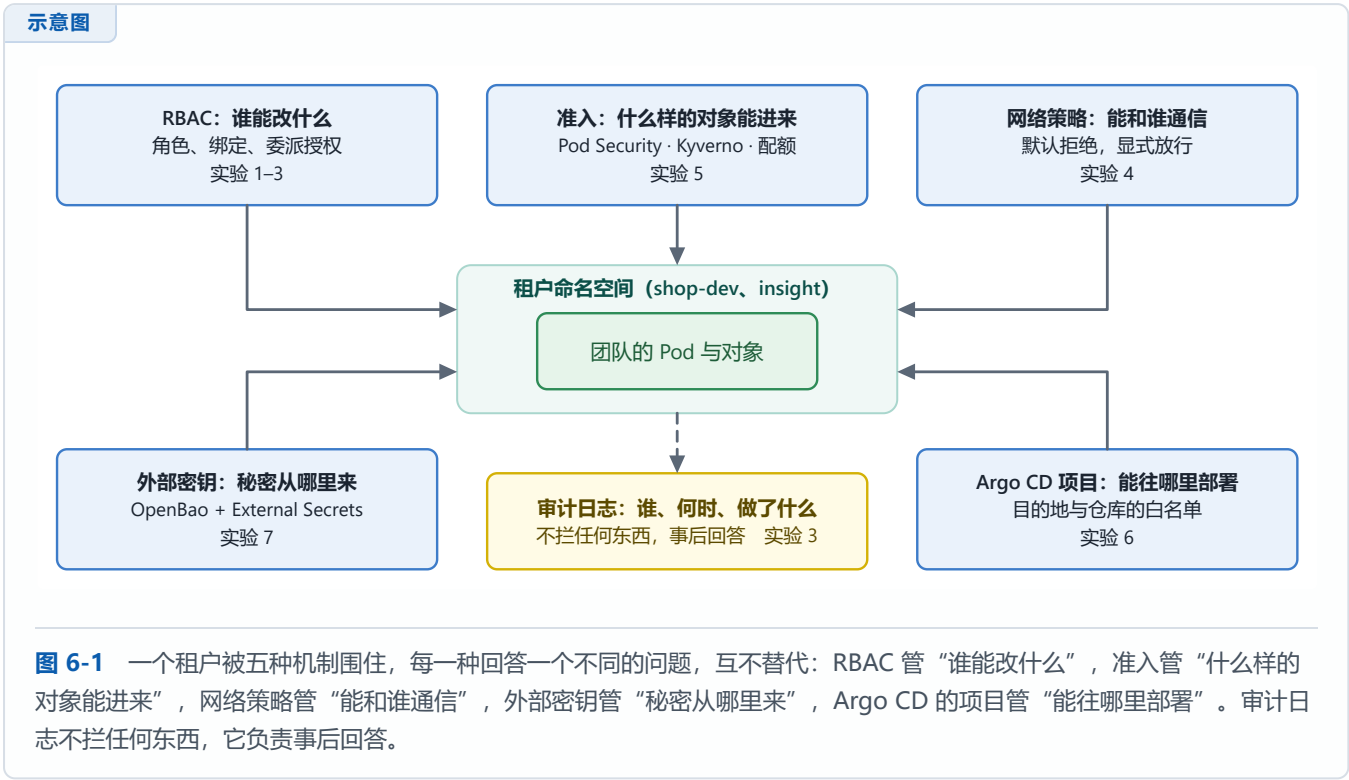


图 6-1 一个租户被五种机制围住，每一种回答一个不同的问题，互不替代：RBAC 管“谁能改什么”，准入管“什么样的对象能进来”，网络策略管“能和谁通信”，外部密钥管“秘密从哪里来”，Argo CD 的项目管“能往哪里部署”。审计日志不拦任何东西，它负责事后回答。

6.2.1 身份、认证与授权

Kubernetes 不保存用户。API 服务器只验证请求者出示的凭据：客户端证书（主体名 CN 是用户名，组织 O 是组）、服务账号令牌，或者外部身份系统签发的 OIDC 令牌。认证之后，授权器（RBAC）只回答一个问题：这个用户（及其所在的组）能不能对这个资源做这个动作。RBAC 只有“允许”，没有“拒绝”：一个权限不存在，是因为没有任何角色授予它。

两条容易被忽略的规则决定了 RBAC 的实际边界：

- **防止提权。**创建 RoleBinding 时，API 服务器要求创建者本身拥有被绑定角色的全部权限，或者拥有对这个角色的 `bind` 动词。这使“团队自己管理成员”成为可能而不危险。
- **能创建 Pod，就能读命名空间里的 Secret。**Pod 可以把任何 Secret 挂载为文件或环境变量，也可以以命名空间中任何服务账号的身份运行。RBAC 对此无能为力。

6.2.2 准入

通过授权的请求还要经过准入：先是修改型（例如 LimitRange 为没有声明资源的容器补上默认值），再是校验型。内置的 Pod Security Admission 按命名空间标签执行三个级别的安全基线；更细的规则由策略引擎实现，要么是外部的 webhook（Kyverno），要么是 API 服务器内置、用 CEL 表达式编写的 ValidatingAdmissionPolicy。webhook 是一个必须在线的外部服务：它不可用时，要么拒绝请求（Fail），要么放行（Ignore）。

示意图

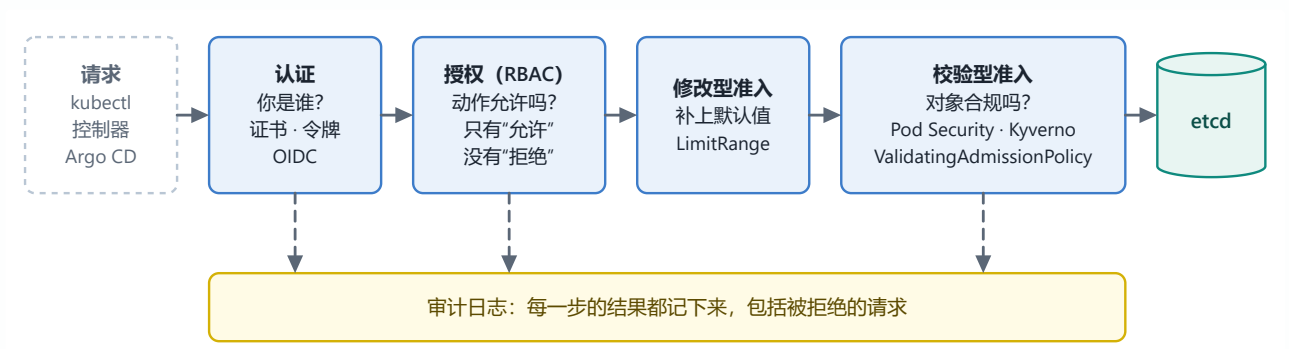


图 6-2 一个请求进入 API server 要过的四道关。认证回答“你是谁”，授权回答“这个动作允许吗”，准入回答“这个对象合规吗”；审计把每一步的结果记下来，包括被拒绝的请求。

6.2.3 网络

Kubernetes 的网络默认全通。NetworkPolicy 按标签选择 Pod，规定允许哪些来源（入站）和目的地（出站）、哪些端口；被任何策略选中的 Pod 只接受被明确允许的流量。执行策略的是 CNI 插件（本环境是 kindnet），它决定了策略的许多细节，例如怎样对待节点自身的流量、怎样对待已经存在的连接。在服务网格中，流量的形态也变了：本环境的 ambient 网格把 Pod 之间的连接封装在端口 15008 的 HBONE 隧道里，经过 waypoint 时，目的地看到的来源是 waypoint。

6.2.4 秘密

Kubernetes 的 Secret 只是 base64 编码的对象，任何能读它、或者能在同一命名空间创建 Pod 的人都能拿到内容。外部密钥管理把秘密放在专门的存储中（本章是 OpenBao，HashiCorp Vault 的开源分支），按工作负载的身份授权，记录每一次读取，再由 External Secrets Operator 把需要的值复制成 Secret。

6.3 实验 1：租户与身份

租户的结构写在 Git 中（`gitops/platform/manifests/tenants/`），由平台的 Argo CD 应用部署：

命名空间	团队	团队的权限
storefront	team-shop（生产）	只读和日志：生产环境的变更只经过 Git
shop-dev	team-shop（开发）	开发者：管理工作负载、进入容器调试，可以把本命名空间的开发者或只读角色授予他人
insight	team-insight	同上

两个新命名空间都强制执行 Pod Security 的 restricted 级别，带有配额和容器默认值（按第 5 章的教训：内存给默认限制，CPU 只给默认请求，不写 `max.cpu`）。平台运维组 platform-sre 可以读整个集群（不含 Secret）并做运维操作（重启、驱逐、封锁节点、调整配额与网络策略），但不能修改 RBAC；审计组 auditors 只读，包括 RBAC 对象；只有 break-glass 组拥有集群管理员权限。

用户由 `platform/tenants/users.sh` 通过 Kubernetes 的证书签名请求接口签发[2]：alice（team-shop）、bob（team-insight）、carol（platform-sre）、dave（auditors），证书有效期 7 天；carol 的 break-glass 证书另外签发，只有 1 小时。私钥和 kubeconfig 只在 `~/.lab-secrets/v2/users/` 中（权限 600），记录里只有主体、组、有效期和指纹。

证书有一个根本的局限：Kubernetes 不能吊销客户端证书，有效期就是唯一的限制。break-glass 证书过期后的请求被拒绝（[3]，证书到期 09:32:35，09:49:01 的请求得到 `error: You must be logged in to the server (Unauthorized)`），这正是期望的行为；生产环境应当用 OIDC，账号在身份系统中停用即失效。

6.4 实验 2：权限矩阵

`experiments/e5/rbac-matrix.tsv` 在测试之前写下每个用户能做什么，`rbac-matrix.sh` 用每个用户自己的证书逐行询问 API 服务器（`kubectl auth can-i`，即 SelfSubjectAccessReview），答案来自授权器本身。

第一次运行有 3 项不符[4]：carol 不能 `create pods/exec`、不能 `create pods/eviction`，alice 却能 `create serviceaccounts/token`。问题出在测试本身：`kubectl auth can-i` 把 `pods/exec` 理解为“名为 exec 的 Pod”，子资源要用 `--subresource`。carol 确实不能创建名为 exec 的 Pod，alice 确实能创建名为 token 的服务账号。更糟的是，看起来通过的 `get pods/log` 也是在测“名为 log 的 Pod”。修正写法之后，52 项全部符合[5]。

实验 3 发现 `kubectl exec` 还需要 `get` 动词（见下文），角色修正后矩阵增加到 55 项；引入外部密钥管理后又增加 3 项（团队可以管理 ExternalSecret，不能管理 SecretStore，仍然不能读 Secret）。最终 58 项全部符合预期[3]，其中包括：

用户	请求	预期	原因
alice	在 storefront 进入容器	否	生产环境不给 shell
alice	在 shop-dev 授予 tenant-viewer	是	委派授权
alice	在 shop-dev 授予 cluster-admin	否	不能授予自己没有的权限
alice	删除自己的命名空间	否	命名空间由平台管理
bob	读 storefront 的 Pod	否	其他团队的命名空间
carol	驱逐 storefront 的 Pod	是	运维：排空节点
carol	读 storefront 的 Secret	否	读 Secret 需要 break-glass
dave	列出所有 RoleBinding	是	审计需要读 RBAC
dave	删除 Pod	否	只读

6.5 实验 3：审计日志回答“谁做了什么”

API 服务器的审计策略是第 1 章建集群时写好的（`platform/cluster/audit-policy.yaml`）：人的读操作记录元数据，写操作记录请求体，RBAC、准入和网络策略的写操作记录请求和响应；Secret、ConfigMap 和进入容器（exec）只记录元数据，以免秘密进入日志。

`experiments/e5/who-did-what.sh` 以四个用户的身份发出一组真实的请求，然后只用审计日志还原同一段时间[6]。可能造成破坏、预期被拒绝的请求用 `--dry-run=server`：授权和审计与真实请求完全相同，即使角色写错了也不会改变任何东西。审计记录中有一个特别有用的字段 `authorization.k8s.io/reason`，它写明是哪个绑定、哪个角色允许了这个请求，例如 `RoleBinding "team-shop-viewer/storefront" of ClusterRole "tenant-viewer" to Group "team-shop"`。

所有越权请求都如预期被拒绝。还原出的时间线中有几处值得细看：

- **能创建 Pod，就能读 Secret。**平台在 shop-dev 中放了一个测试 Secret。alice 直接读取被拒绝（403），随后她创建了一个以环境变量引用这个 Secret 的 Pod（201），Pod 打印出的值的哈希 `91fecd34d314` 与平台写入的值相同。审计日志忠实地记录了两步，但单看任何一步都不是违规。保护 Secret 的办法是让团队根本拿不到不该有的 Secret：按命名空间隔离，Secret 只包含这个团队需要的值，由外部存储按团队授权（实验 7）。
- **“允许”不等于成功。**alice 尝试在 shop-dev 中授予自己 cluster-admin，审计记录是 `403`，授权决定却是 `allow`。授权器允许了“创建 RoleBinding”这个动作，拒绝来自 RBAC 自身的提权检查（“attempting to grant RBAC permissions not currently held”）。分析审计日志时，要看响应码，而不只是授权决定。
- **Pod Security 在 Deployment 上不拒绝。**alice 创建了一个没有 securityContext 的 Deployment，它被接受了，只返回一条警告；被拒绝的是 ReplicaSet 随后创建 Pod 的请求（`violates PodSecurity "restricted:latest"`）。审计记录在 Deployment 的创建上留下了 `pod-security.kubernetes.io/audit-violations` 注解。用户看到的是“部署成功、没有 Pod”。
- **进入容器的命令行在审计日志里。**carol 进入 storefront 的一个 Pod 时，日志中先有一条 `get pods/exec` 被拒绝（403），紧接着一条 `create pods/exec` 成功（101）：kubectl 1.37 先用 WebSocket 方式（GET，以 `get` 动词授权），被拒后回退到 SPDY（POST，`create`）。角色因此改为同时授予两个动词，之后同样的操作只有一条 `get pods/exec 101`[7]。记录的 `requestURI` 包含完整的命令行（URL 编码），`command=python&command=-c&command=print%28%27carol+was+here%27%29`[8]：谁在哪个 Pod 里运行了什么，元数据级别的审计就能回答，这也意味着命令行里不能出现秘密（实验 7 的脚本因此只通过标准输入传递令牌）。
- **冒用身份时两个身份都在。**管理员以 alice 的身份（`--as=alice`）列出 Pod，记录中用户是 `kubernetes-admin`，`impersonatedUser` 是 alice，授权理由是 alice 的绑定。
- **不是每个 403 都是越权尝试。**`kubectl apply` 在创建 Pod 之前读取了命名空间对象，租户没有这个权限，审计中就多了一条 403。告警规则要区分这类“噪声”。

6.5.1 GitOps 的变更是谁做的

通过 Git 发生的变更，在 Kubernetes 的审计日志中只有一个身份：Argo CD 的应用控制器。给 insight 的 Deployment 加一个标签，追溯链是这样的[9]：

时间（UTC）	记录	身份
08:35:32	GitHub 提交 cbe7288	Weilai Wang
08:35:35	GitLab 配置仓库提交 b511e37（发布脚本）	Platform Publisher
08:35:37	Argo CD 自动同步到 b511e37	automated
08:35:37.985	审计：`patch deployments insight/insight-api`	`system:serviceaccount:argocd:argocd-application-controller`

链条是完整的，但有两处薄弱：配置仓库中的提交者是发布脚本，人的名字只在提交信息里；执行变更的控制器拥有 `* on *` 的集群权限（同一记录），Kubernetes 的授权对它不起限制作用。第一处由第 9 章（E8）的合并请求流程解决；第二处见实验 6。

6.6 实验 4：网络隔离

6.6.1 基线：全通

`experiments/e5/reach.sh` 在多个命名空间中放置探测 Pod，逐一尝试连接各个目标，并要求目标在应用层作答：HTTP 状态行、NATS 的 INFO 问候、Valkey 对 PING 的回复、PostgreSQL 对 SSL 请求的回复（`reach_probe.py`）。只看 TCP 握手是不够的：在 ambient 网格中，Pod 本地的 ztunnel 会先接受连接，再去连目的地。没有任何策略时，36 个组合全部可达[10]：任何命名空间的任何 Pod 都能直接连到 PostgreSQL、Valkey、NATS 和 Prometheus。

6.6.2 先弄清执行者：kindnet

在两个一次性的命名空间中逐条加规则（`experiments/e5/np-probe.sh`、`mesh-np.sh`），网格外的结果符合常识：默认拒绝之后连接超时，允许 8080 后恢复[11]。服务端的就绪探针从头到尾都通过，即使在默认拒绝之下。原因在 kindnet 的 nftables 规则[12]：只有被策略选中的 Pod 的地址进入检查集合，其报文交给用户态程序判断；节点上 root 拥有的套接字发出的报文直接放行，所以 kubelet 的探针永远不受影响；一条连接一旦被放行（或者在 Pod 被策略选中之前就已经存在），会被打上连接跟踪标签，之后的报文不再检查。

6.6.3 网格中的三条规则

网格内的第一次测试令人意外：加上默认拒绝之后，客户端仍然能访问服务端。逐一检验后得到三条规则[13, 14]：

实验	设置	结果
A1	无策略，发一个请求	成功；连接跟踪中出现一条 客户端 → 服务端:15008 的连接
A2	加默认拒绝，同一个客户端	**成功**：还是同一条连接（同一源端口，ESTABLISHED）
A3	重建客户端 Pod（新连接）	超时
C1	默认拒绝 + 允许客户端访问 8080（新客户端）	超时
C2	默认拒绝 + 允许客户端访问 15008（新客户端）	成功
C3	网格外的客户端访问 8080	成功
B1	有 waypoint；允许客户端 → waypoint、客户端 → 服务端	超时
B2	再允许 waypoint → 服务端 15008	成功；连接跟踪中的来源是 waypoint 的地址

1. **策略不切断已有的连接。**ztunnel 为每一对 Pod 保持一条长期的 HBONE 连接，新的请求都在这条连接里复用，所以在网格中，一条新策略对已经通信过的 Pod 往往根本不起作用，直到连接因 Pod 重建等原因断开。这也意味着：策略里漏掉的规则，可能要等几小时后某个 Pod 重启才暴露出来。
2. **网格内的流量到达 15008。**只允许应用端口的“经典”规则会挡住网格内的流量；网格外的客户端则仍然到达应用端口。
3. **有 waypoint 时，服务端看到的来源是 waypoint。**允许“客户端 → 服务端”没有用。

示意图

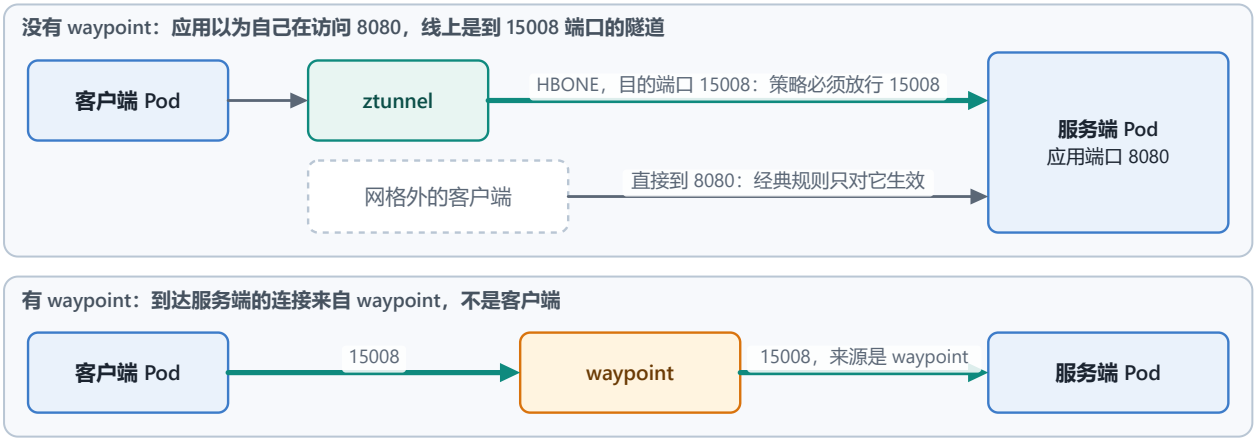


图 6-3 网格中 NetworkPolicy 看到的流量。应用以为自己在访问 8080，线上走的是到 15008 端口的隧道；有 waypoint 时，到达服务端的连接来自 waypoint 而不是客户端。按“客户端 → 服务端:8080”写的经典规则，对网格内的流量不起作用，对网格外的客户端才生效。

得到这些结论之前，探测本身错了两次，都按原样保留在记录中。第一次，探测用 HTTP/1.0，waypoint（Envoy）拒绝 HTTP/1.0 并自己回复 `426 Upgrade Required`[15]，探测把这个回复算作“可达”，于是 waypoint 后面的结果全部偏乐观。修正后的探测使用 HTTP/1.1，并且识别代理自己生成的回复：经过 Envoy

转发的回复带有 `x-envoy-upstream-service-time` 头[16]，Envoy 自己的回复（426、503、403）没有。所有受影响的实验都用修正后的探测重新做了一遍。

6.6.4 平台的策略

按测得的规则，平台为三类命名空间写了策略（`gitops/platform/manifests/tenants/network.yaml`、`gitops/platform/manifests/data/networkpolicy.yaml`）：

- **租户（shop-dev、insight）**：出入都默认拒绝，只放行 DNS 和本命名空间内部。
- **storefront（网格内，有 waypoint）**：本命名空间的所有端口（覆盖 15008 和 waypoint）、monitoring 访问 8080 和 15020（指标），以及网关的 8080 对任何来源开放（负载均衡器的入口）。
- **data**：每个消费者只开放它需要的端口：storefront 访问 5432、6379、4222，Prometheus 访问三个导出器的端口，KEDA 访问 NATS 的监控端口 8222，CNPG 运维程序访问 8000 和 5432，复制和集群流量限于本命名空间。消费者是从正在运行的系统中查出来的（Prometheus 的抓取目标、KEDA 触发器的配置）。

施加策略之前，`experiments/e5/reach-expected.tsv` 写下了 45 个组合的预期并提交（cf28188）。施加之后，45 个组合全部符合[17]：租户只能访问 DNS 和自己；storefront 保有它的数据服务和平台服务，访问不到其他租户；任意其他命名空间只能访问公开的网关；monitoring 能访问服务的指标端口，访问不到数据库的客户端端口。

探测用的都是新连接，平台自身的连接却可能早于策略存在（规则 1）。`experiments/e5/net-validate.sh` 因此强制所有客户端重新连接：逐个替换 storefront 的每个 Pod（每个 Deployment 始终保持副本就绪），重启 KEDA、Prometheus 和 CNPG 运维程序，然后检查[18, 19]：20 迭代/秒、60 秒的订单流量 0 失败；data、storefront、keda 中的所有抓取目标在线；KEDA 的 ScaledObject 就绪，重启后没有错误；PostgreSQL 3/3 就绪，主库看到两个副本都在流复制；运行前后两分钟内确认的订单 305 笔、发出的通知 304 条，积压为 0。脚本本身有两处错误（通知数读错字段、复制状态问了一个副本），已修正并重新核对，原记录保留。

6.7 实验 5：准入策略：Kyverno 与内置策略

Kyverno 1.19.2 经 GitOps 安装（镜像改从 ghcr.io 拉取：默认的 reg.kyverno.io 在本环境没有镜像缓存）。平台的策略只作用于带 `tenant` 标签的命名空间（`gitops/platform/manifests/policies/tenant-policies.yaml`）：镜像必须来自平台的镜像仓库并带有固定的标签（不能是 latest）；每个容器必须有内存限制和 CPU 请求；新建的租户命名空间自动获得默认拒绝的网络策略、配额和容器默认值。

`experiments/e5/admission.sh` 用 `--dry-run=server` 在 shop-dev（Kyverno）和一个绑定了同样规则的 ValidatingAdmissionPolicy（CEL 写成，API 服务器内置）的命名空间中各创建六种 Pod[20]：

Pod	shop-dev	内置策略的命名空间
合规	接受	接受

Pod	shop-dev	内置策略的命名空间
来自 docker.io 的镜像	Kyverno 拒绝	ValidatingAdmissionPolicy 拒绝
标签 latest、没有标签	Kyverno 拒绝	ValidatingAdmissionPolicy 拒绝
没有声明资源	接受，LimitRange 补上了默认值	接受（没有默认值）
没有 securityContext	Pod Security 拒绝	Pod Security 拒绝

顺序体现在结果中：LimitRange 是修改型准入，先于 Kyverno 的校验，所以没有声明资源的 Pod 带着默认值通过了“必须有资源”的规则；Pod Security 是内置插件，在 webhook 之前就拒绝了请求。

两种机制的代价不同。按 API 服务器的指标，Kyverno 的校验 webhook 平均每次 4.04 ms；内置策略只记录了做出拒绝的 3 次检查，平均 0.15 ms（它不记录放行的检查，两组数字不是同一类请求，只能说明数量级）。更重要的发现是调用次数：40 个 Pod 在 shop-dev、40 个在另一个命名空间，Kyverno 的 webhook 被调用了 80 次。它看到了每一个命名空间的每一个 Pod，包括没有任何 Kyverno 策略适用的地方。

6.7.1 当策略引擎重启时

webhook 的失败策略是 Fail：Kyverno 不在线时，请求被拒绝。`admission.sh` 删除唯一的 Kyverno 准入控制器 Pod，同时每秒在三个命名空间各创建一个合规的 Pod：

配置	删除一个准入控制器 Pod 后
1 个副本，webhook 覆盖除 kube-system 外的所有命名空间	**所有命名空间**都不能创建 Pod，34 秒（09:15:54–09:16:28）；包括没有 Kyverno 策略的 storefront
1 个副本，webhook 只覆盖带 `tenant` 标签的命名空间	两个租户命名空间 42 秒不能创建 Pod，storefront 不受影响[21]
2 个副本 + PodDisruptionBudget	只有删除瞬间的 1 个请求失败[22]

单副本、覆盖全部命名空间的策略引擎，每次重启或升级都是一次全集群的“不能创建 Pod”：扩容、滚动更新、被驱逐的 Pod 重建全部失败。缩小 webhook 的范围把影响限制在需要策略的地方，多副本消除了大部分中断；即使如此，正在被删除的 Pod 仍可能让一个请求失败，调用方需要重试。新 Pod 就绪后，准入恢复所需的时间两次分别是 4 秒和 20 秒，本章没有分析差异的原因。

生成规则的表现很好：新建的租户命名空间在 1.1–1.5 秒内获得 4 个默认对象；手工删除其中一个网络策略，3 秒内被恢复[23]。

6.8 实验 6：Argo CD 的项目与权限

Argo CD 有自己的权限：谁能查看、同步、修改哪个项目的应用。本环境没有单点登录，权限规则用与 Kubernetes 相同的组编写（`gitops/bootstrap/argocd-values.yaml`），用 `argocd admin settings rbac can` 离线检验，不需要登录（`experiments/e5/argocd-rbac.sh`）。24 项检查全部符合预期[24]：team-insight 可以管理 insight 项目中的一切，看不到也同步不了 storefront；team-shop 可以查看和同步

storefront，不能修改或删除应用的定义；审计组只读；没有默认角色。第一次运行时，脚本把命令的警告和退出信息当成了答案[25]，修正后重做。

项目本身的边界由 AppProject 规定：insight 项目只能部署到 insight 命名空间，只能使用平台的配置仓库，只能创建五种命名空间级的资源。三次越界尝试都被拒绝[26]：部署到 storefront 命名空间（“do not match any of the allowed destinations”）；使用外部仓库（“is not permitted in project”）；部署平台的租户配置（同步失败，命名空间、集群角色、网络策略、配额等每一项都“not permitted in project insight”），没有任何对象易主。

这道边界完全由 Argo CD 自己执行。真正向 Kubernetes 写入的是应用控制器，它拥有集群的全部权限（实验 3）。Argo CD 被攻破或配置错误时，Kubernetes 的授权不会拦住它。Argo CD 支持同步时冒用每个项目自己的服务账号（impersonation），把边界落到 Kubernetes 的 RBAC 上；这需要为所有项目（包括平台自身）配置服务账号，本章没有启用，列为待办。

6.9 实验 7：外部密钥管理

OpenBao 2.7.2 和 External Secrets Operator 2.14.0 经 GitOps 安装。OpenBao 只有一个节点，使用内置的 raft 存储；初始化、解封和配置由 `platform/secrets/openbao-init.sh` 完成，唯一的解封密钥和根令牌保存在 `~/.lab-secrets/v2/openbao/`，只经标准输入传入容器（实验 3：exec 的命令行会进入审计日志）。部署过程中遇到三处与旧资料不同的行为，都保留了记录（[27] 到 [28, 29]）：OpenBao 2 没有 file 存储（“unknown storage type file”），改用 raft；`bao operator unseal -` 把短横线当作密钥本身，改为 `bao write sys/unseal key=-` 从标准输入读取；OpenBao 2.7 不允许通过 API 启用审计设备（持有令牌的人不能借此改变审计去向），审计设备写在服务器配置中。OpenBao 每次重启后处于密封状态，同一个脚本把它重新解封[28]；生产环境应当用多份密钥或 KMS 自动解封。

每个租户命名空间有一个服务账号 `secrets-reader` 和一个 SecretStore；OpenBao 的 Kubernetes 认证把角色 insight 绑定到 `insight/secrets-reader`，只能读 `secret/insight/*`，shop 同理。只有 External Secrets 所在的命名空间可以连接 OpenBao。两边都在网格中，OpenBao 的监听器不启用 TLS，由 ztunnel 加密：ztunnel 记录的入站连接，来源身份是 `spiffe://cluster.local/ns/external-secrets/sa/external-secrets`，目的身份是 `spiffe://cluster.local/ns/secrets/sa/openbao`。

`experiments/e5/secrets.sh` 的结果[30]：

- **送达**：平台写入 insight 的 API 密钥后 66 秒，insight 命名空间中的 Secret 持有这个值。刷新间隔是 30 秒，多出的时间是因为在值写入之前，ExternalSecret 已经因为路径不存在而失败、处于退避状态。
- **越界请求**：bob 在 insight 中创建了一个指向 `shop/db-password` 的 ExternalSecret。OpenBao 拒绝了，审计日志中记录的是 `kubernetes-insight-secrets-reader (insight/secrets-reader) read secret/data/shop/db-password ERROR`（permission denied）。External Secrets 按退避重试，OpenBao 一共拒绝了 7 次读取[29]；没有生成 Secret。
- **轮换**：平台写入新值后，Secret 在 8 秒内更新；挂载在 Pod 中的文件在 61 秒后更新（kubelet 定期同步挂载的 Secret）；环境变量一直是旧值，只有重启 Pod 才会更新。需要轮换的凭据应当以文件挂载，并由

应用重新读取。

- **审计**：OpenBao 的审计日志按服务账号记录了每一次读取，包括被拒绝的读取；秘密的值以 HMAC 形式出现，不是明文（`openbao-facts.log`）。两次写入的身份是 root：生产环境中写入秘密的人应当用自己的身份登录，根令牌只在初始化时使用。

实测数据

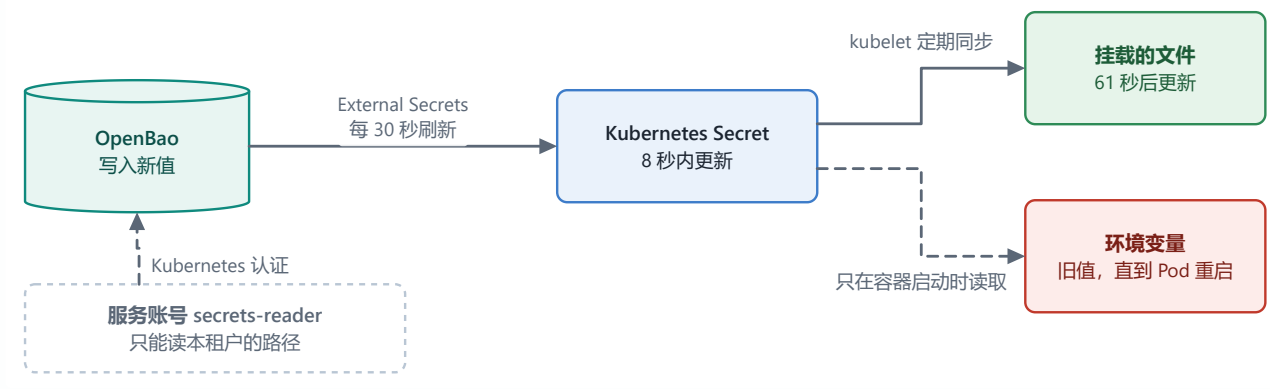


图 6-4 一次密钥轮换的传播（时间为实测）。新值 8 秒内进入 Kubernetes Secret，挂载的文件 61 秒后跟上，环境变量永远不会——需要轮换的凭据应当以文件挂载，并由应用重新读取。

6.10 局限

- **身份是实验证书。**没有单点登录，Argo CD 的权限只能离线检验；客户端证书不能吊销。
- **两个团队是模拟的。**insight 的服务只是一个最小的 HTTP 服务；没有测试团队之间真实的数据依赖。
- **只测了 kindnet。**策略对已有连接、对节点流量的处理方式取决于 CNI；Calico、Cilium 的行为可能不同，结论不能直接搬过去。
- **Argo CD 的同步冒用没有启用**，项目边界只由 Argo CD 执行（实验 6）。
- **OpenBao 只有一个节点、一份解封密钥**；没有测试高可用和自动解封。网格中的服务间授权（Istio AuthorizationPolicy）没有在本章实施。这份唯一的密钥后来在第 12 章的事故 E11-1 中丢失。
- **准入恢复时间的差异**（新 Pod 就绪后 4 秒与 20 秒）没有分析。
- **Kyverno 的策略格式已被标为弃用。**本章的策略是 `kyverno.io/v1` 的 ClusterPolicy；Kyverno 1.19 在读取时提示它“is deprecated and will be removed in a future release”，要迁移到基于 CEL 的 ValidatingPolicy、GeneratingPolicy 等新类型[31]。
- **审计策略不记录服务账号对 Secret 的读取。**排除服务账号和节点读操作的规则排在 Secret 规则之前（`platform/cluster/audit-policy.yaml`），一个被盗用的服务账号令牌读取 Secret 不会留下记录。修改审计策略需要重启 API 服务器，本章没有修改，列为待办（练习 5）。

6.10.1 从实验环境到生产环境

本章的做法	生产环境通常的做法	差别意味着什么
用户是集群 CA 签发的客户端证书（7 天，不能吊销）	企业身份系统经 OIDC 登录，组来自目录，会话可以撤销	证书只能等它过期；真实环境里离职和调岗要立刻生效
OpenBao 单节点、一份解封密钥，用根令牌写入秘密	高可用部署，KMS 自动解封或多人分片，每个人用自己的身份	第 12 章的事故 E11-1 就是单份密钥的代价
CNI 是 kindnet：新策略不切断已有的连接	Calico、Cilium 等，行为各不相同，有的支持第 7 层与全局策略	网络策略的结论要在自己的 CNI 上重新验证
两个团队、四个用户、一个最小的示例服务	几十个团队，自助开通命名空间	配额、策略例外和审计量的管理，本身会成为一个系统

6.11 常见问题

有了命名空间和 RBAC，为什么还要网络策略和准入策略？它们管的是不同的东西。RBAC 管 API 请求：谁能创建什么对象；网络策略管运行中的 Pod 能连到哪里；准入策略管对象的内容：镜像从哪里来、是否以 root 运行。本章的基线中，RBAC 限制得再严，任何 Pod 都能直接连上数据库。

在网格中，NetworkPolicy 和 Istio 的 AuthorizationPolicy 用哪个？两个都用，各管一层。NetworkPolicy 在 L3/L4 按地址和端口划出粗粒度的边界（命名空间之间），由 CNI 执行，不依赖网格；AuthorizationPolicy 按工作负载的身份（SPIFFE）做细粒度的授权（哪个服务能调用哪个服务），由 ztunnel 或 waypoint 执行。在网格中写 NetworkPolicy，要按本章测得的规则：放行 15008，有 waypoint 时放行 waypoint。

Kyverno 还是 ValidatingAdmissionPolicy？内置策略没有额外的组件，不会因为 webhook 不可用而影响集群，适合简单的校验规则；Kyverno 能生成和修改对象、有报告和例外机制，表达力强得多，代价是一个必须高可用的服务。常见的做法是两者结合：关键的、简单的规则用内置策略，其余用 Kyverno，并且限制 webhook 的范围、多副本运行。

为什么 break-glass 证书只有一小时？因为证书不能吊销。紧急访问应当是临时的、可追溯的：单独签发、有效期短、每次使用都在审计日志中带着 break-glass 组出现。

秘密放在 Git 里加密（例如 Sealed Secrets）不行吗？可以，适合少量、很少变化的秘密。外部存储的优势在于集中管理、按工作负载身份授权、每次读取都有审计、轮换不需要改 Git。代价是一个必须可用、必须解封的关键服务。

6.12 练习

练习

1. **（分析）** 一个开发者在某命名空间中可以创建 Pod，但没有读取 Secret 的权限。他能拿到这个命名空间中的 Secret 吗？请说明方法，以及平台可以采取哪些措施（至少三项）降低这个风险。

2. **（排障）** 一个团队给自己的命名空间加了默认拒绝的入站策略，又加了“允许 frontend 访问 api 的 8080 端口”。当天测试一切正常，三天后，api 重新部署之后 frontend 开始超时。两个服务都在 ambient 网格中，api 前面有 waypoint。请解释“当天正常、三天后失败”，并给出正确的策略。
3. **（设计）** 为 storefront 的 orders 服务写一组入站规则：它只应接受网关的调用和 Prometheus 的抓取。分别给出 NetworkPolicy 能做到的部分和需要 Istio AuthorizationPolicy 才能做到的部分，并说明原因。
4. **（计算）** 一个集群的 Kyverno 准入控制器只有一个副本，webhook 覆盖所有命名空间，失败策略为 Fail。每次重启 34 秒不能创建 Pod。如果每月有 4 次升级、2 次节点维护导致重启，一共有多少分钟不能创建 Pod？在这段时间里，哪些自动机制会失效？给出改进方案。
5. **（审计）** 安全团队问：“过去一周谁读取过 storefront 命名空间的 Secret？”请说明需要审计日志中的哪些字段、要注意哪些读取路径（直接读取之外），以及本章的审计策略记录了什么、没记录什么。
6. **（设计）** 一位平台工程师需要在凌晨处理一次数据库事故，必须读取生产环境的数据库密码。请设计这个紧急访问的流程：身份、授权、时限、记录和事后检查。
7. **（分析）** Argo CD 的 AppProject 已经限制了 insight 团队只能部署到自己的命名空间。为什么说这条边界仍然不够牢固？如何把它落到 Kubernetes 本身？

参考答案见附录。

6.13 本章小结

- 身份：用户是集群 CA 签发的客户端证书，组来自证书；证书不能吊销，break-glass 证书只有一小时，过期后被拒绝。
- 权限矩阵在测试前写下预期，用每个用户自己的证书询问 API 服务器，最终 58 项全部符合。第一次运行的 3 项不符来自测试工具的写法（`pods/exec` 被理解为名为 exec 的 Pod）。
- 审计日志能回答谁做了什么、为什么被允许：授权理由写明了是哪个绑定。四处细节：能创建 Pod 就能读 Secret；“允许”的请求也可能被提权检查拒绝（403）；Pod Security 拒绝的是 Pod 而不是 Deployment；exec 的命令行会被记录。GitOps 的变更在审计中只有 Argo CD 的身份，人要到 Git 中去找。
- 网络：基线是全通的。kindnet 执行策略，但不切断已有的连接、不拦节点的探针；在网格中，流量到达 15008，有 waypoint 时来源是 waypoint，而 ztunnel 的长期连接让新策略对通信过的 Pod 不起作用。按这些规则写的策略，45 个组合全部符合事先提交的预期；强制重新连接后，订单、通知、指标、自动扩缩容和数据库复制都正常。探测本身错了两次（把代理自己的 426 当作可达），修正后重做。
- 准入：Pod Security、LimitRange、Kyverno 和内置策略按固定顺序工作。单副本、覆盖全部命名空间的 Kyverno，一次重启让整个集群 34 秒不能创建 Pod；缩小 webhook 的范围、改为两个副本后，只剩一个请求失败。
- Argo CD：项目边界和权限都按预期拒绝了越界操作，但边界只由 Argo CD 执行，它的控制器拥有集群的全部权限。
- 秘密：每个团队只能从 OpenBao 读取自己的路径，越界请求被拒绝并留下审计记录；轮换在 Secret 上 8 秒、挂载文件上 61 秒生效，环境变量要等 Pod 重启。

第 7 章

故障注入与传播（E6）

前几章测量的是系统在正常情况下的表现。本章主动制造故障：让一个依赖变慢、大量返回错误，冻结一个节点，切断一个可用区，观察故障怎样传播、哪些机制挡住了它、哪些机制反而放大了它。每个实验都先写下假设和稳态指标，再执行，最后对照服务等级目标（SLO）计算影响。实验过程中还发生了两次计划外的真实事故，本章把它们和实验一样记录、分析，并写出复盘。

工程问题

故障一定会发生。它会停在出故障的那个服务里，还是沿着调用链扩散成整个系统的故障？超时、重试、熔断这些机制，各自让结果变好还是变坏？

实测结论

- 依赖变慢时，统一 1 秒超时的 resilient 模式让订单全部快速失败；naive 模式全部成功，但订单的 p95 是 1769 ms——“不失败”和“不变慢”要按操作取舍。
- 依赖 90% 失败时，naive 模式的重试让每笔下单打到 inventory 的调用变成 5.56 次；重试预算和熔断器把它压到 0.07 次。
- 冻结一个节点：只有一个副本的 waypoint 让全部请求失败了约 5.5 分钟（54%）；改为两个副本、跨可用区之后是约 55 秒（8.2%）。
- 可用区分区时节点一直 Ready、端点从未移除——Kubernetes 看不见这种故障；用户只损失了 0.22% 的请求，是长连接掩盖了它。

7.1 学习目标

学完本章，你应当能够：

- 按“稳态—假设—注入—观察—恢复”的步骤设计故障实验，并用 SLO 和错误预算衡量影响；
- 解释超时、重试、重试预算、熔断和隔舱怎样决定故障的传播，以及它们在不同故障下的取舍；
- 说出节点失效后 Kubernetes 的处理时间线，以及它为什么看不见网络分区；
- 找出平台中的单点：单副本的组件、被卷固定在节点上的有状态组件、没有余量的资源请求；
- 为真实事故写出不追究个人的复盘，区分根本原因、促成因素和改进项。

实验性质

本章的实验都是 `[真实]` 运行的，故障是 `[注入]` 的：依赖的延迟和错误由服务内置的 `/admin/faults` 接口注入；节点失效用 `docker pause` 冻结节点容器（kubelet、容器运行时和其上的所有 Pod 同时停止响应）；可用区分区用节点容器内的 iptables 规则丢弃与另外两个可用区之间的流量。三个可用区本身是 `[模拟]` 的（第 1 章）。两次事故不是注入的，是实验过程中真实发生的。

7.2 原理

7.2.1 稳态、假设与 SLO

故障实验不是“随便弄坏点什么看看”。每个实验先定义稳态：系统正常时可测量的输出（这里是用户看到的成功率和延迟）；再写下假设：注入某个故障之后，稳态会怎样变化；然后限定爆炸半径和结束条件，执行，对照。本章用下面的 SLO 衡量所有实验：

用户旅程	可用性	延迟
浏览商品	成功率 ≥ 99.5%	p95 ≤ 300 ms
下单	成功率 ≥ 99%	p95 ≤ 1 s

错误预算是 SLO 允许的“坏”的量。以时间计，30 天里 99.5% 的可用性允许约 216 分钟完全不可用，99% 允许约 432 分钟。一次 5 分钟的全面故障，就用掉浏览预算的约 2.5%。

7.2.2 故障怎样传播

服务之间的调用有五个决定传播方式的参数（第 3 章的 storefront 已经实现，`apps/storefront/src/storefront/common/resilience.py`）：

- **超时**：一次调用最多等多久。太长，调用方的资源被慢依赖拖住；太短，依赖稍慢就变成失败。
- **重试**：失败后再试。对偶发错误有效；对过载的依赖，每一层的重试相乘：三层各重试 3 次，最底层的负载可能放大到 $4^3 = 64$ 倍。
- **重试预算**：重试最多占正常请求的一定比例（这里 20%），限制放大。
- **熔断器**：连续失败后一段时间内直接拒绝，给依赖恢复的时间，也让调用方快速失败。
- **隔舱**：限制同时等待某个依赖的调用数，防止一个慢依赖耗尽调用方的全部并发。

示意图

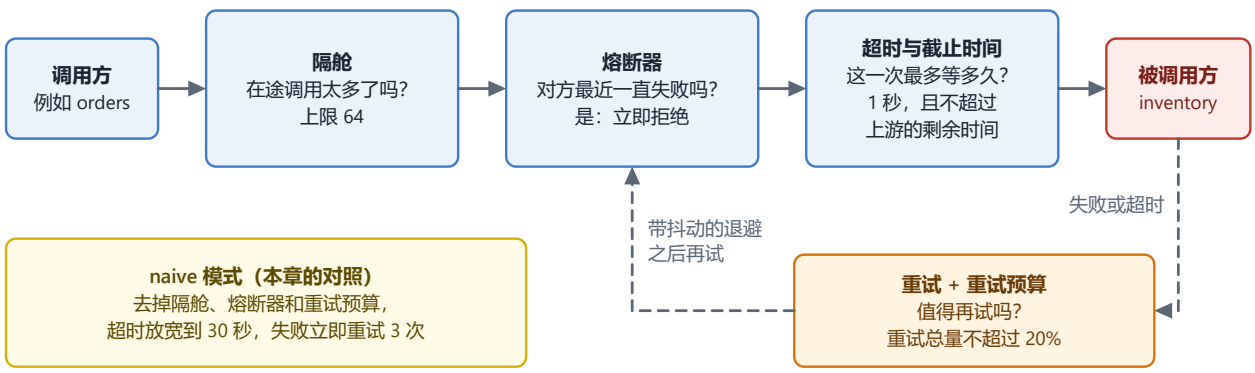


图 7-1 调用方发出一次依赖调用要经过的五道机制，每一道回答一个问题。超时把“慢”变成“失败”；重试把偶发的失败变成成功，也会把负载放大；预算和熔断给这种放大封顶；隔舱防止一个慢依赖占满调用方。去掉隔舱、熔断器和重试预算，并把超时放宽到 30 秒，就是本章用来对照的 naive 模式。

两种模式用于对照：**naive**（超时 30 秒、立即重试 3 次、没有预算、没有熔断和隔舱）和 **resilient**（超时 1 秒、带抖动的退避重试 2 次、20% 的重试预算、熔断器、64 的隔舱）。

7.2.3 节点失效的时间线

节点停止响应后，Kubernetes 按固定的时间表处理：kubelet 不再更新租约，节点控制器在宽限期（本环境测得约 50 秒）后把节点标为 NotReady，节点上的 Pod 随之变为未就绪，从服务的端点中移除；节点被加上 `node.kubernetes.io/unreachable` 污点，Pod 默认容忍它 300 秒，之后才被标记删除，由控制器在其他节点上创建替代者。被卷固定在这个节点上的有状态 Pod 无处可去，只能等节点回来。

7.3 实验 1：依赖变慢与失败：两种韧性模式

`experiments/e6/fault-phase.sh` 在 30 迭代/秒的负载下运行 120 秒稳态、120 秒故障、120 秒恢复，故障施加在 inventory 的全部副本上（订单流程依赖它，浏览流程不依赖），并从 Prometheus 统计每个服务的请求、5xx、服务间调用的结果、重试、快速拒绝，以及“每笔下单在 inventory 上产生多少次调用”（放大倍数）。韧性模式经 GitOps 切换（`gitops/envs/prod/storefront/common.yaml`），对照完成后恢复为 resilient。

故障阶段	resilient	naive
inventory 慢 1.5 秒	17.00% 的请求失败（几乎全部订单）；订单 p50 4.0 ms：快速失败	0.00% 失败；订单 p50 1616 ms、p95 1769 ms
每笔下单在 inventory 上的调用	0.05	1.04
inventory 90% 返回 503	17.26% 的请求失败（全部订单）；订单 p50 4.0 ms	9.80% 的请求失败；订单 p50 705 ms
每笔下单在 inventory 上的调用	0.07	5.56（inventory 每秒 44 个请求，稳态 7.9 个）
浏览（四种情况）	p95 17.5–17.6 ms，不受影响	p95 17.1–17.7 ms，不受影响

原始记录 [4, 5, 6, 7]

依赖变慢时，resilient 把“慢”变成了“全部失败”。1 秒的超时短于 1.5 秒的延迟：orders 对 inventory 的每次调用都超时，加上重试，orders 自己的响应超过了网关对它的 1 秒超时，于是网关到 orders 的熔断器打开，此后网关每秒直接拒绝 8.95 个下单请求。naive 模式下订单全部成功，只是慢了 1.5 秒：可用性目标满足，延迟目标（p95 ≤ 1 s）违反。哪个更好取决于业务：对一个能等的操作，慢比失败好；resilient 的超时应当按依赖的真实延迟分布设定，并留出余量，而不是一个统一的 1 秒。

实测数据

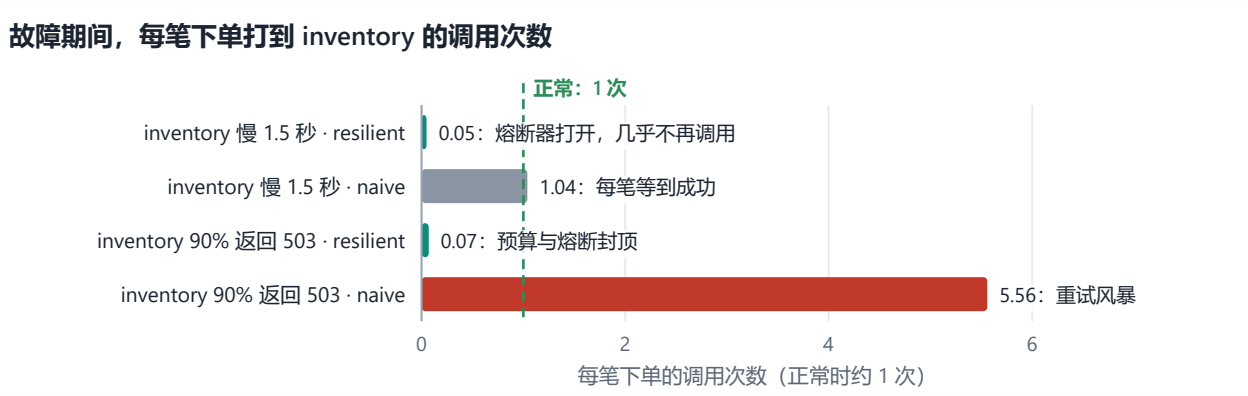


图 7-2 同一个故障，两种模式打到故障依赖上的负载。inventory 已经在失败，naive 模式却把对它的调用放大到平时的 5 倍以上——故障中的服务最不需要的就是更多请求；resilient 模式几乎不再打扰它，代价是订单在这段时间里全部快速失败。

依赖出错时，naive 制造了重试风暴。orders 对 inventory 每秒重试 31.6 次，网关对 orders 每秒重试 5.1 次，inventory 收到的请求是稳态的 5.6 倍。由于注入的错误与负载无关，重试反而救回了一部分订单：naive 的失败率（9.80%）低于 resilient（17.26%，熔断后所有订单都被拒绝）。但如果错误来自过载（第 4 章测得的饱和），5.6 倍的负载只会让依赖更快崩溃，失败从一个服务蔓延到它的所有调用方。resilient 的熔断器和预算把 inventory 收到的调用降到正常的 7%，给了它恢复的空间，代价是故障期间的订单全部快速失败。两种模式下浏览都不受影响：浏览不经过 inventory 和 orders，故障没有越过服务的边界。

7.3.1 验证：事先写下假设的一次运行

上面四次运行是探索性的对照，运行之前没有写下假设。为此又做了一次验证：在还没有运行过的条件下（resilient 模式，inventory **50%** 返回 503），先提交假设[8]，再运行[9, 10]：

	假设	结果
D1	故障不越过服务边界：浏览不受影响	符合：浏览 p95 18.7 ms（稳态 19.2 ms）
D2	重试预算和熔断器限制 inventory 的负载，没有重试风暴：每笔下单不超过 1.3 次调用	符合：0.07 次
D3	失败的订单快速失败	符合：订单 p50 4.1 ms
D4	故障期间违反订单的 SLO，撤除故障后 30 秒内恢复	符合：15.8% 的请求（几乎全部订单）失败；撤除后的第一个完整 30 秒窗口里已经没有 5xx

假设全部得到验证，但这次运行把一个代价看得更清楚：inventory 只有一半的请求失败，仅靠 2 次重试，大约 87.5% 的订单本可以成功（$1 - 0.5^3$）；网关到 orders 的熔断器却打开了，此后几乎拒绝所有订单，只在半开状态下放过少量探测请求（orders 每秒的请求数在 0.6 和 3 之间来回）。按服务的熔断器把部分失败变成了全部失败，这与实验 2 中第二次节点失效的发现是同一个问题。

7.4 实验 2：节点失效

`experiments/e6/node-failure.sh` 在 30 迭代/秒的负载下冻结 ent-worker（zone-a）420 秒，每 2 秒记录节点状态、这个节点上的端点是否就绪、待调度和终止中的 Pod，并按 10 秒统计客户端的失败。选择 ent-worker 是因为 ent-worker3 上有 PostgreSQL 的主库、唯一的 Valkey 和 OpenBao（它们的故障切换属于第 8 章），ent-worker2 上有 Prometheus。假设在运行前提交[1]。

7.4.1 第一次：一个副本的 waypoint

冻结之前，ent-worker 上有 catalog、inventory、orders 各一个副本，还有通知服务和 waypoint 的**唯一**副本。“每个服务都有两个副本，丢一个节点没关系”的直觉不适用于 waypoint：storefront 服务之间的每一次调用都经过它（第 3 章）。结果[11]：

时间（冻结后）	事件
0–10 s	一半的请求失败
10–320 s	**100% 的请求失败**，浏览也不例外
47 s	节点 NotReady，节点上的端点全部移除；但没有其他 waypoint 可用
331 s	节点控制器标记删除节点上的 Pod（容忍 300 秒的不可达污点之后）
约 345 s	新的 waypoint 在 ent-worker3 上运行，失败停止
410 s	解冻后节点恢复 Ready

整个运行 54% 的请求失败。按时间计，5.5 分钟的全面故障用掉了浏览 30 天错误预算的约 2.5%。Grafana 的替代者因内存不足无法调度。通知服务的唯一副本也在冻结的节点上，但没有丢失通知：JetStream 中通知消费者已确认的位置等于流的最后一条（193 792），没有待处理、没有重投[12]。

还有一个计划外的后果：被驱逐的 metrics-server 的替代者（优先级 system-cluster-critical）在 ent-worker2 上找不到位置，于是**抢占**了那里的 Prometheus。Prometheus 的卷固定在 ent-worker2 上，而这个节点的内存已经被其他 Pod 请求满了，它回不去，监控因此中断（下文的事故二）。

7.4.2 修复与第二次

修复按发现的问题逐一进行[13]：waypoint 改为两个副本、按可用区分布、加上中断预算（经 Istio 的参数 ConfigMap，`gitops/platform/manifests/mesh/waypoint-storefront.yaml`）；通知服务至少两个副本；Prometheus 获得 PriorityClass `platform-critical`。waypoint 的滚动更新本身卡住了：严格的可用区分布要求新副本放在 zone-a，而 zone-a 的内存请求只剩 56 Mi（“didn’t match pod topology spread constraints … Insufficient memory”），直到把 metrics-server 的内存请求从 200 Mi 改为 64 Mi（实际用约 40 MiB，[14]），才让出了位置。这正是第 5 章讨论过的取舍：DoNotSchedule 的分布约束在资源紧张时会阻塞部署。

第二次冻结同一个节点（[15]，假设见同一文件的第二部分）：

	第一次	第二次
全部请求的失败率	54%	8.2%
失败持续	约 5.5 分钟	约 55 秒（到端点移除为止）
节点 NotReady	47 s	52 s
Prometheus	被抢占，监控中断	一直运行，记录了全过程

实测数据



图 7-3 两次节点失效的对比。第一次，storefront 命名空间唯一的 waypoint 就在被冻结的节点上：所有经过它的请求全部失败，直到 300 秒的容忍期结束、Pod 在别的节点重建；第二次有两个 waypoint 副本，失败只持续到节点 NotReady、端点被移除为止。决定故障时长的不是节点何时恢复，而是 Kubernetes 何时停止向它发送流量。

故障时间从 5.5 分钟缩短到 55 秒，但第二次的假设 M1 被推翻了：我预计在端点移除之前只有部分请求失败（两个 waypoint 中只有一个被冻结），实际上这 55 秒里 99% 的请求失败。Prometheus 给出了原因[16]：真正因冻结而超时的调用每秒只有几个（一分钟内网关到 catalog 超时 105 次、到 orders 68 次），但它们让网关对 catalog 和 orders 的熔断器打开，此后网关每秒 52.9 次直接拒绝，5xx 正好是 52.8 次。应用的熔断器是**按服务**的，不知道服务背后有两个副本，其中一个完好。按实例剔除故障端点（网格中 Envoy 的异常检测）才能只隔离坏的那一个。

第二次还显示了资源余量的问题：驱逐之后，Argo CD 控制器、Grafana、第二个 waypoint 的替代者都因内存不足无法调度，直到节点回来（这期间 GitOps 停止同步）；节点恢复后，节点控制器取消了对仍在节点上的 Pod 的删除。另外，经过两次节点失效，catalog、gateway、payments 的两个副本都集中到了 zone-c：调度器只在放置新 Pod 时考虑分布，从不重新平衡（第 5 章）。系统每经历一次故障，就变得更脆弱一点，除非有人（或 descheduler）把副本重新分散。

7.5 实验 3：可用区分区

`experiments/e6/partition.sh` 在 ent-worker（zone-a）的节点容器里加入一组 iptables 规则，丢弃它与另外两个工作节点之间的所有流量（节点地址和 Pod 网段），控制面和负载均衡器仍然可达，持续 300 秒，

负载 30 迭代/秒。经过两次节点失效，zone-a 上的 storefront Pod 只剩两个 waypoint 中的一个。假设在运行前提交[2]。

示意图

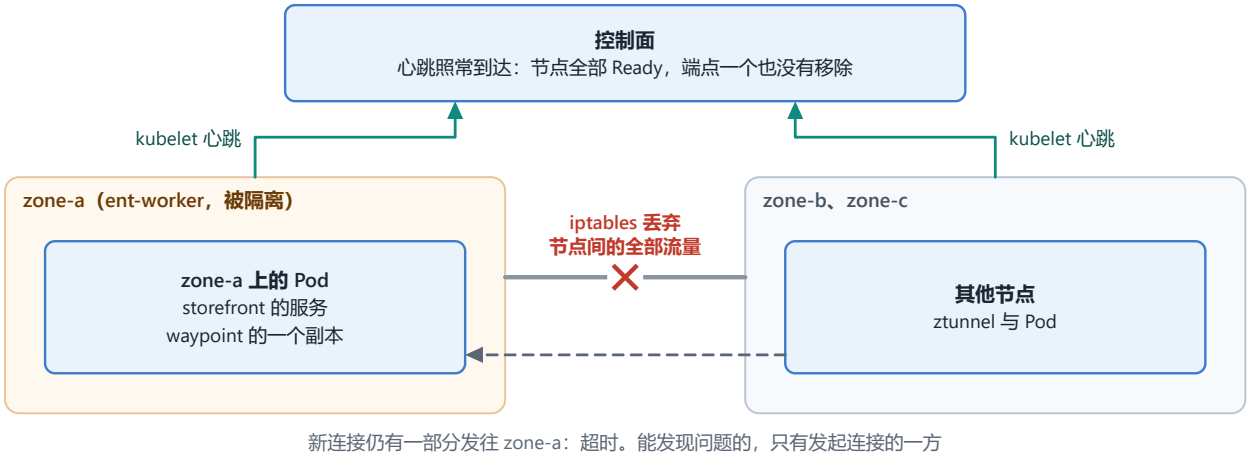


图 7-4 分区与节点失效的区别：数据面断了，控制面没有断。节点的心跳照常到达，Kubernetes 认为一切健康，不会移除任何端点；能发现问题的，只有发起连接的一方。

	假设	结果
Q1	节点一直 Ready，它上面的端点一直就绪：Kubernetes 看不见分区	符合：300 秒内节点一直 Ready，waypoint 的端点一直就绪
Q2	失败持续整个分区期间，不像节点失效那样一分钟后自愈	**不符合**：整个运行只有 0.22% 的请求失败；分区开始后 50–60 秒有一个小高峰（4.7%），其余每 10 秒不到 1%，个别请求等待了 58 秒
Q3	Prometheus 在一分钟内看到 zone-a 的抓取目标下线	符合：12 秒后第一个，24 秒后 10 个
Q4	分区结束后一分钟内失败停止	符合：分区在 13:09:11 解除，13:09:17 所有目标恢复

原始记录 [17]

Q2 为什么错了？zone-c 节点上 ztunnel 的访问日志显示[18]，分区期间它并没有避开不可达的 waypoint：完成的连接中 106 条去了 zone-b 的 waypoint，91 条去了 zone-a 的那个，还有 68 次连接超时（“connection timed out, maybe a NetworkPolicy is blocking HBONE port 15008”）。用户几乎感觉不到，是因为连接是长期复用的：应用与 waypoint 之间、k6 与负载均衡器之间的保持连接大多在分区之前就已建立，只有新建的连接会被分到不可达的一侧；失败之后，应用的重试换一条连接就成功了，失败的频率低到熔断器没有打开。

与第二次节点失效对比，差别在于故障的端点处在哪一层。那里被冻结的有一个 catalog 副本，waypoint 按**请求**在两个 catalog 之间分配，每个请求都有一半的机会碰到它，熔断器很快打开；这里不可达的是 waypoint 本身，ztunnel 按**连接**选择 waypoint，只有新连接受影响。

但隐患并没有消失：Kubernetes 始终把不可达的 waypoint 当作健康的端点，只要出现新连接（Pod 重启、连接池回收），就有一半先撞上超时。分区持续多久，这个隐患就存在多久，不会像节点失效那样在一分钟后被自动处理。分区对监控是可见的（10 个目标下线），对调度器不可见，处理它要靠人或自动化的判断：例如封锁 zone-a 的节点、删除那里的 Pod，或者让网格按实例剔除不可达的端点。

7.6 实验 4：GameDay 与复盘

GameDay 是团队有计划地演练故障：事先约定场景、角色、中止条件和观察的指标，演练后复盘。本章的演练是一个人完成的，按同样的步骤执行了上面的实验。比演练更有价值的，是演练过程中发生的两次真实事故。下面按不追究个人的复盘格式记录它们：发生了什么、影响、时间线、根本原因与促成因素、改进项。

7.6.1 事故一：一次发布让节点越过了驱逐阈值

摘要。把韧性模式从 naive 改回 resilient 的 GitOps 发布，使 ent-worker2 的内存工作集越过 kubelet 的驱逐阈值。kubelet 先后驱逐了 Argo CD 的应用控制器和 Prometheus；Prometheus 的卷固定在这个节点上，在节点的内存压力解除之前无法重新调度，监控中断约 5 分钟，正在进行的实验因此没有指标，只能重做。

时间（UTC）	事件
10:27:45	提交并发布“恢复 resilient”，6 个服务开始滚动更新（新 Pod 先于旧 Pod 退出）
10:27:58	驱逐 Argo CD 应用控制器：使用 538 MiB，请求 512 Mi
10:28:01	ent-worker2 内存压力，节点带上污点
10:29:06	驱逐 Prometheus
10:34:01	内存压力解除，Prometheus 在同一节点上重新启动

原始记录 [19]

根本原因。两个关键组件的内存请求低于实际用量：Prometheus 请求 640 Mi，实际约 940 MiB；Argo CD 控制器请求 512 Mi，实际 538 MiB。按第 5 章测得的驱逐排序，超出请求最多的 Pod 最先被驱逐，它们正是这两个。

促成因素。集群的内存请求已经接近饱和（三个工作节点 95%–99%），滚动更新时新旧 Pod 同时存在，短时间内增加了工作集；Prometheus 的卷固定在一个节点上，没有别处可去。第 5 章的驱逐实验已经指出“超出请求的 Pod 先走”，第 6 章结束时也记录了 ent-worker2 距驱逐阈值只有约 240 MiB，但都没有据此调整请求。

改进项。① 已完成：Prometheus 的内存请求改为 1 Gi，Argo CD 控制器改为 640 Mi，与实际用量一致（`gitops/platform/values/monitoring.yaml`、`gitops/bootstrap/argocd-values.yaml`）。② 已完成：被卷固定的平台组件获得 PriorityClass（见事故二）。③ 待办：为所有组件定期比较请求与实际用量（本章的 `incident-1.log` 和 `spof-fix.log` 中有一次手工比较），并把内存余量作为告警。

7.6.2 事故二：故障后的替代者抢占了监控

摘要。节点失效实验中，被驱逐的 metrics-server 的替代者（system-cluster-critical）在另一个节点上抢占了 Prometheus（优先级 0）。Prometheus 被卷固定在原节点，而那里的内存请求随即被其他替代者占满，它无法回来。两次手工腾出空间都失败了：调度器先把新的 Pod 放进了腾出的空间。监控中断约 27 分钟。

时间（UTC）	事件
10:54:54	节点控制器标记删除被冻结节点上的 Pod
10:54:55	Prometheus 被抢占（“Preempted by pod … on node ent-worker2”）
10:56:33	Prometheus 无法调度：“didn’t match PersistentVolume’s node affinity … Insufficient memory”
11:00–11:21	两次手工移走 ent-worker2 上的 Pod（11:00、11:10，每次等待 10 分钟），新 Pod（包括 Grafana）先占了位置
11:21:38	加上 PriorityClass `platform-critical` 后，Prometheus 在原节点上启动

原始记录 [20, 21, 12]

根本原因。被卷固定在节点上的平台组件与普通工作负载优先级相同（0）。高优先级的系统组件需要位置时，它是合法的牺牲者；而它被抢占之后，无法像无状态的 Pod 那样在别处重建。

促成因素。没有资源余量：一个节点的 Pod 被驱逐后，其余节点放不下它们的替代者；手工腾空间与调度器竞争，没有确定的结果。

改进项。① 已完成：PriorityClass `platform-critical`（`gitops/platform/manifests/policies/priorityclasses.yaml`），Prometheus 使用它，因而能在自己的节点上抢占普通 Pod，第二次节点失效中它一直在运行。② 待办：PostgreSQL、NATS、Tempo、Loki、OpenBao 等同样被卷固定的组件也使用它（修改它们会重启实例，放到第 8 章与数据可靠性一起做）。③ 待办：按“失去一个可用区后其余节点能容纳全部替代者”规划容量，这正是第 4 章算过的冗余。

7.7 局限

- **故障注入是内置的。**依赖故障由服务自己的 `/admin/faults` 注入，与负载无关；真实的过载会随负载加重，重试风暴的后果会比这里更严重。没有使用 Chaos Mesh 一类的工具：内存已经紧张，`docker pause` 和 iptables 足以制造本章的故障。
- **节点失效用 `docker pause` 模拟**：进程冻结、连接保持而无响应，接近一台机器失去网络或卡死；不同于断电（连接被重置）或优雅关机。
- **每种情况只做了一次**（naive 和 resilient 各一次，节点失效修复前后各一次），没有统计意义上的重复。
- **集群没有资源余量。**三个工作节点的内存请求在 95%–99% 之间，节点失效后多数替代者无法调度；这既是本章的发现，也限制了实验能观察到的恢复过程。
- **演练是一个人完成的**，没有值班、升级和沟通的环节；第 10 章（E9）的事故演练再补上。

7.7.1 从实验环境到生产环境

本章的做法	生产环境通常的做法	差别意味着什么
故障由服务自带的开关、`docker pause` 和 iptables 注入	Chaos Mesh、Litmus 或云厂商的故障注入服务，带范围限制与自动中止条件	本章靠脚本的 trap 收回故障；生产演练需要更强的护栏、审批与公告
一台机器上的三个“可用区”	真实的可用区：独立的供电、网络与延迟	整区失联、跨区延迟升高这类相关故障，只能在真实拓扑上演练
按服务的熔断器	按实例的异常检测，加上按服务的熔断	本章中一半的实例失败时，按服务的熔断器把部分故障放大成了全部失败
节点的内存请求已达 95%–99%	按“失去一个可用区仍能容纳全部负载”预留容量	否则故障后的替代 Pod 无处可去——余量是韧性的一部分，不是浪费

7.8 常见问题

resilient 模式在依赖变慢时让订单全部失败，还要用吗？要，但超时不能随便定。超时应当按依赖的延迟分布设定（例如它的 p99.9 加余量），并且沿调用链递减：最外层的截止时间要能容纳内层的超时加重试，否则外层总是先放弃。本章的问题在于每一层都是 1 秒。对能等待的操作，还可以改为异步（接受请求、稍后完成），而不是同步等待。

为什么熔断器反而让节点失效变得更糟？因为它是按服务的：两个副本中一个出问题，熔断器看到的是“这个服务有一部分调用失败”，于是对整个服务打开。故障发生在实例上时，应当剔除实例（Envoy 的异常检测、客户端的负载均衡健康检查），熔断器只用来保护整体不可用的依赖。

为什么节点失效后要等 5 分钟 Pod 才在别处重建？默认的不可达容忍是 300 秒，用来避免网络抖动造成大规模的 Pod 迁移。无状态服务可以缩短（在 Pod 上设置更短的 `tolerationSeconds`），但前提是集群有余量放下替代者，否则缩短只会更早地制造一批无法调度的 Pod。真正缩短影响的，是让服务在端点移除的那一刻就有足够的健康副本（本章第二次实验的 55 秒）。

复盘为什么强调“不追究个人”？因为目的是找出让错误可能发生、让影响扩大的系统条件。事故一的直接触发是一次正常的发布，追究“谁发布的”不会让下一次更安全；把请求改为实际用量、给关键组件优先级、为资源余量设告警，才会。

7.9 练习

练习

1. **（计算）** 一条调用链 A → B → C，每层都立即重试 3 次且没有预算，C 的 90% 请求失败（错误与负载无关）。估算 C 收到的请求是正常时的多少倍；如果 C 的失败来自过载，这个倍数会带来什么后果？本章实测的倍数是 5.56，为什么低于你的估算？
2. **（设计）** 为 orders → inventory 和 gateway → orders 两段调用设定超时，使 inventory 慢 1.5 秒时订单仍能成功、而 inventory 完全不可用时订单在 2 秒内失败。说明你的依据和需要的截止时间传递方式。

3. **（分析）** 第二次节点失效中，两个 waypoint 只冻结了一个，为什么在端点移除之前几乎所有请求都失败？给出两种改进，并说明各自的代价。
4. **（计算）** 浏览的 SLO 是 30 天 99.5%。第一次节点失效约 5.5 分钟完全不可用，第二次约 55 秒。各用掉多少错误预算？如果每月有两次节点失效，按第一次的情况，预算还剩多少？
5. **（设计）** 本环境三个工作节点的内存请求在 95%–99% 之间。按“失去任何一个可用区，其余节点能容纳全部替代者”的原则，每个节点的请求最多能到多少？需要做哪些调整？
6. **（复盘）** 选择事故一或事故二，补全复盘中缺少的部分：检测（是怎样发现的、能否更早发现）、影响的量化，以及每个改进项的负责人和期限应当怎样定。
7. **（分析）** 为什么说可用区分区比节点失效更难处理？结合实验 3 的数据，说明 Kubernetes、负载均衡器和网格各自看到了什么、没看到什么。

参考答案见附录。

7.10 本章小结

- 先写假设，再注入故障：每个实验都有事先提交的假设、稳态指标和 SLO 对照；被推翻的假设（例如“两个 waypoint 就只会部分失败”）和被证实的一样有价值。
- 依赖变慢时，统一 1 秒超时的 resilient 模式让订单全部快速失败，naive 模式让订单全部成功但 p95 达到 1.77 秒；依赖出错时，naive 的重试让 inventory 的负载放大 5.6 倍，resilient 的熔断器和预算把它降到 7%。浏览在所有情况下都不受影响。事先写下假设的验证运行（50% 错误）四项假设全部符合，也暴露出按服务的熔断器在一半请求失败时就拒绝了几乎所有订单。
- 节点失效：单副本的 waypoint 让 5.5 分钟内 100% 的请求失败；改为两个副本并分布到不同可用区后，故障缩短到 55 秒，但按服务的熔断器把部分失败放大成了全部失败。
- 资源余量决定恢复：内存请求在 95%–99% 时，节点失效后的替代者无法调度；分布约束会阻塞部署；副本在每次故障后向幸存的可用区集中。
- 两次真实事故：一次发布让节点越过驱逐阈值，驱逐了内存请求偏低的 Prometheus 和 Argo CD 控制器；一次故障后的替代者抢占了被卷固定的 Prometheus。改进是按实际用量设置请求、为被卷固定的平台组件设置优先级、为余量设告警。
- 可用区分区：节点一直 Ready，Kubernetes 从未移除不可达的端点；用户看到的失败只有 0.22%，因为长期复用的连接只把新连接暴露给分区，应用的重试掩盖了它们。故障端点处在按请求分配的一层（实验 2 的 catalog）还是按连接分配的一层（这里的 waypoint），决定了同一类故障的影响大小。分区对监控可见（24 秒内 10 个目标下线），对调度器不可见。

第 8 章

数据可靠性与灾难恢复（E7）

前几章默认数据一直都在。本章让数据面临风险：主库失效、复制中断、一个清理任务误删了全部订单，最后整个集群消失。每一种情况都测量两个数：恢复花了多久（RTO），丢了多少数据（RPO）。备份放在集群之外，每一次恢复都实际执行，并与事先记录的“真相”逐条核对。实验中出现了两次没有预料到的问题：一次脑裂留下的 WAL 让一个副本悄悄停止了复制；一次重建时，同步波次在全新的集群上死锁。

工程问题

数据最终要落在某个地方，而那个地方会坏。主库失效、有人误删数据、整个集群消失——每一种情况下，服务要中断多久（RTO），会丢掉多少已经确认的写入（RPO）？

实测结论

- 计划内的切换中断写入 8.0–14.4 秒，不丢数据；计划外的失效要 26.9–78.0 秒，大部分时间花在等待租约到期和节点被判定失联。
- 复制中断之后再失去主库：异步复制丢了 3017 次已经确认的写入；同步复制一次不丢，代价是写入停顿 137.9 秒，提交延迟约为 2 倍。
- 误删的 211 350 笔订单用时间点恢复在 103 秒后找回，事故之后的新订单不受影响。
- 备份报告 Completed 不等于数据被保护：Velero 跳过了全部 11 个 hostPath 卷。
- 整个集群删除后从 Git 与集群外的备份重建：第一次 20.1 分钟（两次人工介入），修复之后 15.3 分钟。

8.1 学习目标

学完本章，你应当能够：

- 用 RTO 和 RPO 描述一种故障的代价，说出它们分别由什么决定；
- 解释异步复制与同步复制在故障切换时的取舍，以及结果未知的写入应当怎样处理；
- 配置 PostgreSQL 的连续归档和时间点恢复，在数据被误删后找回它，同时不丢失误删之后的写入；
- 判断一份备份到底保护了什么（Velero 报告“完成”不等于数据在里面）；
- 在集群整体丢失后，用 Git、对象存储和集群外保存的密钥重建平台，并找出重建路径上的隐患。

实验性质

本章的实验都是 `[真实]` 运行的，故障是 `[注入]` 的：强制删除 Pod，用 `docker pause` 冻结节点，用 iptables 切断复制，用 `kind delete cluster` 删除整个集群，再让一个写错了条件的“清理任务”删除订单。“集群外”的备份目标与集群在同一台机器上，是 `[模拟]` 的异地，和第 1 章的三个可用区一样。

8.2 原理

8.2.1 RTO 与 RPO

RTO（恢复时间目标）是从故障发生到服务恢复的时间，RPO（恢复点目标）是恢复之后丢失的数据所对应的时间窗口。目标是事先定的，本章测量的是实际值。

示意图

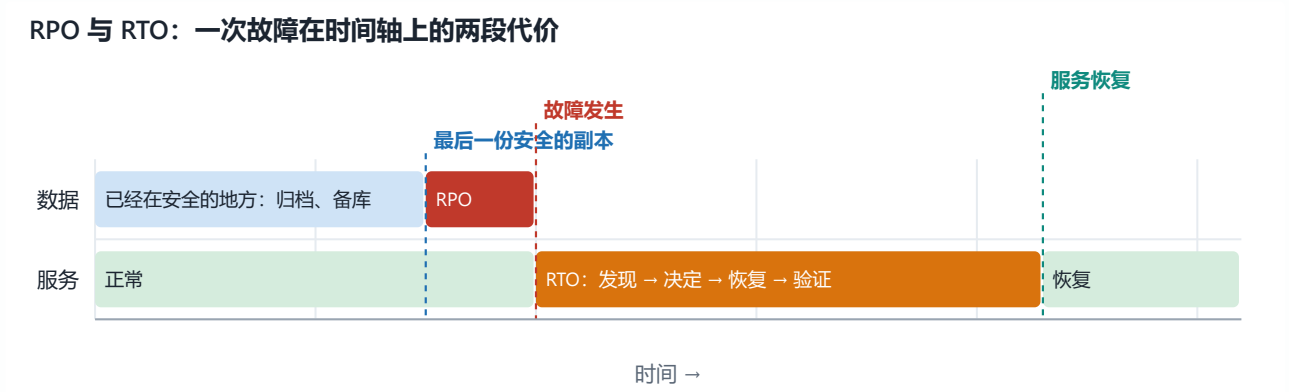


图 8-1 两个指标在时间轴上的位置（示意）。RPO 是故障之前、还没来得及送到安全之处的那一段数据——它由复制与归档的落后决定，在故障发生的那一刻就已经确定；RTO 是从故障到服务恢复的时间——发现、决定、执行、验证，每一步都算在内。

测量用一个专门的写入器（`experiments/e7/writer.py`）：它以每秒 50 次的速度通过 `pg-main-rw` 向数据库 `drill` 插入编号的行，每次尝试记录三种结果之一：

- `ok`：提交得到确认；
- `fail`：肯定没有写入（连不上，或者服务器明确返回了错误，例如降级后的只读）；
- `unk`：语句已经发出，2 秒内没有回答，客户端不知道它是否提交。

演练结束后，把确认过的编号与表中实际存在的行比较：**写入中断**是从第一次没有得到确认的尝试到下一次确认之间的时间，**丢失的写入**是得到了确认、行却不存在的编号。

写入器有两处刻意的设计。第一，它从不取消一个等待中的提交：在同步复制下，取消一个正在等待备库确认的提交，PostgreSQL 会返回“成功”，只附带一条“可能没有复制到备库”的警告。第二，它用 psycopg2 的异步模式给每次等待设截止时间：被冻结的节点的内核仍然会确认 TCP 报文，单靠 keepalive 永远等不到连接断开。

8.2.2 复制：异步、同步与结果未知的写入

异步复制下，主库在本地写盘后就确认提交，备库稍后才收到。故障切换时，被提升的备库没有收到的提交全部丢失；在同一个局域网里差距通常不到一毫秒，但只要备库因为任何原因落后，丢失的就是落后的全部。同步复制（这里是 `ANY 1`：两个备库中任意一个写盘后才确认）保证任何一个实例丢失都不会丢掉已确认的提

交，代价是每次提交多一次往返；`dataDurability: required` 时，两个备库都不可达，写入就停下来等待，而不是悄悄退化成异步。

结果未知的写入是第三种结果，不能简单当作失败：语句可能已经提交，只是确认没有回来。客户端必须能安全地重试（storefront 的订单带幂等键，第 2 章），否则重试会造成重复，不重试又可能丢单。

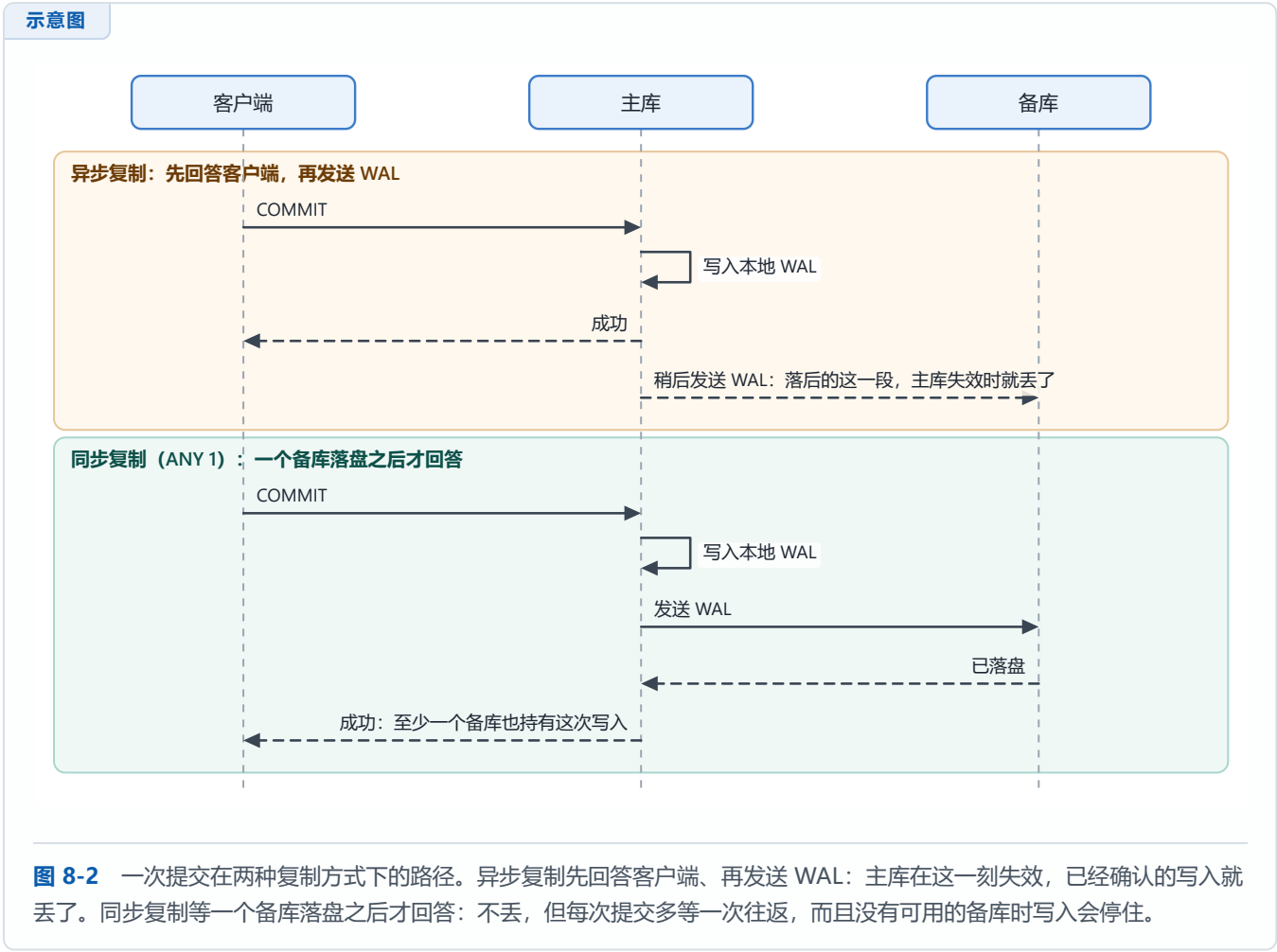


图 8-2 一次提交在两种复制方式下的路径。异步复制先回答客户端、再发送 WAL：主库在这一刻失效，已经确认的写入就丢了。同步复制等一个备库落盘之后才回答：不丢，但每次提交多等一次往返，而且没有可用的备库时写入会停住。

8.2.3 连续归档与时间点恢复

PostgreSQL 把所有修改写进 WAL，每 16 MiB 一个段。连续归档把每个写满的段复制到对象存储，`archive_timeout`（这里 5 分钟）保证即使写得很少，段也最迟在 5 分钟后被关闭并归档；但完全空闲的数据库不产生 WAL，也就没有东西可归档。恢复时先还原一份基础备份，再按顺序重放它之后的 WAL，可以停在任意时刻（时间点恢复，PITR）。每次提升都会开启一条新的时间线，归档中的历史文件记录各时间线从哪里分叉，恢复沿着它们走到最新的时间线。

由此可以直接推出整个集群丢失时的 RPO：归档总是落后于主库，落后的那部分 WAL 只在集群的磁盘上，随集群一起消失。

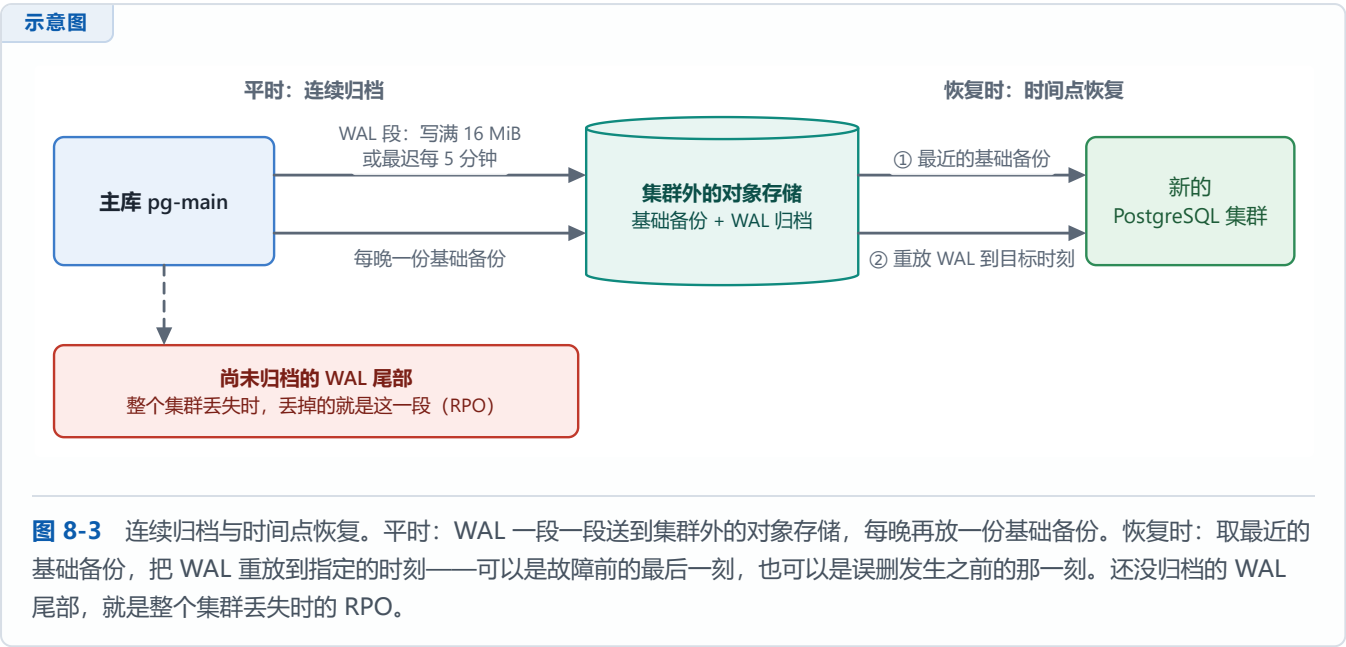


图 8-3 连续归档与时间点恢复。平时：WAL 一段一段送到集群外的对象存储，每晚再放一份基础备份。恢复时：取最近的基础备份，把 WAL 重放到指定的时刻——可以是故障前的最后一刻，也可以是误删发生之前的那一刻。还没归档的 WAL 尾部，就是整个集群丢失时的 RPO。

8.2.4 租约、隔离与脑裂

两个实例同时认为自己是主库并各自接受写入，就是脑裂。CloudNativePG 1.30 用一个 Kubernetes Lease（`data/pg-main`，15 秒，[2]）防止它：新主库在提升之前必须拿到这个租约。隔离（fencing）是让旧主库无法再写入的手段。在物理机房里，最可靠的隔离是在提升新主库之前把旧机器断电（STONITH）；Kubernetes 不能给节点断电，旧主库只能在发现自己不再是主库时自己停下来。

8.2.5 备份放在哪里，谁能读

备份必须在它要防范的故障域之外：保护“整个集群丢失”的备份不能放在集群里。恢复备份所需的凭据也一样：OpenBao 本身要从对象存储恢复，所以访问对象存储的密钥不能只存在 OpenBao 里，否则恢复时就是一个死循环。本章的做法是每个使用者一个桶、一个密钥，副本保存在集群外的 `~/.lab-secrets/v2/backup`，由脚本通过标准输入写入集群（`platform/backup/secrets.sh`）。

8.3 准备：备份目标、容量与插件

备份目标。Garage v2.5.0 是一个 S3 兼容的对象存储，以单节点运行在集群网络 `kind-ent` 上的 Docker 容器 `backup-s3` 里，数据在一个 Docker 卷中，删除集群不影响它（`platform/backup/object-store.sh`）。三个桶各有自己的密钥：`pg-backups`（CloudNativePG）、`velero`、`platform-snapshots`（OpenBao）。CloudNativePG 的密钥能列出 `pg-backups` 的对象，访问另外两个桶时得到 AccessDenied[3]。

容量。第 7 章结束时三个工作节点的内存请求在 91%–98% 之间，放不下备份组件：cert-manager、Barman Cloud 插件（包括每个 PostgreSQL 实例中的一个边车）和 Velero 的节点代理。这里没有挤压已有组件的请求，而是把每个工作节点从 3 GiB 扩到 3.5 GiB：先调高 cgroup 的上限，再调高 kubelet 的驱逐阈值，Pod 不受影响。可分配内存从 2666 MiB 增加到 3178 MiB，请求比例降到 76%–82%（`platform/cluster/resize-workers.sh`、[4]）。这是一次有记录的扩容，第 1–7 章的测量都是在 3 GiB 下做的。

为什么用插件。CloudNativePG 1.30 仍然带着内置的 Barman Cloud 支持，但 API 服务器在接受这样的配置时就给出警告：内置支持已被弃用，“will be completely removed in CloudNativePG 1.31.0”[2]。所以本章使用它的替代品 Barman Cloud 插件 v0.15.1：它通过 CNPG-I（带双向 TLS 的 gRPC，证书由 cert-manager v1.21.3 签发）与 operator 通信，并在每个实例 Pod 中注入一个边车，负责 WAL 归档、基础备份和恢复时取回 WAL。归档目标是一个 `ObjectStore` 对象，每晚 02:00 一次基础备份，第一次在创建时立即执行（`gitops/platform/manifests/data/pg-backup.yaml`）。第一份基础备份压缩后 74 MB[5]。

优先级。第 7 章的事故二之后，所有被卷固定在节点上的组件都使用 PriorityClass `platform-critical`：PostgreSQL、NATS、Valkey、Loki、Tempo、OpenBao。OpenBao 的 StatefulSet 使用 `OnDelete` 更新策略，模板改了，Pod 并不会自动重建：每次重启都会让它进入密封状态，Helm chart 有意把重启交给操作员[6]。

8.4 实验 1：PostgreSQL 的故障切换

`experiments/e7/failover.sh` 在写入器运行时制造一种故障，每秒记录集群的阶段、当前与目标主库、各实例是否就绪、`pg-main-rw` 指向哪个地址（`pg-watch.sh`），结束后汇总写入器的结果、事件和 operator 的日志。每一组运行之前提交假设[7]，全部结果在[8]中；下面先用一张图概括，再逐项列出。

实测数据

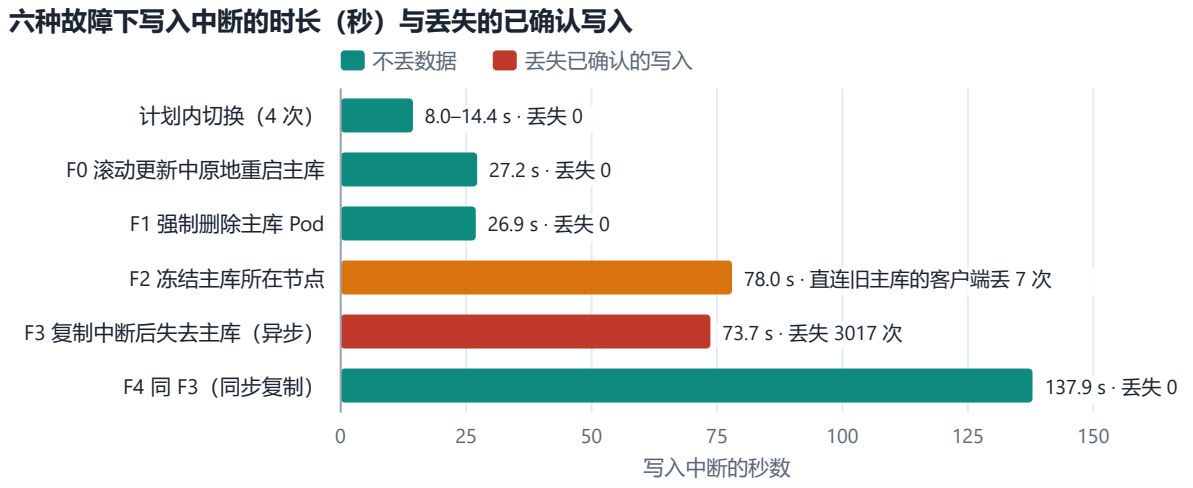


图 8-4 六种故障的两个结果：写入中断多久，丢了多少已经确认的写入。计划内的操作都在 30 秒以内；计划外的失效，时间主要花在“确认旧主库真的不在了”。唯一丢数据的是异步复制下的 F3；同步复制把它换成了更长的停顿——F4 的 137.9 秒里，主库宁可不接受写入，也不确认一个备库没有收到的提交。

运行	故障	写入中断	丢失的已确认写入
F0	引入 WAL 归档边车的滚动更新	27.2 s，之前 3 分钟不能建立新连接	0
F0b–F0e	计划内切换（`kubectl cnpg promote`），F0e 为同步复制	10.7 s、11.7 s、8.0 s、14.4 s	0

运行	故障	写入中断	丢失的已确认写入
F1	强制删除主库 Pod	26.9 s	0
F2	冻结主库所在节点 120 秒	78.0 s	经服务 0；直连旧主库的客户端 **7**
F3	切断复制 57 秒后冻结主库节点（异步）	冻结后 73.7 s	**3017**
F4	同 F3，同步复制	137.9 s	**0**

8.4.1 计划内的切换

F0 的假设是“先更新两个副本，再切换，写入中断 2–10 秒”。实际上 operator 没有切换。它在日志中写道：“Primary is missing the WAL-archiver sidecar; recreating it in place instead of switching over”：主库还没有归档边车时，它不提升副本，而是原地重建主库。删除主库 Pod 先触发 smart shutdown：已有的会话继续运行，新连接一律拒绝，持续 `smartShutdownTimeout`（默认 180 秒）。写入器只用一个长连接，于是继续在旧主库上提交了 3 分钟，而 `pg-main-rw` 在这 3 分钟里没有就绪的端点（14:09:21–14:12:41）。之后是 fast shutdown、首次拉取边车镜像（11.0 秒）、重启，完全中断 27.2 秒[9]。storefront 的连接池和写入器一样：已打开的连接在 smart shutdown 期间一直可用，到 14:12:30 降为 0。**同一次维护，用长连接的客户端看到 27 秒的中断，每次请求新建连接的客户端看到 3 分 20 秒**：测得的 RTO 取决于客户端怎样连接。

真正的计划内切换（F0b–F0e）各用了 8.0–14.4 秒，没有丢失任何写入：旧主库停止并释放租约，副本完成重放、从归档读取时间线历史后提升，operator 修改服务的标签，客户端重新连接。

8.4.2 计划外的失效

强制删除主库 Pod（F1）时，operator 在 0.5 秒内发起故障切换，3.2 秒后选定目标，但写入中断仍有 26.9 秒。新主库的日志给出了原因：“Acquiring primary lease”之后 16 秒才“Acquired primary lease”[10]。被杀死的旧主库没有释放租约，新主库只能等它过期，这是防止脑裂的代价，每一次计划外的切换都要付。

冻结主库所在的节点（F2）时，又多了一段等待：在节点被标记为 NotReady 之前（50.7 秒），没有任何组件发现异常。operator 对冻结实例的状态请求没有很快失败，因为被冻结节点的内核仍然完成 TCP 握手，TLS 握手只是一直等着。随后又是同样的 16 秒租约等待，合计 78.0 秒[11]。两次的等待都是 16 秒，与旧主库已经消失了 3.7 秒还是 51 秒无关：新主库似乎从它自己第一次看到租约记录时开始计时，而不是按记录中的续约时间，这是领导者选举避免节点时钟偏差的常见做法（从两次测量推断）。

F2 的假设中还有一个**直连写入器**：它不经过 `pg-main-rw`，直接连接旧主库的 Pod 地址，代表任何连接活得比故障切换更久的客户端（连接池里的旧连接、缓存的地址）。节点解冻后，旧主库作为主库运行了 2.5 秒，直到它的实例管理器在 14:33:06.466 写下“This is the former primary instance. Shutting it down to allow it to be demoted to a replica”。这 2.5 秒里，直连写入器有 **7 次写入得到确认**，它们在旧的时间线上，之后被 pg_rewind 丢弃。冻结时还在执行的两条 INSERT 也在解冻后完成了提交，其中一条执行了 114 秒；它们的客户端早已放弃（`unk`），这两行同样不复存在。

8.4.3 复制中断之后再失去主库

F3 先切断两个副本到主库 5432 端口的流量 57 秒，再冻结主库的节点。切断之前，主库关闭当前的 WAL 段：下一个段要等 `archive_timeout` 才进入归档，副本无法从归档补上切断期间的 WAL。切断期间没有任何失败，主库照常确认；故障切换提升了一个落后 57 秒的副本，**3017 次已确认的写入丢失**，正是切断期间确认的全部，与假设（约 3000）相符。没有任何机制阻止提升一个落后这么多的副本。也没有任何迹象：集群的阶段一直是“Cluster in healthy state”；副本的延迟指标 `cnpg_pg_replication_lag` 一直是 0，因为它比较的是副本收到的和重放的 WAL，而副本已经把收到的全部重放完了。F3 时的 Prometheus 数据随后随集群重建消失了；重建之后单独切断一个副本 2 分钟复测，结果相同：延迟 0，WAL 接收器 0，主库只剩一个流复制副本，集群仍是“healthy”[12]。

F4 在同步复制下重复同样的双重故障。切断期间没有一次写入得到确认，写入器的每次尝试都在 2 秒后变成 `unk`；故障切换后**没有丢失任何已确认的写入**，写入中断 137.9 秒（切断 59 秒 + 冻结和切换 79 秒）。30 次结果未知的写入中有 2 次其实已经提交[8]：一次恰好在切断生效时发出，备库收到了，确认却没能回来；另一次是新主库上的第一次提交，等了 2 秒多，直到一个备库连上新主库。同样的双重故障，异步复制丢掉了 57 秒的订单，同步复制把它变成了 2 分钟的写入停顿。

同步的代价体现在每一次提交上：提交延迟的 p50 从异步的 2.7–3.1 ms 增加到 5.1–5.9 ms，p99 从 8–13 ms 增加到 15–16 ms，约为两倍。平台保留了同步复制（`ANY 1`，`dataDurability: required`）：任何一个实例丢失都不会丢掉一笔已确认的订单；两个备库同时不可达时，写入等待。

8.5 事故：脑裂之后，一个副本悄悄停止了复制

这不是注入的故障，而是 F2 的后果，谁也没有预料到[13]。F3 开始时，主库报告只有一个副本在流复制，而不是两个。查下去发现，第三个副本 pg-main-3 自 F2 起就没有再复制：

时间	事件
14:33:04.0	F2 的节点解冻，旧主库恢复运行（仍是主库，时间线 5）
14:33:05.291	pg-main-3：“started streaming WAL from primary at 0/A2000000 on timeline 5”
14:33:06.466	旧主库停止
14:33:07.952	pg-main-3：“requested starting point 0/A3000000 on timeline 5 is not in this server’s history. This server’s history forked from timeline 5 at 0/A22C21B0”
14:34–14:50	pg-main-3 的 WAL 接收器指标为 0；Pod 一直 Ready，集群一直“healthy”

pg-main-3 在旧主库可写的那 2.5 秒里从它那里流复制了 WAL，这部分 WAL 不在新主库的时间线上。时间线 5 的服务器在那一刻只剩旧主库，而 `pg-main-rw` 从 14:32:27 起已经指向新主库，所以这次连接一定是在服务还指向冻结节点时发起的：冻结节点的内核完成了 TCP 握手，TLS 握手一直等到解冻（推断，那段时间的副本日志在修复前没有保存）。副本的 `primary_conninfo` 设置了 `tcp_user_timeout='5000'`，没有 `connect_timeout`；面对一个仍在确认报文的内核，TCP 超时永远不会触发[2]。**副本也是客户端，F2 中直连写入器遇到的问题同样会落在它身上。**

历史分叉的副本不会自己恢复（CloudNativePG 只对重新加入的旧主库执行 pg_rewind），而且没有任何东西在看复制状态：副本的就绪不依赖流复制，集群的阶段只数就绪的实例，平台上没有一条针对 `cnpg_` 指标的告警。数据库在 17 分钟（14:33–14:50）里只有一个可用的副本，期间经历了一次计划内切换和 F3。

改进与验证：

- 新增告警（`gitops/platform/manifests/data/pg-alerts.yaml`）：没有 WAL 流的副本、流复制副本少于两个的主库、等待归档的 WAL 段、过旧的基础备份。前两条在部署后立即对这次事故进入 pending（14:47:05），2 分钟后触发[14]。
- 修复：`kubectl cnpg destroy pg-main 3` 删除这个实例及其卷，一个 join 作业从主库克隆出新的 pg-main-3，用了 25 秒，告警随之消失[14]。
- 告警本身也需要验证。第一版的归档告警按“距上次归档超过 10 分钟”触发，在一个空闲的数据库上从 15:34 一直触发到 22:27：空闲的数据库不产生 WAL，没有东西可归档。现在改为按等待归档的 WAL 段触发（`cnpg_collector_pg_wal_archive_status{value="ready"}`，[15]）。
- F4 又暴露了一个盲点：唯一的 Prometheus 和主库在同一个节点上，告警在切断 35 秒后进入 pending，但它们真正触发是在节点解冻之后，那时故障切换早已结束[16]。

8.6 实验 2：时间点恢复：找回被误删的订单

`experiments/e7/pitr.sh` 在 k6 以每秒 5 次迭代下单的同时制造事故：一个本应删除 30 天前失败订单的清理任务，以 orders 的角色执行了 `DELETE FROM orders WHERE created_at < now()`，并在日志中记下自己的开始时间（服务器时钟）。60 秒后事故被“发现”，恢复过程是：

1. 在生产集群旁边建一个单实例的恢复集群 `pg-pitr`，从归档还原到任务开始的时刻（`experiments/e7/pg-pitr.yaml.tpl`；它不归档，以免写进它正在读取的归档）；
2. 把那里的 orders 表经暂存表复制回生产库，用 `INSERT ... ON CONFLICT (id) DO NOTHING`：事故之后的新订单保持不动；
3. 用事故前取得的订单号清单（只用于核对）检查结果，然后删除 `pg-pitr`。

假设在运行前提交[17]，结果[18]：

	pitr-1：任务删除全部订单	pitr-2：只删除最近 5 分钟的订单
删除的行	211 350	141（任务期间只写了 12 kB 的 WAL）
第一次还原	90 秒后成功	**64 秒后失败**
第二次还原	—	强制切换 WAL 段后，95 秒
复制回生产库	211 350 行，事故前的订单无一缺失	141 行，无一缺失
事故后的新订单	212 个，未受影响	289 个，未受影响
从发现到数据回来	**103 s**	**173 s**

我的假设是第一次还原一定失败：事故前刚关闭了一个 WAL 段，下一个段要等 5 分钟才会归档。pitr-1 推翻了它：删除 21 万行本身就写出了足够的 WAL，40 秒内写满了三个段并全部归档（AD–AF，最后一个在 15:07:59）。这个担心只对小事故成立，pitr-2 证实了它：在目标时刻之后的 WAL 还留在主库当前段里，恢复重放完归档中的全部内容后停下，报出“FATAL: recovery ended before configured recovery target was reached”[19]。**恢复之前，先在主库上执行一次 `SELECT pg_switch_wal()`**。两次还原都跨过了故障切换演练留下的多条时间线（基础备份在时间线 2，当时的主库在时间线 10）。

复制回来的行是目标时刻的版本。pitr-1 中有一笔订单在目标时刻之前 175 毫秒创建，之后的确认因为行已被删除而没有落到它身上，于是以 PENDING 状态回来了；而 outbox 里还留着它的 `orders.confirmed` 事件（在目标时刻之后 125 毫秒写入，已经发布，客户收到了确认通知）。按 outbox 中目标时刻之后的事件修正这类订单，这一次修正了 1 笔[20]。如果把生产库本身还原到目标时刻，事故之后的 212 笔和 289 笔新订单都会丢失；在旁边还原、再把行复制回来，才能两者兼顾。

8.7 实验 3：Velero：备份“已完成”，数据却不在

Velero 1.18.4 把 Kubernetes 对象备份到 `velero` 桶，卷的内容由每个节点上的节点代理用 kopia 复制（文件系统备份）。local-path 供给器不支持 CSI 快照，所以没有卷快照。这个平台上的每一个持久卷都来自 local-path，而且都是 hostPath 类型的卷。假设在安装之前提交[21]。

V1：对 data、secrets、observability、monitoring 四个命名空间做一次备份，要求复制所有卷。备份的状态是 **Completed**，612 个对象，13 条警告，0 个错误。11 个持久卷没有一个被复制，日志中每个卷一行：“Volume pgdata in pod data/pg-main-1 is a hostPath volume which is not supported for pod volume backup, skipping”。被复制的反而是 emptyDir：PostgreSQL 的临时目录（每个实例 66–99 MB）、Grafana 的存储（55 MB）。备份中有 22 个 Secret，任何能读这个桶的人都能读到它们[22]。

V2：在测试命名空间 e7-velero 中建两个 PVC，一个使用默认设置（hostPath 卷），一个带注解 `volumeType: local`（供给器随之创建 local 类型的卷），在两个卷中各写一个标记文件，然后备份、删除命名空间、还原。还原开始 12 秒后应用就在运行；local 卷上是原来的标记，hostPath 卷是空的，应用在启动时写了一个新的标记[23]。原因在于节点代理通过 kubelet 的 Pod 目录读取卷，local 卷挂载在那里，hostPath 卷不在。

两个陷阱值得记住。第一，有 11 个卷没被保护的备份状态仍是 Completed，而不是 PartiallyFailed：跳过只算警告。第二，同一份日志对这些卷写着“Skipping snapshot of pv … because it is backed up with PodVolumeBackup”，备份自己的“跳过的卷”摘要只给出快照方面的原因，`velero backup describe --details` 的卷列表里干脆没有它们。只有还原测试（V2）或逐条阅读警告才能发现数据不在。所以在这个平台上，Velero 只负责 Kubernetes 对象（每天一次全部命名空间的备份），不负责任何数据；数据各有自己的备份。

8.8 实验 4：OpenBao 的快照与恢复演练

一个 raft 存储正在运行时，复制它的文件得不到一致的副本，所以 OpenBao 的备份用它自己的快照：一个 CronJob 每小时用自己的服务账号登录（Kubernetes 认证，角色只能读 `sys/storage/raft/snapshot`），保存快照，再上传到 `platform-snapshots`（`gitops/platform/manifests/backup/openbao-snapshot.yaml`）。快照用 OpenBao 的屏障密钥加密：没有集群外保存的解封密钥，它既不能读，也不能恢复。

恢复演练（S1，`experiments/e7/openbao-rehearsal.sh`）在生产实例旁边装一个空的 OpenBao，用 `platform/secrets/openbao-restore.sh` 恢复最新的快照：

1. 用一次性的密钥初始化并解封，因为快照只能恢复到一个正在运行、已解封的服务器上；
2. 从对象存储取回快照，执行 `bao operator raft snapshot restore -force`；
3. 服务器随即重新密封，只能用**原来的**解封密钥打开；
4. 一次性的根令牌被拒绝，原来的根令牌重新有效。

恢复出来的 `secret/insight/api-key` 与生产实例的 SHA-256 相同，整个演练 35 秒[24]。假设 S1a–S1d 全部符合[25]。

8.9 实验 5：整个集群损毁之后

最后一个实验删除整个集群：`kind delete cluster --name ent`。删除时写入器和 k6 都在运行，写入器的日志实时传到集群外。集群之外保留的东西是：Git（GitHub，以及 Argo CD 读取的 GitLab）、本地镜像仓库和拉取镜像、对象存储 `backup-s3`、`~/.lab-secrets` 中的密钥文件。恢复步骤事先写好（`experiments/e7/rebuild.sh`）：

1. 修改 `pg-main.yaml`：数据库从丢失的集群的归档中诞生（最后一份基础备份 + 归档中的全部 WAL），并以新名字归档；一个归档只能有一个写入者，每次重建都把两个名字向前推一步（pg-main → pg-main-r1 → pg-main-r2）；
2. `create.sh`、`bootstrap.sh`，之后一切由 Argo CD 从 Git 收敛；
3. 恢复 OpenBao，为实验用户重新签发证书（新集群有新的 CA），设置负载均衡器的连接数上限。

8.9.1 第一次重建（R1）

假设在运行前提交[26]，结果[27]：

T0 之后	事件
0–54 s	集群被删除
67–230 s	新集群（kind 创建用了 46 秒）、Argo CD、根应用
370 s	**monitoring Degraded**：“pods “prometheus-kps-prometheus-0” is forbidden: no PriorityClass with name platform-critical was found”
545 s	人工介入一：从 Git 手工应用 PriorityClass

T0 之后	事件
695–826 s	**openbao 一直 Progressing**；人工介入二：把计划中的 OpenBao 恢复提前执行
873–1093 s	pg-main 从归档恢复（恢复作业 22:52:07 开始，实例 22:53:36 启动），副本加入
1171 s	34 个应用全部 Synced、Healthy
1205 s	**第一笔订单**（20.1 分钟）；k6 60 秒，509 个请求，0 失败

实测数据

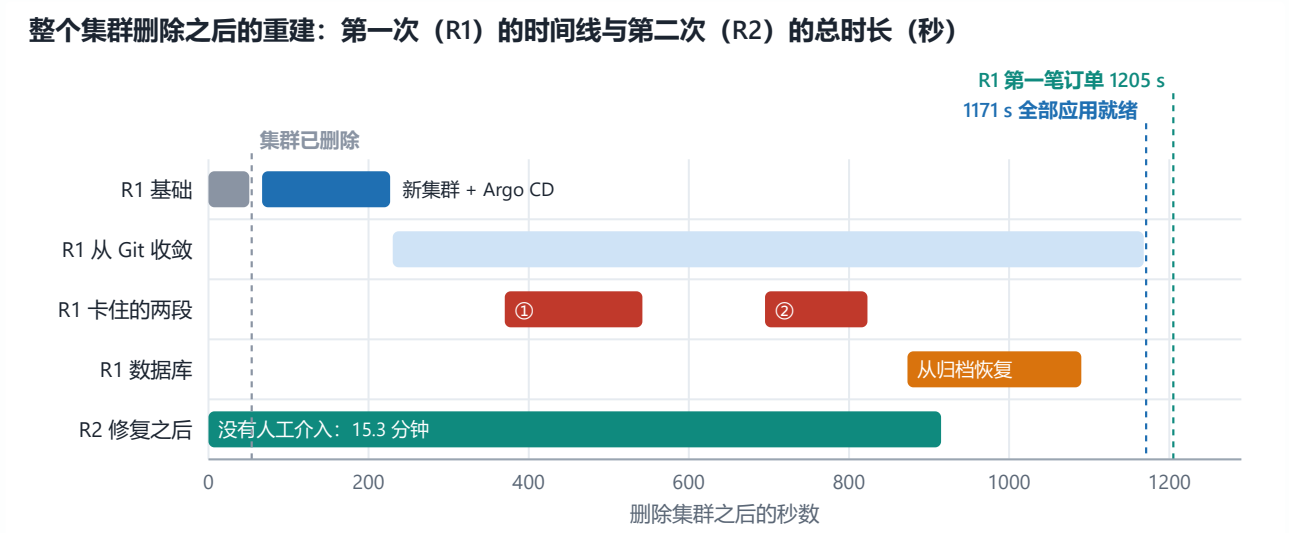


图 8-5 第一次重建的时间线与第二次的总时长。R1 的 20.1 分钟里有两段是“卡住”：① 一个只存在于旧集群里的 PriorityClass，② 一个要先恢复才能就绪的 OpenBao——两者都是平时看不见的顺序依赖，只有在空集群上才会暴露。把它们写进 Git 之后，R2 无人介入，15.3 分钟接到第一笔订单。

数据和第一份假设完全一致：最后一个归档的段在 22:34:01.9 进入归档，最后一条被恢复的写入是 22:34:00.481 确认的；之后到删除时刻的 202.5 秒里确认的 **10 550 次写入全部丢失**，在那之前的一次都没有丢。k6 在删除前得到确认的 147 笔订单（另有 2 笔在删除节点的过程中得到确认）也都在这个窗口里，全部丢失。OpenBao 从删除前 11 分钟的快照恢复，密钥未变，外部密钥重新同步；JetStream 从 204 770 条消息归零，Prometheus 没有任何删除之前的样本[28]。

两次人工介入都是同一类问题：**同步波次在全新的集群上死锁。**

- PriorityClass 是第 7 章（E6）加进 policies 应用（波次 −1）的，而第一批使用它的 Prometheus 和 OpenBao 在波次 −3。在运行着的集群上它一直存在，只有空集群才暴露这个顺序错误；根应用要等波次 −3 健康才创建后面的应用，于是互相等待。
- 一个新的 OpenBao 没有初始化、处于密封状态，按 chart 默认的就绪探针永远不会 Ready，Argo CD 报告它 Progressing，根应用于是不开始任何后续波次：PostgreSQL 连恢复都无法开始，只能等一个人去解封 OpenBao。

8.9.2 修复与第二次重建（R2）

两项修复都写进 Git，并先在重建后的集群上验证[29]：PriorityClass 移到一个新应用 `platform-baseline`（波次 −8），并加上 `Prune=false`，迁移时没有被删除；OpenBao 的就绪探针改为“服务器在运行”，密封和未初始化也算就绪，就绪不再代表能提供密钥。

第二次重建（假设：[30]，结果：[31]）没有任何人工修复：OpenBao 在未初始化时就 Ready，它的恢复与其他组件并行进行；35 个应用在 15.0 分钟时全部健康，**第一笔订单在 15.3 分钟**（R1 为 20.1 分钟），比假设的 12–15 分钟略长。剩下的时间主要花在波次 −3（kube-prometheus-stack、Kyverno、KEDA、cert-manager、External Secrets、OpenBao，共 4.6 分钟）和 PostgreSQL 的恢复（约 2.5–3 分钟）上，各波次依次等待。RPO 仍然等于归档的落后：55.9 秒、2956 次写入、约 62 笔订单。OpenBao 这次从约 45 分钟前的快照恢复：R1 之后的集群只存在了 20 分钟，每小时一次的快照还没来得及执行。

8.9.3 重建清单

一次重建需要的全部东西，都必须在集群之外：

东西	在哪里	丢了会怎样
平台的全部配置	Git：GitHub 上的源，GitLab 上 Argo CD 读取的副本	无法重建
镜像	本地镜像仓库与拉取镜像（第 1 章）	重新构建和下载，慢但可行
PostgreSQL 的基础备份与 WAL	`backup-s3` 的 `pg-backups`	全部订单丢失
OpenBao 快照	`backup-s3` 的 `platform-snapshots`	全部密钥丢失
解封密钥与根令牌	`~/.lab-secrets/v2/openbao`	快照无法打开
数据库角色密码、对象存储密钥、GitLab 令牌	`~/.lab-secrets`	重新生成或重新签发

8.10 局限

- **“异地”在同一台机器上。**对象存储、GitLab 和集群共用一台主机、一块磁盘。真正的异地备份在另一个区域或另一个账号里，并且能防止删除。Garage 没有对象锁，一个泄露的密钥就能删掉全部备份。
- **单节点的对象存储，单副本的 Prometheus。**前者没有冗余，后者与主库同节点时监控会失明（F4）。
- **每种情况只做了一次或两次**，没有统计意义上的重复；时长受这台机器负载的影响。
- **写入器只测数据库这一层。**用户看到的中断还取决于应用怎样连接、怎样重试（F0）；k6 只在重建实验中参与了测量。
- **密钥的保管是实验室级别的。**解封密钥只有一份，与根令牌一起以文件形式保存在一台机器上。生产环境应当把解封密钥分成多份分别保管，或者用 KMS 自动解封，根令牌用后即撤销。第 12 章的事故 E11-1 中，这份唯一的密钥被一个恢复脚本清空，本章的 OpenBao 快照因此全部无法打开。
- 这些数字来自这个实验环境，不代表任何生产系统的规模或经验。

8.10.1 从实验环境到生产环境

本章的做法	生产环境通常的做法	差别意味着什么
“异地”备份是同一台机器上的另一个容器，没有对象锁	另一个区域、另一个账户的对象存储，开启不可删除的保留策略	本机磁盘损坏或密钥泄露会同时带走数据和备份（第 12 章的云迁移分析）
每种故障演练一次，数据库只有几百 MB	定期演练，数据量大几个数量级	恢复时间随数据量和 WAL 量增长，本章的 RTO 不能外推
解封密钥和备份凭据是一台机器上的文件	KMS、分片保管、访问审批与轮换	打不开的备份等于没有备份（第 12 章的事故 E11-1）
三个数据库实例在同一台机器的三个“可用区”里	跨可用区的同步复制，跨区域的异步副本	真实的跨区延迟会直接加到同步复制的每一次提交上

8.11 常见问题

为什么不禁止 F3 那样提升一个落后很多的副本？ CloudNativePG 1.30 总是提升最先进的副本，没有按延迟拒绝提升的选项；Patroni 一类的工具有（`maximum_lag_on_failover`）。拒绝提升就意味着不可用，这本身就是 RPO 和 RTO 之间的选择。本章的选择是同步复制：已确认的提交至少存在于一个备库上，问题从根本上不再出现，代价是 F4 那样的写入停顿。CloudNativePG 的 `failoverQuorum` 进一步保证提升的副本确实拥有全部已同步的提交，本章没有测试它。

同步复制会让写入变慢多少？ 取决于到备库的往返时间。这里三个“可用区”在同一台机器上，提交延迟的 p50 从约 3 毫秒增加到约 6 毫秒；跨真实可用区（往返 1–2 毫秒）时会多出这么多，跨区域（几十毫秒）时同步复制通常就不现实了。

为什么 Velero 不能备份这些卷？ 节点代理只能读挂载在 kubelet Pod 目录下的卷，而 local-path 默认创建的 hostPath 卷不在那里（V1、V2）。可以改用 local 类型的卷，或者使用支持 CSI 快照的存储；但对数据库来说，应用自己的备份（一致的快照、连续归档、时间点恢复）本来就比复制文件更好。

`archive_timeout` 该设多小？ 它决定了整个集群丢失时最坏的 RPO。设为 1 分钟，最坏情况从 5 分钟降到 1 分钟，代价是归档中多出 5 倍的段，好在不满的段压缩后很小。更进一步的做法是把 WAL 实时流到集群外（另一个区域里的备用集群或 `pg_receivewal`），RPO 就接近于零。

恢复演练多久做一次？ 每次改变备份或恢复路径之后都要做，平时定期做。本章的每一项演练都发现了文档里没有写的东西：插件第一次上线时的原地重启、租约的 16 秒、脑裂之后的副本、Velero 的“完成”、空集群上的波次死锁。没有演练过的恢复，只能算一个希望。

8.12 练习

练习

1. **（计算）** 写入器每秒确认 50 次写入，`archive_timeout` 是 5 分钟。整个集群在一个随机时刻丢失，丢失的写入最多是多少？期望是多少？R1 丢了 10 550 次（202.5 秒），R2 丢了 2956 次（55.9 秒），它们落在你算出的范围里吗？
2. **（分析）** F1 中 operator 3.2 秒就选定了新主库，写入中断却有 26.9 秒。把这 26.9 秒分解成几段，指出哪一段不应缩短，并说明理由。
3. **（设计）** F2 中旧主库在解冻后可写 2.5 秒，丢了 7 次直连写入，还让一个副本分叉。设计两种办法缩小或消除这个窗口，并说明各自的代价。
4. **（分析）** 订单服务的 SLO 是 30 天可用性 99%。按 F3（异步）和 F4（同步）的测量，分别计算一次这样的双重故障用掉多少错误预算、丢掉多少数据。你会为订单数据库选哪一种？为商品目录呢？
5. **（操作）** 有人在 10 分钟前误执行了 `DROP TABLE outbox`。写出恢复步骤，并指出它与 pitr-1 有什么不同（提示：被删除的表还在被 relay 读取；事故之后产生的事件去了哪里）。
6. **（设计）** V1 的备份状态是 Completed。写一个检查（告警规则或脚本），能在第一次备份之后就发现有持久卷没有被保护。
7. **（设计）** R2 的 RTO 是 15.3 分钟。提出两项能把它缩短一半的改动，说明代价和风险。

参考答案见附录。

8.13 本章小结

- 先测量，再相信。实验 1 的五组假设中有三组在时长上被推翻：计划内的滚动更新变成了原地重启和 3 分钟拒绝新连接；每次计划外的切换都要多等 16 秒的租约；冻结的节点要 50 秒才被发现。数据丢失方面的假设全部得到证实。
- 异步复制加上副本落后，再失去主库，丢失的就是落后的全部（F3：3017 次已确认的写入）；同步复制把同样的双重故障变成写入停顿（F4：0 丢失，2 分钟的中断），每次提交的延迟约为原来的两倍。结果未知的写入可能已经提交，只能靠幂等的重试处理。
- 脑裂真的会发生：被冻结的旧主库在恢复后可写 2.5 秒，直连它的客户端丢了 7 次写入，一个副本因此分叉并悄悄停止复制 17 分钟。集群报告“healthy”，延迟指标是 0，告警规则也要先验证再信任。
- 时间点恢复找回了误删的 21 万笔订单（103 秒）和 141 笔订单（173 秒），且没有丢失事故之后的订单。归档最多落后 `archive_timeout`，小事故之后恢复前要先切换 WAL 段；复制回来的是目标时刻的版本，之后的变化要从 outbox 补回。
- Velero 在 11 个卷一个都没保护的情况下报告“Completed”。备份是否有效，只有还原才能证明；数据由应用自己的备份保护（PostgreSQL 的归档，OpenBao 的快照）。
- 整个集群丢失后，第一次从 Git 和备份重建用了 20.1 分钟，暴露了两个只有空集群才会出现的同步波次死锁；修复之后第二次用了 15.3 分钟，没有人工修复。RPO 等于归档的落后：202.5 秒和 55.9 秒。

第 9 章

发布治理（E8）

到第 8 章为止，平台已经能扩容、隔离租户、承受故障、在整个集群丢失后重建，但“谁能把什么改动送进生产、送得多快、出了错怎么收回”仍然只靠约定。本章把发布变成一条有关卡的路径：每个改动都是一个合并请求，流水线在合并之前执行与集群准入相同的策略；入口改为按请求分流，金丝雀精确到 5%；Argo CD 以各项目自己的账号写入集群；冻结期由两把钥匙把守；每一次部署都自动记录下来，DORA 指标由这些记录计算。每一项都先写下假设再运行，七条假设被测量推翻；在空集群上重建时，还暴露出两个只有从零开始才会出现的顺序依赖。

工程问题

配置都在 Git 里了，可是“谁能把什么改进生产、出了问题多快能撤回”还没有答案。怎样让每一次变更都经过检查、只影响一小部分用户，并且留下可以度量的记录?

实测结论

- 合并就是发布决定：违规的改动在 31–48 秒内被门禁的流水线拒绝，无法合并；合规的改动合并后 27 秒完成滚动更新。
- 第 7 层入口让扩出的 Pod 立即分到请求，但台阶负载下仍有 57.20% 的请求失败：入口解决了“分担”，没有解决“过载”。三个假设被推翻。
- 5% 的金丝雀实测为 4.93%（k6）和 5.03%（Prometheus）；出故障的金丝雀只伤及 2.45% 的请求，经门禁的撤回在决定后 46.1 秒生效。
- 同步冒用把项目边界交给 Kubernetes 的 RBAC 执行；根应用因为没有对应的账号，7 分钟无法同步。
- 在空集群上重建，才暴露出只写在注释里的顺序假设；修复之后的重建无人介入，636 秒全部就绪。

9.1 学习目标

学完本章，你应当能够：

- 说明在 GitOps 中“合并就是发布决定”意味着什么，把策略检查放到合并之前，并让平台拒绝绕过它的推送；
- 解释第 4 层与第 7 层入口在扩容时的差别，用带权重的路由和请求头做金丝雀与预览，估算一次失败的金丝雀会伤及多少请求；
- 配置 Argo CD 的同步冒用，让 Kubernetes RBAC 而不只是 Argo CD 本身执行项目边界，并避开根应用、命名空间创建和开启顺序上的陷阱；
- 用两把钥匙实施变更冻结，并为紧急变更留出可追溯的通道；
- 从合并请求和部署记录自动计算 DORA 指标，知道这些数字漏掉了什么。

实验性质

本章的实验都是 `[真实]` 运行的：GitLab、流水线、Argo CD 和集群都是真实的软件，负载由 k6 产生。金丝雀的故障是 `[注入]` 的（服务的 `/admin/faults` 让一半的请求返回 503）；冻结演练中的“紧急情况”是 `[模拟]` 的，并没有真的发生连接池耗尽。合并请求由脚本代替两个角色提出：管理员 root 和开发者 alice（团队 shop）。实验时长以分钟计，DORA 指标反映的是一个实验日，不是一个团队的长期交付能力。

9.2 原理

9.2.1 合并就是发布决定

在推送式的持续交付里，流水线最后一步把制品部署出去，审批和检查都可以挂在这一步之前。GitOps 把方向倒了过来：Argo CD 不停地把集群拉向 main 分支所描述的状态（本平台每 60 秒轮询一次，`timeout.reconciliation: 60s`）。于是 main 上出现什么，集群里就会出现什么，**合并本身就是部署决定**。由此得出三条要求：

1. **所有检查必须在合并之前完成。**集群的准入策略（第 6 章的 Kyverno）仍然是最后一道防线，但它在合并之后才起作用：被拒绝的改动已经留在 main 上，Argo CD 会一遍遍重试，main 与集群从此不一致。所以本章把同一套策略“左移”：流水线先按 Argo CD 的方式渲染出将要部署的清单，再用 Kyverno CLI ——集群准入所用的同一个引擎——检查它们，不通过就不能合并。
2. **main 只接受合并。**任何人（包括管理员）直接推送到 main，都会绕过上面的检查，平台必须拒绝它，而不是靠大家自觉。
3. **每一次部署都应当能追溯到一个合并请求。**这是后面计算 DORA 指标的前提，也是冻结期的紧急变更能被事后审查的前提。

9.2.2 第 4 层与第 7 层入口

第 5 章的扩容实验留下一个问题：HPA 已经扩出新的网关 Pod，它们却几乎收不到请求。原因在入口的层次。第 4 层负载均衡（这里是 cloud-provider-kind 的负载均衡器加 Service 的 iptables 规则）分配的是**连接**：连接建立时选定一个后端，此后这条连接上的所有请求都去同一个 Pod。k6 的虚拟用户使用长连接，扩容发生时连接早已建好，新 Pod 只能等新连接。第 7 层入口（本章用 Gateway API，由 Istio 的 Envoy 实现）终结客户端连接，**按请求**在当前就绪的端点之间分配，新 Pod 一就绪就会分到请求。代价是多了一跳代理；本章测量了这一跳的代价。

9.2.3 金丝雀、预览与爆炸半径

金丝雀是新版本的第二个发布（这里是 `gateway-canary`），与稳定版并存。HTTPRoute 的 `backendRefs` 按权重分流（95/5），每个请求独立抽签；带请求头 `x-storefront-preview: canary` 的请求则总是去金丝雀，测试人员可以在零权重时先行验证。

示意图

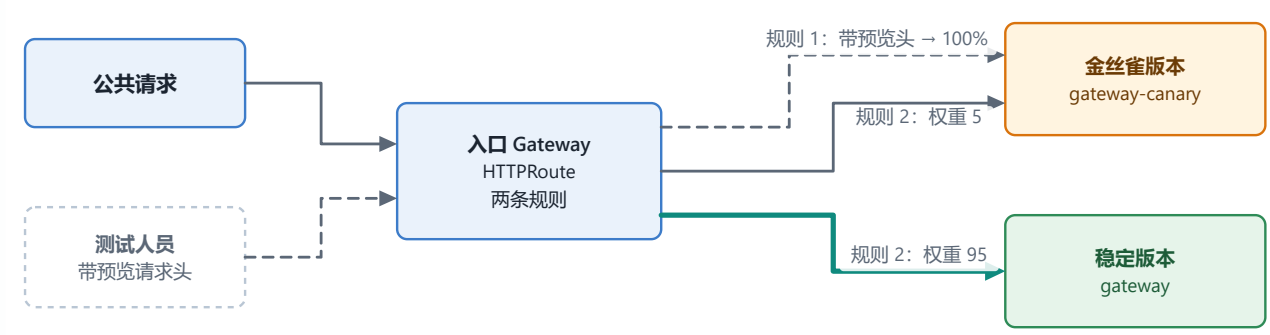


图 9-1 金丝雀的两条路由规则。带预览头的请求全部去金丝雀——团队可以在放量之前自己验证；其余的公共请求按权重分流。新版本出问题时，受影响的比例大约是“权重 × 失败率”，这就是爆炸半径。

一次失败的金丝雀伤及的请求比例约为：

$$\text{受影响比例} \approx \text{金丝雀权重} \times \text{金丝雀的失败率}$$

权重 5%、金丝雀失败一半时约 2.5%。第 7 章的浏览 SLO 是 99.5%，只允许 0.5% 的失败：这样一个金丝雀必须在几分钟之内被发现并撤回，而撤回本身也要经过门禁。

9.2.4 同步冒用：项目边界由谁执行

第 6 章用 Argo CD 的项目（AppProject）限制每个团队能部署到哪些命名空间、能创建哪些种类的对象。但这个限制只由 Argo CD 检查；真正写入集群的是 Argo CD 应用控制器自己的服务账号，它在集群里几乎可以做任何事。一旦项目定义过宽（例如允许“核心组的所有种类”，其中就包括 Secret），或者 Argo CD 自身有缺陷，Kubernetes 不会提供第二道检查。

同步冒用（Argo CD 2.13 引入，仍为 alpha）改变了写入的身份：控制器借助 Kubernetes 的 impersonation 机制，以项目为目标位置指定的服务账号（`destinationServiceAccounts`）执行同步。API 服务器按被冒用的账号做 RBAC 判断，审计日志同时记下真实的请求者和被冒用的用户。前提是控制器有 `impersonate serviceaccounts` 权限，而每个项目都必须有自己的账号——包括创建其他项目的根应用所在的项目。

9.2.5 变更冻结的两把钥匙

在 GitOps 中，冻结需要两把钥匙。第一把在合并之前：冻结期内，普通的合并请求不能合并（GitLab 的部署冻结期让流水线带上 `CI_DEPLOY_FREEZE` 变量，冻结检查据此失败），只有标明 `[emergency]` 的紧急变更可以通过，而且这个标记永久留在历史里。第二把在 Argo CD：同步窗口（`syncWindows`）中的 deny 窗口期间不做自动同步，已经合并进 main 的改动也被挡住，只有操作员手动同步（`manualSync: true`）能放行。只有第二把钥匙时，冻结期间 main 会不断前进，窗口一结束，积压的改动同时生效；只有第一把钥匙时，绕过流水线进入 main 的改动（或者自愈）照样生效。

示意图

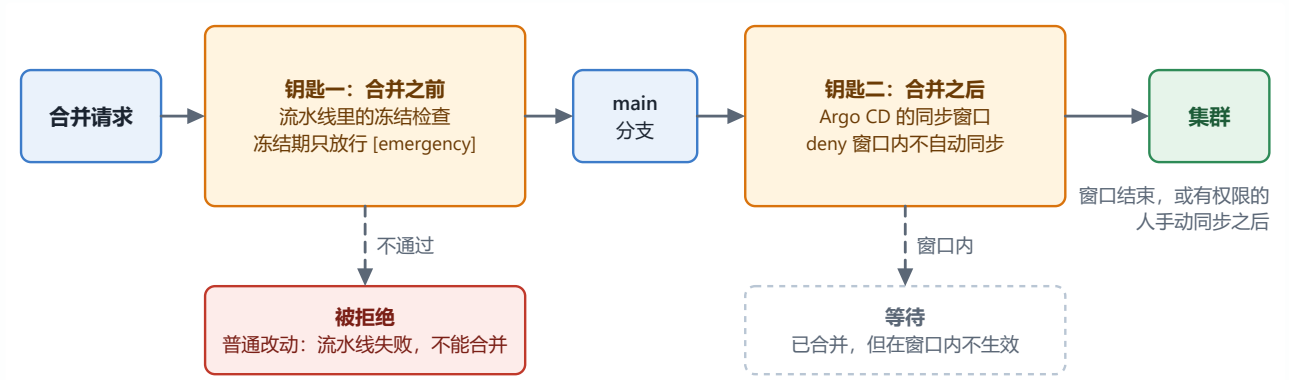


图 9-2 变更冻结的两把钥匙。第一把在合并之前：冻结期间，流水线只放行标明紧急的改动。第二把在合并之后：即使改动已经进入 main，Argo CD 的同步窗口也不把它带进集群。只有两把钥匙都打开，改动才会生效。

9.2.6 DORA 指标从哪里来

DORA 的四个指标衡量交付的速度与稳定性：部署频率、变更前置时间（从提交到在生产运行）、变更失败率（需要回滚或修复的部署所占比例）、恢复时间（失败的部署到恢复）。它们都可以从系统里已有的记录计算：合并请求给出“改了什么、何时提出、何时合并”，部署记录给出“何时在生产生效”。困难在于对应关系——一个合并请求影响哪些应用、哪一次部署包含它——以及不经过合并请求的改动：它们在这些记录里根本不存在。

示意图

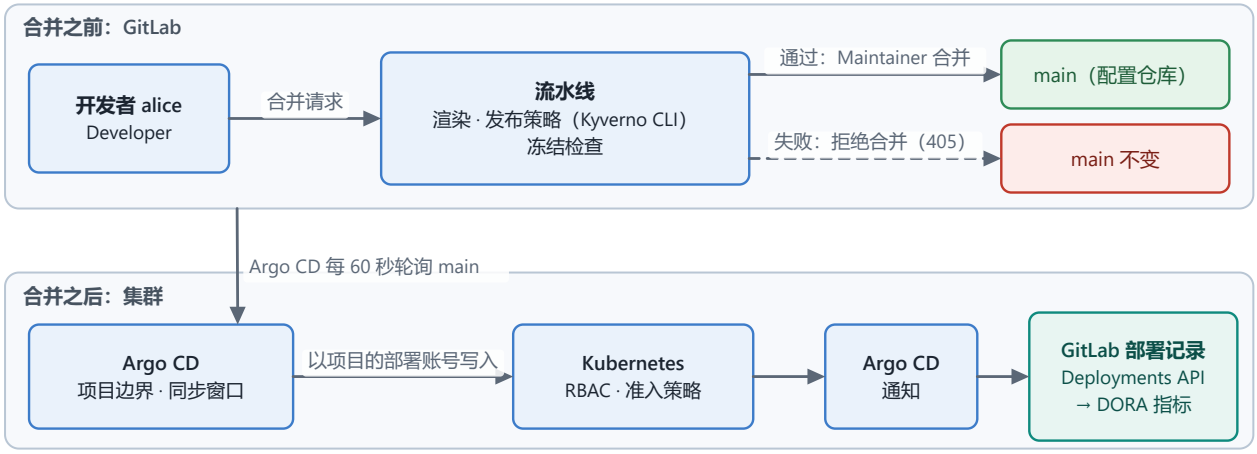


图 9-3 本章的发布路径：每个改动都是一个合并请求，每道关卡都留下记录

9.3 实验 1：合并请求门禁

配置仓库 `root/platform-gitops` 的 main 设为受保护分支：推送权限“无人”，合并权限 Maintainer，并开启“流水线成功才能合并”（`platform/gitops/setup.sh`）。流水线（`gitops/.gitlab-ci.yml`）有两个作业：`render` 用 Helm 4.3.1 按 ApplicationSet 的方式渲染每个服务的发布（`gitops/ci/render.sh`）；

`release-policies` 用 Kyverno CLI 1.19.2 检查渲染结果——镜像来自 `localhost:5001` 且带固定标签，有资源请求和内存限制，有就绪探针；生产环境至少两个副本、自动扩缩容的下限不低于两个。流水线镜像把 Helm 和 Kyverno CLI 装在一起（`platform/gitops/ci-tools/`），因为官方的 kyverno-cli 镜像里没有 shell。

接入门禁之前，先在本地跑了一遍同一条流水线，结果发现了一个问题：团队 insight 的 insight-api 没有就绪探针——门禁一旦打开，这个租户的下一个改动就会被拒绝。修复和门禁本身放在同一个改动里。假设在接入之前提交[2]：

	假设	测量	
G1	引入门禁的改动自己就要通过门禁	合并请求 !1，流水线 50 s 成功，合并	符合
G2	alice 把 catalog 的镜像改为 `:latest`：release-images 失败，GitLab 拒绝合并	流水线 48 s 失败于 `release-images`；合并返回 HTTP 405	符合
G3	去掉 catalog 的内存限制：release-resources 失败	31 s 失败于 `release-resources`；405	符合
G4	生产环境只留一个副本：release-prod-availability 失败	32 s 失败于 `release-prod-availability`；405	符合
G5	合规的改动（catalog 的数据库连接池 10 → 12）：流水线通过；alice 自己不能合并；Maintainer 合并后 Argo CD 在一个轮询周期内完成发布	第二次尝试（!8）流水线通过；alice 合并返回 HTTP 401；root 合并后 27 s，catalog 的滚动更新已经完成（数据库连接池为 12）	符合
G6	直接推送到 main，即使是管理员，也被拒绝	`GitLab: You are not allowed to push code to protected branches on this project`（pre-receive 钩子拒绝）	符合

两处意外值得记下。G5 的第一次尝试（!5）流水线在 15 s 时失败，原因是作业拉取代码时一次没有解释的 HTTP 403；重新提交后通过，这次失败没有复现，原因未查明。更重要的是 G4/G5 的合并请求里混进了无关的文件：`mr.sh` 把工作区的 `v2/gitops` 整个镜像到分支上，而演练进行时工作区里有尚未完成的修改。门禁检查的是“这次合并之后 main 的样子”，而不是“提出者想改什么”——审查者必须看差异，自动化脚本必须从干净的状态出发。此后的演练都遵守一条规则：演练期间不编辑 `v2/gitops`。

门禁把一类事故挡在了合并之前，代价是每个改动多等半分钟到一分钟（本章所有流水线在 15–50 s 之间）。从第 6 章起集群里就有同样的 Kyverno 策略；区别在于失败发生在哪里：在流水线里，main 保持不变，提出者立刻看到原因；在准入控制里，改动已经进入 main，Argo CD 反复同步失败，集群与 main 长时间不一致。

9.4 实验 2：第 7 层入口下重做扩容台阶

第 5 章的结论之一是：HPA 扩出的网关 Pod 收不到请求。本章新增第 7 层入口 `storefront-ingress`（Gateway API 的 Gateway + HTTPRoute，Istio 部署两个 Envoy 副本，`172.20.0.14:8080`），用与第 5

章相同的负载台阶（30 → 170 → 30 迭代/秒，分别持续 120/300/420 s；网关 HPA 2–6 个副本，目标为 500m 请求的 100%）对比两种入口。假设提交在 [3]：

	假设	测量	
H1	第 4 层入口：扩出的 Pod 几乎收不到请求	`hpa-l4`：+48 s 与 +93 s 扩出两个 Pod；高负载阶段的每一分钟都只有原来的两个 Pod 在服务（各 135–150 请求/秒），新 Pod 为 0	符合
H2	第 7 层入口：新 Pod 就绪后约 10 s 内分到请求，负载均匀分布	`hpa-l7`：第一个新 Pod 在它的第一个完整分钟里服务了 79 请求/秒；最后四个 Pod 共同承担（各 22–44 请求/秒）	符合
H3	第 7 层入口：失败只出现在第一分钟，低于该阶段的 5%	高负载阶段失败 57.20%（`hpa-l7`）和 49.18%（复测 run A）	**推翻**
H4	第 7 层入口在中位数上多花 1–2 ms	浏览请求低负载阶段的中位数 11.6 ms（L7）对 12.4 ms（L4），测不出代价	**推翻**
H5	复测：入口重试了大量请求	重试 **0** 次（`envoy_cluster_upstream_rq_retry`）；Istio 默认的路由重试策略里 `retriable-status-codes` 为空，503 不会被重试	**推翻**

真实截图



图 9-4 第 4 层入口（上，`hpa-l4`，2026-11-10 00:17:30–00:32:00 UTC）与第 7 层入口（下，`hpa-l7`，00:34–00:48 UTC）下，同一个 170 迭代/秒台阶中每个网关 Pod 的请求速率（发布看板）。上图：HPA 在 00:21 前后扩到 4 个 Pod，高负载阶段却只有原来的两个 Pod 在服务（各约 135–155 请求/秒），新 Pod 到台阶结束、连接重建之后的 00:26:30 才出现；下图：两个新 Pod 一就绪就分到请求（00:38:45 与 00:41 起），最后四个 Pod 平分负载。

H1 和 H2 说明第 7 层入口做到了第 5 章要求的事：新 Pod 立刻分担负载。H3 的推翻更有意思：负载分散了，失败却没有减少。run A 的高负载阶段，入口把 66 409 个请求转给网关 Pod，得到 40 704 个 503、19 540 个 200、3067 个 201、745 个 504；网关自己的拒绝计数器显示 `circuit_open` 40 010 次——是网关对过载下游的断路器（第 7 章引入的按服务熔断）拒绝了大部分请求。**容量的上限在网关后面**，多几个网关 Pod 并不增加它。

时间花在哪里也出人意料：网关自己测得的 p95 是 387 ms，入口为同一批请求测得的 p95 却是 9446 ms（`istio_request_duration_milliseconds`，reporter 为 source）。请求在入口的 Envoy 与网关应用之间排了队，而入口的待处理请求溢出计数为 0。原因没有查明，候选是过载时网关 Pod 的 accept 队列和 ambient 的 HBONE 路径；入口的连接与待处理指标不在导出的统计之中。k6 在台阶开始后 14–21 s 就达到了 600 个虚拟用户的上限并开始丢弃迭代，因此两种入口的总数不能直接比较。保留的结论是有限的：第 7 层入口解决了负载分配，没有解决超出系统容量的台阶。

9.5 实验 3：5% 的金丝雀、预览与撤回

金丝雀是网关的第二个 Helm 发布 `gateway-canary`：同一个 chart，用新增的 chart 值 `app: gateway` 运行网关程序，镜像为 `8ef5584`，这个版本在每个响应里加上 `X-Storefront-Version` 头（稳定版 `fd38028` 早于这个头，“没有这个头”就表示稳定版）。HTTPRoute 有两条规则：带 `x-storefront-preview: canary` 的请求去金丝雀，其余按权重分配。每一步都是一个经过门禁的合并请求（流水线 32–38 s）：!10 权重 0，!11 权重 5，!12 撤回，!13 提升。假设提交在 [4]：

	假设	测量	
C0	金丝雀通过门禁；权重 0 时不收公共请求，只收预览请求	!10 用 33 s 通过；15 s 探测：300 个不带头的请求全部由 fd38028 回答，300 个带预览头的全部由 8ef5584 回答	符合
C1	权重 95/5：20 000 个请求中 5.0% ± 0.3% 由金丝雀回答	k6 每秒 100 个请求、持续 200 s：**20 001 个中 986 个 = 4.93%**（2 个失败，都由稳定版回答）；同一窗口 Prometheus：20 019 个中 1007 个 = **5.03%**	符合
C2	在 95/5 的权重下，预览头仍然 100% 去金丝雀	400 个中 400 个	符合
C3	有故障的金丝雀（50% 返回 503）伤及约 2.5% 的请求；撤回在合并后 30 s 内生效，从决定起约 1.5 min	决定之前 90 s：金丝雀回答 3596 个中的 178 个（4.95%），**失败 88 个 = 2.45%**，全部来自金丝雀；决定 → 合并 43 s；**最后一个金丝雀回答出现在合并后 2.8 s、决定后 46.1 s**；合并后 154 s 内 0 个失败	符合（更快）
C4	提升时没有失败的请求	提升期间 8697 个请求 **0 个失败**；合并后 55.5 s 最后一次由 fd38028 回答；2.7 min 后 ApplicationSet 删除了金丝雀的 Application 及其资源	符合

C1 的两种测量互相印证：k6 按响应头计数，Prometheus 按服务端计数，两者相差 0.1 个百分点。C3 证实了上面的公式：权重 5%、失败一半，伤及 2.45% 的请求，而且只伤及金丝雀自己的请求。撤回最慢的环节是决定本身和门禁：43 s 中大部分是流水线；路由变更进入集群只用了几秒。对照第 7 章 99.5% 的 SLO，这个金丝雀在被发现之前每多运行一分钟，就多消耗一大块错误预算——金丝雀分析必须在分钟级做出判断，而不是等到第二天看报表。

实测数据

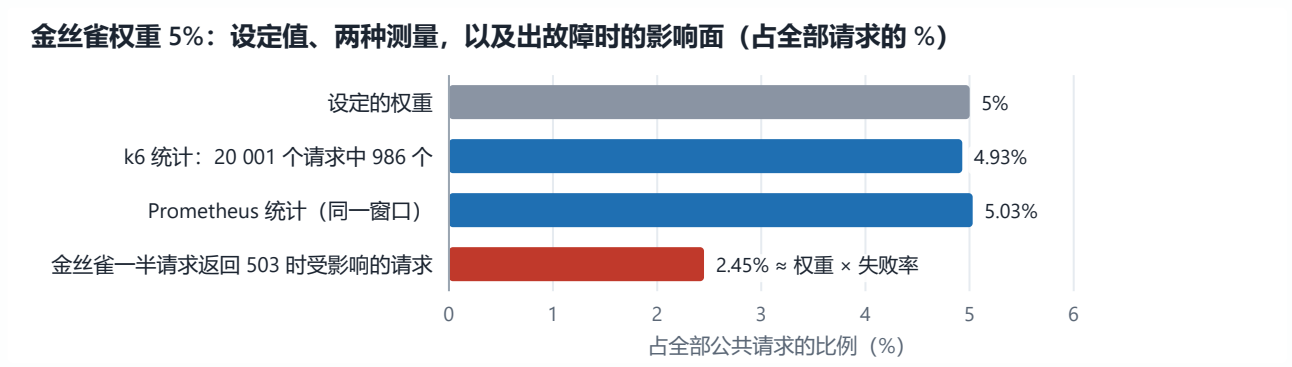


图 9-5 金丝雀实验的四个数字。两种独立的统计都落在设定的 5% 附近；注入“一半请求返回 503”的故障之后，受影响的公共请求是 2.45%——权重（约 5%）乘以失败率（约 50%）。金丝雀没有让故障消失，它把故障的影响面缩小到可以从容撤回的程度。

真实截图



图 9-6 金丝雀的一次完整经历（01:17–01:29 UTC，发布看板）：k6 运行期间金丝雀分到约 5% 的请求（右上）；01:23 的尖峰是只带预览头的 10 秒（C2，100% 去金丝雀）；随后金丝雀被注入 50% 的 503（左下，gateway-canary 的 5xx 最高 1.16 个/秒），01:25:27 撤回的合并之后，它的份额归零。两个发布各有 2 个就绪的 Pod（右下）。

9.6 实验 4：按项目的同步冒用

实验前先确认现状：应用控制器在 `storefront` 命名空间里可以创建 Secret（`kubectl auth can-i create secrets -n storefront --as=system:serviceaccount:argocd:argocd-application-controller` 为 yes），而项目 storefront 允许“核心组的所有种类”。设计如下：

- 团队的部署账号 `storefront/argocd-storefront` 与 `insight/argocd-insight`，Role 只覆盖它们的发布实际渲染出的种类（storefront：Deployment、Service、ServiceAccount、PDB、HPA、ScaledObject、ServiceMonitor；insight：Deployment、Service、ExternalSecret），没有 Secret、Pod 和 RBAC；

- 平台账号 `argocd/argocd-platform`（cluster-admin）随 Argo CD 的 Helm 发布一起创建（`extraObjects`），不放在 Git 里——否则根应用要先同步出自己的权限才能同步；
- 两个临时应用：`e8-probe` 往 storefront 写一个 Secret，`e8-probe-ns` 写一个 ConfigMap 并带 `CreateNamespace=true`。

先在冒用关闭时合并一次（!14），再用 `helm upgrade` 打开 `application.sync.impersonation.enabled`，然后让两个探针再写一次、同时做一个普通发布（!15），最后删除探针（!16）。每一步由 `experiments/e8/impersonation.sh` 记录状态，并从审计日志里取出“谁写了什么”（`experiments/e8/audit_who.py`）。假设提交在 [5]：

	假设	测量	
I1	冒用关闭：`e8-probe` 以 Argo CD 自己的权限创建 Secret，尽管 `argocd-storefront` 无权这么做	03:49:28 Secret 被创建，审计日志：`system:serviceaccount:argocd:argocd-application-controller`，没有被冒用的用户，`create secrets 201`；`argocd-storefront` 的 `can-i create secrets` 为 no	符合
I2	冒用打开：对这个 Secret 的修改被 Kubernetes RBAC 拒绝	同步失败：`secrets "e8-probe" is forbidden: User "system:serviceaccount:storefront:argocd-storefront" cannot get resource "secrets"`——在应用之前读取现状时就被拒绝；每次重试都被拒绝，Secret 保持原样	符合
I3	普通发布以 `argocd-storefront` 的身份成功	orders（把 `LOG_LEVEL: INFO` 写明）：03:54:07 合并，03:55:25 同步完成（78 s），滚动更新完成；审计日志：`patch deployments`、`patch poddisruptionbudgets`，被冒用的用户 `argocd-storefront`，200	符合
I4	平台的应用照常工作；根应用下一次同步以 `argocd-platform` 成功	项目 platform 的应用：是。**根应用：否**——它在项目 `default` 里，`default` 没有账号：`no matching service account found for destination server ... namespace argocd`，重试 5 次；从 !16 合并（03:58）到修复（04:05:33），根应用无法同步	**推翻**
I5	`CreateNamespace=true` 需要读取命名空间的权限	控制器日志：`namespace auto creation failed: namespaces "storefront" is forbidden: ... cannot get resource "namespaces"`；Argo CD 随后打算创建这个命名空间，又被项目检查拒绝（`resource :Namespace is not permitted in project storefront`，这是应用上显示的最终信息）；同一个应用在冒用关闭时同步成功	符合

I4 的推翻揭示了根应用的特殊位置：它创建其他项目，所以不能属于其中任何一个，只能放在 Argo CD 自带的 `default` 项目里——而 `default` 没有部署账号。修复是给根应用一个专属项目 `root`（`gitops/bootstrap/root.yaml`，和根应用一样由人工应用一次），它只允许 `argocd` 命名空间里的 Application、ApplicationSet 和 AppProject，冒用的账号 `argocd-root` 也只拥有这三种对象的 Role。修复后根应用在 04:05:33 同步成功，并以 `argocd-root` 的身份删除了两个探针（`delete applications 200`，2 次）。`default` 保持没有账号：冒用打开之后，放在其中的任何应用都无法同步，这正好关上了一扇后门。

示意图

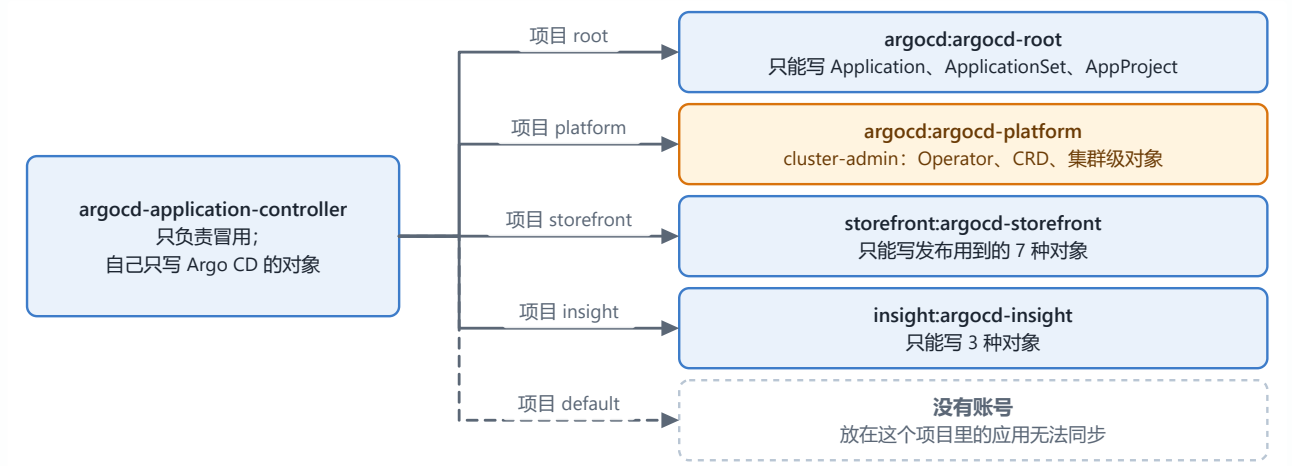


图 9-7 同步冒用之后，Argo CD 以哪个账号写入集群

另外三个发现：

- **开关可能跑在权限前面。**一次 `helm upgrade` 同时带来开关和平台账号：Helm 在 03:50:17.572 写入 `argocd-cm`，在 03:50:19.574 才写入 ClusterRoleBinding。这 2 秒里，一次以 `argocd-platform` 身份执行的试运行被拒绝（403，对象是 Kyverno 的 CRD）。这次无害，但在运行中的系统上应当分两步：先建账号，再打开开关。
- **冒用不只影响同步，也影响比较。**使用 `ServerSideApply=true` 的应用由服务端试运行（`dryRun=All`，字段管理器 `argocd-controller`）计算差异，这些请求同样以项目的账号发出：即使什么都不同步，账号也需要对其应用中所有对象的 patch 权限。
- **审计日志从此能回答“哪个项目改的”。**每次写入都记录被冒用的用户，而不是笼统的“Argo CD”。根应用在约 7 分钟里无法同步，却没有任何告警——同步失败的告警留给第 10 章（E9）。

9.7 实验 5：变更冻结与紧急变更

两把钥匙放在同一段时间上：GitLab 的部署冻结期（项目设置中的 Deploy freezes）和项目 storefront 上的 Argo CD deny 窗口，都覆盖 04:28:00–04:42:00 UTC（`experiments/e8/freeze-drill.sh 5 14`，记录[6]）。窗口和冻结检查本身由合并请求 !18 引入，演练结束后由 !21 移除；项目里保留一个年终冻结窗口（12 月 24–26 日）。冻结检查是流水线的一个作业，只在冻结期且不在 main 上时出现：

```
freeze:
  stage: render
  rules:
    - if: $CI_DEPLOY_FREEZE && $CI_COMMIT_BRANCH != $CI_DEFAULT_BRANCH
  script:
    - |
      case "$CI_COMMIT_MESSAGE" in
        *"[emergency]"*) echo "deploy freeze active - emergency change, allowed: $CI_COMMIT_TITLE"
         ;;
```

```
        *) echo "deploy freeze active - only emergency changes ([emergency] in the commit message) m
         ay merge"; exit 1 ;;
      esac
```

假设提交在 [7]：

	假设	测量	
F1	冻结期内，alice 的普通合并请求在冻结检查中失败，GitLab 拒绝合并	!19（catalog 改为 `LOG_LEVEL: WARNING`）：流水线 17 s 失败于 `freeze`，日志 `deploy freeze active - only emergency changes ... may merge`；合并返回 HTTP 405；随后关闭	符合
F2	紧急合并请求通过，作业日志写明它是紧急变更，一分钟内合并	!20 `[emergency] orders, catalog: DB_POOL_MAX +2 (pool exhaustion)`：流水线 23 s，`emergency change, allowed`；04:29:00 合并，距提出 28 s	符合
F3	deny 窗口挡住已合并的改动至少 120 s；操作员的手动同步在窗口内一分钟之内放行 catalog	合并后 +30/60/90/120 s，两个应用都是 OutOfSync，最近一次同步仍是 03:49 和 03:55 那次；控制器日志 `Sync prevented by sync window`；04:30:59 手动同步，04:31:23 catalog 已同步且健康	符合
F4	窗口结束后，orders 在 2 分钟内自动同步	窗口 04:42:00 结束，orders 04:42:56 开始同步，结束后 **76 s** 已同步且健康；main 与集群不一致持续了 **14 分钟**（04:29:00 合并 → 04:42:57）	符合
D1	每次同步恰好产生一条 GitLab 部署记录，应用恢复健康后 60 s 内出现	#11 prod/catalog 04:31:22.98（同步结束于 04:31:01），#12 prod/orders 04:43:15.25（同步结束于 04:42:57），都指向紧急提交 3783f524，由项目的机器人账号创建；没有多余的记录	符合
D2	GitLab 把紧急合并请求关联到这两条部署	两条部署都关联了 !20 **和 !18**	符合，另有 !18

这次演练的“紧急情况”是 `[模拟]` 的，连接池并没有耗尽；但两把钥匙的行为都是真实的，也印证了原理中的判断：只靠 Argo CD 的窗口，main 会在冻结期间继续前进，窗口一结束，期间合并的所有改动一起生效；只靠流水线，已经进入 main 的改动（以及自愈）照样会被同步。两者一起使用时，紧急通道有两道明确的手续：提交信息里的 `[emergency]`（永久留在历史中）和一次有记录的手动同步。

9.8 部署事件与 DORA 指标

部署事件由 Argo CD 的通知控制器产生（`gitops/bootstrap/argocd-values.yaml` 的 `notifications`）：storefront 的每个应用通过 ApplicationSet 模板上的注解订阅两个触发器，同步成功且应用健康时、或同步失败时，各发送一次（`oncePer` 为这次同步操作的开始时间），内容是一条 GitLab 部署记录——环境为 `prod/<服务>`，提交为本次同步的版本。凭据是配置仓库的一个项目访问令牌（角色 Developer，这是能创建部署记录的最低角色），通过标准输入写入 `argocd-notifications-secret`（`platform/gitops/deploy-events.sh`）。开启时，六个服务各产生了一条记录，代表它们当时的状态（04:20:07–09）。这些记录保存在集群之外：下一节的重建抹掉了 Argo CD 的全部历史，它们仍然在。

D2 暴露了 GitLab 关联方式的局限：它按环境上两次部署之间的提交来关联合并请求，不看合并请求改了什么，所以只改了冻结配置的 !18 也被算进 catalog 和 orders 的部署。DORA 脚本（`platform/gitops/dora.py`）因此自己做对应：

1. 从 GitLab 取出合并到 main 的全部合并请求，以及每个合并请求改动的文件；
2. 对每个 Argo CD 应用，找出它的“输入”：源路径下的文件和它引用的 values 文件（按 Argo CD 的方式解析相对路径和 `$values/` 引用）；一个合并请求改动了某个应用的输入，就“涉及”这个应用；
3. 部署记录来自 Argo CD 的应用历史（`status.history`，每个应用最近 10 次同步，随集群一起消失）或 GitLab 的部署记录；一个合并请求在某个应用上的部署，是该应用第一次在包含其合并提交的版本上的同步（按 main 的 first-parent 历史判断）；
4. 如果在那次同步之前，同一个应用又合并了一个更晚的合并请求，而且两者间隔超过 10 分钟，那次同步归更晚的那个：前一个改动没有改变这个应用的清单，Argo CD 无事可做（记为“no sync”）。

第 4 条是第一次运行后加上的。第一版把 !9（chart 增加 `app` 值，按设计不改变现有服务的清单）算到了 orders 三小时之后的那次部署上，前置时间成了 181.9 分钟；加上规则后，!9 在六个服务上都是 no sync，只部署了平台的 mesh 应用。

重建之前的快照（[8]，Argo CD 历史，2026-11-09 23:58 – 11-10 04:45）：

范围	部署频率	变更前置时间	变更失败率	恢复时间
全部应用（16 个合并请求）	5.5 小时的记录中 56 次应用部署（折合每天 243 次；按合并请求每天 69 个）	中位数 1.2 min，最长 14.3 min（冻结）	1/16 = 6%：!11（金丝雀权重 5%）被 !12 撤回	7.0 min（部署到部署）
storefront 的六个服务	5.3 小时 12 次（5 个合并请求）	中位数 1.4 min，最长 14.3 min	0/5	-

这些数字需要读者带着三个保留去看：

- **它们描述的是一个实验日。**部署频率折合成“每天 243 次”只是换算，不代表一个团队的稳定节奏；前置时间短到一两分钟，是因为合并请求由脚本提出、立即合并，没有人工审查的等待。
- **失败率只认得留下记录的失败。**金丝雀被撤回（!12 的标题含 rollback）被识别出来；但冒用开启后根应用 7 分钟无法同步，是 `helm upgrade` 这一流程外的改动造成、又由流程外的操作修复的，在合并请求和部署记录里都不存在。最危险的改动恰恰最容易逃出度量。
- **恢复时间按“部署到部署”计是 7.0 分钟，**其中包括 C1 中金丝雀健康地运行了 200 秒；从注入故障到最后一个金丝雀回答，实测约 138 秒（故障 01:23:12.7 开始，91.7 s 后做出决定，再过 46.1 s 最后一个金丝雀回答，实验 3）。DORA 的定义是“从失败到恢复”，失败何时开始，部署记录里没有。

9.9 实验 6：空集群上的同步波次

第 8 章的第二次重建（R2）用了 15 分钟。根应用从第一次同步到全部应用就绪用了 692 秒，经过 11 个波次，每个波次都要等上一波中最慢的那个应用健康。按 R2 记录的逐应用耗时，许多波次之间其实没有依赖：

barman-cloud 只需要 cert-manager，却和日志、链路追踪一起排在 -1 波，等 external-secrets（238 秒）把 -3 波拖完；PostgreSQL 的恢复（178 秒）又要再等一个波次。新的布局（9 个波次，合并请求 !23）把没有相互依赖的 Operator 放在一起，让数据服务提前一个波次：

波次	应用	依据
-10、-8	项目；baseline（PriorityClass）、**namespaces**（原 -3）	只是对象，很多应用要用
-6 … -4	Istio 链，不变	CRD → istiod 与 CNI → ztunnel
-3	各 Operator，加上 **cnpg-operator**（原 -2）、**barman-cloud**（原 -1，cert-manager 就绪前靠重试）、**velero**（原 -1）	彼此之间只有 barman-cloud → cert-manager 一条依赖
-2	tenants、mesh、**pg-main**（原 0）、nats、valkey、policies、loki、tempo、otel-collector、dashboards	只需要 Operator 和 CRD
-1、0	storefront、backup-jobs；insight	需要数据库、租户的账号和已解封的 OpenBao

假设[9]：W1 根应用 600 秒内全部就绪（估算 565 秒）；W2 没有死锁、除计划内的 OpenBao 恢复外不需要人工处理；W3 同步冒用从第一次同步就生效；W4 部署事件记录下重建；W5 从删除集群到全部就绪不超过 13.5 分钟。重建由 `experiments/e8/rebuild-run.sh` 驱动：第 8 章的手册步骤一气呵成，中间没有停顿，OpenBao 的 Pod 一出现就开始恢复。

第三次重建（R3）卡在了 -2 波。30 分钟后仍有 3 个应用没有出现，根应用同步失败并放弃了重试。原因有两个，都只会在空集群上出现：

1. **一条路由挡住了整个平台。**HTTPRoute `storefront` 属于 -2 波的 mesh 应用，它的后端是 storefront 的 Service `gateway`——要到 -1 波才创建。后端不存在时路由的 `ResolvedRefs=False`（BackendNotFound），Argo CD 判定它 Degraded，mesh 随之 Degraded；根应用等 -2 波健康，storefront 就永远不会被创建。这条路由是本章实验 2 引入的第 7 层入口（!6），旧布局里同样有这个顺序，只是在已经存在的集群上，后端一直都在。修复：路由移到单独的应用 `storefront-routes`，放在最后一个波次。
2. **一个只写在注释里的顺序假设被打破了。**Kyverno 的生成策略 `tenant-namespace-defaults` 为带 `tenant` 标签的新命名空间生成默认的配额、LimitRange 和网络策略；shop-dev 和 insight 的这些对象由 Git 提供，策略之所以不碰它们，只是因为这两个命名空间一向比策略先存在——这一点只写在策略文件的注释里。新布局把 policies 和 tenants 放进同一个波次，Kyverno 抢先生成了这 8 个对象，Argo CD 和 Kyverno 从此争夺它们。修复：给这两个命名空间加上标签 `tenant-defaults: git`，生成规则显式排除它。在运行中的集群上，Kyverno 拒绝了这次修改（生成规则的匹配条件不可变），只能删除策略、由 Argo CD 从 Git 重建。

两处修复合并之后（!25），R3 在两分钟内恢复了 storefront，删除并重建策略之后 36 个应用全部就绪——这一次花了 55.8 分钟，其中约 35 分钟是等待和诊断。

第四次重建（R4）没有任何人工步骤。

	假设	测量	
W1	根应用 600 s 内全部就绪（R2：692 s）	**636 s**（36 个应用；R2 为 35 个），比 R2 快 56 s（8%）	**推翻**
W2	没有死锁，没有计划外的人工处理	R3：两处，人工修复（推翻）；**R4：没有**（符合）	见左
W3	同步冒用从第一次同步就生效	R4 的全部审计日志：控制器以自己的身份只写了 Argo CD 自己的对象（854 次状态更新），其余写入全部经过冒用——argocd-root 33 次（第一次在 05:54:35，根应用刚开始同步）、argocd-platform 1315 次、argocd-storefront 36 次、argocd-insight 3 次，没有一次被 RBAC 拒绝	符合
W4	部署事件记录下重建	GitLab 部署记录 #19–#24，每个服务一条，成功	符合
W5	从删除到全部就绪不超过 13.5 min（R2：15.05 min）	**14.1 min**；R4 的步骤之间没有停顿，R2 约有 35 s	推翻

新布局带来了它在结构上承诺的东西：barman-cloud 在 Operator 波次里就绪（277 秒，没有自己的波次），PostgreSQL 提前 130 秒开始恢复。但两个布局没有触及的组件合计比 R2 慢了 138 秒：Gateway API 的 CRD 应用用了 121 秒（R2 是 35 秒，R3 是 109 秒，原因没有查明），PostgreSQL 的恢复用了 230 秒（R2 是 178 秒）。同一个组件在三次重建中的耗时差异（CRD 35/109/121 秒、恢复 178/190/230 秒、external-secrets 238/200/约 146 秒）与重排的收益同一量级：每种布局只重建一次，分辨不出两分钟的改进。看得清的是结构上的变化，例如关键路径上少了一个波次；看不清的是总时间。

实测数据

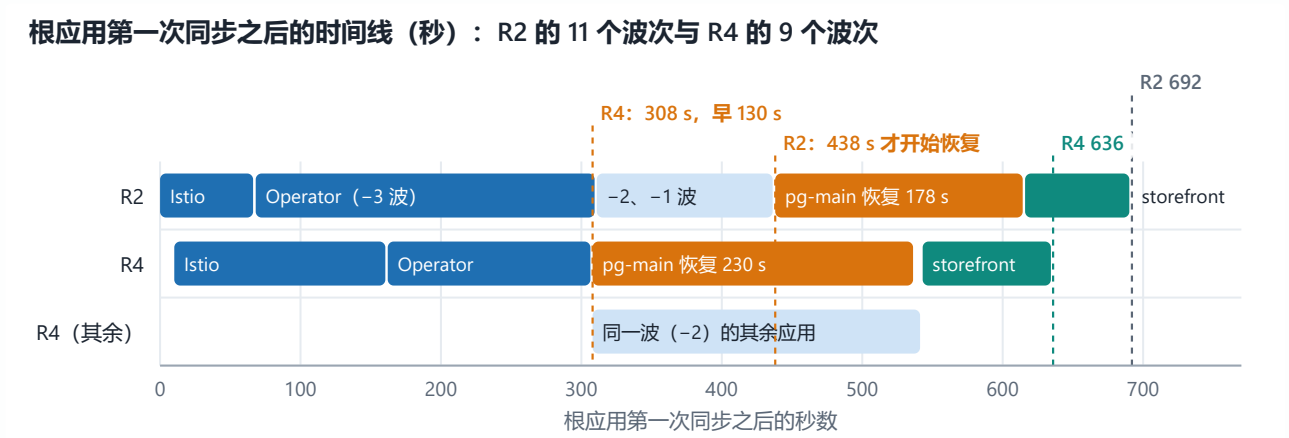


图 9-8 R2 与 R4：根应用第一次同步之后的时间线（秒）。R4 中 PostgreSQL 的恢复（橙色）提前了 130 秒开始，但 Istio 链里的 Gateway API CRD 慢了 86 秒，恢复本身慢了 52 秒；浅色是没有逐项标出的其余波次。

三条经验比秒数更重要：

- **只有空集群才会暴露顺序依赖。**两个问题都在已经存在的集群上隐藏了很久：一个从 E8 开始就埋下，一个由本次重排引出。重建演练不只检验备份，也检验依赖图本身。

- **依赖要写成代码，而不是注释里的约定。** “shop-dev 和 insight 比策略先存在”这个假设藏在注释里，重排时没有人看到；改成标签和排除条件之后，顺序就无关紧要了。
- **健康状态会传染。**一条找不到后端的路由让 mesh、根应用、乃至其后所有波次都停了下来。被等待的应用里，不该有依赖后来者的资源。

另外两个记录：两次重建中，演练写入器都只在删除集群前运行了十几秒（R4 是我的失误：等待时间按 WSL 的本地时区计算，结果早已过去），它们的写入都没有进入任何已归档的 WAL 段，全部丢失——这与第 8 章的结论一致（RPO 最多是一个 `archive_timeout`），不是新的 RPO 测量。审计日志保存在主机上，重建之后仍在；R4 的引导过程写满了一个 100 MB 的文件，统计时要读取全部 `audit*.log`。

9.10 局限

- **两个角色都由脚本扮演。**alice 和 Maintainer 的操作由 `mr.sh` 通过 GitLab API 完成，没有人工审查。真实团队的前置时间主要花在等待审查上，本章的一两分钟不包含这部分。
- **门禁只检查渲染出的发布。**平台自己的应用（Helm chart 加 values）不经过发布策略；Argo CD 本身的升级（`helm upgrade`）更是完全在合并请求之外——本章唯一的一次故障（根应用无法同步）正来自这里。
- **金丝雀的判断是人做的。**C3 中“决定撤回”由演练脚本在 90 秒后做出，没有自动的金丝雀分析；第 7 层入口的排队原因（p95 9.4 s）也没有查明。
- **同步冒用仍是 alpha 功能**，平台账号仍是 cluster-admin：它保护的是团队项目的边界，平台项目内部没有更细的划分。
- **部署事件只覆盖 storefront 的六个服务**，平台应用的同步没有记录；DORA 的失败只认得标题里带 rollback 或 revert 的合并请求。
- 这些数字来自这个实验环境里的一个实验日，不代表任何生产系统的交付能力或规模。

9.10.1 从实验环境到生产环境

本章的做法	生产环境通常的做法	差别意味着什么
开发者与 Maintainer 的操作都由脚本经 API 完成，没有真人评审	至少一名非作者的审批人、代码所有者规则、受保护的环境	本章验证的是机制；评审的质量和速度是组织问题
金丝雀的放量和撤回由人看指标决定	Argo Rollouts、Flagger 等按指标自动分析、逐级放量	自动分析需要可靠的成功率与延迟指标，以及足够的流量
DORA 指标来自 5 个多小时里的 16 个合并请求	按周、按月、按团队统计	这些数字只说明度量方法可行，不代表任何交付水平
门禁检查的是渲染后的配置（Kyverno CLI）	再加上制品签名、漏洞扫描与来源证明（第一册）	配置合规之外，制品本身是否可信也要在合并前回答

9.11 常见问题

集群里已经有 Kyverno 准入策略，为什么还要在流水线里检查一遍？准入在合并之后才起作用：被拒绝的改动已经在 main 上，Argo CD 会反复同步失败，main 与集群长期不一致，而提出者不一定看到原因。流水线在合并之前用同一个引擎检查同样的策略，把失败留在合并请求里。两者不是重复，准入仍然是绕过流水线的改动（例如有人用 kubectl）的最后一道防线。

金丝雀为什么做成第二个发布，而不是用 Argo Rollouts 之类的工具？本章要看清楚每一个机制：路由权重、预览头、撤回经过门禁。第二个发布加上 HTTPRoute 的权重就能做到，没有新的控制器。渐进式交付工具的价值在于自动化——按指标自动推进或撤回——这正是 C3 中人工判断最慢的那一段。

同步冒用开启后，团队想在自己的命名空间里放一个 Secret 怎么办？通过平台提供的通道：ExternalSecret 从 OpenBao 复制（第 6 章），由平台的 SecretStore 决定能读哪些路径。直接给部署账号 Secret 的写权限，等于把 Git 仓库变成密钥的存放处。

为什么根应用不直接放进项目 platform？项目 platform 是根应用自己创建的。第一次同步时它还不存在；以后如果有人在 Git 里改坏了这个项目，根应用就失去了修复它所需的权限。单独的 root 项目和它的账号随 Argo CD 一起安装，与 Git 的内容无关。

冻结期间发现了安全漏洞怎么办？这正是紧急通道的用途：提交信息带 `[emergency]`，流水线照常检查其余的一切；合并后由操作员在 Argo CD 中手动同步。两步都有记录，事后可以审查每一次紧急变更是否名副其实。

9.12 练习

练习

1. **（计算）** 一个金丝雀的权重为 5%，它让 40% 的请求失败。入口每秒处理 100 个请求，浏览 SLO 是 30 天 99.5%。从故障开始到撤回生效用了 3 分钟，这次事故用掉多少错误预算？如果权重是 1%，又如何？
2. **（分析）** H3 被推翻：第 7 层入口让负载均匀分布，失败却没有减少。根据 run A 的数字，指出瓶颈在哪里，并提出两项能让 170 迭代/秒台阶成功的改动。
3. **（设计）** 为 C3 设计自动的金丝雀分析：用哪些指标、多长的窗口、什么阈值判定失败？在权重 5%、每秒 100 个请求时，要多少秒才能有把握区分 0.1% 和 2% 的错误率？
4. **（分析）** 如果把 `argocd-root` 及其 Role 放进 `platform/manifests/` 由 Git 管理，在下一次整集群重建时会发生什么？说明为什么 I4 的修复必须放在 Argo CD 的 Helm 发布里。
5. **（操作）** 团队 shop 需要在 storefront 中新增一个 KEDA 的 TriggerAuthentication。冒用开启后它的同步会怎样失败？写出需要修改的文件和改动。
6. **（设计）** 分别描述只有 GitLab 冻结、只有 Argo CD 窗口时，冻结会怎样被突破。再设计一个办法，让每一次紧急变更都自动生成一条需要事后审查的记录。
7. **（分析）** DORA 脚本没有发现根应用 7 分钟无法同步这次故障。提出一种能让流程外的改动（例如 `helm upgrade`）也出现在变更与部署记录中的办法，并说明它的局限。

8. **（分析）** R4 中 gateway-api-crds 用了 121 秒，R2 中只用了 35 秒。列出三个可能的原因，并说明怎样用已有的记录（审计日志、Argo CD 的操作状态与控制器日志）区分它们。

参考答案见附录。

9.13 本章小结

- 在 GitOps 中合并就是发布决定。合并请求门禁用集群准入所用的同一个引擎检查渲染后的发布，六个假设全部符合：违规的改动在 31–48 秒内被流水线拒绝，GitLab 以 405 拒绝合并，直接推送 main 被拒；合规的改动合并后 27 秒就完成了滚动更新。
- 第 7 层入口让扩出的 Pod 立刻分到请求（第 4 层下，高负载阶段它们一个请求也没有分到），却没有让超出容量的台阶成功：失败来自网关之后的断路器，入口前还有一段原因未明的排队。
- 5% 的金丝雀实测为 4.93%（k6）和 5.03%（Prometheus）；注入一半失败的金丝雀伤及 2.45% 的请求，全部是它自己的；经过门禁的撤回在决定之后 46 秒生效，提升过程零失败。
- 同步冒用让 Kubernetes RBAC 也来执行项目边界：团队账号写 Secret 被拒绝，普通发布照常。代价是每个项目都要有账号——包括创建其他项目的根应用；它所在的 default 项目没有账号，根应用 7 分钟无法同步而无人察觉。
- 变更冻结要两把钥匙：流水线拒绝普通合并、放行标明的紧急变更，Argo CD 的窗口挡住已合并的改动 14 分钟，手动同步是窗口内唯一的通道。
- 部署事件写进集群之外的 GitLab，重建之后仍在；DORA 指标可以从合并请求和部署记录自动计算，但流程外的改动（本章唯一的一次故障）不在记录里。
- 在空集群上重建，暴露出一条依赖后来者的路由和一个只写在注释里的顺序假设；修复后的第四次重建无人工干预，636 秒全部就绪。重排让 PostgreSQL 提前 130 秒开始恢复，但单个组件的波动与这个收益同一量级。

第 10 章

可观测性与事故响应（E9）

到第 9 章为止，平台测量了很多东西，却几乎从不主动告诉任何人：Alertmanager 用的是默认配置，所有告警都送进了“null”。本章让告警离开集群：严重的变成 GitLab 中的事故，一般的变成工单，告警链路本身由集群外的“死人开关”看守。告警规则从 SLO 规格生成，按多窗口、多燃烧率触发；日志改为在每个节点收集，链路改为尾部采样，审计日志开始记录谁读了哪个密钥。最后用一次真实的事故演练检验整条链：从一个看似无害的合并请求，到被呼叫、定位、撤回、关闭，再到复盘与行动项。

工程问题

平台测量了很多东西，却从不主动告诉任何人。怎样让告警在监控系统自己出故障时也能送达、只在值得叫醒人的时候响，并且把一次事故从发现到复盘完整地走一遍？

实测结论

- 告警要离开故障域：集群外的告警桥把严重告警变成 GitLab 中的事故；冻结监控所在的节点之后，死人开关在 183.2 秒时开出事故。
- 启用路由后的第一条呼叫是个误报：虚拟机的时钟漂移让“进程启动时间”10 分钟里变化 14–15 次，而进程一次也没有重启。
- 事故演练：一个测试参数经门禁进入生产，98.1% 的下单失败；影响开始后 2 分 08 秒呼叫，37 秒后开出事故，再过 69 秒撤回。
- 燃烧率告警在故障恢复之后 28.5 分钟才解除；10% 的头部采样让 670 次失败只留下 84 条错误链路。

10.1 学习目标

学完本章，你应当能够：

- 设计一条不与被监控对象同处一个故障域的告警链路，并用死人开关检测它自身的失效；
- 从 SLO 推导多窗口、多燃烧率告警，事先算出一次故障多久之后才会呼叫值班人员；
- 用路由与抑制区分“立即处理”和“工作时间处理”，避免一次故障触发一串告警；
- 在节点上收集日志，使被删除或被驱逐的 Pod 的输出仍可查询；用尾部采样只保留有价值的链路；
- 按“检测—升级—缓解—解决”记录一次事故的时间线，写出带行动项的复盘。

实验性质

本章的实验都是 `[真实]` 运行的：Prometheus、Alertmanager、GitLab、Fluent Bit、OpenTelemetry Collector 和集群都是真实软件，负载由 k6 产生。冻结节点、重启 OpenBao、写入一个注定失败的应用是 `[注入]` 的故障；事故演练的起因——一个测试环境的参数被带进了生产配置——是 `[模拟]` 的情节，之后的检测、通知、诊断与撤回都是真实发生的。值班工程师由我扮演，按脚本记录每一步的时刻。

10.2 原理

10.2.1 告警要能离开故障域

监控系统最容易被忽略的失效，是它自己和被监控的对象一起倒下。第 8 章的 F4 中，唯一的 Prometheus 与 PostgreSQL 主库在同一个节点上，节点被冻结时告警一条也发不出来，直到故障切换结束。解决办法有两层：

1. **通知路径要离开集群。**本章的告警桥是一个运行在集群之外（与 GitLab 同一个 Docker 网络）的小程序：Alertmanager 用 webhook 把通知发给它，它在 GitLab 项目 `root/incidents` 中为每条告警开一个 issue——critical 是事故（incident），warning 是普通工单——告警解除时评论并关闭。
2. **用心跳检测“沉默”。**kube-prometheus-stack 自带一条永远在触发的告警 Watchdog。Alertmanager 每 30 秒把它发给告警桥；告警桥 180 秒没有收到心跳，就自己开一个事故：“集群的告警链路沉默了”。这就是死人开关（dead man’s switch）：它不依赖被监控的系统说出“我坏了”，只依赖“它不再说话”。

示意图

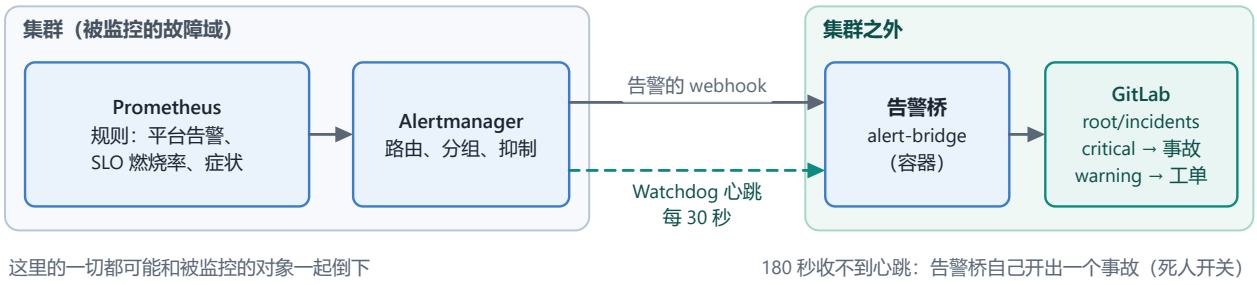


图 10-1 本章的告警链路：通知离开集群，链路本身由心跳看守

10.2.2 从 SLO 到告警：多窗口、多燃烧率

SLO 的错误预算 $b = 1 - \mathrm{SLO}$（下单 99%：$b = 1\%$）。某个时间窗口内的错误比例除以 b，就是燃烧率：燃烧率 1 意味着预算恰好用 30 天；燃烧率 14.4 意味着 1 小时就用掉 30 天预算的 2%（$14.4/720 = 0.02$）。按 Google SRE 工作手册的做法，本章的规则是：

级别	条件（两个窗口同时超过）	含义
呼叫（critical）	1 h 与 5 min 都 > 14.4 b，或 6 h 与 30 min 都 > 6 b	按此速度约 2 天或 5 天用完预算
工单（warning）	1 d 与 2 h 都 > 3 b，或 3 d 与 6 h 都 > 1 b	慢性消耗

长窗口保证告警是“真的在燃烧”，短窗口让燃烧一停告警就解除。代价是检测时间：如果之前一小时的流量全部正常，故障开始后错误比例为 e，1 小时窗口的比例在 t 分钟后达到 $t \cdot e/60$，越过 $14.4b$ 需要

$$t = \frac{14.4\, b \cdot 60}{e} \text{ 分钟}$$

下单全部失败（$e = 100\%$）时 $t \approx 8.6$ 分钟，失败一半时约 17 分钟，失败 20% 时 43 分钟，低于 14.4% 的错误比例在这一组窗口上永远不会呼叫。这不是缺陷，而是设计：呼叫只用于会在几天内耗尽预算的燃烧。这些规则由 `platform/slo/gen.py` 从 `platform/slo/slos.yaml` 生成：浏览可用性 99.5%、下单可用性 99%、浏览延迟（500 ms 内）99%。

模型曲线

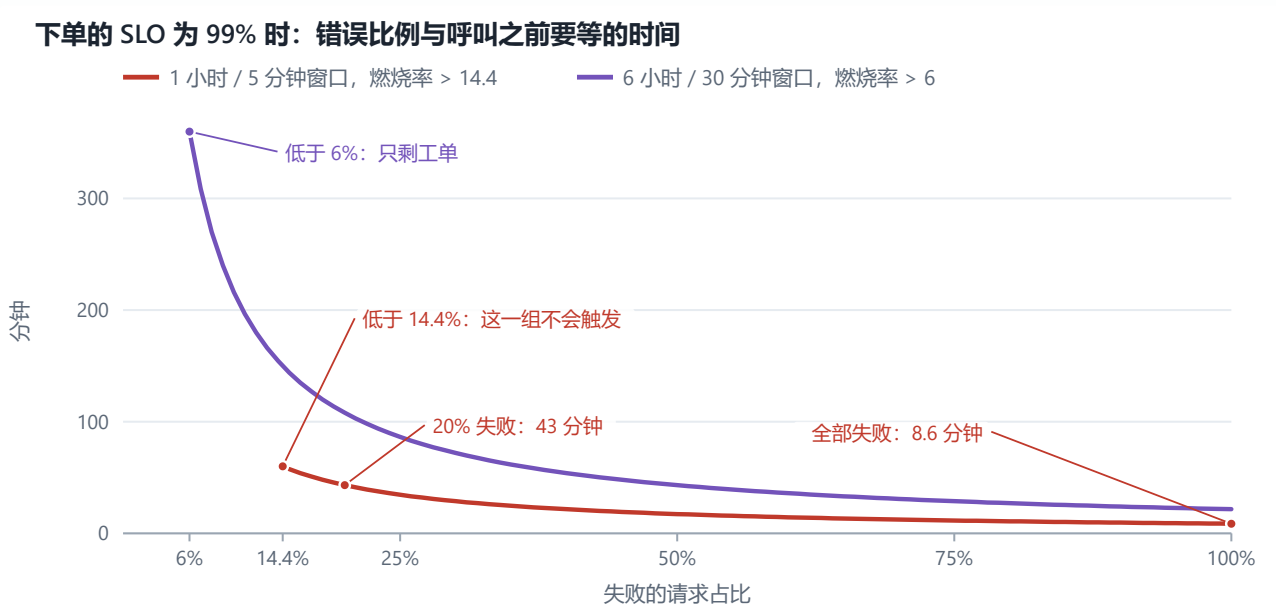


图 10-2 把上面的公式画成曲线（这是公式，不是测量；假设故障之前窗口里装满了正常流量）。错误比例越高，呼叫来得越快：全部失败时 8.6 分钟，20% 失败时 43 分钟。低于 14.4% 的错误永远够不到快的那一组窗口，只能等慢的一组；低于 6% 的错误不会叫醒任何人，只开工单。这是有意的设计：呼叫的速度与预算消耗的速度成正比。

10.2.3 路由、抑制与分级

告警分两级就够了：需要立即处理的（critical，呼叫）和可以等到工作时间的（warning，工单）。Alertmanager 按 `severity` 路由，按 `alertname`、`namespace`、`slo` 分组（同一组只发一次通知，等待 30 秒以收集同时发生的告警）。抑制规则让“原因”压住“后果”：某个 SLO 的呼叫告警抑制同一 SLO 的工单告警；`PostgresPrimaryMissingReplicas` 抑制 `PostgresReplicaNotStreaming`；同一告警的 critical 版本抑制 warning 版本。每条告警都带 `team` 标签和处置手册的链接（`docs/runbooks.md`）。

10.2.4 日志、链路与审计：保留什么

- **日志**以前只有服务通过 OTLP 主动发送的部分；其他容器的标准输出、以及一个已经被删除或驱逐的 Pod 的输出，随 Pod 一起消失（第 5 章的驱逐实验就读不到被驱逐 Pod 的最后几行）。节点级收集在每个节点上读取 `/var/log/containers`，与 Pod 的生命周期无关。
- **链路**在每秒几十个请求时就已经很多。头部采样在请求开始时决定是否保留，无法知道这次请求会不会出错；尾部采样等整条链路结束（这里等 10 秒）再决定：保留全部出错的、全部慢于 1 秒的、其余的

10%。代价是收集器要在内存里暂存每条链路，而且同一条链路的所有 span 必须到达同一个收集器实例。

- **审计**此前不记录服务账号的读操作（数据量太大），因此回答不了“哪个控制器读了哪个 Secret”。本章只为 Secret 打开这一类记录，级别为 Metadata：记录谁、读了哪个对象，从不记录内容。

10.3 实验 1：一条永远在响的告警

启用路由之前，先看现在有哪些告警在触发：除了 Watchdog，只有一条 critical 的 `AlertmanagerClusterCrashlooping`——它在第 8 章就一直在响，经过两次整集群重建（全新的 Alertmanager）仍然在响。Alertmanager 的重启次数是 0。规则的输入是 `changes(process_start_time_seconds[10m])`：进程启动时间在 10 分钟内变化超过 4 次就视为崩溃循环。实际记录[3]显示，这个“启动时间”每半分钟左右往回退 1 秒，10 分钟内变化了 14–15 次，而进程从未重启。

原因在时钟。Linux 计算进程启动时间用的是“开机时刻 + 进程启动时的开机时长”，开机时刻 `btime` = 当前墙上时间 − 开机时长。在这台 WSL2 虚拟机里，单调时钟（开机时长）比墙上时钟快约 3%：60 秒的开机时长里墙上时间只走了 58.3 秒（WSL 里测得 +2.91%，kind 节点里 +2.86%；另一次 45 秒的测量是 +3.74%），墙上时钟被不断校正回主机时间，于是 `btime` 每分钟后退约 2 秒，所有进程的“启动时间”跟着后退。节点导出器报告的时钟偏移却是 0、状态“已同步”——它看的是内核的 NTP 状态，看不到这种虚拟化层面的校正。

处置：关闭这条默认规则（kube-prometheus-stack 的 `defaultRules.disabled`），换成按 kubelet 统计的真实重启次数判断的 `AlertmanagerRestarting`。这个结论对其他测量也有意义：用单调时钟计时的工具（k6 的耗时、Go 和 Python 的计时器）与按墙上时间计算的间隔，在这台机器上可能相差 3% 左右；本书各章的时间点和间隔都来自墙上时间（`date`、Prometheus、审计日志的时间戳）。

10.4 实验 2：告警离开集群

告警桥运行在集群外（容器 `alert-bridge`，172.20.0.15），集群内的 Alertmanager 按名字访问它；GitLab 项目 `root/incidents` 和一个只有 Reporter 角色的项目访问令牌由 `platform/alerting/run-bridge.sh` 创建。路由、抑制和平台告警通过合并请求 !27 生效。假设提交在 [4]：

	假设	测量	
A1	启用路由不会打开任何 issue	第一条通知（06:45:18）就是 **AlertmanagerClusterCrashlooping**：规则已经从 Prometheus 删除，Alertmanager 却还持有这条告警直到它过期；告警桥开了事故 #1，06:49:12 在告警解除后关闭	**推翻**
A2	重启后的 OpenBao 处于密封状态：Pod 启动 2.5 分钟内开出事故，解封 3 分钟内关闭	Pod 06:46:01 运行；31 秒后 `vault_core_unsealed` 为 0（密封时指标端点仍然应答）；41 秒 pending、102 秒 firing；事故 #2 在 **128 秒**后开出；06:48:13 解封，**112 秒**后关闭	符合

	假设	测量	
A3	一个同步失败的应用触发 `ArgoCDSyncFailing`，1.5 分钟内开出工单	探针应用 06:57:26 在 5 次重试后失败——**告警从未触发**：`argocd_app_sync_total{phase="Failed"}` 在第一次失败时才诞生，初值就是 1，`increase()` 看不到任何增长	**推翻**（规则已修正）
A4	冻结同时运行 Prometheus 与 Alertmanager 的节点：最后一次心跳后 180–200 秒死人开关开出事故；解冻一分钟内关闭；数据库写入不受影响	07:16:38 冻结 ent-worker3；45 秒后节点状态 Unknown；事故 #6 在最后一次心跳后 **183.2 秒**开出；解冻后 3 秒关闭；冻结期间写入 14 行	符合
A5	一次带警告的备份 2 分钟内开出工单	备份 e9-warnings 完成，7 个警告；告警 06:52:57 触发，工单 #3 在 34 秒后开出	符合

A3 的修正值得记住：计数器的某个标签组合在第一次发生时才出现，它的第一个样本就是 1，`increase()` 和 `rate()` 都认为“没有增长”。修正后的规则同时匹配“10 分钟前还不存在的失败计数器”（`... unless ... offset 10m`），并用已经记录在 Prometheus 中的数据回测：新规则会在 06:58 到 07:07 之间触发，旧规则始终为空[5]。

另有两个发现比假设本身更重要：

- **一个永远不健康的子应用挡住了根应用。**为了让同步波次等待，平台给 Argo CD 的 Application 配置了健康检查（第 9 章），于是根应用的同步要等每个子应用健康。探针应用永远不会健康，根应用的同步一直“等待 e9-probe 健康”，删除探针的合并（!29）因此迟迟无法生效。新规则 `ArgoCDRootNotSynced` 在 07:13:30 开出了事故 #5——一次真正的呼叫。团队的应用由 ApplicationSet 生成，不是根应用的直接子应用，只有平台自己声明的应用会这样挡住根应用。
- **沉默的 Prometheus 看起来像“一切正常”。**节点冻结期间，Prometheus 不再刷新告警，Alertmanager 让仍在生效的 `VeleroBackupWarnings` 过期并发出“已解除”，告警桥关闭了工单 #3，而问题并没有解决。这一刻唯一说真话的是死人开关。

10.5 实验 3：被删除的 Pod 的日志，和谁读了哪个密钥

Fluent Bit 5.1.3 以 DaemonSet 运行在全部 4 个节点上（每个只请求 16 MiB 内存，ent-worker3 的内存请求已经到了 98%），把容器的标准输出连同命名空间、Pod、容器、节点标签推送给 Loki。审计策略在“服务账号的读操作一律不记录”之前加了一条：服务账号读取 Secret 记录到 Metadata 级别；策略文件在主机上，API 服务器重启后生效（这次重启用了 8 秒）。假设提交在 [6]：

	假设	测量	
L1	被删除的 Pod 的标准输出仍可从 Loki 查到，30 秒之内	Pod 写下标记后被删除，`kubectl logs` 返回 NotFound；标记行**在 Loki 中**，但第一次查询（删除后 19 秒）没有找到，而且所在的流**没有**命名空间和 Pod 标签	符合，有两个保留

	假设	测量	
U1	API 服务器重启后一分钟内，审计日志记录服务账号读取 Secret	一分钟内 30 个服务账号读取 Secret 的记录，例如 External Secrets 读取 `insight/insight-api-key`、`data:pg-main` 读取备份凭据 `pg-backup-s3`、Kyverno 读取自己的 TLS 证书	符合

L1 的两个保留来自同一个细节：Fluent Bit 默认每 60 秒才发现一次新文件。这个只活了 9 秒的 Pod 的日志文件在 Pod 删除之后才被发现（kubelet 恰好还没删除文件），发现的那一刻又正赶上 API 服务器重启，补充元数据失败，于是这一行只带着 job 标签发了出去。当时 Loki 中共有 75 个流，除了 API 服务器重启那几秒发出的行，都带有完整的标签。改为每 5 秒发现一次新文件之后，寿命短于 5 秒、文件又被 kubelet 先删掉的 Pod 仍然会丢失日志——节点级收集缩小了窗口，没有消灭它。

10.6 实验 4：事故演练——一个测试参数进入了生产

情节（`[模拟]` 的起因）：团队 shop 想测试支付服务商出错时的处理，准备了一个测试环境用的参数 `PSP_ERROR_RATE: "1"`（每一笔扣款都失败），这一行却被写进了生产的 `envs/prod/storefront/services/payments.yaml`。alice 提出合并请求，门禁通过（发布策略检查镜像、资源、探针和副本数，不检查环境变量的含义），Maintainer 合并。负载：k6 以每秒 10 次迭代运行（70% 浏览、25% 下单、5% 查询），在改动之前先运行约一小时，让 SLO 的窗口里装满正常流量。值班工程师由我扮演，只通过 GitLab 中的事故和部署记录工作，演练脚本 `experiments/e9/incident-drill.sh` 为每一步打上时间戳。

计划之外发生了一件事：等待期间主机进入了睡眠，从约 07:47 到 08:13，整个实验环境冻结了 26 分钟。醒来时死人开关立即开出了事故 #8（“1548.5 秒没有心跳”），各个 SLO 都出现了短暂的错误尖峰（浏览的 5 分钟错误比例一度达到 73%），下单的慢燃烧工单 #9 被触发后又自行解除。结果是改动发生时，SLO 的窗口里只有约 35 分钟的流量，而不是计划中的一小时——这个偶然恰好成了本节最有教益的部分。

时刻（UTC）	事件	间隔
08:23:40.7	合并请求 !37 合并（流水线 42 s 通过）	改动
08:23:47	Argo CD 同步 payments	
08:24:01.6	GitLab 部署记录 #25：prod/payments，成功	
约 08:24:10	下单开始失败（第一个出现 503 的 30 秒区间结束于 08:24:30）	影响开始
08:24:33.5	慢燃烧工单告警触发，工单 #11 于 08:25:17.6 发出	
08:26:18.5	**呼叫告警触发**（SLOBurnCheckoutAvailabilityPage）	检测：+2 分 08 秒
08:26:55.5	**告警桥开出事故 #12**（分组等待 30 秒）	升级：+37 秒
08:27:20.8	值班工程师确认（GitLab 中的评论）	
08:27:22.7	诊断：prod/payments 的部署记录 #25 关联了 alice 的 !37	
08:28:04.6	**撤回合并**（经过门禁）	缓解：呼叫后 69 秒

时刻（UTC）	事件	间隔
约 08:28:40	最后一批失败的下单	影响结束
08:33:56	下单 5 分钟错误比例回到 0	
08:57:05.5	**事故 #12 关闭**（告警解除）	恢复后 28.5 分钟

实测数据

事故演练的时间线（秒，从合并请求 !37 合并起算）

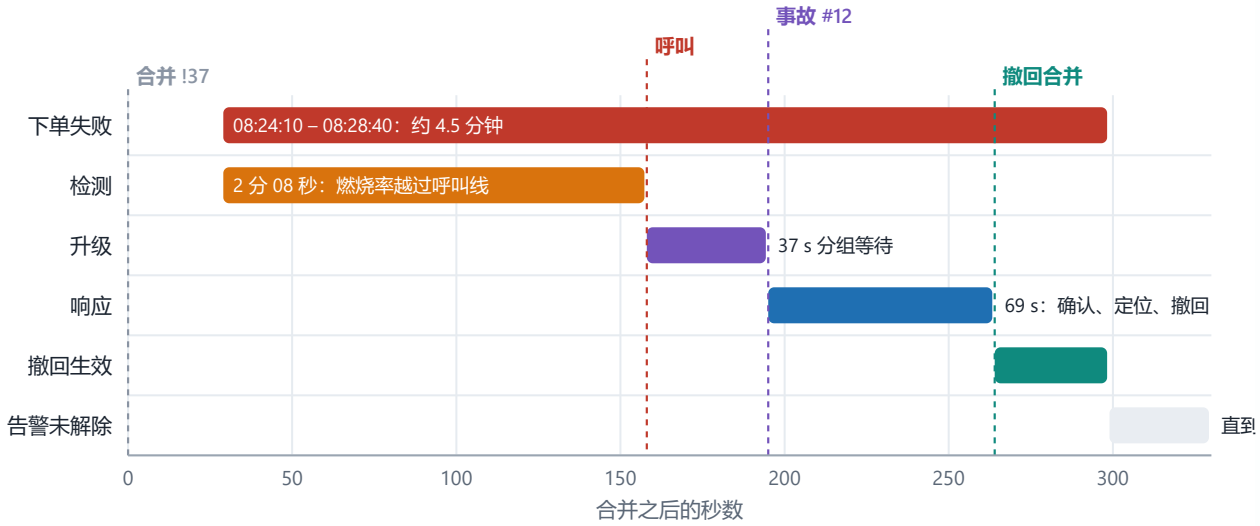


图 10-3 这次演练的时间线。从影响开始到呼叫的 2 分 08 秒是告警规则决定的；从呼叫到开出事故的 37 秒是 Alertmanager 的分组等待；之后的 69 秒是人的部分——确认、从部署记录定位到合并请求、撤回。故障在撤回后很快结束，告警却要等窗口里的错误滑出去：事故在恢复之后 28.5 分钟才自动关闭。

假设提交在 [7]（演练开始之前）：

	假设	测量	
S1	改动通过门禁，2 分钟内到达 payments	流水线 42 s 通过；Argo CD 在合并后 7 秒同步；部署记录在合并后 21 秒	符合
S2	下单几乎全部失败（> 90%），浏览不受影响	08:24:30–08:28:30 网关回答的 638 个下单中 626 个是 503（**98.1%**），其中只有 113 个到达了 orders——网关对 orders 的断路器快速失败；浏览没有 5xx	符合
S3	失败开始 9–10 分钟后呼叫，不会更早；不会有工单	**2 分 08 秒**后呼叫，由 6 小时 / 30 分钟这一组窗口触发；1 小时 / 5 分钟那一组从未触发（1 小时比例最高 13.5%）；**工单先于呼叫**发出	**推翻**
S4	告警触发后 1 分钟内开出 GitLab 事故	37 秒，带 team::shop 与 slo::checkout-availability 标签	符合
S5	不登录集群，只凭 GitLab 找到原因；撤回合并后 3 分钟内失败停止	部署记录 #25 列出了 !37；撤回 08:28:04 合并，36 秒后不再有失败	符合

	假设	测量	
S6	失败停止后 5 分钟内告警解除；消耗当月预算的个位数百分比	告警在失败停止 **28 分钟**后才解除；670 次失败的下单占当月预算的 **0.97%**	解除时间**推翻**，预算符合

S3 的推翻揭示了燃烧率告警的一个性质：它的检测时间取决于窗口里有多少正常流量。改动发生时，所有窗口里都只有约 35 分钟的流量，于是“6 小时”的比例实际上是 35 分钟的比例，11.4% 超过了 6 倍燃烧率的线（6%），两分钟就呼叫了。如果窗口里装满了正常流量，同样 98% 的失败要让 1 小时比例超过 14.4%，需要 $14.4\% \times 60 / 0.98 \approx 8.8$ 分钟——几乎是这次整个故障时长（约 4.5 分钟）的两倍。换句话说，在一个平稳运行的系统里，这次每一笔下单都失败的故障，可能在修复之前根本不会呼叫任何人。S6 的推翻是同一枚硬币的另一面：30 分钟的短窗口让告警在失败停止后又持续了 28 分钟，“已解除”远远晚于“已缓解”。

真实截图

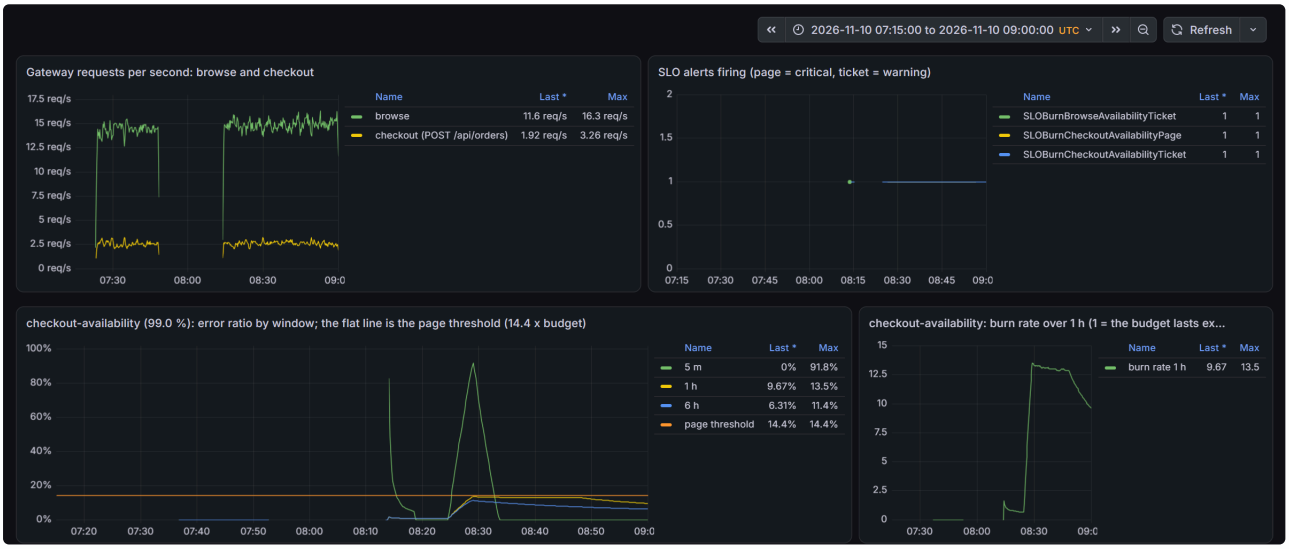


图 10-4 事故演练期间的 SLO 看板（07:15–09:00 UTC，只截取了流量、告警和下单这三块）：上方的流量在 07:47–08:13 之间是空白——主机睡眠；08:14 醒来时出现短暂的尖峰；08:24 起下单的错误比例（下方）5 分钟窗口最高 91.8%，1 小时窗口最高 13.5%，始终没有越过 14.4% 的呼叫线，6 小时窗口最高 11.4%；右下是下单 1 小时的燃烧率，最高 13.5。

做得好的地方：部署事件（第 9 章）让值班工程师一次查询就找到了可疑的改动——prod/payments 在第一笔失败之前 9 秒被部署，GitLab 在这次部署上列出了 !37；撤回同样经过门禁，用时 42 秒；网关的断路器让失败快速返回，638 个下单中只有 113 个到达 orders，失败的支付服务商没有把订单服务和数据库拖进重试风暴。

复盘（[8]，无责）列出了七个行动项，其中四个已经完成并经过验证：

#	行动项	状态
1	发布策略 `no-fault-simulation`：生产环境中 PSP_ERROR_RATE 与 PSP_HANG_RATE 必须为 0	完成；alice 再次提出同样的改动，流水线在 `release-prod-availability` 失败，合并返回 405

#	行动项	状态
2	症状告警 `CheckoutFailing`：2 分钟内超过一半的下单失败（且超过 20 次）即呼叫，不看预算	完成；用演练的数据回测：08:26:15 触发
3	SLO 工单的分组等待改为 5 分钟，让随后的呼叫有机会先抑制它	完成；尚未在真实燃烧中观察到
4	下单的 SLI 只统计 POST 请求	完成
5	对支付等关键服务的配置改动也走金丝雀（第 9 章的加权路由）	未完成
6	事故区分“已缓解”和“已解除”	未完成
7	演练前让主机保持唤醒，并检查 SLO 窗口里有多少历史	未完成（实验环境）

10.7 实验 5：尾部采样与 exemplar

收集器在演练之前就开启了尾部采样（等待 10 秒，保留全部出错的、全部慢于 1 秒的、其余的 10%）。演练之后再看它在事故期间做了什么，结果令人意外：08:23:30–08:35:00 之间它对 4770 条链路做了决定，出错策略只保留了 86 条；Tempo 里这段时间只有 84 条出错的链路，而失败的下单有 670 次。原因在上游：生产环境从第 4 章起就以 10% 的比例做头部采样（`tracesSampleRatio: "0.1"`），十个请求里只有一个的 span 会被发送到收集器——尾部采样保留不了它从未收到的链路。两种采样叠加，最终只有约 1% 的 span 进入 Tempo，而出错的链路只剩十分之一。

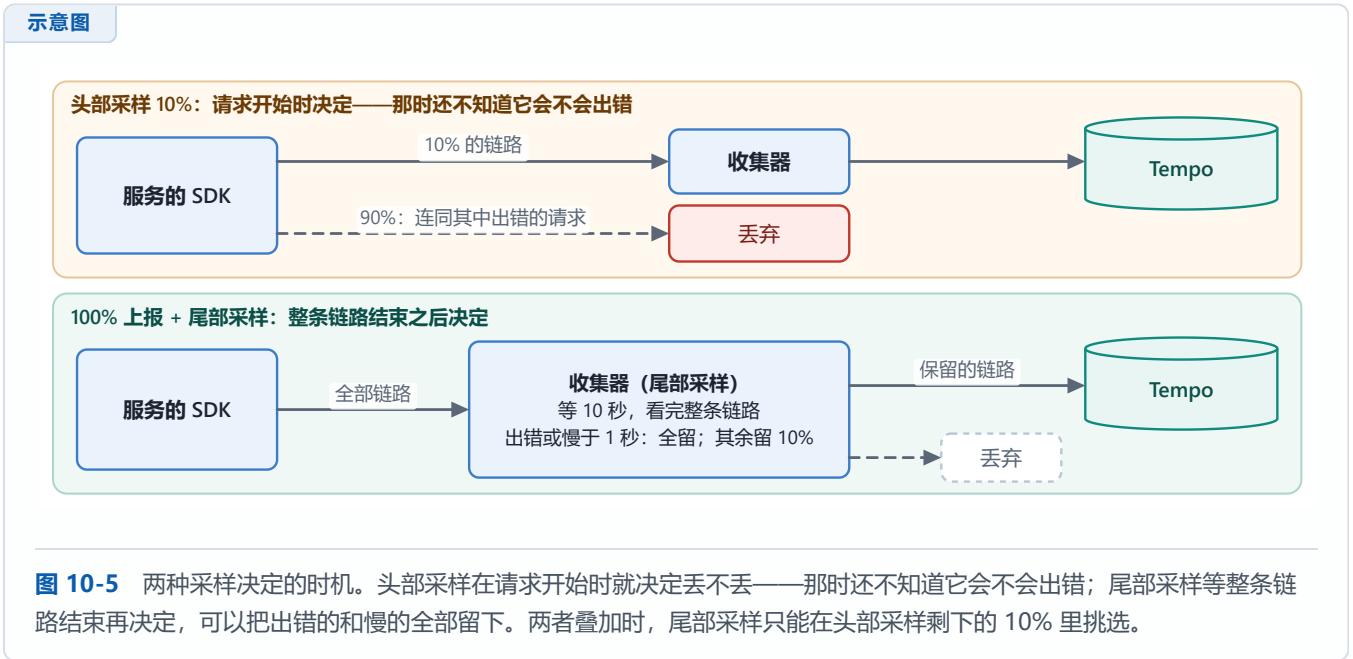


图 10-5 两种采样决定的时机。头部采样在请求开始时就决定丢不丢——那时还不知道它会不会出错；尾部采样等整条链路结束再决定，可以把出错的和慢的全部留下。两者叠加时，尾部采样只能在头部采样剩下的 10% 里挑选。

exemplar 让这个问题更直观。服务从 b13e3cf 起在请求耗时直方图上记录 exemplar：只为出错或慢于 1 秒的请求附上链路 ID，也就是尾部采样一定会保留的那些链路，Prometheus 按 OpenMetrics 格式抓取并保存它们。假设提交在 [9]；检验时给 catalog 注入 45 秒、每个请求 1.2 秒的延迟：

	假设	测量	
X1	注入之后只有 1 秒以上的桶有 exemplar	2 秒桶 18 个、5 秒桶 4 个——**还有 30 个落在 5 毫秒桶**：网关的断路器在约 1 毫秒内返回 503，出错的请求不论快慢都会带上 exemplar（这正是设计）	**推翻**（预测忽略了快速失败）
X2	每个 exemplar 的链路都能在 Tempo 中打开	第一次：5 个里只有 **1 个**。没有被头部采样选中的请求仍然有链路 ID，exemplar 指向了一条从未被记录的链路	**推翻**

修正有两处：exemplar 只在 span 确实被采样（`trace_flags.sampled`）时才记录；头部采样改为 100%，由尾部采样一处决定（镜像 05a04fe）。第二次检验 5 个链路全部能打开（X2 修正后符合）。代价在相同负载下测量[10]：

	头部采样 10%	100%，由尾部采样决定
六个服务的 CPU	0.393 核	0.413 核（+5%，每个请求 +9%）
到达收集器的 span	28.9 个/秒	291 个/秒
写入 Tempo 的 span	3.4 个/秒	29.9 个/秒（8.7 倍）
收集器的 CPU / 内存	0.009 核 / 92 MiB	0.013 核 / 118 MiB

实测数据

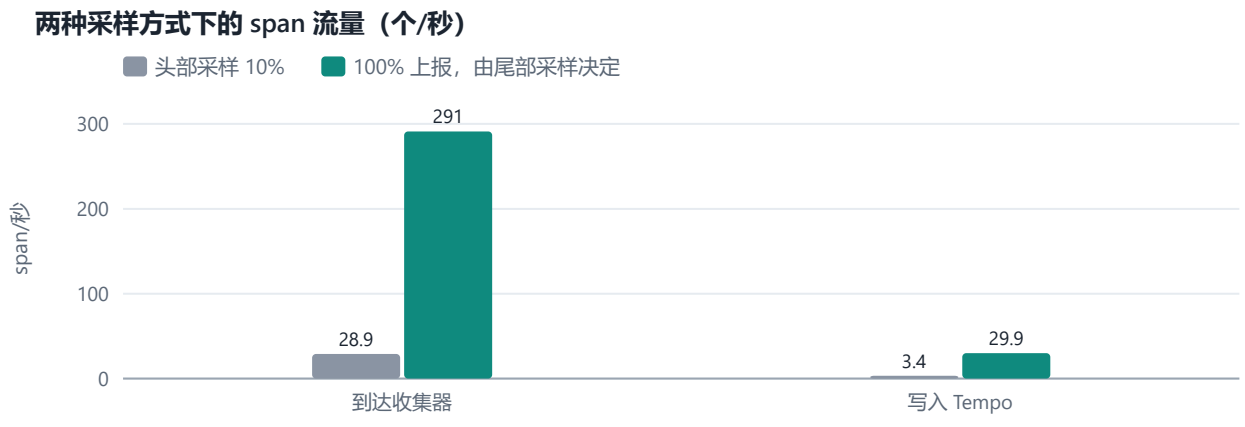


图 10-6 改为 100% 上报之后的 span 流量。到达收集器的 span 是原来的 10 倍，但写入 Tempo 的只有 29.9 个/秒：尾部采样把绝大多数正常的链路丢在了收集器。代价是服务一侧多用约 5% 的 CPU（0.393 核 → 0.413 核），换来的是每一条出错的链路都在。

两个采样器各自都合理，叠加起来却让“保留全部错误”的承诺落空，exemplar 也指向了不存在的链路。这类问题不会自己报错：链路少了，看板照常显示；只有拿具体的事故去核对，才会发现记录里缺了九成。

真实截图

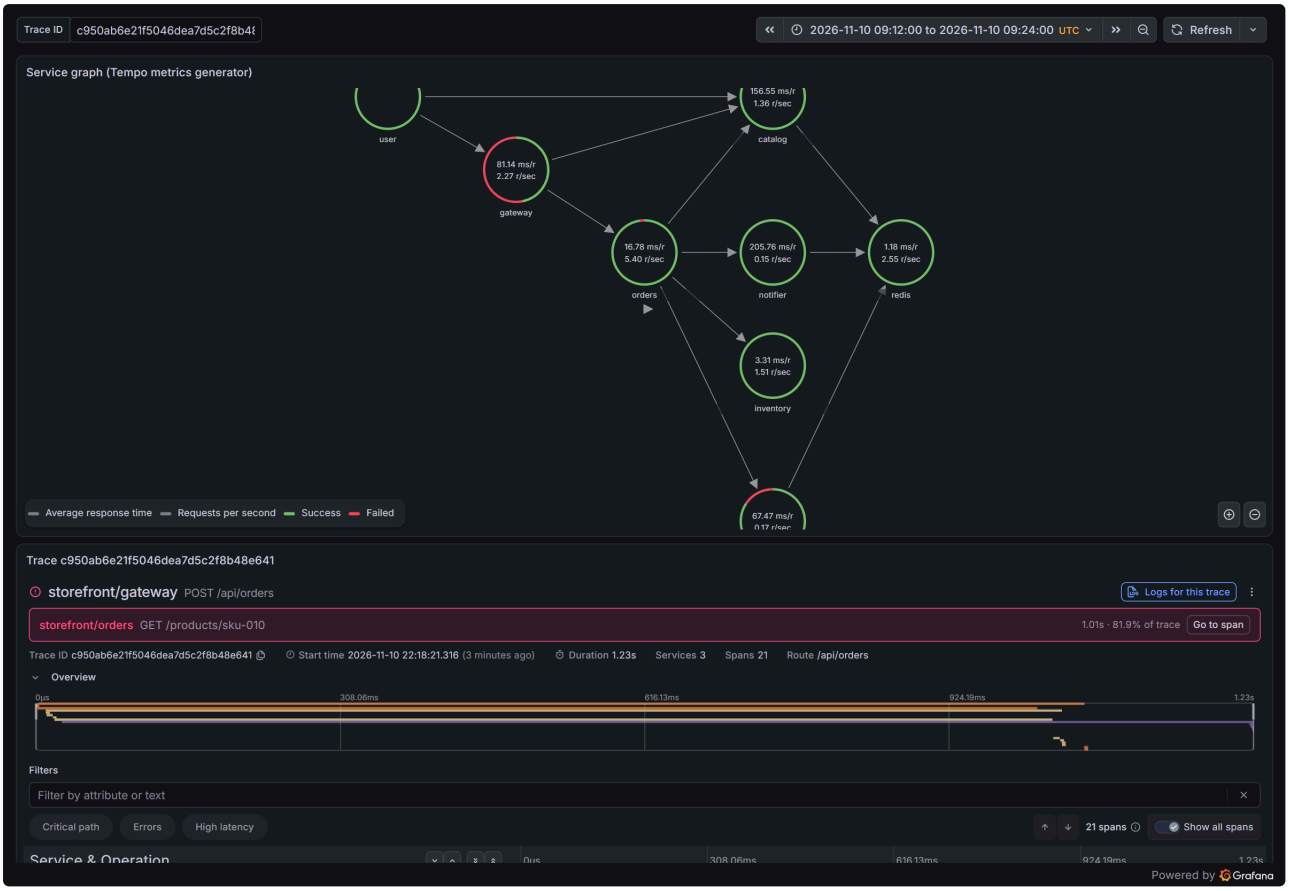


图 10-7 一个 exemplar 打开的链路（catalog 注入 1.2 秒延迟期间）：网关的 POST /api/orders 用了 1.23 秒，Grafana 把关键路径标在 orders 对 catalog 的调用 GET /products/sku-010 上（1.01 秒，占整条链路的 81.9%）——正是注入的延迟。上方是 Tempo 生成的服务依赖图。

10.8 局限

- **告警桥是一个单点。**它运行在集群之外，但只有一个实例，与 GitLab 在同一台主机上；告警桥或 GitLab 倒下时，告警无处可去，死人开关也会一起失效。生产环境会把死人开关交给另一个独立的服务（例如商业的心跳监控），通知也会走多个渠道（电话、即时消息）。
- **值班工程师早有准备。**演练中从呼叫到确认只用了 25 秒，因为扮演值班的我正在等待；真实的确认时间取决于值班安排和通知方式，本章没有测量。
- **事故中的检测时间是偶然的结果。**主机睡眠让 SLO 窗口只有 35 分钟的历史，呼叫因此提前；满一小时历史的检测时间是推算（8.8 分钟），不是测量。
- **每种情况只做了一次。**OpenBao 密封、死人开关、事故演练都只运行了一次，时间受到这台机器负载和睡眠的影响。
- **30 天的预算无法完整呈现。**Prometheus 只保留 3 天，看板显示的是窗口内的燃烧率，不是 30 天里剩余的预算。
- 这些数字来自这个实验环境，不代表任何生产系统的规模或经验。

10.8.1 从实验环境到生产环境

本章的做法	生产环境通常的做法	差别意味着什么
告警桥只有一个实例，与 GitLab 在同一台机器上	托管的值班系统，加一个独立于自己机房的心跳监控	死人开关与它看守的对象共用一台机器：整机冻结时一起失效（第 11 章的事故 E10-1）
值班工程师由作者扮演，按脚本记录时刻	轮值、升级策略、交接与对外沟通	本章测到的是机器的延迟；人的响应时间没有被真正测量
SLO 的窗口里只有几十分钟的历史	28–30 天的滚动窗口	历史不足让告警比设计的更早触发，也更晚解除
Prometheus 只保留几天的数据	长期存储与降采样（Thanos、Mimir 等）	月度的错误预算报告在本环境算不出来

10.9 常见问题

既然有了 SLO 燃烧率告警，为什么还要 CheckoutFailing 这种症状告警？ 燃烧率告警回答的是“预算会不会耗尽”，它的检测时间随窗口里的历史变化，一次短暂但彻底的故障可能在修复之前都不会呼叫。对下单这种直接关系收入的路径，“超过一半失败”本身就值得立即处理。两者不冲突：症状告警负责“现在坏了”，燃烧率告警负责“正在悄悄变坏”。

告警为什么不直接发邮件或即时消息，而要经过一个告警桥？ 本章需要三件事：通知离开集群；每条告警在一个可以追踪的地方有记录（谁确认、何时缓解、何时解除）；告警链路自身的失效能被发现。Alertmanager 的 webhook 加一个很小的桥就能做到，GitLab 的事故页面同时充当时间线。生产环境通常用 PagerDuty、Opsgenie 一类的服务做同样的事。

为什么不把 Alertmanager 的 resolve_timeout 调大，避免冻结期间告警被“解除”？ `resolve_timeout` 只对不带结束时间的告警生效；Prometheus 发送的告警自带结束时间，Prometheus 一停止刷新它们就会过期。根本的办法是承认监控会沉默，并让沉默本身成为告警——这正是死人开关。

尾部采样为什么不能在头部采样之后补救？ 被头部采样丢弃的请求根本不会创建可导出的 span，收集器看不到它们。要让尾部采样保留全部错误，就必须让所有链路都到达收集器，代价是服务和收集器多做的那部分工作，以及更多的网络流量。

10.10 练习

练习

1. **（计算）** 浏览的 SLO 是 99.5%。窗口里装满了一小时的正常流量之后，浏览请求有 30% 失败。1 小时 / 5 分钟这一组呼叫条件多久之后满足？如果只有 3% 失败，会不会呼叫？会不会开工单？
2. **（分析）** 事故演练中，下单的呼叫来自 6 小时 / 30 分钟这一组窗口，而且在失败停止 28 分钟之后才解除。解释这两个现象分别由哪个窗口决定，并说明把 30 分钟换成 10 分钟的利弊。
3. **（设计）** 告警桥只有一个实例，又与 GitLab 在同一台机器上。设计一个方案，让“告警桥本身失效”也能被发现，并说明它依赖什么。

4. **（分析）** `ArgoCDSyncFailing` 的第一版规则从未触发（A3）。找出本章中另一个可能因为同样的原因漏报的指标，并改写对应的规则。
5. **（设计）** 一个新服务每秒只有 0.2 个请求，SLO 是 99.9%。按本章的多窗口规则，它的呼叫告警会有什么问题？提出两种改进。
6. **（分析）** 第 4 章选择了 10% 的头部采样以降低开销，本章改回 100%。分别列出两种做法在“保留错误链路”“服务的 CPU”“存储”三方面的代价，并说明在什么规模下你会重新引入头部采样，以及要配合什么。
7. **（操作）** 为 storefront 的浏览延迟写一个症状告警（类似 `CheckoutFailing`），说明阈值、窗口和最小请求数的选择理由，并用本章 exemplar 实验的数据（catalog 注入 1.2 秒延迟的 45 秒）回测它何时触发。

参考答案见附录。

10.11 本章小结

- 告警离开了集群：critical 变成 GitLab 事故，warning 变成工单，Watchdog 心跳由集群外的死人开关看守。冻结同时运行 Prometheus 与 Alertmanager 的节点后，死人开关在最后一次心跳后 183 秒开出事故，而集群内的告警一条也发不出来，甚至把一张工单错误地标为“已解除”。
- 一条在两次整集群重建后仍在响的 critical 告警，原因是 WSL 虚拟机的单调时钟比墙上时钟快约 3%，进程的“启动时间”随之漂移。启用通知之前先清理误报，否则第一次呼叫就是假的（A1）。
- 规则必须用真实数据检验：同步失败计数器在第一次失败时才诞生，`increase()` 看不见它（A3）；一个永远不健康的子应用挡住了根应用的同步，新告警及时呼叫。
- SLO 告警由规格生成，按多窗口、多燃烧率触发。事故演练中，一个进入生产的测试参数让 98% 的下单失败；因为窗口里只有 35 分钟的历史，呼叫在 2 分钟后到来，否则需要约 9 分钟——比整个故障还长。值班工程师从部署记录中找到了改动，69 秒后撤回；告警却在恢复 28 分钟后才解除。670 次失败约占当月下单预算的 1%。
- 复盘的七个行动项中四个已经完成并验证：发布策略拒绝生产环境中的故障模拟参数，症状告警在 2 分钟内呼叫，工单等待 5 分钟以便被呼叫抑制，SLI 只统计 POST。
- 节点级日志让被删除的 Pod 的输出可以查询，审计日志开始记录服务账号读取了哪个 Secret。10% 的头部采样曾让尾部采样只看到十分之一的错误、让 exemplar 指向不存在的链路；改为 100% 后，服务的 CPU 增加 5%，存储增加 8.7 倍，每一个错误都可以从指标点到链路。

第 11 章

控制面的规模边界（E10）

本书的实验集群只有 4 个节点。企业集群常有几百个节点、上万个 Pod，控制面——etcd、API server、调度器——在那个规模上会遇到 4 个节点时看不到的问题。本章不假装拥有一个大集群：用 KWOK 在同一台电脑上启动一套真实的 Kubernetes v1.37.0 控制面，让它管理 1000 个 `[虚拟]` 节点和 10 000 个 `[虚拟]` Pod——它们只是 etcd 里的对象，状态由一个控制器代写，没有一个容器真正运行。测量的是控制面为这么多对象所做的工作：调度被什么限速，etcd 里一个 Pod 占多少空间、删除为什么先让数据库变大，一万个 Pod 的 LIST 由谁提供，以及 API Priority and Fairness（APF）能否挡住一个失控的客户端。最后这个实验没有按计划进行：第一次运行冻结了整个实验环境（事故 E10-1），之后的六次运行逐一排除了错误的解释，才看清真正的限制——APF 完全按配置限制了并发，可是三个同时执行的大 LIST 就让 API server 多用了约 1.4 GiB 内存。

工程问题

实验集群只有 4 个节点，企业集群有几百个节点、上万个 Pod。控制面在那个规模上先撞到什么限制？手里没有大集群，怎样诚实地回答这个问题？

实测结论

- 默认配置下，调度慢不是算法的问题：10 000 个 `[虚拟]` Pod 用了 193.0 秒，速度恰好是调度器客户端限速的每秒 50 个。
- etcd 里每个最小的 Pod 占 3.73 KiB，严格线性；删除先让数据库变大（92.5 → 172.7 MiB），压缩之后还要碎片整理，文件才变小。
- 一万个 Pod 的 LIST 是 33 MiB、约 1 秒；即使要求一致的读，也由 watch 缓存提供，etcd 不再承担这部分读取。
- APF 完全按配置限制了并发，却救不了 API server：三个并发的大 LIST 让它多用约 1.4 GiB 内存。原来的假设在七次运行中被推翻。

11.1 学习目标

学完本章，你应当能够：

- 说明虚拟节点能验证什么、不能验证什么，并据此设计和解释规模实验；
- 从调度时间线上读出客户端限速，判断大批量创建 Pod 时哪个组件先成为瓶颈；
- 估算 etcd 的容量，说明压缩与碎片整理各做什么、什么时候需要；
- 判断一个 LIST 请求由 watch 缓存还是 etcd 提供，以及这对大集群为什么重要；
- 用 FlowSchema 和优先级隔离一个失控的客户端，并说明 APF 管不住什么。

实验性质

本章的控制面是 [真实] 的：etcd、kube-apiserver、kube-controller-manager、kube-scheduler 都是 Kubernetes v1.37.0 的官方镜像，由 kwokctl v0.8.0 作为 Docker 容器启动。节点和 Pod 是 [虚拟] 的：没有 kubelet，没有容器运行时，没有一个镜像被拉取。洪水客户端是 [注入] 的负载。事故 E10-1 是计划外的真实事件。每种配置只运行了一次，而且这套控制面与前十章的实验集群共用同一个 WSL 虚拟机——所有数字都要在这个前提下读。

11.2 原理

11.2.1 KWOK：只有控制面的集群

KWOK（Kubernetes WithOut Kubelet）有两个部分。kwokctl 用真实的组件启动一个集群的控制面；kwok 控制器代替所有的 kubelet：它看到一个 Pod 被绑定到虚拟节点，就把 Pod 的状态写成 Running，并替每个虚拟节点续租表示心跳的 Lease。对 API server、调度器和各个控制器来说，这些节点和 Pod 与真的一样：同样的对象，同样的写入、监听和调度。

示意图

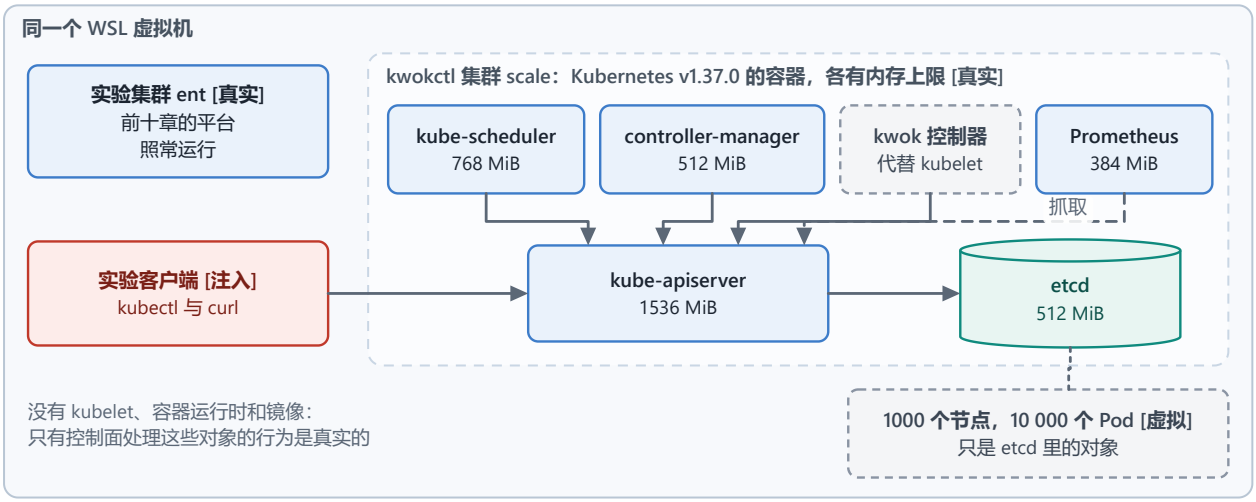


图 11-1 本章的实验环境。同一个 WSL 虚拟机中，前十章的实验集群 ent 与 kwokctl 启动的集群 scale 并存。scale 的控制面组件是真实的 Kubernetes v1.37.0 容器，每个都有内存上限（事故 E10-1 之后加上）；它管理的 1000 个节点和 10 000 个 Pod 只是 etcd 里的对象。

所以 KWOK 能测的是控制面为 N 个对象所做的工作：调度的吞吐、etcd 的体量、LIST 与 WATCH 的代价、限流。它测不了节点一侧：真实的 kubelet 会不断更新 Pod 和节点的状态，节点要拉镜像、启动容器、配置网络和存储，工作负载要消耗 CPU 和内存——这些在这里都不存在。本章的数字说明的是“控制面在这么多对象下的行为”，而不是“这台电脑能运行一万个 Pod”。

11.2.2 调度器：算法之外还有客户端限速

调度一个 Pod 分两步：先在节点中筛选、打分，选出一个节点；再向 API server 写一个绑定（`pods/binding` 子资源）。第二步是一次 API 请求，受调度器客户端的限速约束：`KubeSchedulerConfiguration` 的 `clientConnection.qps` 默认 50、`burst` 默认 100——令牌桶最多攒 100 个令牌，之后每秒补充 50 个。每个 Pod 一次绑定，所以无论算法多快，持续的调度速度都不会超过每秒 50 个。其他控制面组件也有同样的限速，例如 kube-controller-manager 的默认值是每秒 20 次、突发 30 次，它决定了 ReplicaSet 控制器创建 Pod 的速度。

限速保护的是 API server 和 etcd：所有组件都放开限速，写入就可能超过它们的承受能力。提高某个组件的限速是一个容量决定，不是免费的优化。

11.2.3 etcd：每次写入都是一个新版本

etcd 是一个多版本（MVCC）的键值存储：每次创建、修改、删除都产生一个新的修订版本（revision），旧版本仍然留在数据库文件里，直到被压缩（compaction）。kube-apiserver 每 5 分钟压缩一次（`--etcd-compaction-interval`），压缩到它上一次运行时记下的版本，所以 etcd 里通常保留着 5 到 10 分钟的历史。压缩只是把旧版本占用的页面标记为空闲，文件本身不变小；碎片整理（defragmentation）把有效数据重写到一个新文件里，才把空间还给文件系统。碎片整理期间这个成员不能读写，生产环境要逐个成员进行。

文件的大小很重要：etcd 有一个空间配额（`--quota-backend-bytes`，默认 2 GiB）。超过配额时，etcd 触发 `NOSPACE` 告警，整个集群只接受读和删除——不能再创建任何对象，包括事件。恢复的步骤是压缩、碎片整理、解除告警，缺一不可。

示意图

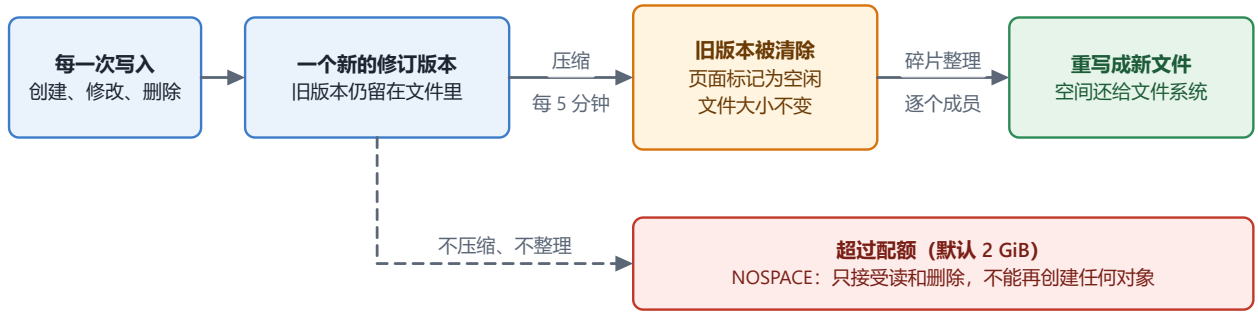


图 11-2 etcd 的空间去了哪里。每次写入都留下一个新版本；压缩只是把旧版本占的页面标记为空闲，文件并不变小；碎片整理才把空间还给文件系统。任由它增长，最终会撞上配额：整个集群只能读和删除，不能再创建任何对象。

11.2.4 watch 缓存：LIST 由谁提供

API server 为每种资源维护一个内存中的缓存，通过一个对 etcd 的 watch 保持更新。客户端的 WATCH 由它提供，带 `resourceVersion=0` 的 LIST 也由它提供（结果可能略旧）。不带 resourceVersion 的 LIST 要求“一致”——看到此刻 etcd 中的数据——过去必须从 etcd 读：一万个 Pod 就是几十 MB 的数据从 etcd 读

出、解码、再编码。在大集群里，大量客户端同时重新 LIST（比如控制器重启之后）曾是 etcd 和 API server 过载的常见原因。从 Kubernetes 1.31 起，一致的 LIST 默认也由缓存提供（KEP-2340）：API server 只向 etcd 询问当前的修订版本，等缓存追上这个版本，再从内存返回结果。

示意图

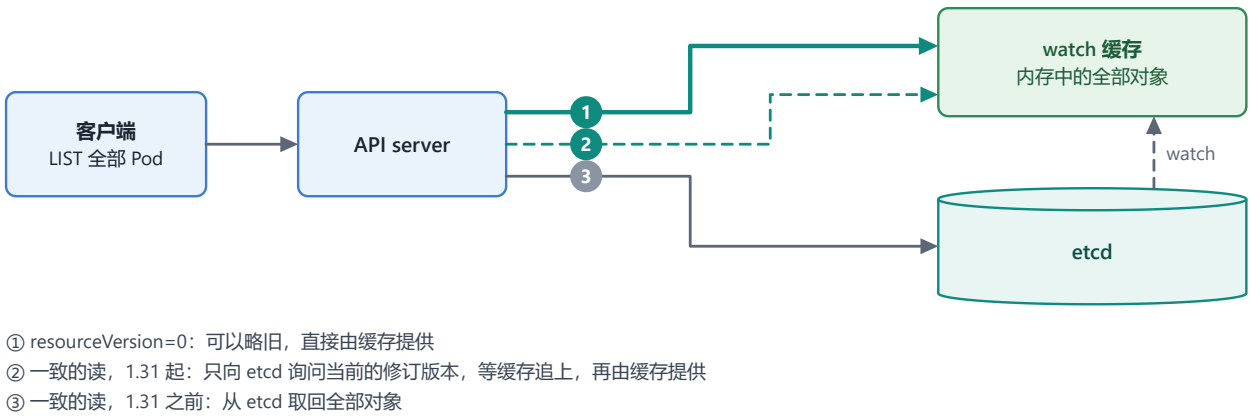


图 11-3 一次 LIST 的三种走法。① 带 resourceVersion=0 的读一直由内存中的 watch 缓存提供；③ 要求“一致”的读过去必须从 etcd 取回全部对象；② 从 Kubernetes 1.31 起改为只向 etcd 询问当前的修订版本，等缓存追上，再从内存返回。

11.2.5 APF：按优先级分配并发

API Priority and Fairness 用 FlowSchema（按用户、服务账号、动词、资源匹配）把每个请求分到一个优先级（PriorityLevelConfiguration）。API server 的总并发——默认 `--max-requests-inflight` 400 加 `--max-mutating-requests-inflight` 200，共 600 个“席位”——按各优先级的份额（`nominalConcurrencyShares`）分配。同一优先级内，不同的“流”（例如不同的用户）排进不同的队列，用混洗分片（shuffle sharding）避免一个流挤占所有队列；队列满了，请求立即得到 429（Too Many Requests）。按设计，一个 LIST 会按估算要处理的对象数量占用多个席位（大约每 100 个对象一个，最多 10 个）；实验 4 会看到实际情况并非总是如此。

份额只决定一个优先级的“名义”席位。空闲的优先级可以把一部分席位借出（`lendablePercent`），繁忙的优先级可以借用（上限由 `borrowingLimitPercent` 决定，不设置就没有上限）。

APF 限制的是“同时处理多少个请求”。它不限制一个请求用多少内存：返回一万个 Pod 的 LIST，无论在哪个优先级执行，都要构造几十 MB 的响应。本章最后一个实验会反复碰到这一点。

示意图

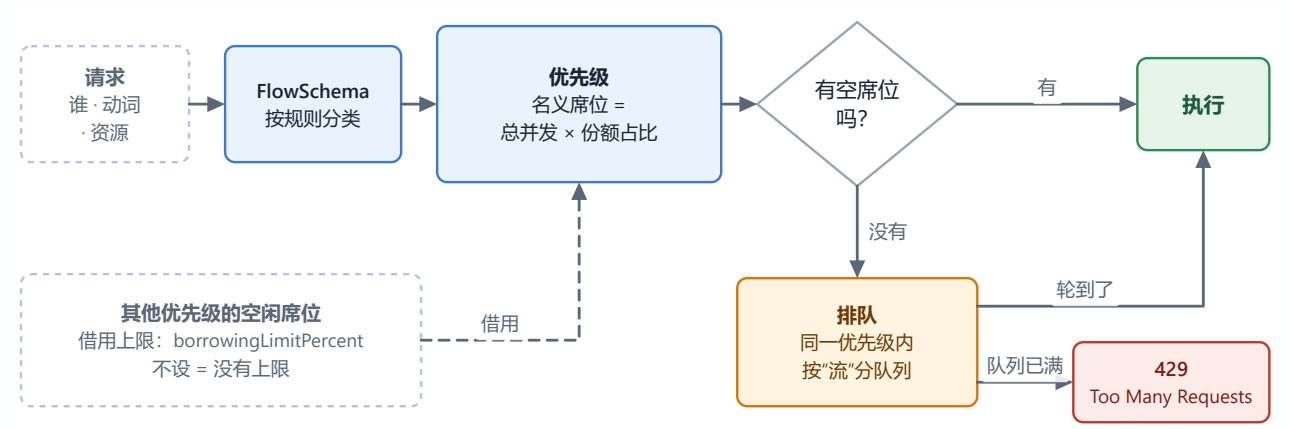


图 11-4 APF 处理一个请求的过程。FlowSchema 把请求分到一个优先级；优先级有自己的名义席位，没有空席位就排队，队列满了立即返回 429。繁忙的优先级还可以借用别的优先级空闲的席位——除非显式把借用上限设为 0。整个过程只数“同时执行多少个请求”，不看每个请求要用多少内存。

11.3 实验 1：一万个虚拟 Pod 的调度（K1、K2）

kwokctl 在 15 秒内建好 1000 个虚拟节点（每个 32 核、256 GiB、最多 110 个 Pod）；8 个并行的 `kubectl create` 创建 10 000 个 Pod，每个请求 0.1 核和 128 MiB，容忍所有污点。脚本每隔几秒数一次：已有多少 Pod、多少还没调度、多少已经 Running。实验前写下的假设：

- **K1**：在调度器的默认限速下（每秒 50 次，突发 100），调度速度约为每秒 50 个，10 000 个至少需要 3.3 分钟；
- **K2**：用 `--disable-qps-limits` 重建集群（调度器改为每秒 5000 次、突发 10 000），同样的 10 000 个 Pod 至少快 5 倍，瓶颈转移到调度器的 CPU 和 API server 的写入延迟。

	K1：默认限速	K2：放开限速
8 个 kubectl 创建完 10 000 个 Pod	51.3 秒	27.5 秒
全部调度完成	193.0 秒	29.4 秒
创建结束后的调度速度	每秒 50.3 个（54.5–164.7 秒）	跟上了创建的速度
全部 Running	193.0 秒	48.6 秒
调度尝试（Prometheus）	10 000 次，全部成功	10 000 次，全部成功

实测数据

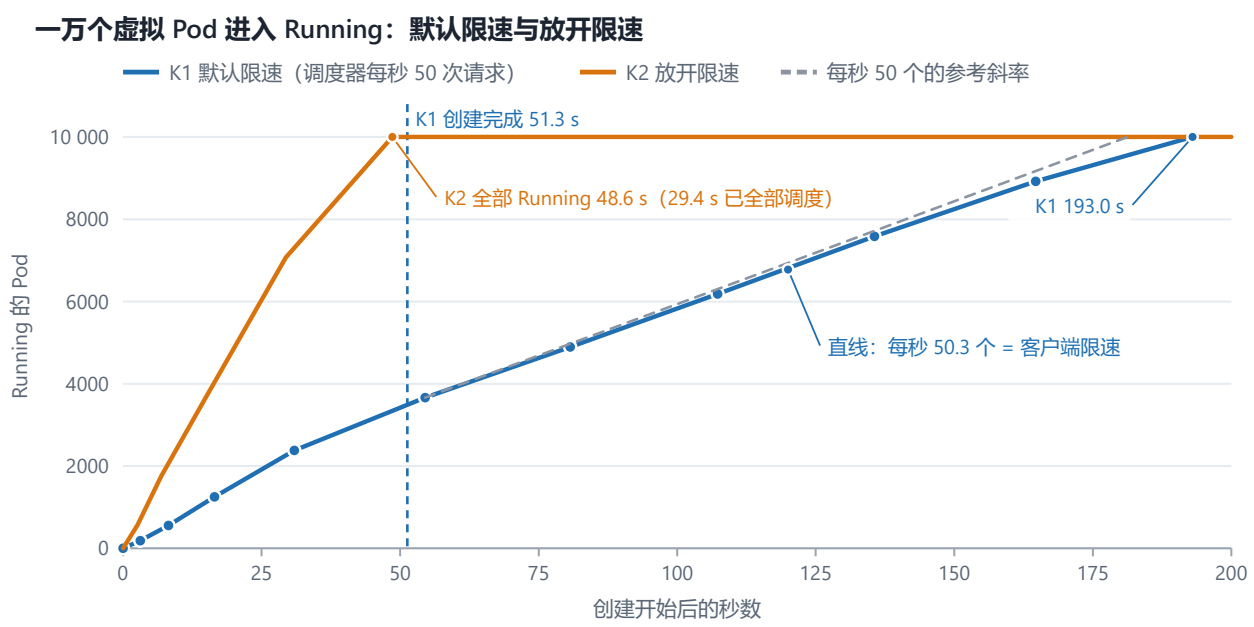


图 11-5 一万个虚拟 Pod 进入 Running 的过程，每个点是一次实际计数。K1 在创建结束之后是一条直线，斜率每秒 50 个——调度器并不忙，它在等客户端限速的令牌。K2 放开限速后调度跟着创建走，29.4 秒全部调度完；Running 晚到 48.6 秒，是因为所有 Pod 的状态都由同一个 kwok 控制器写入。

K1 成立。 创建结束之后，已调度的 Pod 以每秒 50.3 个的速度直线增长，与客户端限速一致：调度器并不忙，它在等令牌。10 000 次调度全部一次成功——虚拟节点共有 32 000 核，这些 Pod 只请求了 1000 核。后来为 K5 新建的集群（同样的默认值）在 31–166 秒之间测得每秒 52 个，重复了这个结果。

K2 只成立了一半。 放开限速后，全部 Pod 在 29.4 秒调度完成，是 K1 的 6.6 倍；但创建本身就用了 27.5 秒，调度器一直跟着创建走——测到的上限是 8 个 kubectl 的创建速度，调度器真正的上限（假设的后半句）没有测到，只能说“至少每秒 340 个”。Running 比调度晚了 19 秒，因为所有状态都由一个 kwok 控制器写入；在真实集群里，这项工作由每个节点上的 kubelet 并行完成。

工程上的含义：大批量创建 Pod 时，先成为瓶颈的往往是某个组件的客户端限速，而不是调度算法。一个 Deployment 从 0 扩到一万个副本，Pod 由 ReplicaSet 控制器创建，它受 kube-controller-manager 的限速约束（练习 1）。批处理和训练任务常常需要调高这些限速，每一次调高，都要确认 API server 和 etcd 承受得住。

11.4 实验 2：etcd 里的一个 Pod（K3）

假设 **K3**：etcd 随对象数量线性增长，每个 Pod（连同它的事件）几 KB；删除 Pod 不会让数据库文件变小，直到碎片整理。数据库的大小直接从 etcd 读取（`etcdctl endpoint status`：文件大小和其中的有效数据）。

先看删除。集群里有 10 000 个 Pod 时，数据库文件是 92.5 MiB；删除全部 Pod 之后，文件变成了 172.7 MiB——**删除让它增大了 80 MiB。**

实测数据

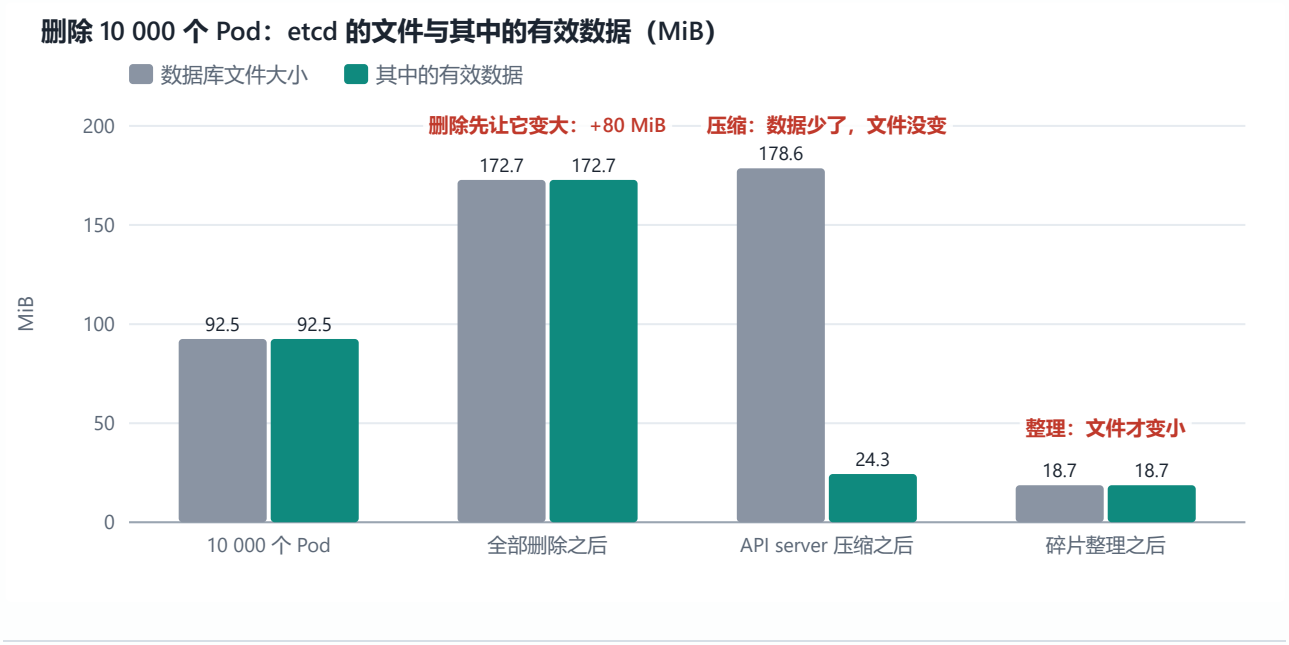


图 11-6 删除 10 000 个 Pod 前后的 etcd。灰柱是数据库文件，绿柱是其中的有效数据：删除让文件增大；API server 的压缩把有效数据降到 24.3 MiB，文件却没有变小；碎片整理之后文件才变为 18.7 MiB。

每删除一个 Pod 会写两次：先给它打上删除标记（优雅删除），kwok 控制器随后真正删除它。两个都是新版本，第一个是对象的完整副本；旧版本要等到压缩才会消失。在后面逐步删除的测量里，每删除一个 Pod，数据库先增大 8.1–8.5 KiB。

接下来的几步各有教训：

- **压缩之前做碎片整理没有用，而且慢。** 删除之后立刻整理，etcdctl 在默认的 5 秒命令超时后失败：172 MiB 全是有效数据，没有什么可以还回去，却有很多要复制。压缩之后再整理，只用了 0.35 秒。脚本此后给 etcdctl 设置 120 秒的超时。
- **API server 的压缩来得比预想的晚。** 它在 10:28:47 出现，正好是这个 API server 上一次重启（10:18:42，K5 第二次运行时被杀）之后 10 分钟：重启后的第一次运行只与 etcd 中记录的压缩进度同步，第二次才真正压缩。之后文件 178.6 MiB，其中有效数据只有 24.3 MiB。
- **只有碎片整理让文件变小**：18.7 MiB。

再看增长是否线性。从 10 000 个 Pod 开始逐步删除，每一步都把 etcd 压缩到当前版本（`compact --physical`，等旧版本真正删除之后才返回）并整理碎片，得到的就是有效数据的大小：

Pod 数	0	5000	6969	7134	10 000
数据库（MiB）	17.65	35.85	43.02	43.62	54.14

0 到 5000 每个 Pod 3.73 KiB，5000 到 10 000 每个 3.75 KiB：严格线性，**每个 Pod 3.73 KiB**。刚创建完时（还带着 5–10 分钟的历史）是 89–93 MiB，比有效数据多出约 70%。K3 成立，而且删除比假设的更糟：它不只是不让文件变小，而是先让它变大。

这里的 Pod 很小：一个容器、pause 镜像，JSON 只有 3.4 KB。真实的 Pod 有多个容器、环境变量、卷、探针和 `managedFields`，常常大好几倍，容量估算要用自己的对象来测（练习 6）。最危险的情形由上面两个结果合成：数据库已经接近配额时，有人批量删除大量对象来“腾空间”——在压缩之前，删除本身就可能把 etcd 推过配额，触发 `NOSPACE`（练习 2）。

测量中犯过两个错误，都留在记录里：第一次“上行”测量用的 `etcdctl compact` 没有加 `--physical`，碎片整理在旧版本被真正删除之前就开始了，那几个“压缩后”的数值仍然包含历史，没有用于上面的表格；一次按名字删除 5000 个 Pod 的操作每秒只删掉约 5 个（原因没有深究，`kubectl delete pods --all` 在同一个集群上每秒至少 84 个），被脚本的超时打断，之后改为 16 个 kubectl 并行删除。

11.5 实验 3：一万个 Pod 的 LIST（K4）

假设 **K4**：Kubernetes 1.37 从 watch 缓存提供一致的 LIST（不带 resourceVersion），它的延迟与 `resourceVersion=0` 接近，etcd 不会被读取；一万个 Pod 的响应有几十 MB。

请求（各 5 次）	耗时	大小
`resourceVersion=0`	0.94–1.23 秒	33 MiB
不带 resourceVersion（一致的读）	0.88–1.06 秒	33 MiB
分页，`limit=500`（20 页）	5.6–7.1 秒（大部分是客户端的时间，见下文）	33 MiB

每种请求单独发一次，在请求前后读取 API server 的计数器：三种请求发往 etcd 的 Pod LIST 都是 +0；一致的读和分页遍历各让“由 watch 缓存提供的一致读”加 1。这个 API server 启动以来，一共只向 etcd 发过 2 次 Pod 的 LIST（填充缓存），却由缓存提供了 1043 次一致读。**K4 成立。**

第一次归因用错了指标：`apiserver_storage_list_total` 在存储层计数，由缓存提供的 LIST 也算在内，据此得出的“etcd +110”是错的。判断请求到底去了哪里，要看真正发往 etcd 的请求（`etcd_request_duration_seconds_count{operation="list"}`）。分页遍历的耗时也要打折扣：脚本为了取出下一页的 continue 令牌，每一页都启动一个新的 Python 进程来解析 1.7 MB 的 JSON，这 5.6–7.1 秒主要是客户端自己的时间（练习 3）。

工程上的含义：即使由内存提供，一次完整的一万个 Pod 的 LIST 也要 API server 工作约 1 秒、传输 33 MiB。一个每 10 秒运行一次 `kubectl get pods -A` 的看板或脚本，就要持续消耗每秒 3.3 MB 的序列化和网络。控制器应当用 informer（LIST 一次，然后 WATCH），脚本应当用标签或字段选择器缩小范围。下一个实验展示 40 个不这样做的客户端会怎样。

11.6 实验 4：一个失控的客户端（K5）与事故 E10-1

设计：服务账号 e10-flood 由一个 FlowSchema（`matchingPrecedence` 500）分到优先级 e10-low：份额 1（折合 3 个席位），不借出（`lendablePercent: 0`），一个长度为 5 的队列。40 个工作进程以它的身份循环 LIST 全部 Pod（`resourceVersion=0`），持续 60 秒。受害者 e10-victim 走默认的 FlowSchema

service-accounts，进入优先级 workload-low（244 个席位），每 0.5 秒 GET 一个 Pod：先单独运行 30 秒，再在洪水期间运行 60 秒。假设 **K5**：洪水的大部分请求得到 429；受害者的 p99 不超过安静时的 2 倍。

这个实验一共运行了七次。每次失败之后，先把新的解释作为假设写下并提交（K5b–K5e，见 [1]），再运行下一次：

运行	与前一次相比的改变	API server	受害者成功的 GET 的 p99：安静 → 洪水
1	原始设计，没有内存上限	整个虚拟机冻结：事故 E10-1	42.8 ms → 10.4 s，之后中断
2	每个容器加内存上限（API server 1536 MiB），但误用了 APF 被关闭的集群	被内核杀死 8 次	40.1 ms → 2081 ms
3	换成默认配置的集群，APF 开启	被杀 6 次	19.4 ms → 219 ms
4	API server 设置 `GOMEMLIMIT=1300MiB`（K5b）	被杀 4 次	19.4 ms → 1800 ms
5	5000 个 Pod 代替 10 000 个（K5c）	被杀 8 次	15.6 ms → 1387 ms
6a	e10-low 设置 `borrowingLimitPercent: 0`（K5d）	被杀 5 次	20.7 ms → 437 ms
6b	去掉借用上限（与 1–5 相同）	被杀 8 次	14.7 ms → 1540 ms
7	同 6a，API server 内存上限 2560 MiB（K5e）	被杀 1 次	20.3 ms → 194 ms

实测数据

K5 的七次运行：洪水期间受害者成功请求的 p99（毫秒，对数刻度）

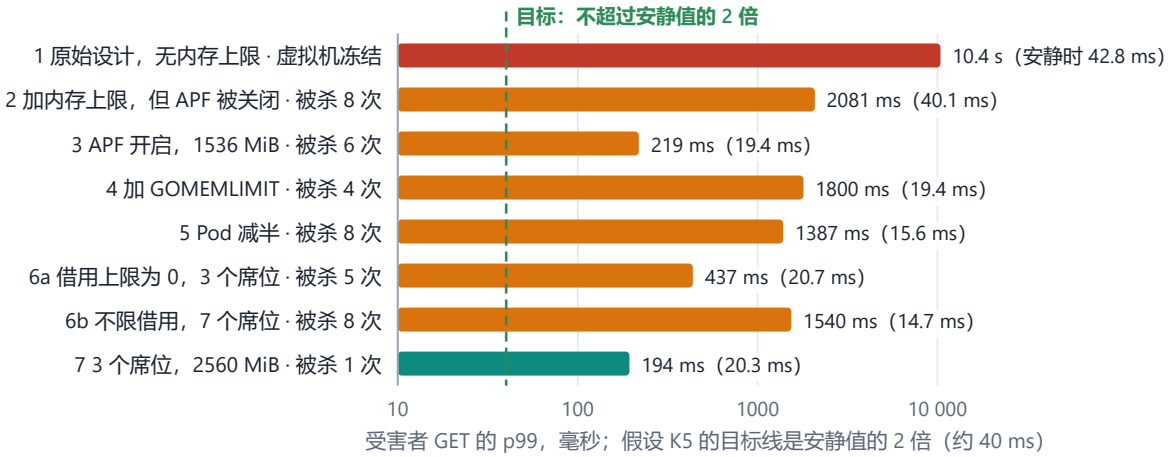


图 11-7 七次运行的结果。条形是洪水期间受害者成功请求的 p99，越短越好；括号里是洪水开始前的 p99。没有一次运行把它保持在安静值的 2 倍以内（绿色虚线）：每一次改动都排除了一个错误的解释，直到第 7 次才看清真正的限制是 API server 的内存。

11.6.1 第一次运行：事故 E10-1

APF 做了它承诺的事：1329 个洪水请求中有 1289 个在约 0.1 秒内得到 429，只有 4 个被接纳。但受害者的 p99 从 42.8 毫秒升到 10.4 秒，几分钟后整个 WSL 虚拟机停止响应——实验集群 ent、GitLab、告警桥，所有东西一起冻结。强制终止虚拟机时，1 分钟负载是 858。

复盘[3]的结论：被测的控制面与实验集群共用一个虚拟机，又没有内存上限；10 000 个 Pod 时它已占用约 1.8 GiB，虚拟机只剩 2.3 GiB 可用。被接纳的 LIST 加上其余负载把虚拟机推进了换页，从那一刻起所有进程一起变慢。

恢复时还暴露了一个隐藏的故障：集群外的告警桥（第 10 章）重启后，死人开关第一次检查就试图在还没启动完的 GitLab 中开事故，HTTP 调用抛出异常，检查线程就此退出——死人开关停了，却什么也没说。修复后每次检查都单独捕获异常并重试，线程不在时 `/healthz` 返回失败。与本实验有关的行动项是：给实验控制面的每个容器设置硬内存上限（`docker update --memory`，交换空间同值），让过载只杀死自己 cgroup 里的进程；洪水脚本监视虚拟机的可用内存，低于 1.2 GiB 时自行停止。

11.6.2 内存上限，以及 API server 重启时客户端看到什么

第二次运行用的是 K2 的集群——它用 `--disable-qps-limits` 创建，而这个选项同时以 `--enable-priority-and-fairness=false` 和 `--max-requests-inflight=0` 启动 API server（事后在一个临时集群上核实并记录）。我事先没有检查：FlowSchema 和优先级被 API 接受了，却没有任何作用，这次运行不能检验 APF。

它检验了内存上限：洪水开始 1.7 秒后，API server 达到 1536 MiB，内核在它自己的 cgroup 里把它杀死（内核日志：`Memory cgroup out of memory: Killed process … (kube-apiserver) … anon-rss:1567152kB`），Docker 重启它，60 秒内重复了 8 次；虚拟机始终有 3.4 GiB 可用，实验集群不受影响。Docker 的 `OOMKilled` 标志却显示 false——它只描述容器当前这一次运行，证据要看内核日志和重启次数。

客户端在每次重启之间看到了什么？专门的探测（重启 API server，然后每 0.2 秒以 e10-flood 的身份 LIST 一次）给出了答案：

终端记录

devops-lab · restart-probe.sh（2026-11-10 11:00 UTC，kwokctl 集群 scale，10 000 个虚拟 Pod） 终端记录渲染

```
== 11:00:13 10000 pods; docker restart kwok-scale-kube-apiserver
  restart returned after 11.7 s
  t=11.8s 000
  t=13.3s 403 pods is forbidden: User "system:serviceaccount:default:e10-flood" cannot list resource "pods" in API group ""
at the cluster scope
  t=13.6s 429 storage is (re)initializing
  t=16.3s 200
  11:00:42 three 200 in a row after 19.6 s
```

图 11-8 API server 重启时客户端看到的响应：先是连接被拒绝（000），然后是 403——服务账号明明有权限，但授权器还没有加载 RBAC 绑定——然后是 429“storage is (re)initializing”——Pod 的 watch 缓存正在填充——最后才是 200。从容器启动到第一个 200 约 4.6 秒。

把 403 当作最终结果的客户端，会放弃一个它其实有权限做的请求。真实的高可用控制面让负载均衡器只把流量发给 `/readyz` 就绪的 API server，客户端就看不到这个窗口；本实验的客户端直接连到 API server。洪水客户端还暴露了另一个问题：16 873 个请求在约 2 毫秒内以连接失败告终——一个没有退避的客户端，对着一个正在重启的服务器空转。

11.6.3 APF 开启，API server 仍然被杀

第三次在一个按默认值新建的集群上运行，APF 确认开启：响应头显示受害者的请求经过 FlowSchema service-accounts、优先级 workload-low，洪水请求经过 e10-flood、e10-low。API server 仍然被杀死了 6 次。洪水开始 20 秒时的一个探测请求得到 429，消息却是“storage is (re)initializing”——它来自重启后的 watch 缓存，不是 APF。APF 的拒绝计数显示 +0：计数器在进程里，每次重启都从零开始。

洪水开始前 API server 已经常驻 976 MiB。第一个解释是 Go 的垃圾回收器：它默认（`GOGC=100`）在堆增长到上次回收后存活数据的两倍时才回收，而且不知道容器的内存上限，除非设置了 `GOMEMLIMIT`。第四次运行给 API server 设置了 `GOMEMLIMIT=1300MiB`，指标 `go_gc_gomemlimit_bytes` 证实它已生效——API server 仍然被杀了 4 次。回收器的余量不是全部原因。

11.6.4 借来的席位，和不等于内存的席位

第五次运行把 Pod 减半，并且每秒记录一次 API server 所在 cgroup 的内存。它仍然被杀了 8 次。内存曲线说明了原因的方向：静止时 662 MiB，洪水开始后的几秒内依次是 825、1172、1523 MiB——然后被杀；每次重启之后，几秒之内又涨过 1.2 GiB。所有 4033 个 429 都带着 APF 的消息“Too many requests, please try again later.”。每秒 200 MiB 以上的增长，不像是一次只执行一个 LIST。

问题出在实验设计上。e10-low 设置了 `lendablePercent: 0`——它不借出席位——却没有设置 `borrowingLimitPercent`。在 v1.37 的 API 文档里（`kubectl explain`），这个字段“为空时，上限实际上是无限的”。默认的优先级都没有设置它，而且都在借出：workload-low 借出 244 个席位中的 90%，workload-high 和 global-default 各 50%，system 33%，node-high 25%。洪水需要席位时，e10-low 可以借用其他优先级空闲的席位。

第六次运行在同一个集群上先后做了两次洪水，每 2 秒记录一次 e10-low 的席位：设置 `borrowingLimitPercent: 0` 时，它的上限始终是 3 个席位（3 个在执行，5 个在排队）；去掉这个设置，上限升到 7 个——借来了 4 个。**借用是真的，但它不是根本原因**：只有 3 个席位时，API server 同样在洪水开始 4 秒内从 938 MiB 涨到 1487 MiB，被杀了 5 次。还有一个意外：采样显示 3 个 LIST 只占 3 个席位，每个 LIST 只算 1 个席位，并没有按要返回的对象数量加权（原因没有深究）。

实测数据

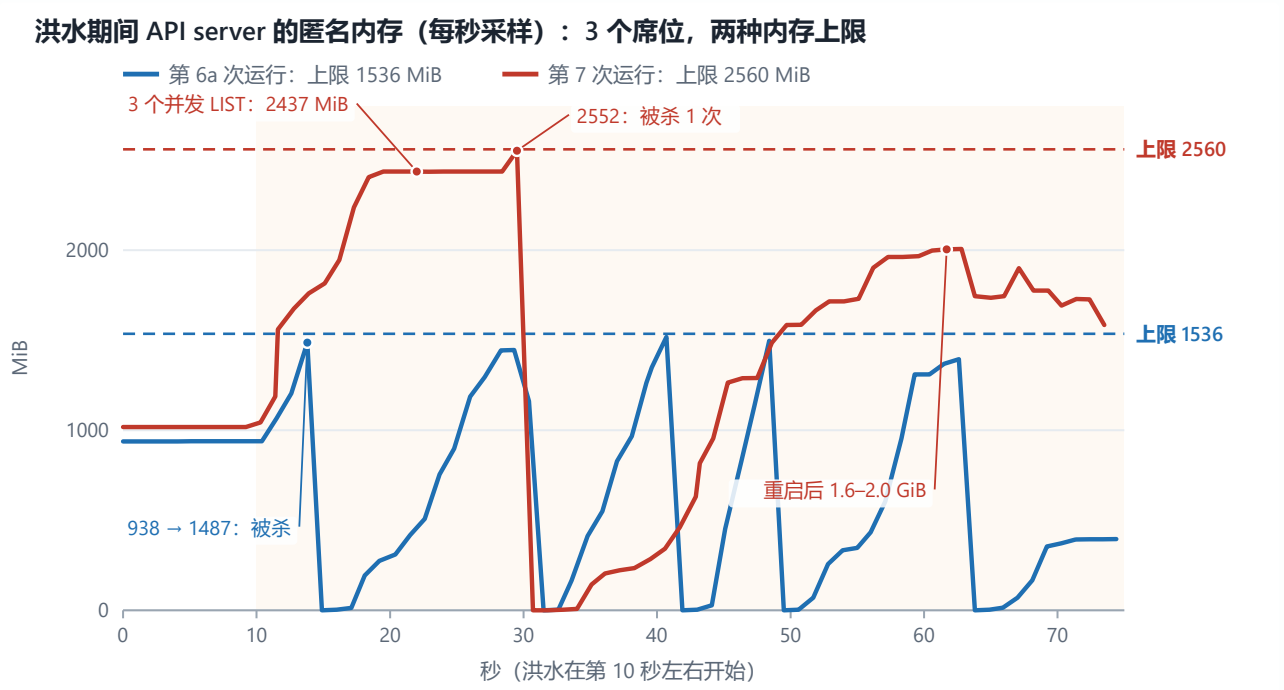


图 11-9 洪水期间 API server 的匿名内存，每秒一个采样点。两次运行中优先级 e10-low 都只有 3 个 LIST 在执行；曲线落到 0 的地方，是进程被内核杀死、Docker 重启它的时刻。上限 1536 MiB 时它在 4 秒内被杀，之后反复被杀；上限 2560 MiB 时，三个并发的 LIST 把它推到 2437 MiB，10 秒后越过上限被杀一次，重启后在同样的洪水下维持在 1.6–2.0 GiB。

第七次运行回答了剩下的问题：席位不变（3 个），只把 API server 的上限提高到 2560 MiB。它的内存在洪水开始 1 秒后是 1560 MiB，8 秒后是 2437 MiB，停在那里 10 秒，然后涨到 2552 MiB，被杀了一次；重启之后，在同样的洪水下维持在 1.6–2.0 GiB，没有再被杀。三个并发的、返回一万个 Pod（每个 33 MiB 的 JSON）的 LIST，连同垃圾回收器的余量，让 API server 多用了约 1.4 GiB。即使在它正常服务的时候，受害者成功的 GET 也慢了：中位数从 13 毫秒变成 71 毫秒，p99 从 20 毫秒变成 194 毫秒。

11.6.5 七次运行说明了什么

K5 不成立：没有一次运行让受害者的 p99 保持在 2 倍以内，每一次 API server 都至少被杀了一次。但这七次运行把问题拆得很清楚：

1. **APF 完全按配置工作。** 分类（由响应头确认）、席位（采样确认 3 个在执行、5 个在排队）、拒绝（429 “Too many requests”）都和配置一致。
2. **份额不是上限。** 不设置 `borrowingLimitPercent` 的优先级会在需要时借用别人的席位；要一个硬上限，就要显式设为 0。
3. **席位不是内存。** APF 数的是同时执行的请求，一个返回一万个 Pod 的 LIST 和一个 GET 在这里都只算 1 个席位，内存代价却相差几个数量级。给大 LIST 的客户端分配席位时，要按“席位数 × 每个 LIST 的内存”留出 API server 的内存，或者干脆把这类客户端的席位压到最少。

4. **CPU 也不在 APF 的管辖之内。** 只有 3 个 LIST 在执行时，另一个优先级里的 GET 也慢了 5 到 10 倍：它们共享同一个进程的 CPU 和垃圾回收。
5. **内存上限把“整台机器冻结”变成了“一个进程被杀”。** 这是正确的方向，但被杀的 API server 重启之后要面对更多的负载：授权器加载之前返回 403，缓存填充期间返回 429，每个组件都要重新 LIST。生产环境的控制面有多个 API server，正是为了让一个实例的重启不变成整个集群的中断。
6. **根本的修复在客户端。** 不要在循环里 LIST 全部对象：用 informer（LIST 一次，然后 WATCH），用分页把每个响应限制在几 MB，遇到 429 和连接失败要退避。

11.7 规模边界

	本章测得（1000 个［虚拟］节点、10 000 个［虚拟］Pod，单机）
调度	默认限速下每秒 50 个；放开限速后至少每秒 340 个（受创建速度限制，调度器的上限未测到）
etcd	每个 Pod 3.73 KiB 有效数据，线性；刚创建时带着历史，多约 70%；删除时每个 Pod 先增大 8.1–8.5 KiB
LIST	10 000 个 Pod 的响应 33 MiB，约 1 秒，一致的读也由 watch 缓存提供
内存	10 000 个 Pod 时 API server 常驻约 0.95 GiB，控制面合计约 1.8 GiB
洪水	APF 按配置分类、限流、拒绝；三个并发的一万 Pod LIST 让 API server 多用约 1.4 GiB，1.5 GiB 和 2.5 GiB 的上限都没能让它撑过 60 秒的洪水

作为参照，Kubernetes 文档给出的大集群上限是 5000 个节点、150 000 个 Pod、每个节点 110 个 Pod。本章的规模只是它的五分之一和十五分之一，而且节点和 Pod 都是虚拟的。没有测量的东西同样重要：kubelet 的负载、真实 Pod 的大小、高可用控制面（3 个 etcd 成员、负载均衡器后面的多个 API server）、真实的网络和工作负载。本章能支持的结论是控制面机制层面的：限速在哪里、存储如何增长、读由谁提供、限流管什么不管什么；不能支持“这套配置能承载多大的生产集群”。

11.8 局限

- 每种配置只运行一次；控制面与实验集群共用一个虚拟机，绝对数值随实验集群的活动而变化。
- 节点和 Pod 是虚拟的：kwok 控制器替 1000 个节点续租 Lease、写 Pod 状态，真实的 kubelet 写入更多；镜像、容器、网络完全不存在。
- K2 没有找到调度器本身的上限；K4 的分页耗时包含客户端的解析时间。
- K5 的七次运行中，前两次各自混入了一个额外的变量（第一次没有内存上限，第二次 APF 被关闭）；第三次没有记录每个 429 的消息，它的 429 中有多少来自 APF、多少来自缓存，无法区分。`GOMEMLIMIT` 只测试了一个值；2560 MiB 是这台机器在实验集群运行时能让出的上限，更大的内存没有测试。
- 每个 LIST 只占 1 个席位、第七次运行重启后 25 次连续的连接失败，这两个现象的原因没有查明。
- etcd 的每 Pod 大小只适用于本实验的最小 Pod。

11.8.1 从实验环境到生产环境

本章的做法	生产环境通常的做法	差别意味着什么
节点和 Pod 是 KWOK 的虚拟对象	真实的 kubelet：状态更新、心跳、镜像与容器	本章只回答控制面的问题；节点一侧的负载要另行压测
控制面是单实例的容器，与实验集群共用一台机器	3 个 etcd 成员，负载均衡器后面的多个 API server	一个实例重启时的 403 与 429 窗口，在生产环境由就绪检查和其他实例遮住
最小的 Pod（3.4 KB 的 JSON）	多个容器、环境变量、卷、探针与 managedFields	etcd 和 LIST 的体量要用自己的对象重新测量
1000 个节点、10 000 个 Pod	Kubernetes 文档给出的上限是 5000 个节点、150 000 个 Pod	本章的规模只是它的五分之一和十五分之一

11.9 常见问题

问：KWOK 的结果能代表真实集群吗？控制面部分可以：同样的二进制，同样的对象，同样的请求路径，调度器的限速、etcd 的增长、watch 缓存、APF 的行为都与真实集群一致。节点一侧不能：没有 kubelet 的状态更新，没有镜像拉取和容器启动。用它回答“控制面在 N 个对象下会怎样”，不要用它回答“集群能跑多少负载”。

问：既然调度器被限速在每秒 50 个，为什么默认值不调高？默认值要适用于所有集群，包括小规模的控制面。调度器每秒的绑定就是 API server 每秒的写入，加上随之而来的事件和状态更新。需要高调度吞吐的集群（批处理、训练任务）应当在确认 API server 和 etcd 余量之后，有意识地调高，并同时观察写入延迟。

问：删除对象为什么会让 etcd 变大？ etcd 保留每个键的历史版本。删除一个 Pod 至少写两个新版本：删除标记（对象的完整副本）和删除本身。旧版本要等 API server 的压缩（每 5 分钟，保留 5–10 分钟的历史）才会被清除，文件要等碎片整理才会变小。所以批量删除应当分批进行，并留出压缩的时间。

问：APF 已经拒绝了 97% 的洪水请求，为什么机器还是被拖垮了？ APF 控制的是同时执行的请求数，不是每个请求的内存，也不是整台机器的资源。一个 LIST 一万个 Pod 的请求要构造 33 MiB 的响应；API server 本身在 10 000 个 Pod 时就占用约 1 GiB。第一次运行时这些压力落在一个没有余量的共享虚拟机上；之后的运行里，同样的压力落在 API server 自己的内存上限上。即使只有 3 个这样的 LIST 同时执行，API server 也多用了约 1.4 GiB；另一个优先级的请求虽然没有被拒绝，也因为共享 CPU 而慢了 5 到 10 倍。

问：API server 刚启动时返回 403，是权限配置错了吗？不是。重启探测中，同一个服务账号在启动后 1.5 秒得到 403，3 秒后就得到 200：授权器加载 RBAC 绑定之前，所有需要 RBAC 授权的请求都会被拒绝。生产环境中，负载均衡器按 `/readyz` 转发流量，客户端看不到这个窗口；直连 API server 的客户端应当对启动期间的 403 和 429 有所准备。

11.10 练习

练习

1. **（计算）** 一个 Deployment 从 0 扩到 10 000 个副本。kube-controller-manager 使用默认的客户端限速（每秒 20 次，突发 30 次），调度器也使用默认值。估算全部 Pod 被调度所需的最短时间，指出瓶颈在哪个组件；如果把 kube-controller-manager 的限速调到每秒 200 次，瓶颈移到哪里？
2. **（计算）** 一个集群有 40 000 个 Pod，按本章的方法测得每个 Pod 在 etcd 中有效数据 12 KiB，其余对象共 300 MiB，etcd 使用默认配额。估算压缩并整理后的数据库大小；一次删除 20 000 个 Pod，在压缩之前数据库会增长多少？给出一个安全的批量删除方案。
3. **（分析）** K4 的分页遍历用了 5.6–7.1 秒，是完整 LIST 的 6 倍。解释为什么这不能说明分页在服务器端很慢，并设计一个能把服务器端和客户端的时间分开的测量。
4. **（设计）** 一个 CI 系统的服务账号每 10 秒列出所有命名空间的 Pod，集群有 30 000 个 Pod。从 API server（FlowSchema 与优先级、借用上限、内存）和客户端（informer、选择器、分页、退避）两方面给出改进，说明每一项防住的是什么、防不住什么。
5. **（分析）** 重启探测中，客户端在 API server 启动后先得到 403，再得到 429。分别说明它们的来源；对一个把 403 当作永久错误的控制器，这会造成什么后果？生产环境如何避免？
6. **（操作）** 用第 2 章 storefront 中任意一个 Deployment 的 Pod 模板代替 pause Pod，重复 K3 的“下行”测量（`experiments/e10/etcd-size.sh down`），得出每个 Pod 的有效数据大小，与本章的 3.73 KiB 比较并解释差异。

参考答案见附录。

11.11 本章小结

- KWOK 让一台电脑运行真实的控制面和虚拟的节点；它能回答控制面的问题，不能回答节点和负载的问题。实验报告必须写清哪些是真实的、哪些是虚拟的。
- 在默认配置下，调度速度由调度器的客户端限速决定：每秒 50 个，10 000 个 Pod 需要 193 秒。放开限速后，测到的上限变成了创建 Pod 的客户端。
- etcd 中每个最小的 Pod 约 3.73 KiB，严格线性；删除先让数据库变大，压缩释放文件内部的空间，只有碎片整理才把空间还给文件系统。
- 一万个 Pod 的 LIST 是 33 MiB，在 Kubernetes 1.37 中即使要求一致，也由 watch 缓存提供，etcd 不再承担这部分读取。
- APF 正确地分类、排队和拒绝了失控的客户端，但它管的是并发：不设置借用上限的优先级会借用别人的席位；一个返回一万个 Pod 的 LIST 和一个 GET 一样只占 1 个席位，内存代价却相差几个数量级。真正的修复在客户端：informer、分页、退避。
- 一次计划外的事故（E10-1）比计划内的实验教得更多：被测系统必须有自己的资源上限，看守者必须在另一个故障域，每一步都要检查而不是假设（APF 是否真的开启）。

第 12 章

基础设施即代码与云迁移（E11）

Argo CD 把集群里的一切拉向 Git，可集群之外还有东西没人看管：配置仓库的门禁——谁能合并、合并前必须通过什么——是第 9 章用一个命令式脚本设置的，被人改了也没有任何东西会发现；Argo CD 自己来自 bootstrap 的一次 helm 安装，而第 9 章唯一一次发布故障，正出自流程之外的一次 helm upgrade。本章用 OpenTofu 接管门禁，把状态加密、加锁保存，并检验它能否发现手工改动；让 Argo CD 管理它自己，经合并请求完成一次升级；最后只用可核实的事实，对照 AWS 与 Azure 分析迁移，重点是第 8 章留下的缺口——不可删除的异地备份。本章还记录了一次计划外的事故：主机重启之后，一个恢复脚本毁掉了 OpenBao 唯一的解封密钥。

工程问题

Argo CD 看守着集群里的一切，可集群之外的门禁没有人看守，Argo CD 自己也是手工安装的。怎样把这些也纳入“声明、比较、纠正”，又不让管理工具本身成为新的风险？

实测结论

- 接管不是重建：OpenTofu 导入 3 个已有的对象，没有改动 GitLab 的任何设置。
- 状态里有秘密，要在客户端加密；锁必须亲手测试——Garage 不实现条件写入，锁什么也没锁住，还删掉了别人的锁；换到 GitLab 的状态存储后，第二次加锁得到 409。
- 手工改动被 `plan` 以退出码 2 报告并恢复；但这次修复是“替换”，main 有约 1 秒没有任何保护。
- Argo CD 管理自身：升级经合并请求完成，合并后 315 秒恢复；流程外的 helm upgrade 被字段所有权拒绝。
- 计划外的事故：一个恢复脚本清空了唯一的解封密钥；密封的 9 小时里，一个原因开出了 54 个事故。

12.1 学习目标

学完本章，你应当能够：

- 把已经存在的基础设施导入 IaC，而不是删掉重建；
- 说明远程状态为什么必须加密和加锁，并亲手验证锁是否真的生效；
- 用 `plan` 的退出码检测漂移，并预见“修复”本身带来的风险；
- 让 GitOps 工具管理自身：先看差异再接管，经合并请求升级；
- 只用可核实的事实做云迁移分析，设计不可删除、并且打得开的备份。

实验性质

本章的软件都是 `[真实]` 的：OpenTofu 1.13.9、GitLab 提供者 19.5.0、GitLab CE 19.5.3、Garage v2.5.0、Argo CD v3.6.1 与它的 Helm chart。漂移和流程外的 helm upgrade 是 `[注入]` 的：由我以 root 身份通过 API 或命令行做出，代替有人在网页上点击。云迁移只是分析，没有创建任何云资源。事故 E11-1 是计划外的真实事件。

12.2 原理

12.2.1 IaC 与 GitOps 的分工

Argo CD 是一个持续运行的控制器：它不停地比较 Git 与集群，并把集群拉回 Git。OpenTofu 是“计划—应用”：只有在有人运行它时才比较，比较的依据是它保存的状态。适合 OpenTofu 的，是没有控制器替我们看守的东西——集群之外的 GitLab 项目设置、云账户、DNS。同一个对象只能有一个管理者：两个工具同时管理，就会互相覆盖。

示意图

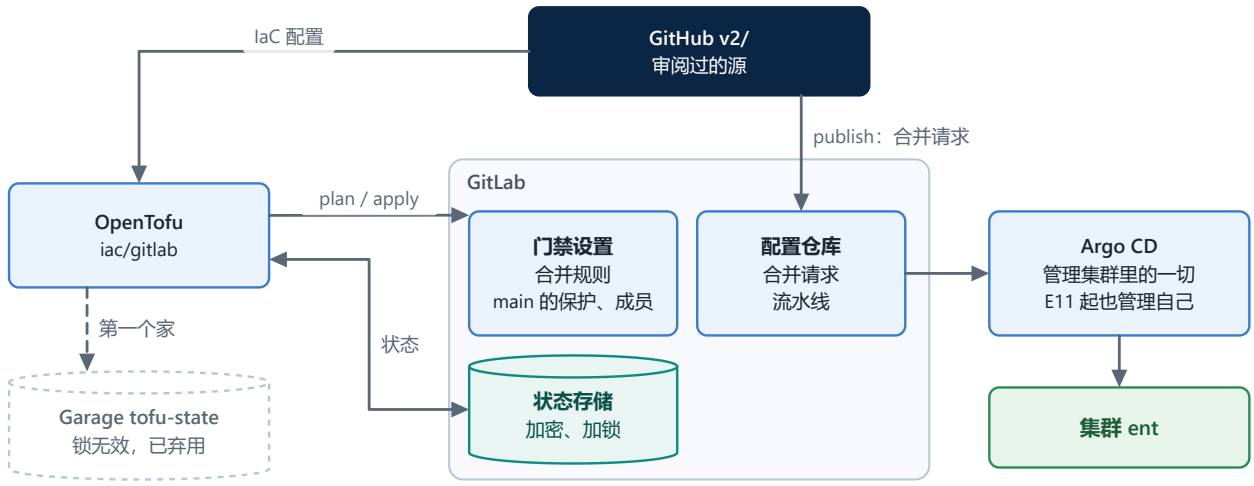


图 12-1 本章的分工。Argo CD 管理集群里的一切，从 E11 起也管理它自己；OpenTofu 管理集群之外的 GitLab 门禁，它的状态加密后保存在 GitLab 的状态存储中，由 GitLab 加锁。Garage 是状态的第一个家，因为锁不起作用而被放弃（实验 2）。

12.2.2 状态：IaC 的记忆

状态把配置中的每个资源对应到真实世界里的对象，并保存 OpenTofu 读到的每一个属性——包括秘密：本章导入的 GitLab 项目，带着它的 runner 注册令牌。所以状态要满足两个条件。

- **加密。** OpenTofu 的状态加密在客户端完成：本章用 AES-GCM，密钥由一段不在 Git 中的口令经 PBKDF2 派生，状态和计划文件都加密，`enforced = true` 让未加密的状态无法写入。存储后端看到的只是密文。
- **加锁。** 两个人同时 `apply`，后完成的会覆盖先完成的状态，丢掉对方记录的资源。锁需要后端提供一个原子操作：“只有不存在时才写入”。OpenTofu 的 S3 后端用 `use_lockfile` 写一个锁对象，并带上 `If-`

`None-Match: *`——已经存在就应得到 412；GitLab 的状态存储提供加锁接口，已被锁住时返回 409；Azure 用 Blob 租约。

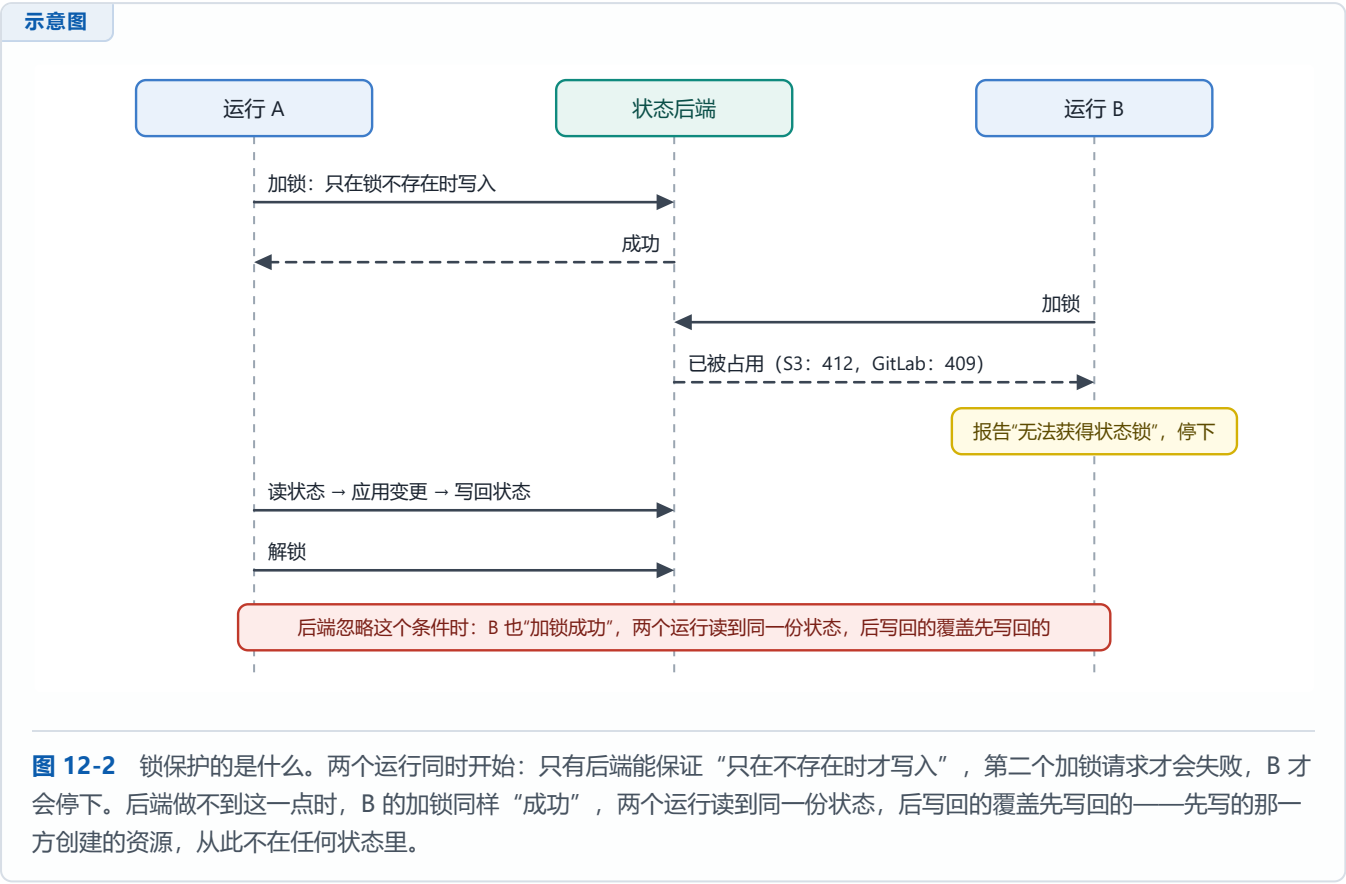


图 12-2 锁保护的是什么。两个运行同时开始：只有后端能保证“只在不存在时才写入”，第二个加锁请求才会失败，B 才会停下。后端做不到这一点时，B 的加锁同样“成功”，两个运行读到同一份状态，后写回的覆盖先写回的——先写的那一方创建的资源，从此不在任何状态里。

12.2.3 漂移

漂移是现实偏离了声明。`tofu plan -detailed-exitcode` 用退出码报告它：0 表示没有差异，1 表示出错，2 表示有差异。它只在运行时才发现漂移，所以检测要定时运行。修复就是 `apply`——但一个改动是“就地修改”还是“替换”（先删除、再创建），由提供者决定，计划中会写明 `must be replaced`。

12.2.4 让 GitOps 工具管理自己

Argo CD 不能安装它自己：第一次只能由 bootstrap 用 helm 安装（先有鸡还是先有蛋）。之后创建一个应用，指向同一个 chart 和同一份 values，从此 Argo CD 的版本和配置都在 Git 里，升级就是一次合并请求。风险在于同步时它会重启自己：同步操作的状态保存在 Application 对象里，新的控制器启动后接着完成。本章的做法是分步接管：先不自动同步，看清差异，再打开自动同步；不开启 prune，避免 Argo CD 删掉自己的一部分；用服务器端应用（server-side apply），因为 Argo CD 的 CRD 太大，放不进客户端应用所需的注解。

12.3 实验 1：接管已有的门禁（I1）

第 9 章的门禁由 `platform/gitops/setup.sh` 设置：项目只在流水线成功时合并（流水线被跳过也不行）、合并后删除源分支；main 不允许任何人推送，只有 Maintainer 能合并，不允许强制推送；alice 是

Developer。OpenTofu 用导入块（`import`）接管这些已经存在的对象，而不是删除后重建。alice 的访问令牌不纳入 IaC——令牌的值会进入状态。

假设 **I1**：导入之后的第一次计划显示配置与现实的差异；修正配置（而不是修正现实）之后，计划不再有差异；接管不会改动 GitLab。

结果：第一次计划就是 `3 to import, 0 to add, 0 to change, 0 to destroy`——配置是照着 API 读出的实际值写的，没有需要修正的地方。应用之后导入了 3 个资源，下一次计划没有差异（退出码 0）；接管前后 GitLab 门禁的快照完全相同。**I1 成立。**在真正的“棕地”接管中，第一次计划几乎总会有差异；规则只有一条：改配置去贴合现实，确认无误之后，才让配置成为现实的标准。

12.4 实验 2：状态加密，锁真的锁住了吗（I2）

假设 **I2**：状态在 Garage 中是密文；`use_lockfile` 在 Garage v2.5.0 上能排斥第二个运行。

加密成立。直接从 Garage 读回状态对象，只有 `encrypted_data`、`encryption_version`、`lineage`、`meta`、`serial` 这几个字段；`platform-gitops`、`runners_token`、`only_allow_merge` 在密文中各出现 0 次，解密后各出现 1 次。

锁不成立。Garage 不理会 `If-None-Match`：同一个键第二次带条件写入，仍然返回 200 而不是 412。于是手工放一个“别人的锁”之后，`tofu plan` 照常运行，而且结束时把那个锁删掉了：

终端记录

devops-lab · experiments/e11/state-check.sh（2026-11-10 12:07 UTC，Garage v2.5.0） 终端记录渲染

```
== 12:07:01 2. does Garage honour If-None-Match on PUT? (what use_lockfile relies on)
  PUT a new object with If-None-Match: *     -> HTTP 200
  PUT the same key again, If-None-Match: *   -> HTTP 200
== 12:07:01 3. another run holds the lock: a lock object placed by hand, then tofu plan
  PUT gitlab/governance.tfstate.tflock (If-None-Match: *) -> HTTP 200
  tofu plan exit code 0:
    No changes. Your infrastructure matches the configuration.
  lock object after the plan: HTTP 404
  DELETE the hand-made lock -> HTTP 204
  tofu plan without the foreign lock: exit code 0, No changes. Your infrastructure matches the configuration.
```

图 12-3 对 Garage 的锁检查：第二次带条件写入仍然成功（应为 412）；放好“别人的锁”之后，tofu plan 照常完成（退出码 0），结束时删除了锁对象（之后 HTTP 404）——第三个运行也就不会再看到任何锁。

这不是 OpenTofu 的错误：它依赖 S3 的一个语义，而一个“兼容 S3”的存储没有实现它。在 AWS 的 S3 上，条件写入从 2024 年 8 月起可用，同样的锁会成立（见本章的云迁移分析）。教训是：**锁必须被测试，而不是被相信**——一个不起作用的锁，在出事之前与正常的锁毫无区别。

于是状态迁到了 GitLab 的状态存储（项目 `root/platform-iac`，`http` 后端，GitLab 的加锁接口），用 `tofu init -migrate-state`。同样的检查，这次锁住了：

终端记录

devops-lab · experiments/e11/state-lock-gitlab.sh（2026-11-10 13:49 UTC，GitLab CE 19.5.3 的状态存储） 终端记录渲染

```
== 13:49:42 1. another run holds the lock
  POST http://localhost:8929/api/v4/projects/5/terraform/state/gitlab/lock -> HTTP 200
  the same lock once more -> HTTP 409 {"Operation":"OperationTypeApply","Info":"","Version":"1.13.9","Path":"","ID":"e11-tes
t-0001","Who":"root","Created":"2026-11-10T13:49:42.803Z"}
  tofu plan exit code 1:
    Error: Error acquiring the state lock
    Error message: HTTP remote state already locked: ID=e11-test-0001
    Lock Info:
      ID:        e11-test-0001
      Operation: OperationTypePlan
      Who:       root
  release: DELETE http://localhost:8929/api/v4/projects/5/terraform/state/gitlab/lock -> HTTP 200
  tofu plan without the foreign lock: exit code 0, No changes. Your infrastructure matches the configuration.
== 13:49:50 2. two plans started at the same moment (each holds the lock for its refresh and plan)
  plan 1: exit code 0, No changes. Your infrastructure matches the configuration.
  plan 2: exit code 1, Error: Error acquiring the state lock
```

图 12-4 对 GitLab 状态存储的锁检查：第二次加锁请求得到 409；tofu plan 报告“Error acquiring the state lock”并列出锁的持有者；释放之后计划正常；两个同时开始的计划，一个成功，一个被锁挡住。

迁移恰好被一次中断打断：迁移之后的那次计划运行时，Windows 主机失去响应被注销，虚拟机随之停止。重启后先做一致性检查，再继续：目标中的状态完整（serial 1，3 个资源和 1 个数据源），没有残留的锁，工作目录指向 `http` 后端，重新运行被打断的计划，没有差异。中断之后要回答的四个问题是：状态在哪里、是否完整、是否被锁住、工作目录指向哪个后端——在回答之前，不要再导入，也不要再应用。

这个选择有代价：GitLab 门禁的状态保存在 GitLab 里。GitLab 坏了，修复它的 IaC 也拿不到自己的状态。Garage 中的副本留作第一个家，GitLab 也保存状态的历史版本。

12.5 实验 3：手工改动能被发现吗（I3）

假设 **I3**：三处手工改动——main 向 Developer 开放推送、允许强制推送、alice 升为 Maintainer——让 `plan -detailed-exitcode` 以 2 退出，并显示 2 个资源需要修改；`apply` 恢复声明的值，之后的计划以 0 退出。

检测与恢复成立，但预测错了一处。提供者认为 `push_access_level` 的改变需要替换整个分支保护，于是计划是 `1 to add, 1 to change, 1 to destroy`，而不是两个就地修改；应用时先删除了保护（1 秒），再重新创建——**在这一秒里，main 没有任何保护。**修复打开了一个它本来要关上的窗口。

这类问题只能靠读计划发现：一个安全控制旁边出现 `must be replaced`，就意味着修复期间控制会短暂消失。可以选择的做法有：在低风险的时间应用、改用就地修改的 API 修复、或者接受窗口但让它尽可能短（练习 3）。另一个要点是检测的时机：本章在手工改动后立即运行计划；实际环境需要定时运行，并把退出码 2 变成工单或告警（练习 2）。

终端记录

devops-lab · experiments/e11/drift.sh（2026-11-10 13:51 UTC，[注入] 三处手工改动） 终端记录渲染

```
== 13:51:26 0. before: plan exit code 0
== 13:51:32 1. [注入] the drift
  13:51:32 main re-protected with push: Developers, force push: allowed
  13:51:32 alice: Maintainer
  GitLab now: {"push":[30],"force":true,"alice":40}
== 13:51:33 2. detection: tofu plan -detailed-exitcode
  exit code 2 (0 = no changes, 1 = error, 2 = changes)
    # gitlab_branch_protection.main must be replaced
        ~ allow_force_push            = true -> false
        + allowed_to_merge            = (known after apply)
        + allowed_to_push             = (known after apply)
        + allowed_to_unprotect        = (known after apply)
        ~ branch_protection_id        = 5 -> (known after apply)
        ~ id                          = "3:main" -> (known after apply)
        ~ push_access_level           = "developer" -> "no one" # forces replacement
    # gitlab_project_membership.alice will be updated in-place
        ~ access_level    = "maintainer" -> "developer"
  Plan: 1 to add, 1 to change, 1 to destroy.
== 13:51:37 3. tofu apply: back to the declared values
  exit code 0
  gitlab_branch_protection.main: Destroying... [id=3:main]
  gitlab_project_membership.alice: Modifying... [id=3:5]
  gitlab_branch_protection.main: Destruction complete after 1s
  gitlab_branch_protection.main: Creating...
  gitlab_project_membership.alice: Modifications complete after 1s [id=3:5]
  gitlab_branch_protection.main: Creation complete after 0s [id=3:main]
  Apply complete! Resources: 1 added, 1 changed, 1 destroyed.
== 13:51:44 4. after: plan exit code 0
  GitLab is back to the state after the adoption (i1-gitlab-after.json = i3-gitlab-restored.json)
```

图 12-5 漂移实验：注入三处改动之后，计划以 2 退出；分支保护“必须被替换”（push_access_level 的改变强制替换），成员关系就地修改；应用时先删除、再创建分支保护；之后的计划以 0 退出，GitLab 回到接管后的状态。

12.6 实验 4：Argo CD 管理自己（I4）

假设 **I4**：应用 `argocd`（chart argo-cd 10.14.0，values 来自 Git）接管 bootstrap 安装的版本，不重启 Argo CD；升级到 10.17.1 经过合并请求、流水线通过，10 分钟内所有应用回到 Synced 和 Healthy；之后一次流程外的 helm upgrade 被 selfHeal 撤销。

接管分两步。第一个合并请求（!43）只创建应用，不自动同步。Argo CD 报告 59 个资源中有 56 个 OutOfSync——看起来差异很大。对同一份渲染结果做服务器端试运行（`kubectl diff --server-side`），只有 1 个对象不同：helm 安装后已删除的钩子任务 `argocd-redis-secret-init`；其余差异只是 Argo CD 自己的跟踪注解。确认之后，第二个合并请求（!44）打开自动同步：同步用了 22 秒，60 个资源全部一致，Argo CD 没有一个 Pod 重启。

流程外的 helm upgrade 被拒绝了两次。按第 9 章那次故障的方式，用 helm 把 argocd-server 改成 2 个副本——这一次 helm 失败了：helm 4 也用服务器端应用，而这些字段已经属于 Argo CD 的字段管理者 `argocd-controller`，API server 报告冲突（例如 `.spec.replicas`），什么也没有改变。这是预测之外的一层保护。用 `--force-conflicts` 强行覆盖之后，helm 成功了，Argo CD 的 selfHeal 在几秒内把副本数改回 1，`.spec.replicas` 又归 `argocd-controller` 所有。只是 helm 的历史从此记着“第 4 版，chart

10.14.0，2 个副本，已部署”，而升级之后的集群是 chart 10.17.1、1 个副本——一份不再描述集群的记录。

升级经过合并请求。10.14.0 到 10.17.1 两个版本的 Argo CD 都是 v3.6.1。先用同一份 values 渲染两个版本比较：镜像不变，7 个对象改变——6 个工作负载的 Pod 加上了 `seccompProfile: RuntimeDefault`，以及一个标签。所以 Argo CD 的每个组件都会滚动，包括正在执行同步的应用控制器。门禁的流水线这次多了一步：用我们的 values 渲染新版本的 chart，chart 不接受这份 values 就在合并之前失败。

实测数据

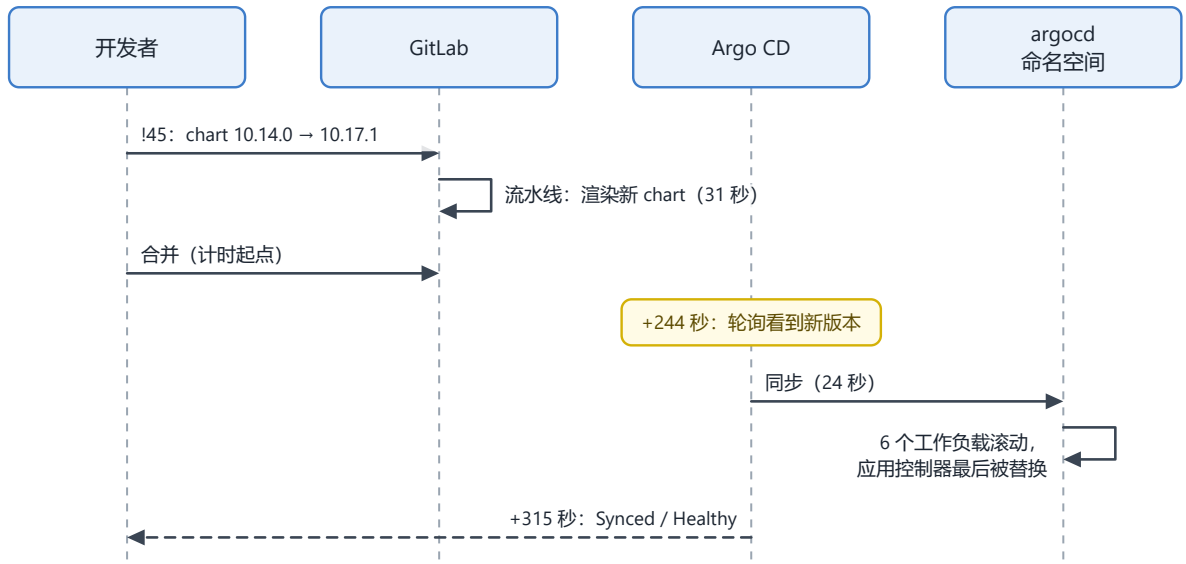


图 12-6 Argo CD 经合并请求升级自己（2026-11-10，合并请求 !45；门禁的流水线 31 秒）。时间从合并起算：Argo CD 轮询 Git，244 秒后才看到新版本；同步本身 24 秒；它的组件依次滚动，应用控制器在自己的同步完成之后才被替换；315 秒时回到 Synced 和 Healthy。

I4 成立，并多了一个发现。从合并到 Synced 和 Healthy 用了 5 分 15 秒；其他应用的状态没有变化——35 个 Healthy，以及 4 个 Degraded，它们来自下一节的事故，与升级无关。

12.7 事故 E11-1：恢复脚本毁掉了唯一的解封密钥

本章进行到一半，Windows 主机失去响应，被注销；虚拟机随之停止。重启后的例行步骤之一是运行 `openbao-init.sh` 为 OpenBao 解封。这一次它毁掉了 OpenBao 唯一的解封密钥[2]。

时间（UTC）	经过
13:43	虚拟机启动，带重启策略的容器（包括 4 个 kind 节点）随之启动，节点很快 Ready
13:44:03	运行 `openbao-init.sh`。Pod 的状态还停留在重启之前的 Running，脚本的等待随即通过，可容器还不存在；状态检查吞掉了错误，把“没有回答”当成“没有初始化”，执行了 `bao operator init … > init.json`——shell 在命令失败之前就清空了文件。这个文件是解封密钥和根令牌唯一的副本
13:44:24	容器启动；再运行一次，用空文件解封，得到 400

时间（UTC）	经过
13:45	发现文件为 0 字节、修改于 13:44:03；机器上没有其他副本；上一次恢复留下的临时密钥打不开恢复后的密钥环（试过，被拒绝）
13:47:39	告警 OpenBaoSealed 经告警桥在 GitLab 中开了事故 #14——第 10 章的告警链路如期工作

影响：OpenBao 的数据再也无法读取——配置可由脚本重建，数据只是第 6 章写入的两个随机测试值；但**第 8 章每小时一次的 OpenBao 快照全部失去意义**，它们用同一个密钥环加密。没有工作负载受影响：唯一的使用者保留着最后一次同步的 Secret。第 6 章和第 8 章都写过“解封密钥只有一份……快照无法打开”，这一次风险成了现实。

为什么：状态检查分不清“没有回答”和“没有初始化”；等待条件被一个过期的 Pod 状态满足；输出重定向在命令运行之前就截断了目标文件；而密钥只有一份。

改了什么：两个脚本都改为等 OpenBao 自己回答；状态只在有回答时才算数；存在密钥文件时拒绝初始化；新密钥先写临时文件再移动到位，设为只读并保留第二份——只读之后，误写的 `>` 会失败，而不是清空文件。

恢复需要所有者的决定。清除永久密封的存储是不可逆的删除，本次会话的权限检查两次拦下了它，连“保留旧数据”的做法也一样。所有者批准之后（约 22:37），旧卷的回收策略改为 Retain——9.5 MiB 的密文留在节点上，万一旧密钥的副本出现还能重新挂载——OpenBao 换上新卷，由修复后的脚本初始化、解封、配置，重新写入两个测试值；3 秒后 External Secrets 恢复同步，用新密钥做的快照上传到 Garage，22:39:39 所有 39 个应用回到 Synced 和 Healthy，22:40:15 事故 #14 关闭。新密钥的副本由所有者自己保管。

还有一个教训来自告警。OpenBao 密封的 9 个小时里，每小时一次的快照任务都失败一次，告警桥为这一个原因在 GitLab 中开了 54 个事故：16 个应用 Degraded、10 个任务失败、27 个容器等待，每小时关闭一批、再打开一批。

实测数据

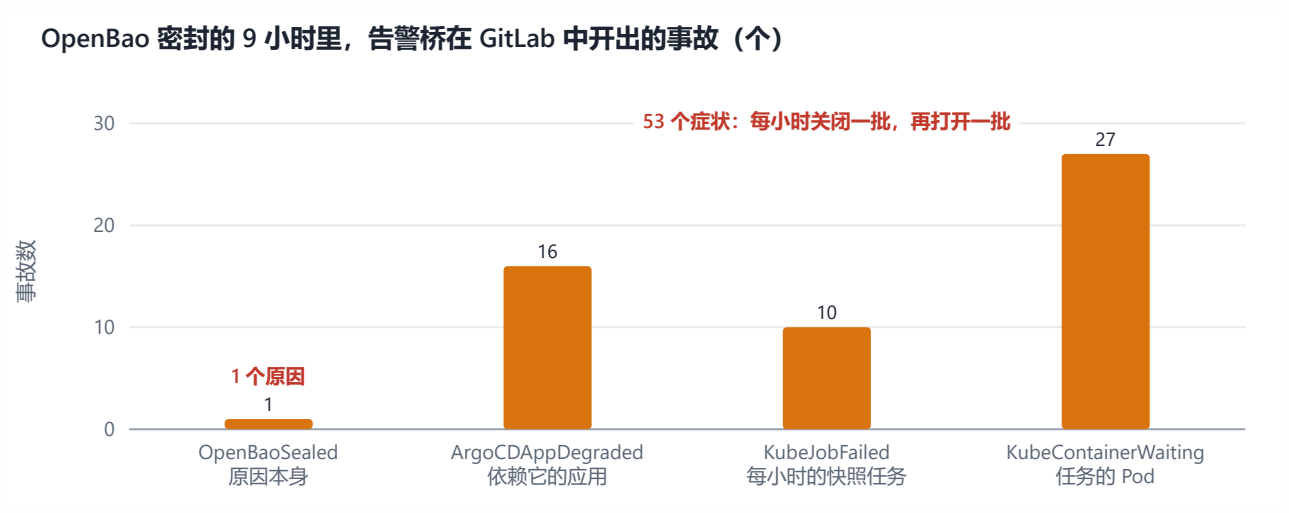


图 12-7 一个原因、54 个事故。OpenBaoSealed 是原因，只响了一次；其余 53 个都是它的症状：依赖 OpenBao 的应用变为 Degraded，每小时的快照任务失败，任务的 Pod 起不来——每过一小时，旧的一批关闭，新的一批打开。值班的人看到的是一墙告警，真正需要处理的只有第一条。

第 10 章的抑制规则只覆盖同一告警的不同级别、SLO 的呼叫与工单，以及 PostgreSQL 主库与副本这一对因果，没有覆盖“一个依赖坏了，它的症状全都在叫”。改进是让 `OpenBaoSealed` 触发时抑制它的下游症状，并通过故意密封 OpenBao 来测试——这一项留作后续行动（第 13 章的实操题完成了规则与合成告警的检验）。

这次事故与本章的主题相连：备份能否恢复，取决于两样东西——备份本身不能被删除，打开它的密钥也不能丢失。下一节的云迁移分析把这两点放在首位。

12.8 云迁移分析：只陈述可核实的事实

分析全文在 `docs/cloud-migration.md`。每一条关于云服务的陈述都附官方来源，核实日期为 2026-11-10；没有核实的写明“未核实”；价格不在分析之内。与本书实验直接相关的几条：

实验环境	AWS	Azure
Istio ambient + Gateway API	自行安装；AWS App Mesh 已于 2026-09-30 停止支持	AKS 的 Istio 插件尚不支持 ambient 模式，也不支持用 Gateway API 管理网格流量——保留 ambient 就要自行安装开源 Istio
Argo CD	自管理，或 EKS Capabilities 的托管 Argo CD（2025-11-30 正式可用）	自管理（托管选项未核实）
Garage 与 Velero	S3 Object Lock；AWS Backup 自 2025-11-10 起支持 EKS，2026-03-12 起可用逻辑隔离的保管库	Blob 不可变存储；Azure Backup for AKS 的保管库层与不可变保管库
OpenTofu 状态	S3 后端 + `use_lockfile`：S3 支持条件写入，实验 2 中失败的锁在这里成立	azurerm 后端（Blob 租约锁，未核实）

不可删除的异地备份——第 8 章的缺口——有两种成熟的做法。S3 Object Lock 的合规模式下，受保护的对象版本不能被任何用户覆盖或删除，包括 AWS 账户的 root 用户；保留期不能缩短，到期前删除它的唯一方法是删除整个账户。Azure 的不可变存储在锁定基于时间的保留策略之后，策略不能删除，保留期只能延长，数据连账户管理员也不能修改或删除。两者都需要先开启版本控制或选择 WORM 的范围；两者的不可逆也是代价：保留期设长了，误存的数据在到期前删不掉（练习 6）。再加上事故 E11-1 的教训：密钥的保管要和备份一起设计、一起演练。

12.9 局限

- IaC 只覆盖一个 GitLab 项目的门禁，而不是一个组织的几百个项目；OpenTofu 使用 root 的令牌——企业中应是权限最小的专用服务账号。
- 漂移检测是手动运行的，没有定时任务；修复窗口的长短只测了一次（约 1 秒）。
- 状态的家：Garage 的锁无效，GitLab 的锁有效，但状态与被管理的对象在同一个系统里。迁移前后外层 `lineage` 字段不同，原因没有深究（解密后的状态完整）。
- Argo CD 的升级是一次 chart 升级，应用版本没有变化（两个版本都是 v3.6.1）；没有测试跨大版本、CRD 结构变化的升级。
- 云迁移分析没有核实所有组件（表中写明），没有价格和性能数据。
- 事故 E11-1 已经恢复，但留下三项后续：旧的密封数据仍在一个保留的卷上（是否删除由所有者决定）；新密钥只有虚拟机中的两份和所有者自己保管的副本，没有做成多人分片或托管的 KMS；抑制下游症状的告警规则还没有实现和测试。

12.9.1 从实验环境到生产环境

本章的做法	生产环境通常的做法	差别意味着什么
OpenTofu 用 root 的令牌管理一个项目	权限最小的专用服务账号，按环境分开状态	状态和执行身份的泄露面，要按“它能改多少东西”来划分
`plan` 由人手动运行	流水线定时运行 `plan`，在合并请求里展示计划，审批之后再 `apply`	漂移只在运行时才被发现；不定时运行，就等于没有检测
状态存放在被管理的 GitLab 自己里面	独立的、带版本和锁的状态存储	GitLab 出故障时，修复它的 IaC 拿不到自己的状态
解封密钥是一个文件，恢复脚本直接读写它	KMS 自动解封或多人分片保管；恢复流程经过评审与演练	事故 E11-1：一个脚本缺陷就毁掉了唯一的一份密钥

12.10 常见问题

问：既然有了 Argo CD，为什么还要 OpenTofu？ Argo CD 管的是集群里的对象。GitLab 的项目设置、云账户、DNS 这些没有控制器的东西，需要一个“计划—应用”的工具记录和比较。两者分工明确，同一个对象只能属于其中一个。

问：OpenTofu 的状态为什么要加密？后端不是已经有访问控制了吗？状态里有秘密（本章是 runner 注册令牌，在别处是数据库密码）。后端的访问控制保护的是“谁能读”，加密保护的是“读到了也没用”——备份、日志、误配置的权限都可能让状态落到别人手里。

问：S3 兼容的存储都能做 OpenTofu 的锁吗？不一定。`use_lockfile` 依赖条件写入：已存在的键必须返回 412。本章的 Garage v2.5.0 不实现它，锁形同虚设，而且没有任何报错。换用一个存储之前，先用本章的方法测一次。

问：Argo CD 管理自己，出问题时怎么办？接管时不开 prune，Argo CD 不会删掉自己的组件；升级前在门禁中渲染新版本；最后的退路是 bootstrap：它从 Git 中读取同一个 chart 版本，用 helm 重新安装。

问：恢复脚本为什么会毁掉密钥？这不是很基础的错误吗？是基础的错误，而且组合起来才致命：一个吞掉错误的状态检查、一个过期的 Pod 状态、一次截断文件的重定向、一份没有副本的密钥。任何一处设计成防御式的，事故都不会发生。恢复脚本往往在最混乱的时候运行，最需要防御式编写。

12.11 练习

练习

1. **（分析）** 实验 2 中，Garage 上的 `tofu plan` 不但没被锁挡住，还删除了别人的锁。设想两个 `apply` 同时运行，写出一条导致“更新丢失”的时间线；在支持条件写入的 S3 上，同样的时间线会在哪一步停下？
2. **（设计）** 为本章的 GitLab 门禁设计定时漂移检测：在哪里运行、用什么身份和凭据、发现漂移后如何通知、是否自动修复。结合实验 3，说明自动修复的风险。
3. **（分析）** 实验 3 中分支保护被“替换”而不是“修改”，main 有约 1 秒没有保护。给出两种消除或缩短这个窗口的做法，并说明各自的代价。
4. **（分析）** Argo CD 接管自身时，59 个资源中 56 个显示 OutOfSync，而服务器端试运行只有 1 个对象不同。解释两个数字为什么不同，以及为什么接管之前要做试运行。
5. **（设计）** 如果一次 Argo CD 升级改变了某个 CRD 的结构，本章的流程（合并请求 → 门禁渲染 → 自身同步）在哪一步能发现问题、哪一步发现不了？给流程补充一项检查。
6. **（设计）** 用 S3 Object Lock 的合规模式保存第 8 章的备份：保留期设多长？分别说明设得过长和过短的代价；结合事故 E11-1，说明密钥应当如何保管。

参考答案见附录。

12.12 本章小结

- IaC 管理没有控制器的东西；接管已有的对象用导入，改配置去贴合现实，而不是反过来。
- 状态保存着秘密，要加密；状态要加锁，而锁必须被测试：Garage 不实现条件写入，`use_lockfile` 在那里什么也锁不住，还会删掉别人的锁；GitLab 的状态存储返回 409，锁住了。
- `plan -detailed-exitcode` 能发现手工改动；读计划时要看清“替换”——修复本身可能短暂移除一个安全控制。

- Argo CD 可以管理自己：先看差异再接管，经合并请求升级，流程外的 helm upgrade 被字段所有权拒绝，被强行覆盖时由 selfHeal 撤销。
- 云迁移分析只写可核实的事实：不可删除的备份在两家云上都有成熟的做法；而事故 E11-1 说明，打开备份的密钥必须和备份一起设计。

第 13 章

能力考核（E12）

前十二章回答的是“这些技术在一个受控环境中是否被正确地设计、实施、测量和排查”。本章把它们变成可以重复使用的考核：五道架构设计题，三道故障排查题——脚本注入故障、只告诉考生一句症状、脚本自动判分——以及一道实操题：关闭事故 E11-1 留下的一个行动项。所有题目都在本书的实验环境中真实运行过一遍：判分脚本在故障存在时判 FAIL，修复之后判 PASS，环境在每道题前后都是 39 个应用全部 Synced 和 Healthy。需要说明的是，这一遍的考生是本书的作者：它证明的是考核本身能用，不证明任何人的能力。

工程问题

读完十二章之后，怎样检验“会了”？考核能不能像实验一样可以重复：注入真实的故障，只给一句症状，由脚本判分？

实测结论

- 三道故障排查题和一道实操题都在真实环境中运行过：故障存在时判 FAIL，修复之后判 PASS，环境每次都回到 39 个应用全部健康。
- 判分要检查原因，不只是症状：OpenBao 已经密封时，“密钥同步”一项仍然通过；故障刚注入的几秒里，单次下单探测仍会成功。
- 实操题关闭了事故 E11-1 的一个行动项：抑制规则经合并请求部署，用被静默的合成告警检验，判分从 3/8 变为 8/8。
- 这一遍的考生是本书的作者：它证明考核可以运行、判分正确，不证明任何人的能力。

13.1 学习目标

学完本章，你应当能够：

- 把“会不会”变成可以重复的考核：注入真实的故障，只给一句症状，由脚本判分；
- 设计判结果与原因、不判命令的判分规则，并说明只判症状会漏掉什么；
- 为每一次注入准备对应的恢复动作，让环境在每道题前后回到同一个状态；
- 说明“出题人自己作答”的考核能证明什么、不能证明什么。

实验性质

本章的故障都是 `[注入]` 的，其余都是 `[真实]` 的：真实的集群、Argo CD、OpenBao、Istio、Alertmanager 和 GitLab。

13.2 考核的设计原则

- **判结果，不判命令。**判分脚本只看环境的最终状态和行为：凭据是否正确（比较哈希）、应用是否同步、下单是否成功、告警是否被抑制。考生用什么命令、走什么路径都可以，但修复必须真实有效。
- **判原因，不只判症状。**这是本章运行时得到的教训：T2 中，OpenBao 已经密封，“密钥同步”这一项却仍然通过——External Secrets 的状态要过一会儿才更新；T3 中，单次下单探测在故障刚注入的几秒里仍会成功。所以每道题都同时检查原因（OpenBao 是否解封、是否存在 Git 之外的对象）和症状。
- **故障来自本书的真实事件。**T1 是 GitOps 的数据通路断了；T2 是事故 E11-1 本身；T3 是第 12 章“流程外的改动”的另一种形态；实操题是事故 E11-1 的行动项 5。
- **安全。**每个注入只触及一个对象，都有对应的恢复动作；凭据只经文件或标准输入传递，判分只比较哈希；实操题的行为检验使用被静默的合成告警，不会通知任何人。

13.3 架构设计题

题目	场景	主要依据
D1 跨区域灾备	下单 RPO ≤ 5 分钟、RTO ≤ 30 分钟	第 8 章的重建与备份、第 12 章的不可删除备份、事故 E11-1 的密钥保管
D2 十倍流量	营销活动，2 小时，10 倍流量	第 4 章的容量与第一个瓶颈、第 5 章 HPA 的反应时间、第 7 章节点的余量
D3 保护控制面	CI 系统每分钟列出 3 万个 Pod	第 11 章的 LIST 代价、APF 与借用、客户端的修复
D4 发布与治理	Argo CD、OpenTofu、门禁都进入合并请求，并保留应急路径	第 9 章的门禁与冻结、第 12 章的漂移与字段所有权
D5 告警治理	一个原因开了 54 个事故	第 10 章的路由与抑制、事故 E11-1、实操题 P1

每道题 20 分，评分要点写在 `assessment/design.md` 中，并且都要求引用本书的测量数据：例如 D2 的满分答案必须从第 4 章测得的饱和点出发，而不是从一个假设的数字出发。

13.4 故障排查题

题目	告诉考生的症状	实际注入的故障（考生不知道）	判分
T1	十分钟前合并的改动没有到达集群	Argo CD 仓库凭据的密码被换成随机字符串	凭据正确（哈希）、没有 ComparisonError、全部应用同步
T2	一次重启之后，平台的密钥不再刷新	重启 openbao-0，它回来时是密封的	OpenBao 已解封、密钥文件的哈希不变且只读、密钥同步恢复
T3	下单失败，浏览正常，Git 和 Argo CD 都显示一切正常	在 Git 之外手工创建的 Istio 拒绝策略，作用于 payments	没有 Git 之外的策略对象、连续 5 次下单成功、浏览正常

三道题的运行结果：

题目	注入后判分	修复	修复后判分
T1	0/3：37 个应用报告 `authentication required: HTTP Basic: Access denied`	先确认令牌本身在 GitLab 上有效（HTTP 200），再把集群中的凭据从实验室的秘密文件经标准输入恢复	3/3，25 秒后
T2	2/3：OpenBao 已密封，但密钥同步一项仍通过（状态滞后）	用修复后的 `openbao-init.sh` 解封，5 秒	3/3，密钥文件不变
T3	2/4：存在 Git 之外的策略，连续下单失败；Argo CD 始终显示 39 个应用正常	删除那条 Argo CD 不跟踪的拒绝策略	4/4，7 秒后

实测数据

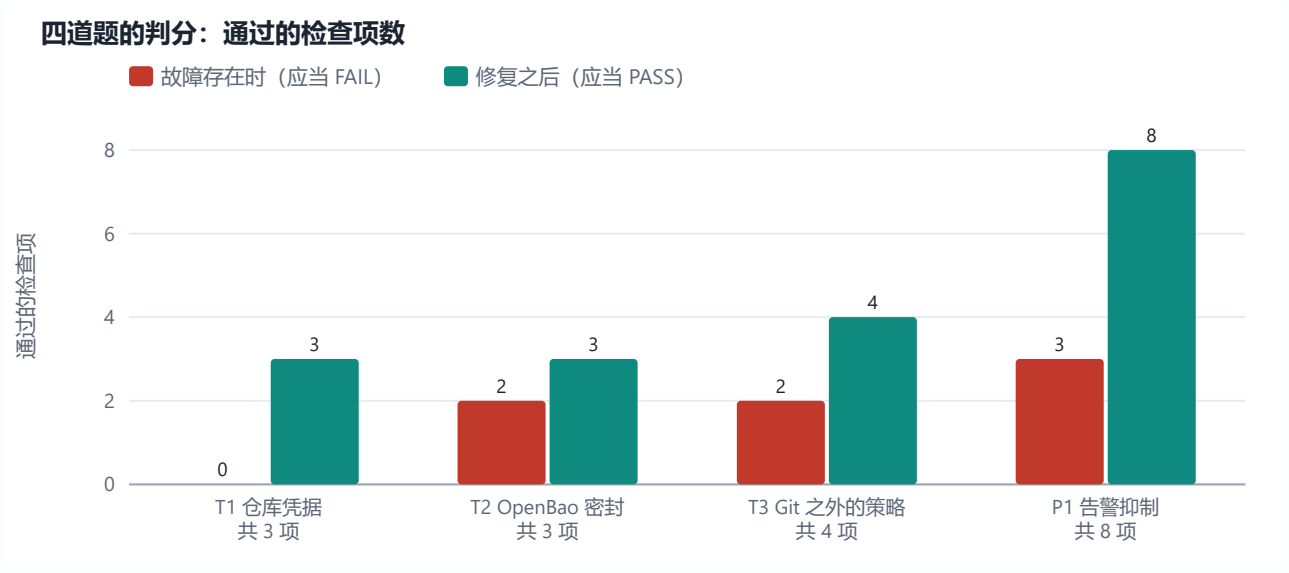


图 13-1 四道题在两种状态下的判分。红色是故障存在时通过的检查项数，绿色是修复之后：每一道题都是“故障存在时不能全过，修复之后全过”，判分脚本才算可信。T2 和 T3 在故障存在时仍有两项通过——那两项检查的是症状，而症状有时会滞后或被掩盖。

T3 最能说明“从症状到原因”的路径。考生得到的症状里，Argo CD 一切正常——第 12 章说过，Argo CD 只管它跟踪的对象。线索来自三处：orders 的访问日志全是 503；ztunnel 的日志里有一条 `handling RBAC update e12-quarantine`；而这条策略的字段管理者是 `kubectl-client-side-apply`，不是 Argo CD。在 storefront 中列出所有没有 Argo CD 跟踪注解的对象，原因就出现了：

终端记录

devops-lab · assessment/t3-out-of-band.sh（2026-11-10 22:49 UTC，考生：本书作者） 终端记录渲染

```
22:49:09 T3 injected - candidate: "checkout fails, browsing works, Git and Argo CD say everything is in order"
  22:49:28 checkout answers 200 200 200; browsing answers 200; applications  39 Synced/Healthy;
== 22:49:29 grade right after the injection (NOWAIT, expected FAIL)
== 22:49:29 T3 grade
  FAIL  no object in storefront outside Git (untracked AuthorizationPolicy or NetworkPolicy)
  FAIL  5 checkouts in a row answered 2xx or 402
  PASS  browsing answers 200
  PASS  all applications Synced and Healthy
== 22:49:30 result: 2 passed, 2 failed -> FAIL
  exit code 1
== 22:49:30 the candidate: evidence
  orders: 22:49:28.812 POST /orders 503  (JSON log, shortened)
  orders: 22:49:29.013 POST /orders 503  (JSON log, shortened)
  orders: 22:49:30.361 POST /orders 503  (JSON log, shortened)
  ztunnel: 2026-11-10T22:49:09.180458Z  info    xds:xds{id=24}  handling RBAC update e12-quarantine
  objects in storefront that Argo CD does not track: AuthorizationPolicy [e12-quarantine ] NetworkPolicy []
  e12-quarantine: action DENY, selector {"app.kubernetes.io/name":"payments"}, created 2026-11-10T22:49:08Z, managers istio-
ambient-ztunnel-policy kubectl-client-side-apply
== 22:49:31 the candidate: repair - remove the object that is not in Git
  authorizationpolicy.security.istio.io "e12-quarantine" deleted from storefront namespace
== 22:49:31 grade after the repair
== 22:49:31 T3 grade
  PASS  no object in storefront outside Git (untracked AuthorizationPolicy or NetworkPolicy)
  PASS  5 checkouts in a row answered 2xx or 402
  PASS  browsing answers 200
  PASS  all applications Synced and Healthy
== 22:49:38 result: 4 passed, 0 failed -> PASS
  exit code 0
```

图 13-2 T3 的一次完整运行：注入后连续下单失败、浏览正常、Argo CD 显示 39 个应用正常；判分 2/4；证据来自 orders 的日志（503）、ztunnel 的策略更新日志、未被跟踪的对象列表和字段管理者；删除这条 Git 之外的策略后判分 4/4。orders 的 JSON 日志行在图中被缩短，并已标注。

13.5 实操题：关闭事故 E11-1 的行动项 5

任务：OpenBao 密封时，让 Alertmanager 抑制它的症状——命名空间 `secrets` 中的任务失败和容器等待告警，以及依赖它的 4 个应用的 Degraded 告警——而且不影响其他告警；改动必须经合并请求进入配置仓库，由 Argo CD 部署。

这道题有一个陷阱。已有的 4 条抑制规则都用 `equal` 要求源告警和目标告警的某个标签相等（例如 `namespace`），照抄它们的写法看起来很自然。但 `OpenBaoSealed` 是用 `max(...)` 计算的，聚合掉了所有标签，它没有 `namespace`——带 `equal: [namespace]` 的规则永远不会匹配，而且不会有任何报错。正确的写法是不用 `equal`，直接在目标的匹配条件中写出命名空间和应用名。

判分因此必须检验行为，而不是检验文本。脚本先在 Alertmanager 中建一个静默（被静默的告警不会发出通知，但仍然可以作为抑制的源），再用 `amtool` 发送一组合成告警：一个 `OpenBaoSealed`、三个应当被抑制的症状、两个不应被抑制的对照（storefront 的任务失败、storefront-gateway 应用 Degraded），标签与真实告警相同；然后检查每个告警的 `inhibitedBy`；最后把合成告警标为已解决，并让静默过期。

	规则提交前	规则经合并请求 !46 部署后
main 中有以 `OpenBaoSealed` 为源的规则	FAIL	PASS

	规则提交前	规则经合并请求 !46 部署后
main 受保护（只能经合并请求进入）	PASS	PASS
运行中的 Alertmanager 已加载规则	FAIL	PASS（合并后 38 秒）
3 个症状被抑制	FAIL ×3	PASS ×3
2 个对照不被抑制	PASS ×2	PASS ×2
判分	3/8	8/8

两次判分期间，告警桥收到的通知都是 0 条：考核没有在 GitLab 中开任何事故。还没有做的是用一次真实的密封来检验这条规则——那需要让 OpenBao 密封几个小时，等待每小时的快照任务失败。

13.6 局限

- **这一遍的考生是作者。**题目由作者设计，作者也知道注入了什么；运行证明的是“考核脚本可运行、判分正确”，不能证明任何人的排查能力。真正的考核需要一位不了解注入内容的考生，并按 `assessment/README.md` 的规则进行。
- 每道题只运行了一遍；判分在“故障存在”和“修复之后”两种状态下得到验证，没有验证所有错误的修复方式（例如一个只修复了症状的考生）。
- 架构设计题没有由第三方评分；评分要点由作者给出。
- 实操题的规则只用合成告警检验过，没有经历一次真实的 OpenBao 密封。

13.7 第二册的边界

第二轮验证的是：在一台笔记本上，用真实的开源软件搭建一个接近企业生产的平台，并对架构、容量、弹性、安全、故障、数据可靠性、发布、可观测性、控制面规模和基础设施即代码做可重复、有证据的实验。它没有验证的是：真实的用户与流量、跨团队的协作、合规审计、长期运行带来的问题，以及比这台电脑大几个数量级的规模。书中所有的数字都来自这台电脑上的一次或几次运行；两次计划外的事故（E10-1、E11-1）和第 10 章中主机的一次睡眠同样是真实的，它们教给我们的，往往比计划中的实验更多。

全部 13 个阶段的完成情况、关键结果、证据位置、局限与剩余风险，汇总在独立的验收报告中（`docs/acceptance-report.md`，v2.0.0 的技术验收）；出版版在此之上所做的图表核对、逐页审查与复现性评估，另见 `docs/publication/ACCEPTANCE.md`。

附录 A　练习参考答案

第 1 章

1. `system-reserved cpu = 16 − 2 = 14`，allocatable CPU = 16 − 14 − 0.25 = 1.75。驱逐从用量达到 4096 − 150 = 3946 MiB 开始，所以 `eviction-hard memory.available < 15642 − 3946 = 11696Mi`；allocatable 内存 = 15642 − 256 − 11696 = 3690 MiB。节点容器相应地改为 `docker update --cpuset-cpus <一个物理核心的两个线程> --memory 4096m --memory-swap 4096m`。
2. 实际用量 2.8 GiB（2867 MiB）尚未达到驱逐起点 2922 MiB（信号为 15642 − 2867 = 12775 MiB，高于阈值 12720 MiB），kubelet 什么也不做；requests 之和 2.5 GiB 也没有超过 allocatable 2666 MiB，调度器仍可能放入新 Pod。再分配 400 MiB 后用量将超过 3072 MiB 的内核上限。kubelet 默认每 10 秒才检查一次驱逐信号，3 秒内来不及反应，最可能的结果是内核 OOM：在节点 cgroup 内按 `oom_score` 挑选进程，BestEffort（1000）最先，其次是超出 requests 最多的 Burstable，容器以 137 退出（`OOMKilled`）并原地重启。只有当用量在 2922–3072 MiB 之间停留到 kubelet 的下一次检查时，才会发生有序驱逐。结论：驱逐余量必须大于“一个检查周期内可能增长的内存”。
3. 先 `kubectl get --raw /api/v1/nodes/<节点>/proxy/configz | jq .kubeletconfig.evictionHard`，只看到三个磁盘信号。在 Kind 配置中加入 `kubeadmConfigPatches: ["kind: Kubelet Configuration\nevictionHard:\n memory.available: \"500Mi\"\n nodefs.available: \"0%\"\n nodefs.inodesFree: \"0%\"\n imagefs.available: \"0%\""]`，重建集群后再查一次。保留三个 `0%` 是为了明确地关闭磁盘驱逐：节点容器看到的是整个主机的磁盘，若将来 `mergeDefaultEvictionSettings` 被设为 true 或默认行为改变，没写的信号会套用默认值（如 `nodefs.available<10%`、`imagefs.available<15%`），主机磁盘被别的东西占满时，所有节点都会开始驱逐 Pod、回收镜像，而原因与节点本身无关。
4. 配额（`--cpus 4`）下进程仍能看到 16 个 CPU。Go 1.25 之前的程序会按 16 启动 P（GOMAXPROCS），16 个线程并行时 25 ms 就用完 100 ms 周期内的 400 ms 配额，随后被暂停 75 ms，表现为周期性的延迟尖峰，可观测节点 cgroup 的 `cpu.stat` 中 `nr_throttled`、`throttled_usec` 增长；cpuset 下 GOMAXPROCS 为 4，不会被节流。Python 多进程程序若按 `os.process_cpu_count()` 启动工作进程，cpuset 下是 4 个，配额下是 16 个，内存占用翻几倍，进程之间争抢 4 个 CPU 的配额，可观测进程数、RSS 与 `nr_throttled`。CPU 密集型 Pod 在 cpuset 下只在 CPU 4-7 上竞争，饱和时表现为调度延迟（`cpu.pressure` 的 some 值上升）；配额下它可以跑在全部 16 个线程上，与其他节点的进程共享物理核心（SMT 兄弟线程、缓存），形成跨节点的吵闹邻居，节点总用量超过 4 个 CPU 时整个节点被节流。
5. ① 负载均衡器本身：`kubectl get svc` 看 EXTERNAL-IP 是否已分配，`docker ps --filter label=io.x-k8s.cloud-provider-kind.cluster=ent` 看 envoy 容器是否在运行，`docker logs <envoy 容器>` 看健康检查事件（`add_healthy_event`）和访问日志中的响应标志（`UH` 表示没有健康的后端）。② 健康检查的目标：envoy 的集群配置（`/home/envoy/cds.yaml`）中的检查间隔与阈值，以及每个节点上 `curl`

`http://<节点 IP>:10256/healthz`（kube-proxy 的健康端点）是否返回 200。③ 服务端点与转发规则：`kubectl get endpointslices -l kubernetes.io/service-name=<服务>` 看就绪的端点数，Pod 的就绪探针是否通过，`docker exec <节点> iptables-save | grep <服务>` 看 KUBE-SVC/KUBE-SEP 链及其概率，kube-proxy 日志中规则同步的时间。

6. 审计日志在 Metadata 级别已经记录了“谁、何时、对哪个 Secret、做了什么操作”，缺的只是“值是否被篡改”。做法：① 让人不直接写 Secret，由外部密钥管理系统（Vault/OpenBao、云的 Secrets Manager）保存值和版本历史，并由 External Secrets 之类的控制器同步到集群；密钥系统自己的审计日志记录谁改了值，集群审计日志中写 Secret 的只应是同步控制器，出现其他写入者即为异常。② 需要在集群侧比对时，记录值的指纹而不是值本身：用只有审计系统持有的密钥计算 HMAC（不要用普通哈希，短密码的 SHA-256 可以被穷举），由准入控制器或同步控制器写进注解或外部的防篡改日志，定期与密钥系统中的版本比对。③ 配合 etcd 静态加密，防止绕过 API 直接读取存储。

第 2 章

1. naive：每层最多 4 次尝试，三层相乘，payments 最多收到 4 × 4 × 4 = 64 次请求。resilient：每层最多 3 次尝试，但稳态下重试量被预算限制在请求量的 20%，每层放大不超过 1.2 倍，三层合计约 $1.2^3 \approx 1.73$ 倍（启动时每个调用方另有至多 10 个令牌的突发）。实际上 payments 完全不可用时，orders 的熔断器在 10 次调用、失败率 ≥ 50% 后断开，之后每 5 秒只放过 3 次试探，payments 收到的请求反而远少于用户请求。
2. 订单已是 CONFIRMED，outbox 中有这条事件：中继（本副本重启后或另一个副本）会把它发布出去，事件不会丢；outbox 中只有一行，流中也只有一条（中继重发时由 `Nats-Msg-Id` 去重）。客户端看到连接错误或网关返回的 502/504；它（或网关的自动重试）用同一个幂等键再次提交时，orders 按键查到已有订单，返回 `200` 和 `Idempotent-Replayed: true`。不会产生第二笔订单，也不会第二次扣款。
3. 用 CronJob 每 5 分钟运行：查询 `status = 'FAILED' AND failure_reason LIKE 'payments%'`、创建时间早于“支付锁过期时间”（防止与仍在进行的请求竞争）且在 24 小时内的订单；对每一笔调用 `GET /charges/{order_id}`。没有扣款记录则无需处理；已扣款时按业务规则二选一：用同一订单号重新预留库存（inventory 按订单号幂等），成功则把订单改为 CONFIRMED 并写 outbox 事件；否则以 `refund-{order_id}` 为幂等键发起退款，并把订单标为 REFUNDED。状态变更用条件更新（`UPDATE ... WHERE id = $1 AND status = 'FAILED'`），多个对账进程并发或重复运行也只会生效一次。输出每轮处理的数量作为指标，异常增多时告警。
4. 中继只在 outbox 为空时休眠，所以空闲时一笔订单的 PRODUCER span 与 COMMIT 之间的间隔近似在 [0, 轮询间隔] 内均匀分布：200 ms 时平均约 100 ms，改为 1 秒后平均约 500 ms，`outbox delay p95` 接近 0.95 秒。负载较高时中继几乎不会休眠，两种设置的差别会变小。这说明轮询间隔直接决定了异步路径的延迟下限。
5. ① 流量上升：即使依赖的延迟不变，并发数 = 请求速率 × 延迟（利特尔定律），速率翻倍并发也翻倍，可对比请求速率与 `bulkhead_inflight`。② 调用方自身变慢：事件循环被阻塞或 CPU 被节流时，从调用方看每次调用的耗时变长，可对比调用方的 `upstream_request_duration_seconds` 与依赖服务端的 `http_server_request_duration_seconds`，再看调用方的 CPU 节流指标。③ 计数泄漏或连接问题：

`bulkhead_inflight` 在流量很低时仍居高不下说明计数没有释放（代码缺陷）；连接建立慢（DNS、连接池上限）也会延长占用时间，可在链路中看请求发出前的空白。

6. 为平台组件建立与业务服务同等的监控：容器重启次数的增长（`kube_pod_container_status_restarts_total`）、上一次终止原因为 OOMKilled（`kube_pod_container_status_last_terminated_reason`）、工作集与内存上限之比超过 90%、节点的 OOM 计数（`node_vmstat_oom_kill`），以及 Argo CD 自身的指标（对账耗时、同步失败数）。这些告警发给平台团队，以工单级别处理；另外定期（例如每周）检查平台命名空间中的重启和 OOM 记录，因为“自愈”只是让故障不再立刻可见。
7. ① 为 CloudNativePG 的 Cluster 写一个自定义健康检查，主库可用（例如 `status.readyInstances >= 1`）即视为 Healthy，服务因此提前约 5 分钟启动。风险：副本还在加入时主库发生故障，没有可提升的副本；服务的连接和迁移与基础备份同时压在主库上。② 让副本加入不再等待平缓检查点：在主库初始化后立即执行一次 `CHECKPOINT`，或改用从备份/卷快照创建副本（如果所用的运营商支持）。风险：一次集中的写入会短时间拖慢主库；快照方式依赖存储的能力，本地 local-path 存储并不支持。

第 3 章

1. 每条连接独立、均匀地落到 3 个副本之一，共 $3^3 = 27$ 种等可能的分配。三个副本都至少有一条连接的分配是 3! = 6 种，所以“至少有一个副本没有任何连接”的概率是 $1 - 6/27 = 21/27 \approx 78\%$。即使每条连接承载的请求一样多，最忙的副本也有很大概率承担 2/3 或全部的请求。结论：k 与 N 相当时，L4 负载均衡几乎必然失衡；要么让 k 远大于 N（更多连接、定期重建），要么改用按请求的 L7 负载均衡。
2. kube-proxy 的 DNAT 发生在节点的网络命名空间里，Pod 自己的连接表记录的是它连接的目的地址（Service 的 ClusterIP），看不到转换后的后端。服务端 Pod 看到的是真实的对端（客户端 Pod 的 IP，未做源地址转换）。在客户端一侧也能得到对应关系：在节点上查看连接跟踪表（`conntrack -L -p tcp --dport 8080`），每条记录同时包含原始目的地址（ClusterIP）和回复方向的源地址（实际的后端 Pod IP）。
3. 交替进行时，每一对 A、B 在几分钟之内先后运行，主机的慢变化（其他进程、温度、后台任务）对两者的影响近似相同，配对差把它抵消掉；先 A 后 B 时，漂移完全混入了差值。仍未控制的因素：每对内部的先后顺序（可以每轮随机或交替 A/B 的先后）、切换后的预热（本实验每次切换后等待 20 秒）、单次运行的抽样波动（延长每次运行或增加轮数）。判断结论是否可靠，看配对差在各轮之间是否方向一致、离散程度是否远小于差值本身。
4. ① 让 Prometheus 所在的 monitoring 命名空间加入网格：它的抓取经 ztunnel 以 mTLS 发出，STRICT 下可以通过，还可以在授权策略中只允许 Prometheus 的身份访问 `/metrics`；代价是监控系统依赖网格，网格出问题时监控也可能受影响（这正是最需要监控的时候）。② 把指标放在单独的端口上，并为这个端口设置端口级的 PERMISSIVE；代价是要修改应用（多一个监听端口），而且该端口上的明文流量没有身份认证，需要用 NetworkPolicy 限制只能从 monitoring 命名空间访问。也可以让外部的 Prometheus 通过联邦或远程写入从网格内的采集器取数，但会增加组件和延迟。
5. 建议：网格层只保留 Istio 默认的连接级重试（connect-failure、refused-stream：请求没有送达，重试是安全的），不在网格上为 5xx 配置重试；业务级的重试只在应用层做，只对幂等调用，用重试预算和熔断器限制总量；超时按“外层大于内层”的原则配置（外层超时 > 内层超时 × 内层尝试次数），并通过截

止时间传播让内层知道还剩多少时间。如果某个服务没有应用层重试（例如第三方语言写的旧服务），才在网格上为它单独配置重试，并在容量规划中把放大倍数（尝试次数 + 1）计入下游。Envoy 也支持重试预算（按活动请求的比例限制重试），可以作为网格层的保护。按这套策略，payments 完全不可用时：熔断器断开之前，每个下单请求最多 1 + 2 次尝试，但重试预算把稳态下的总量限制在请求量的 1.2 倍；熔断器在 10 次调用、失败率 ≥ 50% 后断开，之后每个 orders 副本每 5 秒只放过 3 次试探，两个副本合计约每秒 1.2 次，与用户的请求量无关。本章 S4 场景中 61 笔订单只让 payments 收到 6 次请求，就是这种保护的结果。

6. ① 把 orders 调用 payments 的超时与 payments 的延迟分布对齐（高于其 p99.9），或者在超时后先用 `GET /charges/{key}` 查询结果，再决定确认、退款还是重试。② 把“结果不明”作为一个独立的状态：409 或超时时订单保持 PENDING（库存继续预留），由后台任务查询支付结果后再确认或释放，而不是立即标为 FAILED。③ 对账：定期找出“支付 approved、订单 FAILED 或不存在”的组合，按规则补单或退款（第 2 章练习 3）。④ gateway 在重放 PENDING 订单时返回 202（已受理、处理中）而不是 200，让客户端知道需要稍后查询，而不是把它当作最终结果。
7. 不进入应用容器的做法：① 在节点上读连接跟踪表（`conntrack -L`），一次得到所有经过 kube-proxy 的连接及其实际后端，不触碰应用进程；② 用网格的指标：ztunnel 以源和目的工作负载为标签记录打开的连接数与字节数（`istio_tcp_connections_opened_total` 等），可以直接按 Pod 对统计；③ 用 eBPF 工具（如 Cilium Hubble、Inspektor Gadget 的 `trace tcp`）在内核里观察连接，开销低且不需要进入容器。代价分别是需要节点权限、依赖网格、需要额外的工具。不过本章的实验也表明，`kubectl exec` 一个短小的进程并不会让应用停顿（`evidence/e2/exec-stall.log`），关键是用实验去确认，而不是想当然。

第 4 章

1. 最忙的副本每次迭代消耗 0.58 × 1.6 × 6 ms = 5.57 ms CPU，达到 0.95 核时的速率是 0.95 / 0.00557 ≈ 171 迭代/秒。3 个副本、最忙的承担 40% 时，每次迭代 0.40 × 1.6 × 6 = 3.84 ms，饱和点约 247 迭代/秒，是原来的 1.45 倍而不是 1.5 倍：一组副本的容量受其中最忙的一个限制，3 个副本时最忙的一个承担了平均值（33%）的 1.2 倍，比 2 个副本时（58% 对 50%，1.16 倍）更不均衡。实际的收益还会被下一个瓶颈截住：本章的模型中 orders 约在 287 迭代/秒饱和，节点的 CPU 也随之接近上限。
2. 这两条 503 序列是被这次爆发本身创建的：prometheus_client 在一个标签组合第一次被使用时才创建它，Prometheus 第一次抓到时，序列的值已经包含了整个爆发（118 和 106），此后不再变化。`increase()` 只计算一条序列在窗口内各个样本之间的变化，“从没有这条序列到 118”的跳变不在任何两个样本之间，所以结果是 0。办法：① 启动时为需要告警的组合（路由 × 错误类别）预先调用 `.labels(...)`，让序列从 0 开始；② 让 Prometheus 利用客户端暴露的创建时间，在序列开头补一个零样本（`created-timestamp-zero-ingestion`，需要能携带创建时间的抓取格式）；③ 用始终存在的序列构造指标（例如“全部请求减去成功请求”），或者以客户端一侧的测量（合成探测、k6）作为服务水平指标，它们不依赖服务端序列是否已经存在。
3. 先后测量时，机器的慢漂移（其他进程、温度、频率、邻居）全部混入差值：本章同一个旧版本相隔三小时分别测得 11.3 ms 和 7.4 ms。并排时，所有 Pod 在同一时间承受同一股流量，漂移对它们的影响相

同，两个旧 Pod 组成 A/A 对照，给出方法本身的噪声。仍未控制的因素：① Pod 所在的节点与核不同（本章第二次把金丝雀放在一个旧 Pod 所在的节点上，结果相同）；② 每个 Pod 分到的负载不同，而每个请求的开销可能随负载变化（第一次金丝雀分到每秒 51 个请求，旧 Pod 是 25–31；第二次金丝雀负载更低，结论不变）；③ 金丝雀是新启动的（缓存、连接池、垃圾回收的状态不同），本章在统计窗口前预热了 20 秒；④ 每种配置只有一个样本，靠重复两次弥补；⑤ 金丝雀相对其依赖的位置（是否与 catalog 同节点）没有控制。

4. ① 进程或事件循环被阻塞：那样 `/healthz` 也会慢，而它一直是 2–7 ms，排除。② 数据库本身慢或不可达：那样 ent-worker3 上的 Pod 也会失败，从节点自己的网络命名空间直连数据库也会慢；两者都正常，排除。③ catalog 自己的连接池耗尽：连接池指标显示使用中的连接只有 0–2 个，排除。④ 从这个节点上的 Pod 到数据库的路径慢：Pod 内部执行 `SELECT 1` 很慢而节点直连很快，证实；再用网卡丢包计数和 TCP 重传计数区分“网络丢包”和“代理内部停顿”：丢包为 0、重传极少，指向经过 ztunnel 的那一段。
5. $N = X \cdot R = 64 \times 0.0265 \approx 1.70$。Prometheus 每 15 秒读一次计数器，90 秒窗口中每个 Pod 只有 6 个读数，每个读数是一个每隔几毫秒就变化的量的快照：6 个快照的平均值方差很大，如果抓取时刻与请求的到达节奏锁相，还会有系统偏差。在 Pod 内部每秒采样 20 次，平均值与 $X \cdot R$ 相差不到 5%。正确的做法是对时间求积分：`rate(http_server_request_duration_seconds_sum[窗口])` 就是窗口内（已完成请求的）平均在途请求数。
6. 两个网关副本分在两个可用区时，失去其中一个可用区就只剩一个副本。实测一个副本在 100 迭代/秒时仍能服务（失败 0–3%，浏览 p95 41–73 ms），110 迭代/秒时饱和（0.96 核，失败率 33%），即容量从约 170 降到约 100 迭代/秒。目标“最忙的网关进程不超过 0.7 核”：网关每次迭代合计约 9.2 ms CPU（第二阶段实测），一个副本时上限约 0.7 / 0.0092 ≈ 76 迭代/秒；两个副本、最忙的承担约 53% 时约 0.7 / (0.53 × 0.0092) ≈ 144 迭代/秒。所以要在失去一个可用区之后仍承受约 140 迭代/秒，网关需要 3 个副本，并用硬性的拓扑分布约束（`maxSkew: 1`、`whenUnsatisfiable: DoNotSchedule`）保证三个可用区各有一个，失去任何一个都还剩两个。本章的软约束（`ScheduleAnyway`）允许副本挤在两个可用区中，catalog 就是这样（实验 3 的异常三）。还要检查下一个瓶颈：三个副本、负载均匀时网关约可承受 210 迭代/秒，而 orders 约在 287 迭代/秒饱和，工作节点合计约 1 核 + 每次迭代 32 ms，在 210 迭代/秒时约 7.7 核（共 12 核），余量仍在，但已经不大。
7. py-spy 根据操作系统的线程状态判断线程是否空闲，按线程号查找。它在节点的 PID 命名空间中运行，看到的是节点命名空间的线程号（例如 21387），解释器报告的却是 Pod 命名空间中的线程号（1、12、13……），对不上，于是退回到启发式判断，而启发式不认识 uvloop 的等待，空闲的事件循环被算作活跃。`--gil` 只保留某个线程持有 GIL 的样本，它漏掉的是释放了 GIL 的 C 代码所用的 CPU（uvloop/libuv 的 I/O 处理、系统调用、TLS 握手、压缩等），以及非阻塞模式下读取失败而丢失的样本。核对的办法是同时从 `/proc` 读每个线程的 CPU 时间：网关主线程用了 0.43 核，而持有 GIL 的时间只占 25%。

第 5 章

1. 0.53 核是请求的 106%，比值 1.06 在 10% 的容差之内，HPA 不动作。0.58 核是 116%，超出容差，期望副本数 = ⌈2 × 1.16⌉ = 3。要让 0.55 核一定触发扩容，比值必须大于 1.1，即目标低于 0.55 ÷ 1.1 = 0.5 核：目标 100%（0.5 核）时比值恰好是 1.1，仍不触发，应当设为 90% 或更低（0.45 核时比值 1.22）。也可以缩小容差（集群级参数，或较新版本中按 HPA 设置的容差），但那会让 HPA 对噪声更敏感。
2. ① 在入口按请求分配（L7：入口网关、网关 API 路由）：新副本立刻分到请求；代价是多一跳代理和它的运维。② 限制连接的寿命：服务端在若干请求或若干秒后关闭保持连接（例如代理的最长连接时长），客户端重连时被分配到新副本；代价是更多的连接建立，以及客户端的 TIME_WAIT（本章的对照就因端口耗尽而失败）。③ 提前扩容：按日常曲线或计划事件预先提高最小副本数，副本在流量到来之前就已经有连接；代价是闲置容量。同一个原因还要求 HPA 之外的观察：按副本看负载是否均匀，而不是只看平均值。
3. 只按积压扩缩容，消费者跟上之后积压接近零，期望副本数就会落到最小值，稳定窗口一过就缩容，积压重新积累，形成以稳定窗口为周期的锯齿（实验 3）。规则要让稳态由到达速率决定：用 Prometheus 触发器按“每秒新增的消息 ÷ 每个副本在目标利用率下的处理能力”计算副本数（本章：确认订单数 ÷ 8），积压触发器保留，用来消化突发；KEDA 取要求最多的那个。验证方法是在恒定负载下保持比稳定窗口长得多的时间（本章 12 分钟，稳定窗口 5 分钟），记录副本数与通知的端到端延迟：修正后副本数应当稳定在到达速率 ÷ 处理能力附近，延迟不再出现周期性的尖峰。本章修正后副本数稳定在 3，p95 保持在 0.48–0.73 秒（台阶开始、从 1 个副本扩容的头两分钟除外，那段积压与触发器无关，要靠更高的最小副本数或提前扩容）。
4. 先看被限流的周期比例：`rate(container_cpu_cfs_throttled_periods_total[5m]) / rate(container_cpu_cfs_periods_total[5m])`，以及被限流的时间；在节点上读容器 cgroup 的 `cpu.stat`（`nr_throttled`、`throttled_usec`）和 `cpu.max`。尖峰与 100 ms 周期对齐、事件循环延迟或 p99 随被限流比例上升，就是限流。确认的办法是并排对照：一个副本去掉限制（或提高限制），其余不变，比较延迟（实验 4 的方法）。还要检查命名空间的 LimitRange：只写了 `max` 而没写默认限制时，每个容器都会被加上等于最大值的 CPU 限制（实验 8）。
5. 按 kubelet 的换算，权重分别是 79（2 核）、20（500m）和 2 × 3（50m）。竞争时按权重分配：批处理约 79/105 = 75%（3 核），网关约 19%（0.76 核），每个 50m 的服务约 1.9%（0.08 核）。网关平均只需要约 0.3 核，按这个估算应当够用；可是它是单线程的事件循环，每次被唤醒都要排在某个批处理线程之后，等它用完时间片才能运行，延迟由排队决定而不是由平均份额决定，所以实验 5 中这个节点上网关的 p50 是 131 ms，同一时刻其他节点上的副本只有 14 ms。50m 的服务连平均需要的 CPU 都分不到，延迟到了秒级。
6. 第一个 Pod（请求 0、用 300 MiB）和第二个（请求 64Mi、用 500 MiB）都超出了请求，第三个（请求等于限制 300Mi、用 200 MiB）没有超出。优先级相同时，超出请求的 Pod 按“用量 − 请求”从大到小排序：第二个超出 436 MiB，第一个超出 300 MiB，所以先驱逐第二个（Burstable），而不是 BestEffort 的第一个。QoS 类别不是排序键，实验 6 测到的正是这个顺序。kubelet 每次只驱逐一个 Pod，压力解除就停止：第二个 Pod 释放约 500 MiB 后，如果可用内存回到阈值之上（加上最小回收量），第一个就不会被驱逐。第三个只有在所有超出请求的 Pod 都处理完、节点仍有压力时才轮到，那通

常说明系统进程的用量超出了预留。如果第二个 Pod 的优先级更高：先看是否超出请求（两个都超出），再看优先级，低的先走，于是先驱逐第一个，再驱逐第二个。

7. 参考方案：① 平台组件用 `system-cluster-critical` 或自建的高优先级类（例如 1 000 000）；② 在线服务一个明确的类（例如 10 000），默认的抢占策略；③ 批处理用负的优先级（低于没有指定类的 Pod 的 0），`preemptionPolicy: Never`，它永远不会抢占别人；④ 实验性工作负载放在自己的命名空间，优先级最低。每个团队的命名空间有 ResourceQuota（Pod 数、CPU 与内存请求、内存限制的总量），LimitRange 同时写默认请求和默认限制：内存给默认限制，CPU 只给默认请求、不给默认限制，也不写 `max.cpu`（否则它会成为每个容器的默认 CPU 限制）；需要上限时用配额控制总量，而不是用限制控制单个容器。还可以要求所有 Pod 都指定 PriorityClass（准入策略，第 6 章），避免默认的 0 混在中间。

第 6 章

1. 能。他可以创建一个 Pod，以环境变量或文件引用这个 Secret，再读 Pod 的日志或进入容器（实验 3：alice 不能读 Secret，她的 Pod 却打印出了相同的哈希）；也可以让 Pod 以命名空间中任何服务账号的身份运行，继承它的权限。措施：① 按信任级别划分命名空间，一个命名空间里只放这个团队必须有的 Secret；② 用外部密钥存储按团队和工作负载的身份授权，并审计每一次读取（实验 7）；③ 用准入策略限制 Pod 能引用哪些 Secret、能使用哪些服务账号；④ 生产命名空间只给团队只读权限，变更经过 Git；⑤ 对引用敏感 Secret 的 Pod 创建和 exec 告警。
2. 当天正常，是因为 frontend 和 api 之间的 HBONE 连接在加策略之前已经存在，kindnet 不会重新检查已经放行的连接，旧连接一直在用（实验 4，规则 1）。api 重新部署后出现新 Pod、新连接，策略这才真正生效，而这条规则是错的：网格内的流量到达 15008 而不是 8080，并且经过 waypoint 时来源是 waypoint 而不是 frontend。正确的做法：允许 waypoint 访问 api 的 15008（如果这个服务的流量不经过 waypoint，则允许 frontend 访问 15008），保留 8080 给网格外的客户端；或者在 NetworkPolicy 中只按命名空间放行，把“谁能调用谁”交给 AuthorizationPolicy。验证策略时，一定要用新的连接（新的探测 Pod，或者重建客户端）。
3. NetworkPolicy：允许本命名空间访问 15008（网格内的调用，来源是 waypoint 或 ztunnel），允许 monitoring 访问 8080（Prometheus 在网格外，明文到达应用端口）。它做不到“只允许网关”：经过 waypoint 时，所有调用者在网络层看起来都是 waypoint。“只有网关能调用 orders”要用 Istio 的 AuthorizationPolicy，挂在 waypoint 上（L7），按调用者的身份 `cluster.local/ns/storefront/sa/gateway` 放行；Prometheus 的抓取不带网格身份，要按端口单独放行，或者让抓取也进入网格。
4. 每月 6 次 × 34 秒 = 204 秒，约 3.4 分钟。这期间所有需要新建 Pod 的机制都失效：HPA 和 KEDA 扩容、滚动更新、被驱逐或崩溃的 Pod 重建、Job 和 CronJob、新节点上的 DaemonSet Pod。如果恰好发生在高峰扩容或故障恢复时，影响远大于 3.4 分钟这个平均数。改进：① 至少 2–3 个副本，加上 PodDisruptionBudget 和跨可用区分布（实验 5：2 个副本时只剩 1 个请求失败）；② 用 namespaceSelector 把 webhook 限制在需要策略的命名空间，排除平台的关键命名空间；③ 非安全性的规则用审计模式或 Ignore；④ 简单的校验改用内置的 ValidatingAdmissionPolicy；⑤ 监控 webhook 的失败次数和延迟，并在升级前演练。

5. 字段：`user.username`、`user.groups`、`impersonatedUser`、`verb`（get、list、watch：list 一次返回全部 Secret）、`objectRef`（resource=secrets、namespace=storefront、name）、`responseStatus.code`、`requestReceivedTimestamp`、`sourceIPs`、`userAgent`，以及授权理由的注解。直接读取之外，还要查：在 storefront 中创建 Pod 并引用 Secret 的请求、对 storefront 中 Pod 的 exec（容器里的环境变量可以读到）、能读 Secret 的服务账号（以及谁用过它们的令牌），以及外部存储自己的审计（实验 7）。本章的审计策略对人的 Secret 读取记录元数据（谁、何时、哪个 Secret，不含内容）；但它对服务账号和节点的 get、list、watch 完全不记录，而且这条规则排在 Secret 规则之前，所以一个被盗用的服务账号令牌读取 Secret，不会留下任何记录。这是一个需要权衡的缺口：记录所有服务账号的读取会产生大量日志，至少应当为 Secret 单独记录。
6. 参考流程：① 身份：工程师用自己的身份申请，加入一个临时的 break-glass 组，经第二个人批准（身份系统中的审批流程）；② 授权：只授予需要的最小范围，例如只读这一个密码的外部存储策略，而不是集群管理员；③ 时限：凭据或授权自动过期（本章的 break-glass 证书一小时）；④ 记录：申请理由关联工单，所有操作都带着 break-glass 组出现在审计日志中，任何 break-glass 的使用都实时告警；⑤ 事后：导出这段时间的审计记录做复盘，轮换被读取过的密码（它已经暴露给了人），检查是否需要把这类操作做成常规的、受控的工具。
7. 因为这条边界只由 Argo CD 自己执行。真正写入集群的是应用控制器，它拥有 `* on *` 的权限（实验 3）。AppProject 配置得过宽、Argo CD 的缺陷、有人能修改 AppProject 或 argocd-cm、Argo CD 本身被攻破，都会让边界消失，而 Kubernetes 的授权不会拦住它。落到 Kubernetes 本身的办法：① 启用 Argo CD 的同步冒用，为每个项目指定目标命名空间中的服务账号，控制器以这个服务账号的身份同步，权限由 RBAC 限定；② 对多个不互信的团队，使用各自的 Argo CD 实例；③ 准入策略按对象的内容校验，不论是谁创建的；④ 严格限制谁能修改 argocd 命名空间中的 AppProject、Application 和配置。

第 7 章

1. B 对 C 的每次调用平均尝试 1 + 0.9 + 0.81 + 0.729 = 3.44 次，最终失败的概率是 $0.9^4 = 0.66$；A 对 B 的每次调用平均尝试 1 + 0.66 + 0.43 + 0.28 = 2.37 次。C 收到的请求约为正常时的 3.44 × 2.37 ≈ 8.1 倍。如果 C 的失败来自过载，8 倍的负载会让它更快地完全不可用，失败率升高又带来更多重试，形成正反馈，失败从 C 蔓延到所有调用它的服务。本章实测 5.56 倍，低于估算：并非每次下单都会走到调用 inventory 的那一步，网关只在 orders 返回特定错误时才重试，而且“每笔下单”的分母与估算的口径不同；本章没有逐一分解这些因素。
2. 两个要求同时满足，超时要落在 1.5 秒和 2 秒之间：orders 对 inventory 的超时取约 1.8 秒，超时后不重试（重试只留给快速返回的错误，并受预算限制），连续超时后熔断器打开，之后快速失败。网关对 orders 的超时要大于 orders 的最坏情况，例如 2.5 秒；更好的做法是传递截止时间：网关在请求中写入截止时刻，orders 调用 inventory 之前检查剩余时间，取“1.8 秒”和“剩余时间减去余量”中较小的一个，剩余时间不够时直接失败（storefront 已经有按截止时间拒绝的机制，即 `upstream_rejected_total` 的 `deadline` 原因）。依据是依赖的延迟分布，而不是一个统一的数字。
3. 因为熔断器是按服务的。冻结的节点上还有一个 catalog 副本，waypoint 按请求在两个 catalog 之间分配，每秒几个调用超时，就让网关对 catalog（以及 orders）的熔断器打开，此后所有调用都被直接拒绝

（5xx 52.8 次/秒，正好等于熔断拒绝 52.9 次/秒）。改进一：在网格中按实例做异常检测（例如 DestinationRule 的 outlierDetection），只剔除超时的那个端点；代价是需要足够的健康副本承接流量，剔除的判断本身也可能误伤。改进二：调整熔断器的触发条件（更长的统计窗口、更高的失败比例阈值），让一部分失败不至于打开它；代价是依赖真的全部不可用时，熔断器反应更慢。还可以缩短连接超时，让坏端点上的调用更快失败并重试到另一个副本，代价是网络抖动时更多的误判。

4. 30 天 × 24 × 60 × 0.5% = 216 分钟。第一次约 5.5 分钟，用掉约 2.5%；第二次约 55 秒（0.92 分钟），约 0.4%。每月两次第一次那样的节点失效，用掉约 11 分钟，即 5.1%，还剩约 205 分钟（94.9%）。错误预算还要留给发布、依赖故障和其他事故，节点失效的成本是可以压缩的（第二次实验）。
5. 三个节点平均分布时，失去一个可用区，剩下两个节点要容纳全部 Pod：总请求 ≤ 2 × 单节点可分配量，即每个节点的请求最多约为可分配量的 2/3（约 67%），本环境约 1.78 GiB。现在是 95%–99%，要减掉约三成：按实际用量下调过高的请求（例如 metrics-server），减少不必要的副本和组件，或者增加节点和内存。另一种放宽的方式是分级：只要求关键工作负载（高优先级）满足 67%，低优先级的工作负载在失去可用区时可以等待调度，这需要 PriorityClass 把两者分开。
6. 以事故一为例。检测：是做实验的人发现“实验没有指标”才察觉的，大约在 10 分钟后；更早的检测方式包括对 kubelet 驱逐（`kube_pod_status_reason{reason="Evicted"}`）、节点内存压力状态、Prometheus 自身的“看门狗”告警（发往集群外，Prometheus 停止时由外部发现）告警。影响：监控缺失约 5 分钟（10:29–10:34），一次实验的指标丢失、需要重做，Argo CD 控制器重启约 1 分钟；没有用户可见的影响。改进项要有明确的负责人和期限，记录在问题跟踪系统中，在下一次例会上检查，“完成”的标准是有证据，例如事故二的改进项以第二次节点失效中 Prometheus 一直运行作为验证。
7. 节点失效会触发 Kubernetes 的处理：节点在约 50 秒后变为 NotReady，端点被移除，问题在一分钟左右被“自动处理”。分区中节点一直 Ready（kubelet 与控制面之间是通的），Kubernetes 没有任何动作；负载均衡器的健康检查看的是节点，也没有发现；网格的 ztunnel 继续把约一半的新连接发往不可达的 waypoint，5 分钟内在一个节点上就有 68 次连接超时。只有 Prometheus 看到了：24 秒内 10 个目标下线。本次用户的失败只有 0.22%，是因为连接大多在分区之前建立、长期复用，应用的重试掩盖了新连接的失败；这不是自愈，隐患在整个分区期间一直存在。处理分区需要基于监控的判断和操作（封锁并清空受影响可用区的节点）或网格层面的实例健康检查。

第 8 章

1. 最坏情况：丢失发生在下一次段切换之前的一刻，丢掉整整一个 `archive_timeout` 的写入，5 × 60 × 50 = 15 000 次（再加上段上传所需的几秒）。丢失时刻在 5 分钟的窗口里均匀分布时，期望是一半，约 7500 次。R1 的 202.5 秒和 R2 的 55.9 秒都落在 0–300 秒之内；按写入器实际的速率（10 550 / 202.5 ≈ 52 次/秒，略高于标称的 50 次），丢失的次数也与时间窗口一致。时间窗口才是 RPO，次数只是它乘以写入速率。另外，一个完全空闲的数据库不产生 WAL，此时没有东西可丢。
2. 删除后约 2.2 秒旧的 PostgreSQL 才真正停止（kubelet 杀死容器）；operator 在 0.5 秒内发起切换，等待各副本的 WAL 接收器停止，3.2 秒时选定 pg-main-1；新主库等待租约 16 秒（3.7–19.7 秒）；提升约 5 秒（重放 2 秒，从归档读取时间线历史 3 秒）；operator 修改服务标签 0.5 秒；写入器的下一次连接成

功又过了约 4 秒（每次连接尝试最多等 2 秒）。不应缩短的是租约等待：它防止的正是 F2 那样的脑裂，缩短它会让一次网络抖动就可能产生两个主库，除非有可靠的手段确认旧主库已经不在（例如节点被确认关机）。可以缩短的是客户端的重连（更短的连接超时、立即重试）和提升时读取归档的开销。

3. 办法一：在提升新主库之前，从外部确认旧主库不可能再写，即 STONITH，通过虚拟化平台或带外管理接口关闭旧节点，Kubernetes 中对应的是确认节点已关机后给它加上 `node.kubernetes.io/out-of-service` 污点。代价是需要一个可靠的带外接口，隔离本身失败时宁可不切换，故障切换变慢。办法二：让旧主库无法被用到。客户端只通过服务连接，连接池设置最长寿命并在借出连接时检查连接的服务器是否仍是主库；副本的流复制连接设置连接超时，不让一次挂起的握手在解冻后完成。代价是更多的连接开销，而且仍然只是缩小窗口：冻结的进程在恢复的一瞬间就是可写的，只有外部隔离或者写入路径上的隔离令牌（每次提升递增的代数，写入时校验）才能彻底消除它。
4. 30 天的错误预算是 30 × 24 × 60 × 1% = 432 分钟。F3：切断期间写入都成功，冻结后中断 73.7 秒，约 1.2 分钟，用掉约 0.28%；但 3017 次已确认的写入（57 秒的订单）永久丢失，这不是错误预算能衡量的。F4：中断 137.9 秒，约 2.3 分钟，用掉约 0.53%，没有丢失任何已确认的数据。订单数据库选同步复制：几分钟下单失败，客户可以重试；已经扣款、已经确认的订单消失，就需要人工对账和补偿。商品目录写得很少、可以从源头重新导入，异步就够了。两者在同一个集群里时，不必为此分成两个集群：同步与否可以按数据库设置，例如 `ALTER DATABASE catalog SET synchronous_commit = local`，让目录的提交不等待备库，订单仍然等待。
5. 先止血：`DROP TABLE outbox` 不只是丢数据，每一笔新订单的事务都要写 outbox，所以从删表那一刻起下单全部失败。第一步按迁移脚本重建一张空表，让下单恢复；从 orders 服务的错误（“relation outbox does not exist”第一次出现的时间）确定删表的时刻。然后按 pitr-1 的方法在旁边恢复到删表之前，把 outbox 的行复制回来。与 pitr-1 不同的地方：（1）删表期间的订单事务整体失败，订单本身不存在，没有需要找回的事件；（2）恢复回来的未发布事件（`published_at` 为空）会被 relay 再次发布，下游要按 `event_id` 去重；（3）重建的表的 `bigserial` 序列从 1 开始，与恢复回来的旧 id 冲突，复制回来之前要把序列调到旧的最大值之后，或者复制时不带 id 列。
6. 两种做法可以结合。告警：Velero 导出备份警告的计数（`velero_backup_warning_total`），对它在一天内的增加告警；V1 的备份有 13 条警告。检查脚本：每次备份之后，列出备份所涵盖的命名空间中所有已绑定的 PVC，再列出这次备份中状态为 Completed 的 PodVolumeBackup 对应的卷，数量不一致就失败，并列出缺少的 PVC；或者直接查 `kubectl get pv` 中 `spec.hostPath` 不为空的卷。最可靠的检查仍然是定期还原（V2）：还原到一个测试命名空间，检查数据是否在。
7. 改动一：减少波次之间的串行等待。PostgreSQL 的恢复不需要等 kube-prometheus-stack、Kyverno 和 KEDA 都健康：把 cert-manager、CloudNativePG 和 Barman Cloud 插件放在最前面的波次，pg-main 紧随其后，让最慢的数据恢复与其余组件并行；R2 中波次 −3 就用了 4.6 分钟，数据库的恢复又用了约 2.5–3 分钟。代价是依赖关系要更仔细地声明，顺序错误在重试中表现为一段时间的报错，而不是一眼可见的死锁。改动二：不重建，而是切换。在另一个“区域”保持一个备用集群，用 CloudNativePG 的副本集群从归档或流复制持续跟随，灾难时提升它，RTO 降到分钟以内，RPO 接近于零。代价是双倍的基础设施、两边配置的持续同步，以及一套必须同样定期演练的切换流程。

第 9 章

1. 30 天、每秒 100 个请求，共 2.592 亿个请求，99.5% 的 SLO 允许其中 0.5%，即 129.6 万个失败。3 分钟内入口处理 18 000 个请求，金丝雀分到 5%（900 个），失败 40%（360 个），用掉预算的 0.028%。这 3 分钟里的错误率是 2%，燃烧速率为 2% ÷ 0.5% = 4 倍；权重 1% 时错误率 0.4%，燃烧速率 0.8，连预算的平均速度都没超过。结论有两层：一次短暂的金丝雀事故在 30 天预算里微不足道；正因如此，按全局 SLO 燃烧速率设计的告警（例如 1 小时窗口 14.4 倍）根本不会为它触发——金丝雀必须用金丝雀自己的指标与稳定版对比来判断，而不是等全局告警。
2. 瓶颈在网关之后：run A 的高负载阶段网关返回 40 704 个 503，它自己的计数器显示断路器打开拒绝了 40 010 次，说明下游服务（及其数据库连接池）已经过载，网关的按服务熔断把部分失败变成了全部失败；再加网关 Pod 只会让更多请求冲向同一个瓶颈。可行的改动：其一，提高瓶颈本身的容量——为下游服务配置 HPA 并按数据库的连接上限调整连接池，或者为读请求加缓存；其二，在入口做准入控制或限流，超出容量的请求尽早以 429 拒绝，而不是在 Envoy 与网关之间排队 9 秒。两项都要用同一个台阶重新测量，并把 k6 的虚拟用户上限调高，避免负载生成器先成为瓶颈。
3. 指标：金丝雀与稳定版各自的 5xx 比例和 p95 延迟，按发布（`service` 标签）分开；判定基于两者的对比，而不是绝对阈值。窗口：滑动 1 分钟，至少 300 个金丝雀请求才做判断。金丝雀每秒分到 5 个请求，60 秒约 300 个：错误率 0.1% 时期望 0.3 个错误，出现 3 个及以上的概率约 0.4%；错误率 2% 时期望 6 个，出现 3 个及以上的概率约 94%。所以“60 秒内 3 个及以上错误，且比例高于稳定版的两倍”就能以约 94% 的把握在一分钟内发现，误报约 0.4%；要更有把握，就延长窗口或提高权重——这正是权重、发现时间和爆炸半径之间的取舍。
4. 重建时根应用第一次同步就以 `argocd-root` 的身份进行，而这个账号的 Role 要由根应用同步出来：没有权限就无法创建权限，根应用永远卡住，后面的一切都不会发生（与 I4 中 default 项目没有账号的情形相同）。放在 Argo CD 的 Helm 发布里，账号和 Role 在根应用存在之前就已创建，与 Git 的内容无关；代价是这部分配置不经过合并请求门禁，需要用别的方式审查（例如 Argo CD 本身的发布也走评审）。
5. 先被 Argo CD 拒绝：项目 storefront 的 `namespaceResourceWhitelist` 在 `keda.sh` 组只允许 ScaledObject，同步报告该种类不被项目允许；即使项目允许了，冒用的 `argocd-storefront` 也没有 `triggerauthentications` 的权限，Kubernetes 会返回 forbidden。需要两处改动，都经过门禁并由平台审查：`gitops/platform/apps/00-projects.yaml` 中加入 `{group: keda.sh, kind: TriggerAuthentication}`；`gitops/platform/manifests/tenants/argocd-deployers.yaml` 中 keda.sh 那条规则的 resources 加入 `triggerauthentications`。如果它引用 Secret，Secret 应通过 ExternalSecret 由平台提供。
6. 只有 GitLab 冻结：冻结开始前已合并、尚未同步的改动照样生效；自愈会在冻结期间改写集群；流程外的改动（`helm upgrade`、kubectl）完全不受影响。只有 Argo CD 窗口：main 在冻结期间继续前进，窗口结束时积压的改动一起生效，这个组合从未被单独测试过；`manualSync` 允许操作员同步任何已合并的内容。自动记录的办法：流水线在 `[emergency]` 的合并请求合并后，通过 GitLab API 创建一个带“emergency-review”标签的议题，链接合并请求、流水线和随后的部署记录；冻结期结束后，一个定时作业列出窗口期间的所有部署及其合并请求，没有对应议题的就报警。

7. 办法之一是让 Argo CD 管理它自己：把 Argo CD 的 Helm 发布也写成一个 Application，升级它就是一个合并请求，自然出现在 DORA 的记录里；根信任（第一次安装和 root 项目）仍然要人工应用一次。办法之二是从审计日志发现流程外的写入：不是由 Argo CD 冒用的账号、而是 `kubernetes-admin` 之类的身份修改了平台对象，就生成一条变更记录。局限：自管理有引导顺序的问题，Argo CD 升级失败时可能无法用它自己修复；审计日志只能事后发现，噪声也多，需要排除日常操作。
8. 三个候选：其一，引导期间 API 服务器和 etcd 忙于处理大量并发写入，每个 CRD 的服务器端应用都变慢——审计日志里每个请求都有接收时间和完成时间（`requestReceivedTimestamp` 与 `stageTimestamp`），比较 gateway-api-crds 的 patch 请求在 R2 和 R4 中的服务器端耗时即可判断；其二，Argo CD 这一侧的同步变慢或失败重试（冒用之后的服务器端应用与试运行、冲突后带退避的重试）——应用的 `operationState` 记录了开始、结束时间和重试次数，控制器日志记录每次尝试；其三，同步早已完成，但健康状态到下一次刷新才被重新评估（本平台每 60 秒轮询一次，另有抖动）——比较 CRD 的 `Established` 条件的 `lastTransitionTime` 与 wait-apps 中应用变为 Healthy 的时刻。三者可以叠加；本章没有在集群删除之前保存这些记录，原因因此没有查明，这本身就是下一次重建前要补上的观测。

第 10 章

1. 浏览的预算 $b = 0.5\%$，呼叫线 $14.4b = 7.2\%$。窗口里装满一小时正常流量后，30% 失败时 1 小时比例在 t 分钟后为 $0.3t/60$，越过 7.2% 需要 $t = 14.4$ 分钟；5 分钟比例一分多钟就越过了，所以约 14.4 分钟后呼叫。3% 失败时，1 小时比例最多 3%，低于 7.2%；6 小时那一组的线是 $6b = 3\%$，比例只会逼近 3% 而不会超过，因此永远不会呼叫。工单的条件是 2 小时与 1 天都超过 $3b = 1.5\%$：2 小时比例 1 小时后超过，1 天比例要 12 小时后（$0.03t/1440 > 0.015$，$t > 720$ 分钟）才超过，所以约 12 小时后开工单；3 天 / 6 小时那一组（线 0.5%）同样在约 12 小时后满足。
2. 触发由长窗口决定：6 小时比例超过 6% 才可能呼叫，而这次窗口里只有 35 分钟的历史，6 小时比例实际上就是这 35 分钟的比例，所以很快越线。解除由短窗口决定：两个窗口都超过才算触发，30 分钟窗口里只要还装着足够多的失败，告警就不会解除，于是失败停止后又持续了约 28 分钟。换成 10 分钟，解除会快得多，但短窗口的作用是确认“燃烧还在继续”：太短时，错误比例在阈值附近波动的慢性燃烧会反复触发、解除（抖动），而且一次短暂的停顿就会让告警解除。SRE 工作手册的经验是短窗口取长窗口的 1/12。
3. 监控告警桥的东西必须在另一个故障域里，否则它会和告警桥一起倒下。可行的做法：让告警桥每分钟向一个独立的心跳服务报到（另一台机器上的小程序，或商业的心跳监控），超时由那个服务发出通知；或者部署两个相互检查的告警桥实例，放在不同的主机上，并各自走不同的通知渠道。这个方案依赖的是“第二个故障域”本身——同一台主机上的第二个容器（或者依赖同一个 GitLab 的定时流水线）不算。
4. 任何“第一次发生时才出现”的计数器都有同样的问题。例如 `velero_backup_failure_total` 和 `velero_backup_partial_failure_total` 的某个 schedule 第一次失败时可能才出现；SLO 的错误计数器 `http_server_requests_total{code="503"}` 也是在第一个 503 时才诞生，第一个样本的值不会被 `rate()` 计入，开始时会略微低估。改写方式与 A3 相同，例如 `(sum by (schedule) (increase(velero_backup_failure_total[1h])) > 0) or (sum by (schedule) (velero_backup_failure_total unless`

`velero_backup_failure_total offset 1h) > 0)`；对自己的服务，也可以在启动时把已知的标签组合预先初始化为 0。

5. 每秒 0.2 个请求，一小时只有 720 个，5 分钟只有 60 个。99.9% 的预算是 0.1%，呼叫线 1.44%：5 分钟里一次失败就是 1.7%，1 小时里 11 次失败就会呼叫——几次偶然的失败就会把人叫醒，统计上毫无意义。改进：给告警加上最少失败次数的条件（例如 1 小时内至少 20 次失败）；用合成流量（定期的探测请求）增加样本；把几个低流量的服务合并成一个 SLO；或者为低流量服务选择更长的窗口或更低的目标，并说明理由。
6. 10% 头部采样：错误链路只能保留十分之一（尾部采样补救不了），服务的 CPU 约少 5%（本章测量），存储只有约 1% 的 span。100% 加尾部采样：所有错误和慢请求都保留，服务的 CPU 多约 5%，收集器多出缓冲链路的内存（本章 +26 MiB），存储约为 10% 的 span（是之前的 8.7 倍）。当请求量大到记录全部 span 的 CPU 和网络开销不可接受时（例如每秒数千个请求、收集器需要多个实例），可以重新引入头部采样，但要配合：错误与延迟由指标（以及采样之前生成的 span 指标）统计，而不是从链路数；exemplar 只引用确实被采样的链路（本章的修正）；多个收集器之间按链路 ID 路由，保证同一条链路到达同一个实例。
7. 一种写法：`(histogram_quantile(0.95, sum by (le) (rate(http_server_request_duration_seconds_bucket{namespace="storefront", service="gateway", route=~"/api/products.*"}[2m]))) > 1) and on() (sum(increase(http_server_request_duration_seconds_count{namespace="storefront", service="gateway", route=~"/api/products.*"}[2m])) > 50)`，`for: 1m`：p95 超过 1 秒说明相当一部分顾客在等待，2 分钟窗口和 50 个请求的下限避免为几个慢请求呼叫。用 exemplar 实验（catalog 注入 1.2 秒延迟 45 秒）回测，它**一次也不会触发**（`evidence/e9/exercise7-backtest.log`）：网关的断路器把大部分慢调用变成了约 1 毫秒的 503（exemplar：33 个快速失败对 24 个慢请求），p95 始终低于 1 秒。延迟问题以错误的形式出现——延迟的症状告警必须和可用性的告警一起使用，而且 45 秒的短暂问题本来就不该呼叫。

第 11 章

1. ReplicaSet 控制器为每个 Pod 发一次创建请求，受 kube-controller-manager 的客户端限速约束（每秒 20 次，突发 30 次），10 000 个 Pod 至少约 500 秒；它还按“慢启动”分批创建、每次同步最多创建 500 个，实际只会更慢。调度器每秒 50 个，比创建快，所以瓶颈在 kube-controller-manager，全部调度要 8 分钟以上。把它的限速调到每秒 200 次，创建只需约 50 秒，瓶颈移到调度器：10 000 / 50 = 200 秒。再往上就要同时调高调度器的限速，并确认 API server 和 etcd 承受得住每秒几百次写入。
2. 有效数据约 40 000 × 12 KiB ≈ 469 MiB，加上其余对象约 0.75 GiB；刚做完大批量变更时还带着 5–10 分钟的历史（本章测得刚创建完时多约 70%），可能接近 1.3 GiB。本章测得每删除一个 Pod，压缩之前数据库增加约有效大小的 2.2 倍（8.1–8.5 KiB 对 3.73 KiB），20 000 个就是约 20 000 × 12 × 2.2 KiB ≈ 0.5 GiB——在 1.3 GiB 的基础上离 2 GiB 的配额已经很近。安全的做法：分批删除（例如每批 2000 个），批次之间等 API server 的压缩（最长 10 分钟，或观察 `etcd_mvcc_db_total_size_in_use_in_bytes` 下降）；对 `etcd_mvcc_db_total_size_in_bytes` 与配额之比设告警（例如 80%）；删除完成并压缩之后逐个成员碎片整理；如有需要，事先调大配额（etcd 建议不超过 8 GiB）。

3. 脚本为了取出下一页的 continue 令牌，每一页都启动一个 Python 进程解析约 1.7 MB 的 JSON：20 次进程启动和解析就是几秒，全是客户端的时间。要分开测量：对每一页记录 curl 的 `time_starttransfer` 和 `time_total` 并求和，与整个循环的耗时比较；或者用一个长期运行的客户端（例如 `kubectl get pods --chunk-size=500 -v=6`，它记录每个请求的延迟）；服务器端可以比较 `apiserver_request_duration_seconds` 中 LIST 的增量。分页在服务器端也有额外代价（20 次认证、授权、APF 和缓存快照），但它把每个响应限制在约 1.7 MB——这正是分页的意义。
4. API server 一侧：为这个服务账号建一个 FlowSchema 和专用的优先级，份额小、队列短，`lendablePercent: 0` 且 `borrowingLimitPercent: 0`——它防住的是“占满别人的席位”，防不住内存和 CPU：本章中 3 个席位的大 LIST 就让 API server 多用约 1.4 GiB，所以 API server 的内存要按“这个优先级的席位数 × 每个 LIST 的内存”留出余量，或者把席位压到 1 个。客户端一侧（根本的修复）：用 informer（LIST 一次再 WATCH）代替轮询；用标签或字段选择器、只列需要的命名空间；需要完整列表时分页（`limit=500`），让每个响应只有几 MB；遵守 429 的 `Retry-After`，对连接失败做带抖动的指数退避——本章的洪水客户端在 60 秒里向一个已经停止的服务器发了上万个请求。
5. 403 来自授权器：API server 启动后、加载 RBAC 绑定之前，按 RBAC 授权的请求都被拒绝（探测中，403 之后 3 秒就得到了 200）；429“storage is (re)initializing”来自 watch 缓存：缓存填充完成之前，需要它的请求被拒绝并带 `Retry-After: 1`。把 403 当作永久错误的控制器会停止调和或把对象标记为失败，部署流水线会报“权限不足”，运维人员甚至可能去“修复”一个本来没坏的 RBAC。避免的方法是让负载均衡器按 `/readyz` 转发流量（启动钩子完成之前不就绪），优雅关闭时先摘除流量，并运行多个 API server；客户端对 429 和 5xx 按 `Retry-After` 重试。
6. 步骤：把 `experiments/e10/etcd-size.sh` 中生成 Pod 的模板换成 storefront 某个 Deployment 的 Pod 模板（`kubectl get deploy <名字> -o jsonpath='{.spec.template}'`），在 kwokctl 集群中按“上行”创建 10 000 个，再运行 `etcd-size.sh down`，用相邻两步的数据库大小之差除以 Pod 数。结果通常明显大于 3.73 KiB，差异来自更多的容器、环境变量、卷和探针、标签和注解，以及记录字段归属的 `managedFields`；真实的 kubelet 还会写入更多状态（每个容器的状态、条件）。用这个数字（而不是本章的最小 Pod）做容量估算。

第 12 章

1. 时间线：A 读取状态（serial 5）并开始 apply；B 的锁请求在 Garage 上同样“成功”，B 也读取 serial 5；A 创建资源 X，写回 serial 6；B 创建资源 Y，写回 serial 6——覆盖了 A 的状态，X 从此不在任何状态里：它还存在，却不再被管理，下一次计划甚至可能再创建一个 X。何况 A 结束时删除锁对象，连 B 的锁也一起删掉了，第三个运行同样畅通无阻。在支持条件写入的 S3 上，B 写锁对象时得到 412，OpenTofu 报告“Error acquiring the state lock”，B 在读取状态之前就停下。
2. 一种设计：在 GitLab 的计划流水线（例如每小时）中运行 `tofu plan -detailed-exitcode -lock-timeout=60s`；身份用一个只有所需权限的专用服务账号或项目令牌，令牌和状态口令放在受保护、隐藏的 CI 变量中，流水线只在受保护的分支上运行；退出码 2 时开一个工单（或经告警桥开事故），附上计划输出（注意输出中不能有秘密值）。默认不自动修复：实验 3 说明，修复可能是“先删除、再创建”，自动

执行会在无人察觉时打开窗口；可以只对计划中没有 `destroy`、没有 `must be replaced` 的情况自动应用，其余交给人确认。

3. 做法一：不用 OpenTofu 修复，而是用 API 就地修改（GitLab 的受保护分支接口可以调整访问级别），再运行计划确认没有差异——没有窗口，代价是修复绕过了 IaC 的流程，需要记录。做法二：在没有合并和推送的时间窗口内应用，并在应用前暂时冻结推送（例如把项目设为只读或暂停 runner）——窗口仍在，但没人能利用它，代价是短暂的停工。另一个方向是向提供者反馈或升级到支持就地更新的版本。
4. 56 个 OutOfSync 来自 Argo CD 自己的跟踪方式：它在期望的对象上加跟踪注解，而 helm 安装的对象没有这个注解，于是几乎所有对象都“不同”。服务器端试运行只问 API server“如果应用这份渲染结果，对象会怎样变化”，跟踪注解不在渲染结果里，所以只剩真正的差异——helm 已经删除的钩子任务。接管前做试运行，是为了区分“元数据的差异”和“规格的差异”：后者意味着接管时会真正改变正在运行的 Argo CD（重启、配置变化），必须先弄清楚。
5. 门禁的渲染能发现 chart 不接受我们的 values（模板错误、校验失败），但发现不了 CRD 的结构变化对现有对象的影响：渲染只生成清单，不与集群里的对象比较。同步时 API server 会拒绝不符合新结构的对象，或者旧对象中的字段被静默丢弃——这时问题已经在生产中。补充的检查：在流水线中把新 chart 的 CRD 与集群当前的 CRD 做差异比较（字段被删除、类型改变即失败），并用 `kubectl diff --server-side` 对一个测试集群做试运行；跨大版本时先在临时集群（例如第 11 章的 kwokctl 集群）中完整同步一次。
6. 保留期至少要覆盖“发现问题所需的时间”加上恢复点目标：例如勒索软件或误删可能几天甚至几周后才被发现，第 8 章每天的备份就需要保留 30 天以上才能在发现后找到一份干净的副本。设得过长：存储成本上升；误存的个人数据在到期前无法删除，可能与“被遗忘权”等法规冲突；合规模式下连账户的 root 用户也删不掉，唯一的办法是删除整个账户。设得过短：在问题被发现之前，所有干净的副本都已过期。事故 E11-1 说明，备份能恢复还需要密钥：加密密钥（或解封密钥）应当有多份、分开保管（例如 Shamir 分片交给不同的人，或托管在独立账户的 KMS 中），访问需要审批，并且定期用这些密钥完成一次真实的恢复演练。

附录 B　术语表

按拼音与字母顺序排列；括号内是首次展开讲解的章。

allocatable（可分配资源）　节点容量减去系统预留和驱逐阈值之后，调度器可以分给 Pod 的部分。（第 1 章）

ambient 模式　Istio 不向 Pod 注入 sidecar 的数据面：每个节点一个 ztunnel 负责第 4 层，按需部署的 waypoint 负责第 7 层。（第 3 章）

APF（API Priority and Fairness）　API server 按优先级分配并发席位、排队并在过载时返回 429 的机制。它限制同时执行的请求数，不限制内存。（第 11 章）

app-of-apps　一个 Argo CD 应用指向一个目录，目录里的每个文件又是一个应用；整个平台由此从 Git 引导。（第 2 章）

burn rate（燃烧率）　错误预算被消耗的速度；1 表示恰好在 SLO 周期结束时用完。多窗口、多燃烧率告警同时看长短两个窗口。（第 10 章）

compaction（压缩）与 defrag（碎片整理）　etcd 清除旧修订版本，以及把数据库文件重写、把空间还给文件系统的两个操作。（第 11 章）

DORA 指标　部署频率、变更前置时间、变更失败率、恢复时间。（第 9 章）

drift（漂移）　真实状态偏离了声明的状态。（第 12 章）

eviction（驱逐）　节点资源不足时，kubelet 按顺序终止 Pod 以保护节点。（第 1、5 章）

exemplar　直方图桶上附带的一条链路 ID，把指标上的一个点连到具体的请求。（第 10 章）

FlowSchema / PriorityLevelConfiguration　APF 的两种对象：前者把请求分类，后者定义一个优先级的席位、队列与借用。（第 11 章）

GitOps　以 Git 为唯一的声明来源，由控制器持续把运行状态拉向它。（第 2、9 章）

HBONE　ambient 网格节点之间的隧道：mTLS 之上的 HTTP/2 CONNECT，端口 15008。（第 3 章）

HPA / KEDA　按指标（HPA）或按事件源如队列积压（KEDA）自动调整副本数。（第 5 章）

informer　客户端的缓存模式：LIST 一次，然后靠 WATCH 保持更新。（第 11 章）

KWOK　不运行 kubelet 的 Kubernetes：真实的控制面管理虚拟的节点与 Pod。（第 11 章）

MVCC　多版本并发控制；etcd 为每次写入保留一个修订版本。（第 11 章）

outbox（事务性发件箱）　把要发的消息与业务数据写进同一个事务，再由中继发布，解决“双写”问题。（第 2 章）

PITR（时间点恢复）　基础备份加上重放 WAL，把数据库恢复到指定的时刻。（第 8 章）

QoS 类别　Guaranteed、Burstable、BestEffort：由 Pod 的请求与限制决定。（第 5 章）

RPO / RTO　故障时最多丢失多少数据（恢复点目标）与多久恢复服务（恢复时间目标）。（第 8 章）

saga　用一串可补偿的本地事务完成跨服务的业务流程。（第 2 章）

selfHeal　Argo CD 发现集群偏离 Git 时自动改回。（第 12 章）

SLI / SLO / 错误预算　衡量服务好坏的指标、它的目标值，以及目标允许的失败量。（第 7、10 章）

server-side apply 与字段所有权　由 API server 合并配置，并记录每个字段由哪个管理者设置；别的管理者改它会产生冲突。（第 12 章）

状态（state）与状态锁　IaC 工具记录“配置中的资源对应哪个真实对象”的文件，以及防止两次运行同时修改它的锁。（第 12 章）

同步波次（sync wave）　Argo CD 应用之间的先后顺序。（第 2、9 章）

同步冒用（sync impersonation）　Argo CD 以项目指定的服务账号而不是自己的高权限身份写入集群。（第 9 章）

尾部采样 / 头部采样　在整条链路结束之后，或在请求开始之时，决定是否保留链路。（第 10 章）

WAL（预写日志）　PostgreSQL 在修改数据之前先写的日志；复制、归档与时间点恢复都靠它。（第 8 章）

watch 缓存　API server 内存中的对象缓存，为 WATCH 和大多数 LIST 提供数据。（第 11 章）

waypoint　ambient 网格中按命名空间或服务部署的第 7 层代理（Envoy）。（第 3 章）

熔断器 / 隔舱 / 重试预算　调用方保护自己和下游的三种机制：对持续失败的依赖快速拒绝；限制在途调用数；限制重试占总请求的比例。（第 2、7 章）

死人开关（dead man’s switch）　一个永远在响的心跳告警；它停了，说明告警链路自己出了问题。（第 10 章）

协调遗漏（coordinated omission）　封闭模型的压测在系统变慢时自动少发请求，从而漏记最慢的那些请求。（第 4 章）

准入（admission）　请求通过认证和授权之后、写入之前的检查与修改。（第 6 章）

ztunnel　ambient 网格在每个节点上的第 4 层代理，负责 mTLS 与身份。（第 3 章）

附录 C　代码与实验索引

每一章的实现都在实验仓库的 `v2/` 目录下（`github.com/miku-wwl/devops-delivery`，分支 `v2/publication`）。下表按章列出产出与位置，路径相对于 `v2/`；前三章另附当时的验收记录。

第 1 章　平台基线：在一台电脑上模拟三可用区集群（E0）

产出	位置（相对于 `v2/`）
集群创建脚本（渲染 Kind 配置、节点尺寸、镜像缓存、负载均衡）	`platform/cluster/create.sh`
API 审计策略	`platform/cluster/audit-policy.yaml`
分层资源测量脚本	`platform/cluster/measure.sh`
暂停与恢复 v1 集群	`platform/cluster/v1-lab.sh`
渲染后的集群配置与全部测量记录	[1]

验收标准（PLAN E0）： 集群可重复创建；资源预算来自实测。已通过：脚本可重复执行（第二次执行只补齐缺失的部分，见 [2]），本章的预算表中每个数字都有测量记录。

第 2 章　多服务系统：storefront 与它的平台（E1）

产出	位置（相对于 `v2/`）
六个服务的源代码与测试（一个镜像）	`apps/storefront/`
GitOps 配置仓库的内容（平台组件、服务 Chart、环境配置）	`gitops/`
引导脚本（密钥、镜像、配置仓库、Argo CD）	`platform/bootstrap/`
看板即代码	`platform/dashboards/gen.py`
负载脚本与分布统计	`experiments/load/`
测试、收敛、链路、负载与排障的全部记录	[1]

验收标准（PLAN E1）： 一笔订单的完整链路（同步 + 异步）在 Tempo 中可见；单元测试与集成测试通过。已通过：订单 `f9c9a183` 的链路包含 6 个服务、65 个 span，从网关的请求一直延续到通知服务消费消息 [2]；28 个测试（其中 4 个使用真实的 PostgreSQL、Valkey 和 NATS）全部通过，覆盖率 89% [3]。

第 3 章　服务网格：身份、加密与 L7 流量（E2）

产出	位置（相对于 `v2/`）
Istio 1.31.4 ambient、Gateway API v1.6.3 的 GitOps 配置	`gitops/platform/apps/istio.yaml`、`gitops/platform/values/`

产出	位置（相对于 `v2/`）
命名空间加入网格、waypoint、网格监控	`gitops/platform/manifests/namespaces/`、`gitops/platform/manifests/mesh/`
实验脚本：连接普查、线上抓包、单跳成本、交替 A/B、重试叠加	`experiments/load/connections.sh`、`experiments/e2/`
全部测量记录	[1]

验收标准（PLAN E2）： 有网格时长连接上的请求均匀分布；mTLS 有抓包或指标证据。已通过：经过 waypoint 后，各服务两个副本承担的请求在 46.9%–53.1% 之间，而无网格时 inventory 为 97.1% / 2.9%[2, 3]；加入网格后，节点线上可读的服务间请求行从 140 行变为 0[4]。

第 4 章　负载与容量规划（E3）

产出	位置（相对于 `v2/`）
按时间窗口统计资源、响应时间与排队的工具	`experiments/e3/window_stats.py`
阶梯负载（一次运行、带预热与过渡、按档位统计）	`experiments/e3/step-ramp.sh`、`experiments/load/storefront.js`
不改动 Pod 的 CPU 剖析（只取持有 GIL 的样本，附线程 CPU）	`experiments/e3/profile.sh`、`experiments/e3/top_frames.py`
新旧版本并排对比（金丝雀）	`experiments/e3/ab-canary.sh`
容量模型，以及在实验之前提交的预测	`experiments/e3/capacity_model.py`、[1]
实验环境负载均衡器的修正	`platform/cluster/lb-limits.sh`
容量看板	Grafana “Storefront: load and saturation”（`platform/dashboards/gen.py`）
全部测量记录	[2]

第 5 章　弹性与调度（E4）

产出	位置（相对于 `v2/`）
可选的 HPA 与 KEDA 模板（storefront Chart）	`gitops/charts/storefront-service/templates/`
KEDA（经 GitOps 安装）	`gitops/platform/apps/keda.yaml`
扩缩容、限流、邻居干扰、驱逐、抢占、配额、拓扑分布的实验脚本	`experiments/e4/`
全部测量记录	[1]

第 6 章　多租户与安全治理（E5）

产出	位置（相对于 `v2/`）
租户命名空间、RBAC 角色、配额与默认值、网络策略	`gitops/platform/manifests/tenants/`
实验用户（集群 CA 签发的客户端证书）	`platform/tenants/users.sh`
Kyverno 与准入策略	`gitops/platform/apps/kyverno.yaml`、`gitops/platform/manifests/policies/`
OpenBao 与 External Secrets Operator	`gitops/platform/apps/secrets.yaml`、`platform/secrets/openbao-init.sh`
权限矩阵、可达性矩阵、审计、准入、密钥实验	`experiments/e5/`
全部测量记录	[1]

第 7 章　故障注入与传播（E6）

产出	位置（相对于 `v2/`）
故障注入实验脚本（依赖故障、节点失效、可用区分区）	`experiments/e6/`
实验前提交的假设	[1, 2]
修复：waypoint 两个副本与中断预算、通知服务至少两个副本、PriorityClass、资源请求	`gitops/platform/manifests/mesh/`、`gitops/platform/manifests/policies/priorityclasses.yaml`、`gitops/platform/values/`
全部测量记录与两份事故记录	[3]

第 8 章　数据可靠性与灾难恢复（E7）

产出	位置（相对于 `v2/`）
集群外的备份目标：Garage 对象存储，每个使用者一个桶和一个密钥	`platform/backup/`
cert-manager 与 Barman Cloud 插件、基础备份与连续 WAL 归档、同步复制、数据库告警	`gitops/platform/apps/backup.yaml`、`gitops/platform/manifests/data/`
OpenBao 每小时的 raft 快照与恢复脚本	`gitops/platform/manifests/backup/`、`platform/secrets/openbao-restore.sh`
Velero（Kubernetes 对象的每日备份）	`gitops/platform/values/velero.yaml`
演练工具：写入器、故障切换、时间点恢复、Velero、集群重建	`experiments/e7/`
每组实验前提交的假设、全部测量记录与一份事故复盘	[1]

第 9 章　发布治理（E8）

产出	位置（相对于 v2/ ）
配置仓库的合并请求门禁：受保护的 main、流水线（渲染 + 发布策略 + 冻结检查）、开发者 alice	platform/gitops/setup.sh、platform/gitops/mr.sh、gitops/.gitlab-ci.yml、gitops/policies/release/
第 7 层入口（Gateway API）与带权重、按请求头预览的金丝雀	gitops/platform/manifests/mesh/ingress-storefront.yaml、gitops/platform/manifests/routes/
Argo CD 按项目的同步冒用（sync impersonation）：每个项目一个部署账号	gitops/bootstrap/、gitops/platform/apps/00-projects.yaml、gitops/platform/manifests/tenants/argocd-deployers.yaml
变更冻结：流水线的冻结检查与 Argo CD 同步窗口	gitops/.gitlab-ci.yml、gitops/platform/apps/00-projects.yaml
部署事件（Argo CD 通知 → GitLab 部署记录）与 DORA 指标脚本	gitops/bootstrap/argocd-values.yaml、platform/gitops/deploy-events.sh、platform/gitops/dora.py
9 个同步波次的布局与两处顺序依赖的修复；无停顿的重建驱动	gitops/platform/apps/、experiments/e8/rebuild-run.sh
演练工具与发布看板	experiments/e8/、platform/dashboards/gen.py
每组实验前提交的假设与全部测量记录	[1]

第 10 章　可观测性与事故响应（E9）

产出	位置（相对于 v2/ ）
集群外的告警桥：Alertmanager → GitLab 事故与工单，死人开关	platform/alerting/bridge.py、platform/alerting/run-bridge.sh
告警路由、抑制与平台告警（Argo CD、OpenBao、Velero、Alertmanager 重启），处置手册	gitops/platform/values/monitoring.yaml、gitops/platform/manifests/alerts/、docs/runbooks.md
SLO 规格与生成器：记录规则、多窗口多燃烧率告警、SLO 看板	platform/slo/、platform/dashboards/gen.py
节点级日志（Fluent Bit → Loki）、尾部采样、记录服务账号读取 Secret 的审计规则	gitops/platform/values/node-logs.yaml、gitops/platform/values/otel-collector.yaml、platform/cluster/audit-policy.yaml
事故演练脚本、时间线与复盘	experiments/e9/、[1]
每组实验前提交的假设与全部测量记录	[2]

第 11 章　控制面的规模边界（E10）

产出	位置（相对于 v2/ ）
kwokctl 安装脚本（固定版本，按发布页的摘要校验 SHA-256）	platform/kwok/install.sh、platform/versions.env

产出	位置（相对于 v2/ ）
规模实验驱动：建集群并设内存上限、批量创建 Pod、记录状态、LIST、清理	`experiments/e10/scale.sh`
etcd 体量、压缩与碎片整理	`experiments/e10/etcd-size.sh`
APF 洪水实验、API server 重启探测	`experiments/e10/apf.sh` 、 `experiments/e10/restart-probe.sh`
实验前提交的假设、全部测量记录与结果	[1, 2]
事故 E10-1 的复盘	[3]

第 12 章　基础设施即代码与云迁移（E11）

产出	位置（相对于 v2/ ）
OpenTofu 安装脚本（固定版本，两处来源校验 SHA-256）	`platform/iac/install.sh` 、 `platform/versions.env`
GitLab 治理的配置：项目设置、main 的保护、成员；导入块；状态加密；状态后端	`iac/gitlab/` 、 `iac/tofu.sh` 、 `platform/iac/state-project.sh`
状态与锁的检查、迁移后的一致性检查、漂移实验	`experiments/e11/state-check.sh` 、 `state-lock-gitlab.sh` 、 `state-consistency.sh` 、 `drift.sh`
Argo CD 管理自身：应用、门禁中的渲染、bootstrap 的唯一版本来源	`gitops/platform/apps/argocd.yaml` 、 `gitops/ci/render.sh` 、 `platform/bootstrap/bootstrap.sh` 、 `experiments/e11/watch-argocd.sh`
云迁移分析（含不可删除的备份）	`docs/cloud-migration.md`
假设、记录、结果与事故 E11-1	[1]

第 13 章　能力考核（E12）

产出	位置（相对于 v2/ ）
考核说明：规则、运行方式、安全措施	`assessment/README.md`
架构设计题（5 道，每道 20 分，附评分要点）	`assessment/design.md`
故障排查题（注入、判分、恢复）	`assessment/t1-argocd-repo.sh` 、 `t2-openbao-restart.sh` 、 `t3-out-of-band.sh` 、 `lib.sh`
实操题（自动判分）	`assessment/p1-inhibition.sh`
假设、运行记录与结果	[1]

附录 D　证据索引

正文中的上标数字（如 [3]）是证据编号：每一章单独编号，对应下面列出的原始记录——终端输出、查询结果、时间线与复盘。文件都在实验仓库的 `v2/` 目录下，路径相对于 `v2/`。全书另有 572 处具体陈述与这些文件逐条自动比对（`v2/manual/spot-checks.json`）。

第 1 章　平台基线：在一台电脑上模拟三可用区集群（E0）

[1] evidence/e0/
[2] evidence/e0/cluster-create-rerun.log
[3] evidence/e0/v1-pause.log
[4] evidence/e0/baseline.log
[5] evidence/e0/host.log
[6] evidence/e0/node-view-details.log
[7] evidence/e0/audit-volume.log
[8] evidence/e0/ent-cluster.rendered.yaml
[9] evidence/e0/cluster-create.log
[10] evidence/e0/cgroup-tree.log
[11] evidence/e0/smoke.log
[12] evidence/e0/lb-cold-start.log
[13] evidence/e0/v1-restart-check.log

第 2 章　多服务系统：storefront 与它的平台（E1）

[1] evidence/e1/
[2] evidence/e1/one-order-trace.log
[3] evidence/e1/app-tests-3.log
[4] evidence/e1/bootstrap.log
[5] evidence/e1/converge.log
[6] evidence/e1/pg-checkpoints.log
[7] evidence/e1/pg-join-timing.log
[8] evidence/e1/baseline-k6.txt
[9] evidence/e1/latency-buckets.log
[10] evidence/e1/baseline-distribution.log
[11] evidence/e1/v1-came-back.log
[12] evidence/e1/cp-restart.log
[13] evidence/e1/reboot-pg-failover.log
[14] evidence/e1/reboot-pg-after.log

第 3 章　服务网格：身份、加密与 L7 流量（E2）

[1] evidence/e2/
[2] evidence/e2/mode-ambient-l7.log
[3] evidence/e2/mode-none.log
[4] evidence/e2/capture-compare.log
[5] evidence/e2/chart-repos.log
[6] evidence/e2/istio-install.log
[7] evidence/e2/capture-none.log
[8] evidence/e2/enrol.log
[9] evidence/e2/enrol-cni.log
[10] evidence/e2/mode-ambient-l4.log
[11] evidence/e2/compare-none-l4.log
[12] evidence/e2/hop-cost-ambient.log
[13] evidence/e2/hop-cost-ambient-2.log
[14] evidence/e2/ab-mesh.log
[15] evidence/e2/ab-waypoint.log
[16] evidence/e2/retry-matrix.log
[17] evidence/e2/istio-default-retry.log
[18] evidence/e2/strict.log
[19] evidence/e2/ambiguous-outcome-trace.log
[20] evidence/e2/ambiguous-outcome-2.log
[21] evidence/e2/charged-but-failed.log
[22] evidence/e2/exec-stall.log
[23] evidence/e2/app-tests-2.log
[24] evidence/e2/soak-stalls.log
[25] evidence/e2/soak-gc.log

第 4 章　负载与容量规划（E3）

[1] evidence/e3/capacity-prediction.md
[2] evidence/e3/
[3] evidence/e3/app-tests.log
[4] evidence/e3/tracing-cost.log
[5] evidence/e3/tempo-gc.log
[6] evidence/e3/tempo-gc-2.log
[7] evidence/e3/tempo-heap-before.pprof.txt
[8] evidence/e3/tempo-heap-top.log
[9] evidence/e3/step-load-1.log
[10] evidence/e3/step-start-burst.log
[11] evidence/e3/lb-crash.log
[12] evidence/e3/lb-storm-180.log
[13] evidence/e3/lb-storm-180-fixed.log
[14] evidence/e3/lb-storm-180/readiness.log
[15] evidence/e3/storm-probe-180.log
[16] evidence/e3/storm-net-180.log
[17] evidence/e3/burst-connect.log
[18] evidence/e3/profile-60.log
[19] evidence/e3/profile-method.log
[20] evidence/e3/profile-gil-60-summary.log
[21] evidence/e3/app-tests-2.log
[22] evidence/e3/ab-canary-gateway-60.log

[23] evidence/e3/ab-canary-gateway-60-samenode.log
[24] evidence/e3/steps-run2-low/window-40.json
[25] evidence/e3/little-40.log
[26] evidence/e3/step-ramp-low.log
[27] evidence/e3/capacity-model.log
[28] evidence/e3/step-ramp-high.log
[29] evidence/e3/step-ramp-high-events.log
[30] evidence/e3/step-ramp-high-traces.log
[31] evidence/e3/one-replica.log
[32] evidence/e3/one-replica-2.log
[33] evidence/e3/one-replica-2-analysis.log
[34] evidence/e3/prometheus-retention.log

第 5 章　弹性与调度（E4）

[1] evidence/e4/
[2] evidence/e4/hpa-switch.log
[3] evidence/e4/hpa-timeline-collect.log
[4] evidence/e4/hpa-timeline-2.log
[5] evidence/e4/hpa-2-analysis.log
[6] evidence/e4/hpa-2-pods.log
[7] evidence/e4/hpa-timeline-3.log
[8] evidence/e4/host-cpu.txt
[9] evidence/e4/drift-check.log
[10] evidence/e4/keda-install.log
[11] evidence/e4/keda-crd-size.log
[12] evidence/e4/keda-steady.log
[13] evidence/e4/keda-steady-2.log
[14] evidence/e4/keda-compare.log
[15] evidence/e4/keda-steady-failures.log
[16] evidence/e4/keda-steady-rejections.log
[17] evidence/e4/ab-limit-300m.log
[18] evidence/e4/ab-limit-300m-table.log
[19] evidence/e4/noisy.log
[20] evidence/e4/noisy-rejections.log
[21] evidence/e4/cgroup-weights.log
[22] evidence/e4/eviction-prediction.md
[23] evidence/e4/eviction-1.log
[24] evidence/e4/eviction-2.log
[25] evidence/e4/eviction-2/watch.txt
[26] evidence/e4/eviction-2-oom-check.log
[27] evidence/e4/eviction-2-pagecache.log
[28] evidence/e4/eviction-2-vm-memory.log
[29] evidence/e4/eviction-2-logs.log
[30] evidence/e4/preemption.log
[31] evidence/e4/quota.log
[32] evidence/e4/spread.log

第 6 章　多租户与安全治理（E5）

[1] evidence/e5/
[2] evidence/e5/users.log
[3] evidence/e5/rbac-matrix-5.log
[4] evidence/e5/rbac-matrix-1.log
[5] evidence/e5/rbac-matrix-2.log
[6] evidence/e5/who-did-what.log
[7] evidence/e5/rbac-matrix-3.log
[8] evidence/e5/who-1/audit.jsonl
[9] evidence/e5/gitops-chain.log
[10] evidence/e5/reach-baseline-2.log
[11] evidence/e5/mesh-np-2.log
[12] evidence/e5/kindnet-nft.log
[13] evidence/e5/mesh-np-diag2-2.log
[14] evidence/e5/mesh-np-port.log
[15] evidence/e5/mesh-np-diag5.log
[16] evidence/e5/probe-headers.log
[17] evidence/e5/reach-after.log
[18] evidence/e5/net-validate.log
[19] evidence/e5/net-validate-2.log
[20] evidence/e5/admission.log
[21] evidence/e5/admission-restart-a.log
[22] evidence/e5/admission-restart-b.log
[23] evidence/e5/admission-generate-b.log
[24] evidence/e5/argocd-rbac-2.log
[25] evidence/e5/argocd-rbac-1.log
[26] evidence/e5/argocd-projects.log
[27] evidence/e5/openbao-init-1.log
[28] evidence/e5/openbao-init-3.log
[29] evidence/e5/openbao-facts.log
[30] evidence/e5/secrets.log
[31] evidence/e5/cleanup.log

第 7 章　故障注入与传播（E6）

[1] evidence/e6/node-failure-prediction.md
[2] evidence/e6/partition-prediction.md
[3] evidence/e6/
[4] evidence/e6/slow-resilient.log
[5] evidence/e6/slow-naive.log
[6] evidence/e6/errors-naive.log
[7] evidence/e6/errors-resilient-2.log
[8] evidence/e6/dependency-prediction.md
[9] evidence/e6/errors50-resilient.log
[10] evidence/e6/errors50-recovery.log
[11] evidence/e6/node-1.log
[12] evidence/e6/incident-2c.log
[13] evidence/e6/spof-fix.log
[14] evidence/e6/metrics-server-usage.log
[15] evidence/e6/node-2.log
[16] evidence/e6/node-2-analysis.log
[17] evidence/e6/partition-1.log
[18] evidence/e6/partition-1-analysis.log
[19] evidence/e6/incident-1.log
[20] evidence/e6/incident-2.log
[21] evidence/e6/incident-2b.log

第 8 章　数据可靠性与灾难恢复（E7）

[1] evidence/e7/
[2] evidence/e7/facts.log
[3] evidence/e7/s3-keys.log
[4] evidence/e7/resize.log
[5] evidence/e7/incident-1/archive.txt
[6] evidence/e7/openbao-restart.log
[7] evidence/e7/failover-prediction.md
[8] evidence/e7/failover-results.md
[9] evidence/e7/f0-rollout/analysis.md
[10] evidence/e7/f1-delete/analysis.md
[11] evidence/e7/f2-freeze/analysis.md
[12] evidence/e7/lag-metric.log
[13] evidence/e7/incident-1/postmortem.md
[14] evidence/e7/incident-1/repair.log
[15] evidence/e7/alert-false-positive.log
[16] evidence/e7/f4-cut-freeze-sync/alerts.log
[17] evidence/e7/pitr-prediction.md
[18] evidence/e7/pitr-results.md
[19] evidence/e7/pitr-2/archive-lag.log
[20] evidence/e7/pitr-1/reconcile.log
[21] evidence/e7/velero-prediction.md
[22] evidence/e7/v1-coverage/
[23] evidence/e7/v2-restore/run.txt
[24] evidence/e7/s1-openbao/run.txt
[25] evidence/e7/openbao-prediction.md
[26] evidence/e7/rebuild-prediction.md
[27] evidence/e7/r1/results.md
[28] evidence/e7/r1/verify.txt
[29] evidence/e7/r1/fixes.log
[30] evidence/e7/rebuild-2-prediction.md
[31] evidence/e7/r2/results.md

第 9 章　发布治理（E8）

[1] evidence/e8/
[2] evidence/e8/gate-prediction.md
[3] evidence/e8/hpa-prediction.md
[4] evidence/e8/canary-prediction.md
[5] evidence/e8/impersonation-prediction.md
[6] evidence/e8/freeze/run.txt
[7] evidence/e8/freeze-prediction.md
[8] evidence/e8/dora-*.txt
[9] evidence/e8/waves-prediction.md

第 10 章　可观测性与事故响应（E9）

[1] evidence/e9/incident/
[2] evidence/e9/
[3] evidence/e9/clock-drift.log
[4] evidence/e9/alerting-prediction.md
[5] evidence/e9/a3-backtest.log
[6] evidence/e9/logs-audit-prediction.md
[7] evidence/e9/incident-prediction.md
[8] evidence/e9/incident/postmortem.md
[9] evidence/e9/exemplars-prediction.md
[10] evidence/e9/tracing-cost.log

第 11 章　控制面的规模边界（E10）

[1] evidence/e10/scale-prediction.md
[2] evidence/e10/scale-results.md
[3] evidence/e10/incident-1.md

第 12 章　基础设施即代码与云迁移（E11）

[1] evidence/e11/
[2] evidence/e11/incident-1.md

第 13 章　能力考核（E12）

[1] evidence/e12/